
BACHELOR OF SCIENCE IN INFORMATION TECHNOLOGY

ARTIFICIAL INTELLIGENCE

Search, Knowledge and Reasoning

UNIFIED LECTURE HANDOUT

Complete Course in One Volume • Fall 2026



Every example drawn from IT project management

Task dependencies and release plans, sprints and staffing, tickets and escalations, defects and risk.

What this volume covers. The whole of the HEC course *Artificial Intelligence* — every topic of the HEC 2017 outline, every topic of the HEC 2023 outline, and every learning outcome HEC 2025 attaches to the course — in one resequenced volume, taught from first principles to working code.

The course is deliberately *symbolic*: it teaches the agent that is told the rules and must reason with them. Learning is one signpost chapter, not a unit, because *Machine Learning* owns it three semesters later. Appendix F states every boundary.

Six Parts • Twenty-Two Chapters • Seven Appendices

How to Use This Handout

This is not a reference manual to be searched; it is a **path** to be walked. The chapters are ordered so that every idea rests on ideas you have already met, and the order is not the order of the HEC outline — three deliberate departures are set out at the end of the Course Specification.

The six parts

Part I — Foundations of Intelligent Systems (Chapters 1–3) fixes what an agent is, how to specify a task before building anything, and the four data structures every later algorithm is written in. Chapter 3 is the toolkit chapter: everything after it says “push the successor onto the frontier” and means something precise.

Part II — Problem Solving by Search (Chapters 4–6) is the engine room. Uninformed search and why it cannot scale; heuristics and A^* , which is the single most useful algorithm in the book; and local search for the problems where only the final state matters.

Part III — Constraints, Games and Plans (Chapters 7–9) applies that engine three ways: to problems where the answer must satisfy rules simultaneously, to problems where another party is choosing against you, and to problems where the answer is a sequence of actions.

Part IV — Knowledge, Logic and Reasoning (Chapters 10–13) changes the question from *what should I do* to *what follows from what I have been told*. It ends with a working rule engine that can explain its own conclusions — the artefact an auditor accepts.

Part V — Reasoning Under Uncertainty (Chapters 14–16) handles the case where what you have been told is not certain: probability and expected utility, Bayesian networks, and fuzzy logic.

Part VI — Learning, Language and Responsible AI (Chapters 17–22) closes the course. One signpost chapter on learning, the symbolic half of natural language processing, the four classic systems read through the machinery you now have, current trends, the limits, and the project.



Why search comes before logic, when both published outlines list reasoning first. Search is the part of AI you can *run* on the first day, and Chapters 7, 8 and 9 are all search with extra structure — so teaching it first means three chapters cost almost nothing to introduce. Logic then arrives as a change of question rather than a change of subject. The cost of this choice is that Chapter 10 is nine weeks in; the benefit is that by then students have written five search algorithms and know what a state space is in their hands, not just on a slide.

The furniture of every chapter

Every chapter has the same parts, in the same order. Learn to read them once.

Box	What it is for
🎯 Outcomes	What you will be able to <i>do</i> . Bloom verbs, not topics. Use it as a checklist before the examination.
🔗 Prerequisites	Which earlier chapters this one stands on. If these are shaky, go back; nothing here is improved by pressing on.
📖 Key terms	The vocabulary introduced. Every one appears in the glossary of Appendix D.

Box	What it is for
 Definition	The canonical statement, with its source. Learn these verbatim.
 Concept	The judgement call — what an experienced practitioner would decide here, and why.
 Warning	Something that will bite you, and usually silently.
 Worked example	Real arithmetic, every step shown. Do them with a pen.
 Common pitfalls	The mistakes that are actually made, with the symptom each produces.
 Try it yourself	The laboratory. Each is a runnable file in <code>code/</code> , named for its chapter.
 Checkpoint	Three questions. Attempt them before reading the answers in Appendix C.
 Chapter summary	What to retain.
 Review questions	Examination practice: multiple choice, short answer, applied.

How to study with it

1. Read the outcomes first. They tell you what the chapter is *for*.
2. Read the prose once without stopping, then again with a pen on the worked examples. Every number in this book was computed before it was typed; you should be able to reproduce each one.
3. Run the lab *before* the lecture on that chapter if you can. The code is short and the questions it raises are the questions the lecture answers.
4. Do the checkpoint from memory. Then read Appendix C and argue with it.
5. Attempt the applied review questions in pairs. They are written to be discussed, not looked up.

On the laboratories

Every lab in this book runs on a plain Python 3.10 installation. Chapters 10–13 and 18–21 use nothing but the standard library; the rest add NumPy. No lab needs a GPU, a network connection or an account with anybody. That is a deliberate property of a symbolic course, and it is worth noticing: the methods in this book were designed when a megabyte was a lot of memory, and they still fit in one.

Course Specification

This page is the course file. It states what HEC requires of a course called *Artificial Intelligence*, and shows where in this handout each requirement is met. Three editions are in force in different programmes, so all three are traced.

Course particulars

Item	Value
Course title	Artificial Intelligence
Programme	BS Information Technology. Under HEC 2023 this is a <i>computing core</i> course — compulsory in every computing degree — taught in Semester 3; under HEC 2017 it was a BSCS domain core absent from BSIT; under HEC 2025 it is a major core in Semester V
Credit hours	3 (2–3): two credit hours of lectures and one credit hour of laboratory, which is three contact hours of lab per week (HEC 2023). HEC 2017 wrote the same course as 3+1
Prerequisite	Object Oriented Programming (HEC 2023). HEC 2017 required <i>Data Structures and Algorithms</i> , which is the stronger requirement; Chapter 3 supplies the data structures this course actually needs, so the weaker prerequisite is sufficient here
Duration	Sixteen teaching weeks, with a midterm after Week 8 and a final examination after Week 16 (Appendix G)
Course introduction	The HEC 2023 aim, verbatim: “An Introduction to Artificial Intelligence and its applications towards Knowledge Based Systems . . . Python programming language will be used to explore and illustrate various issues and techniques in Artificial Intelligence.”
Set texts	S. J. Russell and P. Norvig, <i>Artificial Intelligence: A Modern Approach</i> , 3rd ed., Prentice Hall, 2010 (both editions); G. F. Luger and W. A. Stubblefield, <i>AI Algorithms, Data Structures, and Idioms in Prolog, Lisp and Java</i> , Pearson, 2009; R. O. Duda, P. E. Hart and D. G. Stork, <i>Pattern Classification</i> , Wiley, 2001 (HEC 2017); D. Poole and A. Mackworth, <i>Artificial Intelligence: Foundations of Computational Agents</i> , 2nd ed., 2017

Course learning outcomes

HEC 2025 publishes five outcomes for this course and no topic outline; HEC 2017 and HEC 2023 publish outlines and expect Bloom-levelled outcomes. The seven below are the union, and each is traced to the chapters that deliver it.

CLO	By the end of the course the student will be able to	Bloom	Chapters
CLO-1	Describe core AI concepts, including intelligent agents, search algorithms, knowledge representation and reasoning	C2 Understand	1, 2, 4, 5, 10, 11
CLO-2	Formulate a real problem as a search, constraint-satisfaction or planning problem, and implement an algorithm that solves it	C3 Apply	3–9
CLO-3	Represent knowledge formally and derive conclusions from it by inference	C3 Apply	10–13
CLO-4	Reason under uncertainty and select an action by expected utility	C3 Apply	14–16
CLO-5	Analyze AI systems and the applications of AI algorithms in real-world scenarios	C4 Analyze	19, 20, and every lab
CLO-6	Develop and test a working AI solution of moderate size using appropriate tools	C6 Create	3, 22, and every lab
CLO-7	Discuss the ethical considerations and the limitations of AI systems	A3 Valuing	21, and the audit in 22

Coverage of the HEC 2017 course outline

Every topic in the published outline, in the order HEC lists it, with the chapter that teaches it. Nothing is omitted.

HEC 2017 outline topic	Chapter	Where exactly
Introduction; basic components of AI	1	The agent definition; the four classical definitions on two axes
Identifying AI systems	1	<i>Is It AI? A Checklist</i> — the five-point test, applied to five tools in the chapter's lab
Branches of AI	1	The branch diagram, and which branch each course in the degree owns
Reasoning and knowledge representation	10	Semantic networks, inheritance, frames and defaults, ontologies
Propositional logic	11	Syntax, semantics, entailment, model checking, DPLL, resolution
First-order logic	12	Quantifiers, unification with the occurs check, skolemisation, resolution refutation
Problem solving by searching: uninformed	4	Breadth-first, depth-first, depth-limited, iterative deepening, uniform cost
Problem solving by searching: informed	5	Greedy best-first, A^* , admissibility, consistency, dominance, relaxation
Problem solving by searching: local	6	Hill climbing, random restart, simulated annealing, beam search, genetic algorithms

HEC 2017 outline topic	Chapter	Where exactly
Constraint satisfaction problems	7	Backtracking, minimum remaining values, degree heuristic, forward checking, arc consistency
Adversarial search: minimax	8	Game trees and the minimax value
Adversarial search: alpha–beta pruning	8	Pruning, move ordering, and the best case it achieves
Adversarial search: game playing	8	Cutting off search, evaluation functions, the horizon effect
Learning: supervised	17	<i>Supervised Learning</i> — the perceptron worked numerically, and linear separability
Learning: unsupervised	17	<i>Unsupervised Learning</i> — grouping without labels, one figure, no algorithm
Learning: reinforcement	17	<i>Reinforcement Learning</i> — the agent–reward loop, as a signpost to <i>Machine Learning</i>
Uncertainty in AI	14	Probability as degree of belief, Bayes’ rule, conditional independence, expected utility
Fuzzy logic	16	Membership functions, fuzzy operators, Mamdani inference, defuzzification
Recent trends in AI	20	Large language models, tool-using agents, neuro-symbolic systems
Case study of AI systems	19	The General Problem Solver, Eliza, Student and Macsyma, each reconstructed in code
Analysis of AI systems	19	The six-column framework — task, environment, representation, inference, knowledge source, failure mode — applied to all four

Coverage of the HEC 2023 course outline

HEC 2023 outline topic	Chapter	Where exactly
Introduction to AI and its applications towards knowledge-based systems	1, 13	The field and its branches in Chapter 1; a working rule-based expert system in Chapter 13
Introduction to reasoning and knowledge representation	10	What a representation must support, and four that do
Problem solving by searching: informed, uninformed, heuristics, local	4, 5, 6	The whole of Part II
Minimax algorithm, alpha–beta pruning, game playing	8	Chapter 8 entire

HEC 2023 outline topic	Chapter	Where exactly
Case studies: General Problem Solver	9, 19	Means–ends analysis in Chapter 9, where the machinery belongs; the system analysed in Chapter 19
Case studies: Eliza	18, 19	Pattern substitution in Chapter 18; the system analysed in Chapter 19
Case studies: Student	19	Reconstructed using the unification of Chapter 12
Case studies: Macsyma	19	Reconstructed as rule rewriting, using Chapter 13
Learning from examples	17	The three paradigms, and where this course stops
Artificial neural networks	17	<i>Artificial Neural Networks</i> — the perceptron, the delta rule by hand, and the exclusive-or limit
Natural language processing	18	The symbolic pipeline: grammars, chart parsing, pattern substitution, n-grams
Recent trends in AI and applications of AI algorithms	20	Chapter 20 entire
Python will be used throughout	3, all labs	Chapter 3 fixes the representations; all 22 labs are runnable Python in <code>code/</code>

Coverage of the HEC 2025 learning outcomes

HEC 2025 publishes outcomes only — no prerequisite, no outline, no reading list.

HEC 2025 outcome	Chapters	Met by
Describe core AI concepts including search algorithms, knowledge representation, and reasoning	1, 2, 4–5, 10–12	CLO-1 and CLO-3
Implement basic AI algorithms for problem-solving and decision-making	3–9, 14	CLO-2 and CLO-4. <i>Decision-making</i> is met specifically by <i>From Belief to Decision</i> in Chapter 14: expected utility and the value of information
Analyze the applications of AI in real-world scenarios	19, 20	CLO-5, and the six-column analysis framework
Discuss ethical considerations and limitations of AI systems	21	CLO-7. <i>Limitations</i> is read broadly: Chapter 21 covers computational limits and brittleness as well as ethics
Develop simple AI models using appropriate tools and frameworks	3, 22	CLO-6. The “tools” here are the Python standard library and NumPy, deliberately — see the note below

 On “tools and frameworks”

HEC 2025’s fifth outcome asks for AI models built with appropriate tools and frameworks. This course uses the Python standard library and NumPy and no AI framework at all, and that is a deliberate choice that should be defended rather than hidden.

A framework is appropriate when it saves work you understand. For a first course in symbolic AI it does the opposite: a constraint solver called through a library teaches nothing about arc consistency, and a planner behind an API teaches nothing about means–ends analysis. Every algorithm in this book is short enough to write from scratch in one laboratory session, and writing it is the learning. Students who go on to *Programming for Artificial Intelligence* and *Machine Learning* meet the frameworks there, with the understanding to judge them.

Topics taught here that the published outlines do not name

Four chapters go beyond the letter of both outlines. They are listed here so that the departure is explicit, each with the clause it rests on.

Chapter	Why it is here	Clause relied on
9 Planning	Search with structure. It reuses Part II wholesale, and it is where the General Problem Solver — which HEC 2023 names as a required case study — actually belongs	HEC 2023, “Case Studies: General Problem Solver”
10 Knowledge Representation	Semantic networks, frames and ontologies are the representations that Chapters 11–13 reason over	HEC 2017 and 2023, “Reasoning and Knowledge Representation”
13 Rule-Based Expert Systems	The working knowledge-based system the 2023 aim points at, and the only place explanation is built rather than discussed	HEC 2023, “applications towards Knowledge Based Systems”
15 Bayesian Networks	The representation that makes uncertainty computable rather than merely acknowledged	HEC 2017, “Uncertainty in AI”



These four overlap with the HEC 2023 elective *Knowledge Representation and Reasoning* (3 (3-0), prerequisite *Artificial Intelligence*), and that is intended rather than accidental. This course teaches each of them *once, at first pass*, because reasoning is not teachable without a representation to reason over. The elective goes considerably further — description logic, reasoning over description logic, ontology languages, logical agents, commonsense reasoning and explainable AI — and none of that appears here. Appendix F sets the line out in full.

Three departures from the outline’s grouping



The content of both outlines is taught in full. The *grouping* departs in three places, each deliberately.

1. Search is taught before logic. Both outlines list reasoning and knowledge representation first. Here Part II comes first, so that Chapters 7, 8 and 9 can reuse its machinery and so that students write working code in Week 3.

2. Planning sits with search, not with knowledge. STRIPS progression search *is* the frontier of Chapters 4 and 5 applied to a new state space. Teaching it in Part III keeps means-ends analysis next to the General Problem Solver that uses it.

3. Learning gets one signpost chapter, not a unit. HEC 2017 names three learning paradigms and HEC 2023 names artificial neural networks. All four clauses are covered, in named sections of Chapter 17, and then the course stops: the perceptron's exclusive-or limit is the last thing taught, and *Machine Learning* (Semester 6) takes over. Teaching more would duplicate that course and leave less room for the symbolic material no other course covers.

The Course at a Glance

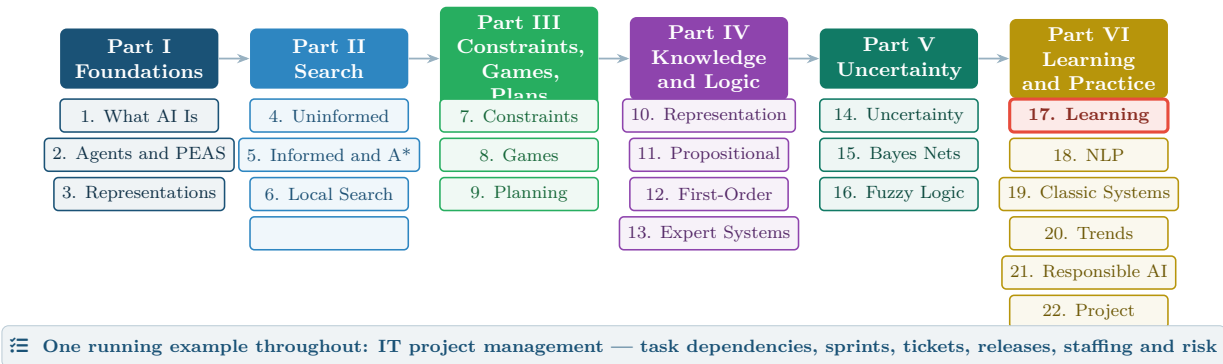


Figure F.1: The six parts and the chapters in each. Chapter 17 (outlined) is the signpost to *Machine Learning*: the three learning paradigms and the perceptron, and no further.

This course is the published prerequisite for Machine Learning, Programming for Artificial Intelligence, Knowledge Representation and Reasoning and Introduction to Data Science. Appendix F states what it hands to each of them. A one-page map of the whole Artificial Intelligence → Machine Learning → Data Science progression is distributed alongside this handout.

The Running Example

Every example in this book comes from IT project management. There are no other application domains and no lens panels. The point is not that project management is the most interesting use of AI; it is that a reader who has to learn a new domain in every chapter is spending attention on the domain instead of on the method. One domain, met repeatedly, becomes invisible — which is what you want.

Chapter	The recurring example	What it is used to teach
1	Five tools in a project office	Which software is an AI system, and which is merely useful
2	Overnight ticket triage	PEAS, environment properties, and why an agent needs internal state
3	The nine-task release graph	States as values; the frontier; the Problem interface
4	Ordering the release tasks	Uninformed search, and the explored set
5	Manual against automated testing	Heuristics, A^* , and why an investment is invisible to greedy search
6	Staffing four workstreams	Local maxima, ridges, annealing and evolution
7	Sprint scheduling	Variables, domains, constraints, and propagation
8	Negotiating a release slot	Minimax, alpha–beta, and move ordering
9	Building a release plan	STRIPS operators and means–ends analysis
10	The delivery ontology	Semantic networks, inheritance, frames and defaults
11	The release gate	Propositional inference three ways
12	Who blocks whom	Quantifiers, unification and resolution
13	The escalation adviser	Forward and backward chaining, and explanation
14	Ship now or delay a week	Expected utility and the value of information
15	Diagnosing a defect’s cause	Bayesian network inference, four ways
16	Rating project risk	Membership functions and Mamdani inference
17	Predicting a late task	The perceptron, and the limit where this course stops
18	Ticket subject lines	Grammars, chart parsing and pattern substitution
19	Four classic systems	Read through the machinery of Chapters 9, 12, 13 and 18
20	A tool-using delivery agent	What current systems add, and what they do not change
21	Auditing the escalation rules	Bias, explanation as evidence, and computational limits
22	Your own project	The whole course, end to end

Notation

Symbol	Meaning
s, s'	A state, and a successor of it
b, d, m	Branching factor, depth of the shallowest solution, maximum depth of the space
$g(n)$	Cost of the path from the start to node n
$h(n)$	Heuristic estimate of the cost from n to a goal
$h^*(n)$	The true cheapest cost from n to a goal
$f(n)$	$g(n) + h(n)$, the estimate used by A^*
C^*	The cost of an optimal solution
X_i, D_i	A constraint variable and its domain
α, β	The bounds carried by alpha–beta pruning
$\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$	Not, and, or, implies, if and only if
$\models$	Entails: every model of the left is a model of the right
$\vdash$	Derives: the right is provable from the left by the inference procedure in use
$\forall, \exists$	For all, there exists
θ	A substitution, as produced by unification
$P(a), P(a \mid b)$	Probability of a ; probability of a given b
$EU(a)$	Expected utility of action a
$\mu_A(x)$	Degree of membership of x in fuzzy set A
w_i, η	A perceptron weight and the learning rate

⚠ One notational warning

$\models$ and $\vdash$ look similar and mean different things. $KB \models \alpha$ says α is *true* in every world where the knowledge base is true — a fact about meaning. $KB \vdash \alpha$ says α can be *derived* by a particular procedure — a fact about that procedure. Chapter 11 shows why an inference procedure is worth having exactly when the two coincide, and what it means when they do not.

Prerequisite Self-Check

Ten questions. If you can answer eight, you are ready for Chapter 1. If you cannot, the chapter named beside each one is where to look first — and every one of them is in Appendix A.

1. What is the difference between a Python `list` and a `tuple`, and why can only one of them be a dictionary key? (Chapter 3)
2. What does `{1,2} <= {1,2,3}` evaluate to, and what does the operator mean for sets? (Chapter 3)
3. Write a class with one method that subclasses override. What is this pattern for? (Object Oriented Programming)
4. What does the `yield` keyword do, and how does a generator differ from a list? (Chapter 3)
5. What is the time complexity of removing the first element of a Python list, and what should you use instead? (Chapter 3)
6. Given a dictionary mapping each task to the set of tasks it depends on, write one expression giving the tasks with no unmet dependency. (Chapter 3)
7. What is recursion, and what must every recursive function have? (Programming Fundamentals)
8. If a fair coin is tossed twice, what is the probability of at least one head? (Chapter 14)
9. What does it mean for two events to be independent, and write the consequence as an equation. (Chapter 14)
10. A truth table for two variables has how many rows? For n variables? (Chapter 11)



Notice what is *not* on that list: calculus, linear algebra, statistics and any experience of machine learning. This course needs none of them. It needs you to be fluent with sets, dictionaries, recursion and classes, and to be willing to work an example by hand before trusting the code.

Contents

How to Use This Handout	ii
Course Specification	iv
The Course at a Glance	x
The Running Example	xi
Notation	xii
Prerequisite Self-Check	xiii
I Foundations of Intelligent Systems	1
1 What Artificial Intelligence Is	3
1.1 Two Programs in a Project Office	3
1.2 Four Definitions, Two Axes	4
1.3 Is It AI? A Checklist	5
1.4 The Branches of AI	6
1.5 Sixty Years in Five Waves	6
1.6 What This Course Teaches, and What It Does Not	8
2 Intelligent Agents and Task Environments	13
2.1 Agent, Environment, Percept, Action	13
2.2 PEAS: Specify the Task Before You Build the Agent	14
2.3 Six Properties of a Task Environment	15
2.4 Five Kinds of Agent Program	16
2.5 Rationality Is Not Omniscience	17
3 Representations and Search in Python	22
3.1 Why This Chapter Exists	22
3.2 A State Is a Value	23
3.3 The Problem Interface	23
3.4 Frontiers: Three Structures, Three Strategies	24
3.5 Representing the Project	25
3.6 Generators and Memoisation	25
II Problem Solving by Search	32
4 Problem Formulation and Uninformed Search	34
4.1 From a Task to a State Space	34

4.2	The State Space and the Search Tree	35
4.3	Breadth-First Search	36
4.4	Depth-First and Depth-Limited Search	36
4.5	Iterative Deepening	36
4.6	Uniform-Cost Search	37
4.7	Why Uninformed Search Cannot Scale	37
5	Informed Search and Heuristics	43
5.1	A Sprint Where the Cost Matters	43
5.2	Heuristics: Telling the Search Where the Goal Is	44
5.3	Greedy Best-First Search, and Why It Is Not Enough	45
5.4	A* Search	45
5.5	Inventing a Heuristic	47
6	Local Search and Optimisation	54
6.1	When the Path Does Not Matter	54
6.2	The State-Space Landscape	55
6.3	Hill Climbing	55
6.4	Three Ways Out	56
6.5	Genetic Algorithms	57
6.6	Comparing Them Honestly	57
III	Constraints, Games and Plans	66
7	Constraint Satisfaction	68
7.1	A Sprint Plan as a Constraint Problem	68
7.2	Backtracking Search	69
7.3	Which Variable Next, and Which Value	70
7.4	Propagation: Forward Checking	70
7.5	Propagation: Arc Consistency	71
7.6	The Measurement	71
7.7	When There Is No Solution	72
8	Adversarial Search and Game Playing	79
8.1	When the Other Party Is Choosing Too	79
8.2	Minimax by Hand	80
8.3	Alpha–Beta Pruning	80
8.4	Move Ordering Is Everything	81
8.5	Cutting Off the Search	82
9	Planning	88
9.1	From a Task List to Operators	88
9.2	Planning as Progression Search	89
9.3	Means–Ends Analysis and the General Problem Solver	90
9.4	Goal Clobbering	91
9.5	What the Representation Buys	91

IV	Knowledge, Logic and Reasoning	99
10	Knowledge Representation	101
10.1	The Change of Question	101
10.2	Semantic Networks	102
10.3	Frames, Slots and Defaults	102
10.4	When Inheritance Has No Answer	103
10.5	Ontologies	104
10.6	Where This Course Stops	104
11	Propositional Logic and Inference	110
11.1	The Release Gate, Written Down	110
11.2	Syntax, Semantics and Models	111
11.3	Entailment and Derivation	111
11.4	Deciding It: Model Checking	112
11.5	Deciding It Better: DPLL	112
11.6	Conjunctive Normal Form	113
11.7	Resolution	114
11.8	When the Answer Is No	114
12	First-Order Logic and Resolution	121
12.1	Where Propositional Logic Runs Out	121
12.2	Syntax and Semantics	122
12.3	Unification	122
12.4	Clause Form	123
12.5	Resolution Refutation	124
12.6	What This Buys, and What It Costs	125
13	Rule-Based Expert Systems	132
13.1	The Escalation Adviser	132
13.2	Forward Chaining	133
13.3	Backward Chaining	133
13.4	Conflict Resolution	135
13.5	What Conflict Resolution Does Not Fix	136
13.6	Why the Industry Collapsed	136
V	Reasoning Under Uncertainty	144
14	Uncertainty and Decision Making	146
14.1	What Logic Cannot Say	146
14.2	Bayes' Rule	147
14.3	Conditional Independence	148
14.4	From Belief to Decision	148
14.5	A Change of Belief Is Not a Change of Decision	149
14.6	The Value of Information	149

15 Bayesian Networks	156
15.1 Diagnosing a Defect	156
15.2 Exact Inference by Enumeration	157
15.3 Variable Elimination	158
15.4 Approximate Inference by Sampling	158
15.5 Explaining Away	159
16 Fuzzy Logic	166
16.1 Vagueness Is Not Uncertainty	166
16.2 Membership Functions and Linguistic Variables	167
16.3 The Fuzzy Operators	167
16.4 Mamdani Inference	168
16.5 Defuzzification	168
16.6 Why Smoothness Is the Point	169
VI Learning, Language and Responsible AI	175
17 Learning from Examples	177
17.1 One Chapter, and Why	177
17.2 Three Kinds of Learning	178
17.3 Supervised Learning	178
17.4 Artificial Neural Networks	179
17.5 Linear Separability and the Exclusive-Or Limit	179
17.6 Unsupervised Learning	181
17.7 Reinforcement Learning	181
17.8 Where This Course Hands Over	181
18 Natural Language Processing	187
18.1 Ticket Subject Lines	187
18.2 The Symbolic Pipeline	188
18.3 Context-Free Grammars	188
18.4 Parsing with CYK	188
18.5 Ambiguity Is the Point	189
18.6 Pattern Substitution: Eliza	189
18.7 N-Grams and Their Limit	190
19 Classic AI Systems and How to Analyse Them	197
19.1 Four Systems, Four Mechanisms	197
19.2 The General Problem Solver	198
19.3 Eliza	198
19.4 Student	198
19.5 Macsyma	199
19.6 A Framework for Analysing an AI System	199

20 Recent Trends and Applications	208
20.1 Where the Field Is Now	208
20.2 Large Language Models, Briefly	208
20.3 Tool-Using Agents	209
20.4 Neuro-Symbolic Systems	210
20.5 Reading a Claim About AI	210
21 Responsible AI and Its Limits	217
21.1 Nobody Intended It	217
21.2 Where Bias Enters a Rule Base	217
21.3 Auditing a Deployed Rule Set	218
21.4 Explanation as Accountability	219
21.5 Brittleness and the Edge of Competence	220
21.6 Computational Limits	220
22 The Artificial Intelligence Project	228
22.1 The Brief	228
22.2 Choosing a Problem	229
22.3 The Proposal	229
22.4 Building It	230
22.5 Reporting It	230
22.6 The Demonstration	231
Appendices	237
A Python Reference for Artificial Intelligence	238
A.1 Setting Up	238
A.2 The Four Structures That Matter	238
A.3 Frontiers	238
A.4 Generators, Memoisation and Recursion	239
A.5 Exact Arithmetic for Probability	239
A.6 NumPy, Where It Earns Its Place	239
A.7 Reproducibility	239
A.8 Style Rules the Labs Follow	239
B Algorithm and Formula Sheet	241
B.1 Search	241
B.2 Constraint Satisfaction	242
B.3 Adversarial Search	242
B.4 Planning	242
B.5 Logic	242
B.6 Uncertainty	243
B.7 Fuzzy Logic	243
B.8 The Perceptron	244

C	Answers to Checkpoints	245
D	Glossary	255
E	Resources	258
E.1	The Set Texts	258
E.2	Where Each Chapter Sits in Russell and Norvig	258
E.3	The Primary Sources Worth Reading	259
E.4	Departmental Materials	259
E.5	Tools	260
F	The Boundary with the Neighbouring Courses	261
F.1	Why This Course Is Symbolic	261
F.2	Machine Learning (Semester 6)	261
F.3	Knowledge Representation and Reasoning (elective)	262
F.4	Data Science (Semester 7) and the 2025 Arrivals	263
F.5	Programming for Artificial Intelligence	263
F.6	The Summer 2026 Handout	263
G	Sixteen-Week Teaching Schedule	265
G.1	The Schedule	265
G.2	Assessment Weights	266
G.3	Dependency Map	267
G.4	Laboratory Sessions	267

List of Figures

F.1	The six parts and the chapters in each. Chapter 17 (outlined) is the signpost to <i>Machine Learning</i> : the three learning paradigms and the perceptron, and no further.	x
P1.1	Reading path through Part I. Chapter 3 (highlighted) fixes the state representation and the frontier; Chapters 4 to 9 are short only because it did.	2
1.1	The four classical definitions of artificial intelligence, on the two axes of thought against behaviour and human performance against rational performance. This course works in the bottom-right quadrant throughout.	4
1.2	The branches of artificial intelligence, and which of them this course teaches. The five branches on the upper row are the body of the course; the lower row is covered by a signpost chapter or by another course entirely.	6
1.3	Artificial intelligence, machine learning and data science, separated by what each is given and what each must work out. The distinction is not the technique; it is where the rules come from.	8
2.1	The agent and its environment. The <i>agent function</i> is the mathematical object — what the agent ought to do in every situation it could ever be in. The <i>agent program</i> is the code that runs on the machine.	14
2.2	Five agent architectures. Each keeps more state than the one before it. Choose the leftmost one that can do the job.	17
3.1	Three state representations, and the situation each one is for. All three are immutable and hashable; that is the property the algorithms depend on.	23
3.2	Three frontier structures. Choosing the container chooses the strategy, and the operation costs are the reason each one is matched to its search.	25
P2.1	Reading path through Part II. Each chapter relaxes something the one before it insisted on: Chapter 5 drops blind expansion, Chapter 6 drops the path.	33
4.1	The state space and the search tree for a three-task fragment where c depends on both a and b . The tree repeats every state once per path that reaches it.	35
4.2	Iterative deepening against breadth-first search at solution depth $d = 5$. The repeated work is a modest overhead that shrinks as the tree widens — 91% extra at $b = 2$, 11% at $b = 10$, 9% at $b = 12$ — while the memory requirement falls from exponential to linear.	36
5.1	The sprint network. Ten of the eleven tasks are unavoidable; the testing step can be reached two ways, and the cheaper route is not the shorter one in number of tasks.	44
5.2	The two routes as cost. Only an algorithm that adds the cost already spent to the cost still expected can see that three days of harness buys four days of testing.	45
6.1	The state-space landscape. Local search sees only the ground next to it. A <i>local maximum</i> has no better neighbour but is not the best state; a <i>plateau</i> is flat, so there is no gradient to follow, and a <i>shoulder</i> is a plateau that does lead upward — which is why forbidding flat moves loses solutions and allowing them without limit loops forever. A <i>ridge</i> is a run of local maxima along which no single move goes up, though a combination of two would.	55

6.2	Simulated annealing’s acceptance probability $\exp(\Delta/T)$. High temperature accepts almost any move; as T falls the method converges on hill climbing. Note that an <i>improving</i> move is always accepted, at every temperature.	56
6.3	Mean final score at an equal budget of 4,000 objective evaluations per run. Hill climbing alone is far behind; the three escape strategies are close together, with the genetic algorithm ahead.	58
P3.1	Reading path through Part III. Chapter 7 (highlighted) is the one that gives a <i>guarantee</i> — no constraint violated — which no learned model can offer.	67
7.1	The constraint graph. Twelve variables, twelve binary constraints, and one n -ary capacity constraint that no edge can represent.	69
7.2	Forward checking after one assignment. The pruning is the visible effect; giving the ordering heuristic something to measure is the valuable one.	70
8.1	A three-ply negotiation. Squares are our choices, circles theirs. Minimax evaluates all eight leaves to establish that the position is worth 5.	80
8.2	Alpha–beta on the tree of Figure 8.1. Green leaves are evaluated, grey are never looked at, and the crosses mark the two cutoffs.	81
9.1	One STRIPS operator. Applying it removes the delete list and then adds the add list; everything else in the state is untouched.	89
9.2	Goal clobbering, in two operators. Means–ends analysis is complete only if it happens to take the goals in a workable order.	91
P4.1	Reading path through Part IV. Chapter 11 (highlighted) is the hinge: once entailment and inference are separated, everything after it is engineering.	100
10.1	A fragment of a delivery ontology. Rounded boxes are individuals, square boxes are classes; solid arrows are is-a , dashed are instance-of	102
10.2	A multiple-inheritance conflict — the “Nixon diamond” of the literature, in a delivery setting. The lab detects it rather than guessing.	104
11.1	Entailment as containment between sets of models. The picture is the definition; the algorithms in the rest of the chapter are three ways of checking it.	112
11.2	DPLL deciding the release gate. Unit propagation cascades from the four given facts and closes the question without a single branch.	113
12.1	The delay chain. Quantifiers buy the ability to state a rule once, which is the difference between a knowledge base a team can maintain and one it cannot. . . .	122
12.2	Three unification problems. The third is why every implementation needs an occurs check, and why many production Prolog systems omit it for speed and document the risk.	123
13.1	The architecture. Working memory grows as conclusions are asserted, which makes further rules applicable, and the cycle repeats until nothing new can be derived. .	133
13.2	The proof tree for <i>why is T-4471 escalated?</i> Rounded green boxes are facts from the ticket; blue boxes are derived, each labelled with the rule that derived it. . . .	135
P5.1	Reading path through Part V. Chapter 16 branches from Chapter 14 rather than following Chapter 15: fuzzy logic answers a different question, and confusing the two is the standard error.	145

14.1	The same calculation as Bayes' rule, in whole releases rather than fractions. Natural frequencies make the base-rate effect visible, which is why this is the form to use when explaining a result to a team.	147
14.2	The two expected utilities as the probability of a defect varies. The alarm moves the belief a long way and leaves the decision alone, because $1/3$ falls just short of the crossing at $10/29$	149
15.1	The diagnosis network. Causes point to effects, so the arrows run the way the world does; inference then runs against them, from rollback back to cause.	157
15.2	Enumeration against variable elimination on a chain. At six nodes enumeration wins; at fourteen it needs 8,192 terms against 186 operations, and beyond that it stops being an option.	158
16.1	A fuzzy partition of the uncertainty scale. The memberships sum to one at every point, so each input is described by at most two words and the description moves smoothly as the input does.	167
16.2	Mamdani inference on the project <i>Beacon</i> , effort 45 and uncertainty 4. Four of the nine rules fire, and the output is a single number.	168
16.3	The same knowledge as a fuzzy controller and as a threshold rule. The fuzzy output moves continuously; the threshold output is flat, flat, and then an eighty-point jump between two adjacent inputs.	169
P6.1	Reading path through Part VI. Chapter 17 (outlined in red) is a signpost, not a unit; Chapter 21 (highlighted) is where the limits of everything in the book are stated plainly.	176
17.1	The three learning paradigms, with a delivery example of each. This section discharges the HEC 2017 outline's <i>Learning</i> clause; the sections that follow take the first one further.	178
17.2	The twelve tasks and the boundary the perceptron learned. Everything above the line is predicted on time, everything below it late.	179
17.3	The exclusive-or arrangement. The two <i>late</i> tasks sit at opposite corners, and so do the two <i>on time</i> tasks. Any straight line has at least one point on the wrong side. .	180
17.4	The reinforcement learning loop. One figure is all this course gives it; <i>Machine Learning</i> Chapters 18–20 develop Markov decision processes, Monte Carlo methods and Q-learning.	181
18.1	The symbolic pipeline. Stages 1 to 3 are bookkeeping; stage 4 is where the knowledge lives.	188
18.2	Prepositional-phrase attachment ambiguity. The words and their order are identical; only the structure differs, and the structure is the meaning.	189
19.1	Six columns for analysing an AI system. Applying them to a system you did not build is the skill this chapter is for.	201
20.1	A tool-using agent. The loop is Chapter 2' goal-based agent; only the decision procedure at its centre has changed.	209
21.1	Four ways bias enters a system whose rules mention nothing sensitive. The first — a proxy variable — is the one that survives every attempt to fix the problem by deleting columns.	218

G.1 Chapter dependencies. The outlined chapters (3, 4, 5 and 11) are the ones later chapters rely on most: the search toolkit is reused by constraint satisfaction, adversarial search and planning, and propositional inference underpins first-order logic and the rule engine. 267

PART I

Foundations of Intelligent Systems

Before any algorithm, three questions: what makes a program intelligent rather than merely useful, how do you specify what an agent is supposed to do before you build it, and what does a state look like in memory? Part I answers them, and the third answer is the one every later chapter is written on top of.

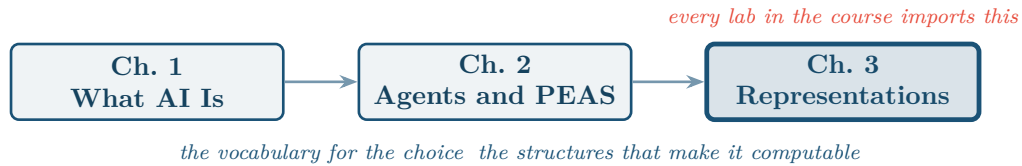


Figure P1.1: Reading path through Part I. Chapter 3 (highlighted) fixes the state representation and the frontier; Chapters 4 to 9 are short only because it did.

What Artificial Intelligence Is

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part I — Foundations of Intelligent Systems

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Define** artificial intelligence, and **distinguish** the four classical definitions along the two axes of thinking versus acting and humanly versus rationally.
- **Identify** whether a given piece of software is an AI system, by applying a five-point test rather than by reputation.
- **Name** the branches of AI and **say** which of them this course teaches, which it signposts, and which belong to other courses.
- **Describe** the field's history as a sequence of promises, disappointments and recoveries, and **explain** what caused each turn.
- **State** the boundary between artificial intelligence, machine learning and data science in terms of what each one is given and what each one must work out for itself.

🔗 Before You Start

Object Oriented Programming, the published prerequisite for this course: classes, objects, inheritance, collections and recursion. No mathematics beyond school algebra is assumed in this chapter. Appendix A is a reference for every Python construct the labs use.

📖 Key Terms in This Chapter

artificial intelligence • agent • percept • action • rationality • the Turing test • symbolic AI • knowledge representation • search • heuristic • combinatorial explosion • weak and strong AI • narrow and general AI • AI winter • expert system

1.1 Two Programs in a Project Office

A software house runs its delivery work out of a project management office. Two programs run there every night.

The first recalculates the Gantt chart. It is given the task list, each task's duration and each task's predecessors, and it computes the earliest start and latest finish of every task, the critical path, and the float on everything off it. It is a few hundred lines long, it is completely deterministic, and if it ever disagrees with a hand calculation the program is wrong. Nobody in the office calls it intelligent.

The second decides which of tonight's three hundred open tickets to escalate to the on-call engineer. It is given the tickets, the service level agreements, who is on shift, what each engineer has already been handed, and a history of which escalations turned out to have been worth making. There is no formula in the brief. Two reasonable people, given the same three hundred tickets, would escalate different ones and each could defend the choice. The program has to decide anyway, tonight, before midnight.

That difference — not the difficulty of the arithmetic, not the size of the codebase, and not the fashionability of the technique — is what this book is about. The first program computes an

answer that was already implied by its input. The second must work out what to do when its input does not determine the answer.

Artificial intelligence

Artificial intelligence is the study and construction of **agents**: programs that perceive an environment through **percepts** and act on it through **actions**, choosing those actions so as to do well by some measure of success. The central question of the field is not “can a machine think?” but “given what it can perceive, what should this agent do next?”

After S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd edition, Prentice Hall, 2010 — the set text for this course in both the HEC 2017 and HEC 2023 outlines.

The definition is deliberately modest. It says nothing about consciousness, nothing about brains, and nothing about whether the agent’s reasoning resembles ours. It asks only that the agent take in a situation and choose. Everything in this course is machinery for making that choice: search when the agent must look ahead (Chapters 4 to 6), constraint reasoning when the choice must satisfy rules at once (Chapter 7), game reasoning when another agent is choosing against you (Chapter 8), planning when the choice is a sequence (Chapter 9), logic when the agent must draw conclusions from what it has been told (Chapters 10 to 13), and probability when what it has been told is not certain (Chapters 14 to 16).

1.2 Four Definitions, Two Axes

The field has never had one definition. It has had four, and they differ along two axes: whether the goal is to replicate *thought* or to produce the right *behaviour*, and whether the standard of correctness is *human* performance or an ideal standard of *rationality*. Figure 1.1 sets them out.

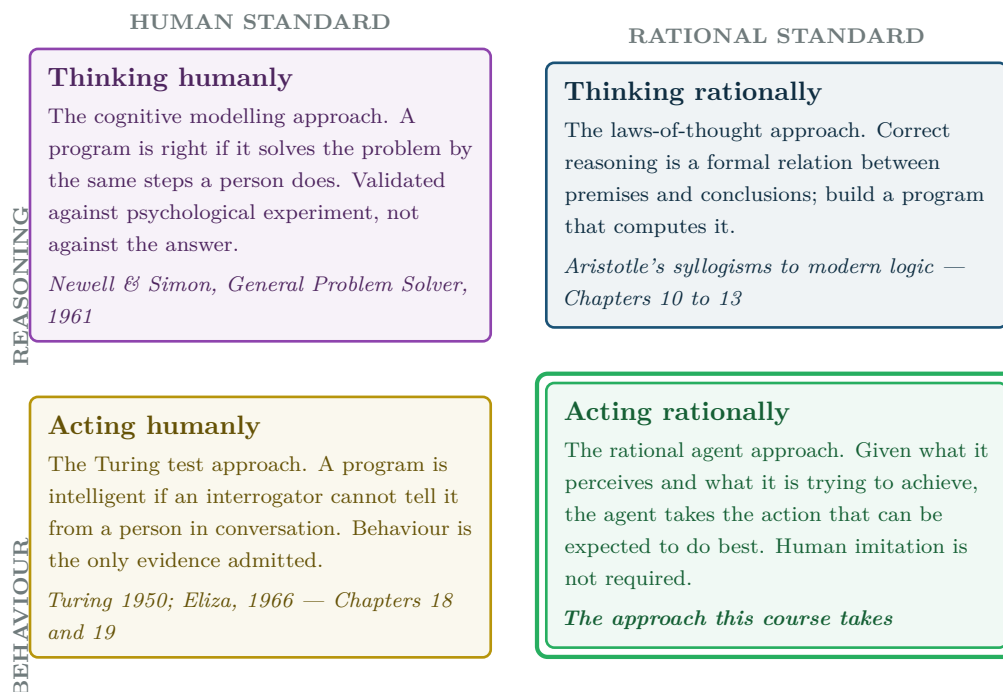


Figure 1.1: The four classical definitions of artificial intelligence, on the two axes of thought against behaviour and human performance against rational performance. This course works in the bottom-right quadrant throughout.



The four quadrants are not four opinions about the same question; they are four different questions, and they license different evidence. If your goal is *acting humanly*, then a program that answers a service-desk query the way a tired engineer would — including the shortcuts — is a success. If your goal is *acting rationally*, the same program is a failure, because the tired engineer was wrong.

This matters in practice, and the practical consequence is this: *decide which quadrant you are in before you choose your evaluation*. Chapter 19 shows Eliza, a program that is a triumph in the bottom-left quadrant and worthless in the bottom-right, and the difference is not in the code.

1.3 Is It AI? A Checklist

“Artificial intelligence” is now a procurement word. Vendors apply it to spreadsheet macros, and sceptics withhold it from systems that plainly reason. The HEC outline asks you to be able to *identify* AI systems, which means having a test you can apply to software you did not write. Here is one. Five questions; score each one and count.

1. **Goal.** Is the program given an *objective* rather than a procedure? A program told *minimise the number of breached service level agreements* has a goal. A program told *sort the tickets by deadline, then assign the first ten* has a procedure.
2. **Search.** Does it *consider alternatives it does not act on*? An agent that generates candidate assignments, evaluates them and picks one has searched. An agent that computes one answer directly has not.
3. **Knowledge.** Does it hold an explicit, inspectable *representation* of its domain — facts, rules, constraints, a probability model — that a domain expert could read and correct without editing the algorithm?
4. **Adaptation.** Does its behaviour change with *experience*, without being reprogrammed?
5. **Explanation.** Can it say *why*? Can it produce the chain of facts and inferences that led to its answer?

A program scoring zero is ordinary software, however valuable. A program scoring one or two is doing something interesting. Three or more, and the vocabulary of this course is the right vocabulary for talking about it.

Two warnings about the test. First, it is not a quality measure: a badly built system can score five, and the Gantt recalcuator that scores zero is the most useful program in the office. Second, question 4 is neither necessary nor sufficient. Much of the most successful AI ever built — the chess programs of Chapter 8, the schedulers of Chapter 7, the diagnostic systems of Chapter 13 — scores zero on adaptation. Learning is one route to intelligent behaviour, not its definition. That is worth saying plainly at the start of a course that will spend fifteen weeks on the other routes.

⌕ Five tools in one project office

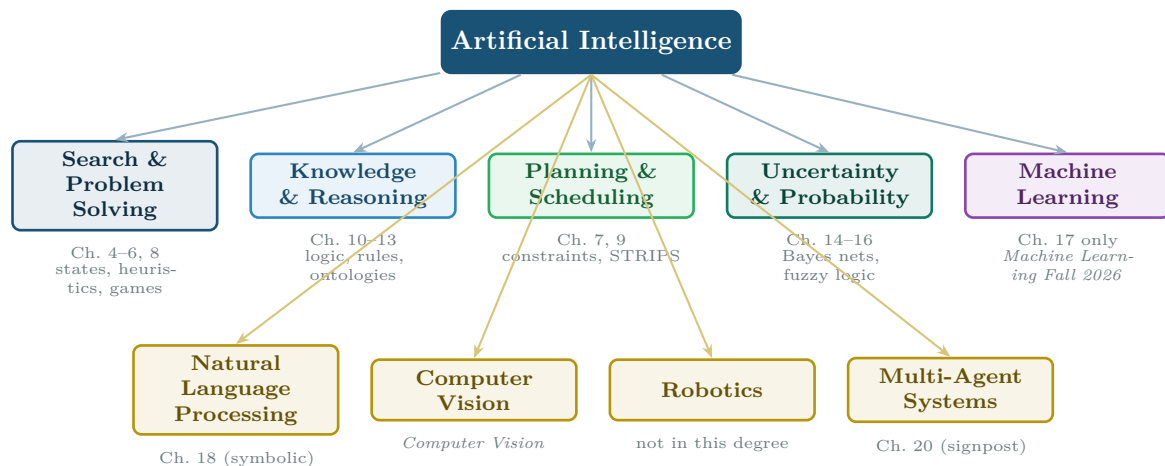
Scores on the five tests of Section 1.3, for the five programs the office actually runs. This is the audit the chapter’s lab reproduces in code.

Program	Goal	Srch	Know	Adapt	Expl	Score
Gantt recalcuator	—	—	—	—	—	0
Burndown chart generator	—	—	—	—	—	0
Sprint scheduler	yes	yes	yes	—	yes	4
Escalation adviser	yes	yes	yes	—	yes	4
Duplicate-ticket finder	—	—	—	yes	—	1

The two zeroes are not failures; they are the right tools for questions that have formulas. The sprint scheduler is Chapter 7, the escalation adviser is Chapter 13, and the duplicate finder is the one tool of the five that this course hands to Chapter 17 and then to Machine Learning.

1.4 The Branches of AI

The field is conventionally divided as in Figure 1.2. The division is untidy, because the branches are not disjoint — planning uses search, natural language processing uses both logic and probability — but it is the division the HEC outline uses and the one job advertisements use.



Bold branches are taught here. Grey captions name the course that owns the branch instead; Appendix F states every boundary in full.

Figure 1.2: The branches of artificial intelligence, and which of them this course teaches. The five branches on the upper row are the body of the course; the lower row is covered by a signpost chapter or by another course entirely.

1.5 Sixty Years in Five Waves

AI has a distinctive history: repeated cycles in which a genuine advance is over-promised, fails to generalise, and is followed by a collapse in funding severe enough to have earned its own name, an **AI winter**. Knowing the shape of the cycle is professionally useful, because it is happening again now and you will be asked to make judgements inside it.

Period	What was achieved	Why it stalled
1943–1956 <i>Foundations</i>	McCulloch and Pitts’ artificial neuron; Turing’s 1950 paper and his test; the 1956 Dartmouth workshop, which named the field.	Nothing stalled yet. The founding claim — that every aspect of intelligence could be “so precisely described that a machine can be made to simulate it” — set the expectations that the next twenty years failed to meet.
1956–1974 <i>Search and the general solver</i>	The Logic Theorist and the General Problem Solver (Chapter 9); Samuel’s checkers player; Eliza (Chapter 18); early perceptrons (Chapter 17).	Combinatorial explosion. The methods worked on toy problems and died on real ones: a search that is fine at depth 6 is hopeless at depth 20. Minsky and Papert’s 1969 analysis of the perceptron’s limits closed the neural line for fifteen years.
1974–1980 <i>First winter</i>	—	The Lighthill report (1973) in the UK and DARPA’s withdrawal in the US ended general AI funding. The diagnosis was correct: the field had solved the easy half of its problems and called it progress.
1980–1987 <i>Expert systems</i>	Knowledge, not cleverness, as the engine: MYCIN, XCON, and a commercial industry in rule-based systems (Chapter 13). Japan’s Fifth Generation project.	The knowledge acquisition bottleneck . Rules had to be elicited from experts one at a time, could not be learned, grew brittle at a few thousand rules, and degraded catastrophically rather than gracefully at the edge of their competence.
1987–1993 <i>Second winter</i>	—	The specialised hardware market collapsed; “AI” became a word firms avoided in prospectuses. The research continued under other names — “knowledge-based systems”, “decision support”.
1993–2011 <i>The quiet, successful period</i>	The rational-agent reframing that this course follows; probability and Bayesian networks (Chapter 15); Deep Blue (1997); statistical machine translation; search engines and recommenders. AI methods everywhere, under other names.	It did not stall. It stopped claiming to be AI, which is why this era is the least famous and the most useful.
2012–present <i>Learning at scale</i>	Deep networks, then transformers and large language models; systems that write fluent text and usable code (Chapter 20).	Still open. The symbolic questions this course teaches — can it prove it, can it explain it, can it guarantee the constraint holds — are exactly the ones the current wave answers least well (Chapter 21).

Table 1.1: Five waves and two winters. Each collapse followed a real achievement that was mistaken for a general one.

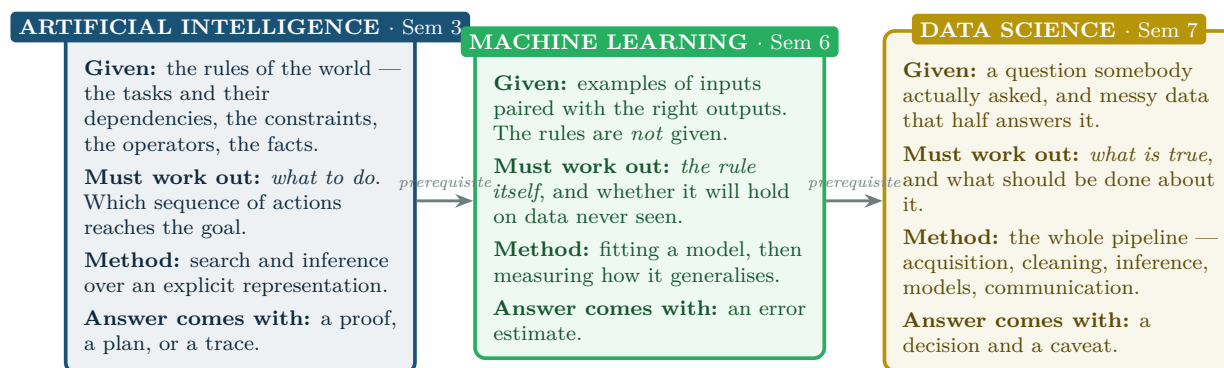
⚠ The pattern worth taking from the history

Every winter arrived the same way: a method that worked at small scale was sold as a method that worked, and the gap was discovered by customers rather than by researchers. The methods themselves were not discredited — minimax, resolution and rule engines are all in production today, in this book's Chapters 8, 12 and 13. What was discredited each time was a claim about their *reach*.

You will be asked to make that kind of claim in your own career, probably about something you built over a weekend that worked beautifully on twenty examples. Chapter 21 gives you the vocabulary to make it honestly.

1.6 What This Course Teaches, and What It Does Not

Artificial intelligence, machine learning and data science are three courses in this degree, taught in that order, and students routinely confuse them because all three are described in job advertisements with the same six words. Figure 1.3 draws the boundary in a way that survives contact with a real question, and Appendix F states it chapter by chapter.



One question, three courses. Which tickets should be escalated tonight? — AI writes the rules down as constraints and infers the escalation set, with the chain of reasoning attached (Chapter 13). Machine Learning is given ten thousand past escalations and their outcomes and learns which features predict a worthwhile one. Data Science asks whether “worthwhile” was measured correctly in the first place, and what the escalation policy is doing to the on-call engineers.

Figure 1.3: Artificial intelligence, machine learning and data science, separated by what each is given and what each must work out. The distinction is not the technique; it is where the rules come from.

So: this course is *symbolic*. It teaches the agent that is told the rules and must reason with them. That is not a historical curiosity, for three reasons. It is where guarantees live — a constraint solver can prove no engineer is double-booked, and no learned model can (Chapter 7). It is where explanations live — a rule engine can print the chain that produced its answer, and that chain is what an auditor accepts (Chapter 13). And it is where you go when you have no data, which is the situation at the start of every new project.

Chapter 17 is the one chapter about learning. It covers the three paradigms the HEC 2017 outline names, works the perceptron numerically, shows why a single perceptron cannot represent exclusive-or — the result that caused the first neural winter — and then stops and hands over. It is a signpost, not a unit.

📊 Worked Example: Auditing one tool against the five tests

The office's *duplicate-ticket finder* takes each new ticket and flags the three existing tickets most similar to it, so the service desk can merge them. Its similarity measure is tuned monthly against the merges the desk actually made. Score it.

1. Goal? *No.* The brief is a procedure: compute similarity, return the top three. There is no objective it is trying to do well by; if the desk wanted the top five it would be a different program.

2. Search? *No.* It scores every candidate and returns the best three. Scoring all alternatives and ranking them is not search — there is no point at which the program commits to a partial choice and explores its consequences. Compare Chapter 4, where a decision about task 3 changes which states are reachable at all.

3. Knowledge? *No.* The similarity weights are numbers in a configuration file. A service-desk manager cannot read them and say “that is wrong, a database timeout is not like a login failure”.

4. Adaptation? *Yes.* Behaviour changes monthly on the evidence of past merges, with nobody editing the code.

5. Explanation? *No.* It reports a similarity score. A score is a number, not a reason.

Score: 1 of 5 — and the one point is adaptation, the single test that this course does not teach. That is the correct verdict, and the useful one: the duplicate finder is a good tool and it is not an AI problem in the sense of this course. It is the machine learning course’s problem. Contrast the sprint scheduler, which scores 4 and whose one missing point is adaptation.

! Common Pitfalls

- **Treating “AI” as a synonym for “machine learning”.** It is the most common error in this subject, and it makes whole categories of problem invisible. If a constraint must *never* be violated, no amount of training data is the answer; you need Chapter 7.
- **Reading the four quadrants of Figure 1.1 as a ranking.** They are different goals with different evidence. A program can be excellent in one and worthless in the neighbouring one.
- **Assuming a program that scores well on the checklist is a good program.** The checklist identifies the *kind* of system, not its quality. The highest-scoring system in an office can be the one that should be deleted.
- **Believing the winters were caused by the methods failing.** Minimax, resolution and rule engines all still work and are all still deployed. What failed was a claim about how far they reached.
- **Calling a system “general” AI because it is fluent.** Fluency is the bottom-left quadrant — acting humanly. Chapter 19’s Eliza is two hundred lines long and was mistaken for a therapist in 1966.

🔧 Try It Yourself: Is It AI? A Five-Point Audit

The audit of the *examplebox* above, in code. Five tools from the project management office, five tests, one verdict each. Pure standard library: the point of this lab is the classification, not the computation.

```
"""Chapter 1 lab -- Is it AI? A five-point audit.
```

```
Score five programs from a project management office against the five
tests of Section 1.3, and classify each one. Nothing here learns
anything; the exercise is to separate programs that decide from
programs that compute.
"""
```

```
TESTS = ("goal", "search", "knowledge", "adapts", "explains")
```

```

# One row per tool: which of the five tests it passes, and why the
# most interesting answer is what it is.
TOOLS = {
    "Gantt recalcuator": {
        "passes": (),
        "note": "given a procedure, not an objective; one answer, "
                "computed directly",
    },
    "Burndown chart generator": {
        "passes": (),
        "note": "arithmetic and a plot; no alternatives considered",
    },
    "Sprint scheduler": {
        "passes": ("goal", "search", "knowledge", "explains"),
        "note": "constraints are declared, not coded; reports which "
                "constraint blocked a story (Chapter 7)",
    },
    "Escalation adviser": {
        "passes": ("goal", "search", "knowledge", "explains"),
        "note": "rules an operations lead can read and correct; prints "
                "its firing chain (Chapter 13)",
    },
    "Duplicate-ticket finder": {
        "passes": ("adapts",),
        "note": "weights retuned monthly from past merges; the one "
                "tool here that belongs to Machine Learning",
    },
}

def verdict(score):
    """The reading of a score given in Section 1.3."""
    if score == 0:
        return "ordinary software"
    if score <= 2:
        return "doing something interesting"
    return "an AI system"

def audit():
    width = max(len(name) for name in TOOLS)
    head = "TOOL".ljust(width) + " " + " ".join(
        t[:4].upper().rjust(5) for t in TESTS) + " SCORE VERDICT"
    print(head)
    print("-" * len(head))
    for name, row in TOOLS.items():
        marks = ["yes".rjust(5) if t in row["passes"] else "-".rjust(5)
                 for t in TESTS]
        score = len(row["passes"])
        print(name.ljust(width) + " " + " ".join(marks) +
              " %5d %s" % (score, verdict(score)))
    print()
    for name, row in TOOLS.items():
        print("%s\n %s" % (name, row["note"]))

def check():
    """The two claims the chapter makes about this table."""
    scores = {n: len(r["passes"]) for n, r in TOOLS.items()}
    # Adaptation is the only test the AI systems here fail ...
    for name in ("Sprint scheduler", "Escalation adviser"):
        assert "adapts" not in TOOLS[name]["passes"], name

```

```

    assert scores[name] == 4, name
    # ... and the only test the machine learning tool passes.
    assert TOOLS["Duplicate-ticket finder"]["passes"] == ("adapts",)
    print("\nchecks passed: intelligent behaviour here does not "
          "require learning.")
    print("and the one tool that learns is the one this course "
          "hands on.")

if __name__ == "__main__":
    audit()
    check()

```

Now add a sixth tool of your own — something you have actually used, a CI pipeline or an IDE’s autocomplete — and score it. The interesting cases are the ones that score 2 or 3: write down which test you were least sure about and why.

? Checkpoint — Test Yourself

1. A program is given the sentence *minimise total project delay* and a list of tasks with durations and dependencies. Which of the five tests of Section 1.3 does it pass on that description alone, and which cannot be judged until you see the implementation?
2. A chatbot answers service-desk questions so convincingly that users believe it is a person, but it has no representation of the service catalogue and cannot say why it gave an answer. Which quadrant of Figure 1.1 is it succeeding in, and which is it failing in?
3. The expert-systems boom ended in the knowledge acquisition bottleneck. Name the chapter of this handout that builds an expert system, and state which single one of the five tests such a system fails.

✓ Chapter Summary

- An **agent** perceives an environment and acts on it. Artificial intelligence is the study of choosing those actions well; the field’s question is “what should this agent do next?”.
- The four classical definitions differ on two axes: thinking against acting, and the human standard against the rational standard. This course works throughout in the *acting rationally* quadrant.
- A program is an AI system to the extent that it is given a goal rather than a procedure, considers alternatives, holds an inspectable representation of its domain, adapts, and can explain itself. Three or more of the five, and this course’s vocabulary applies.
- Learning is one route to intelligent behaviour, not the definition of it. Chess programs, schedulers and diagnostic systems score zero on adaptation and are unambiguously AI.
- The history is a cycle: a real advance, an over-general claim, a winter. Combinatorial explosion ended the first wave; the knowledge acquisition bottleneck ended the expert-system wave.
- Artificial intelligence is given the rules and must work out what to do; machine learning is given examples and must work out the rule; data science is given a question and must work out what is true. This course is the first of the three, and it is symbolic.

✍ Review Questions

1. (MCQ) A program that beats a grandmaster at chess but cannot learn from its games is: (a) not AI, because it does not learn (b) AI, and scores zero on the adaptation test (c) AI only in the “thinking humanly” sense (d) machine learning.

2. **(MCQ)** The Turing test belongs in which quadrant? (a) thinking humanly (b) thinking rationally (c) acting humanly (d) acting rationally.
3. **(MCQ)** Which ended the first wave of AI research? (a) the knowledge acquisition bottleneck (b) combinatorial explosion (c) the collapse of the Lisp machine market (d) Minsky and Papert's book alone.
4. **(MCQ)** A system is given the tasks, the dependencies and the objective, and returns a sequence of actions with a proof that no dependency is violated. Which course owns this problem? (a) Artificial Intelligence (b) Machine Learning (c) Data Science (d) Data Mining.
5. **(Short)** State the difference between being given a goal and being given a procedure, using two versions of the same ticket-triage requirement as your example.
6. **(Short)** Why is "it adapts" neither necessary nor sufficient for a program to be an AI system? Give one example each way.
7. **(Short)** The expert-system industry collapsed even though rule engines work. Explain what actually failed, and name the chapter of this handout in which you will build one anyway.
8. **(Applied)** Take a piece of software you have used this week. Score it against the five tests, writing one sentence of justification per test, and give the verdict. Then state which single change to the system would raise its score by one — and whether that change would make it better.

Intelligent Agents and Task Environments

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part I — Foundations of Intelligent Systems

🕒 Learning Outcomes

By the end of this chapter you will be able to:

- **Define** an agent in terms of percepts, actions, an agent function and an agent program, and **explain** why the distinction between the last two matters.
- **Specify** any task for an agent as a **PEAS** description — performance measure, environment, actuators, sensors — before writing a line of code.
- **Classify** a task environment on the six standard properties, and **predict** which agent architectures the classification rules out.
- **Distinguish** the five kinds of agent program and **choose** the simplest one that can do a given job.
- **Explain** rationality as doing the best that can be expected given the percepts so far, and **separate** it from omniscience and from perfection.

🔗 Before You Start

Chapter 1 (the definition of an agent, the acting-rationally quadrant, the five-point checklist). The agent vocabulary set up here is used by every later chapter: search is what a goal-based agent does to choose its action (Chapter 4), and utility appears again as expected utility in Chapter 14.

📖 Key Terms in This Chapter

agent • percept • percept sequence • actuator • sensor • agent function • agent program • PEAS • performance measure • rational agent • fully and partially observable • deterministic and stochastic • episodic and sequential • static and dynamic • discrete and continuous • single- and multi-agent • simple reflex agent • model-based reflex agent • goal-based agent • utility-based agent • state estimation

2.1 Agent, Environment, Percept, Action

Everything in this course is an agent doing one thing: taking in what it can see and choosing what to do. Figure 2.1 is the picture, and it is worth looking at carefully, because four of its labels are technical terms that get used precisely from here on.

📄 Agent function and agent program

The **agent function** maps every possible **percept sequence** to an action:

$$f : \mathcal{P}^* \longrightarrow \mathcal{A}$$

where $\mathcal{P}^*$ is the set of all finite sequences of percepts and $\mathcal{A}$ the set of available actions. It is an abstract object, usually infinite, and it is what we are trying to specify.

The **agent program** is a concrete implementation of f that runs inside the agent, with finite memory and a deadline.

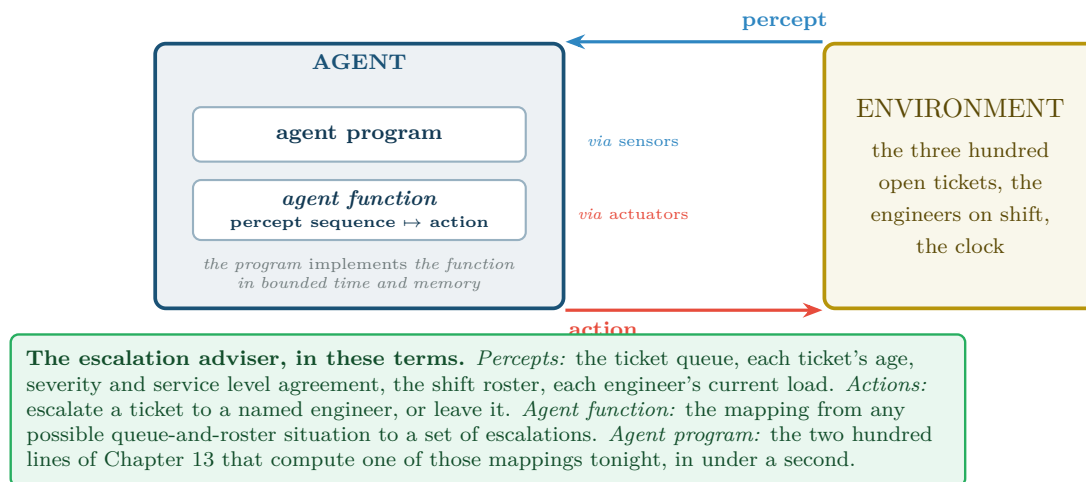


Figure 2.1: The agent and its environment. The *agent function* is the mathematical object — what the agent ought to do in every situation it could ever be in. The *agent program* is the code that runs on the machine.

The distinction looks pedantic and is not. The agent function for the escalation problem could be written as an enormous table: every possible state of the queue in the left column, the right set of escalations in the right column. That table would be a complete, correct specification — and with three hundred tickets it has more rows than there are atoms in the observable universe. So the interesting work of AI is never “what is the right answer in every case”; it is “what program computes a good enough answer in the time available”. Every method in this book is an answer to that second question.



Notice also that the agent function takes the whole *percept sequence*, not the latest percept. That single choice is the difference between the first two agent architectures in Section 2.4. An agent that reacts only to what it can see right now cannot escalate “the ticket that has been quiet suspiciously long”, because quietness is not visible in a snapshot — it is a property of the history.

2.2 PEAS: Specify the Task Before You Build the Agent

Before any code, write down four things. The mnemonic is **PEAS**.

- **P**erformance measure — what counts as success, stated so that two people would score the same run identically.
- **E**nvironment — what the agent operates in, including the parts it cannot see.
- **A**ctuators — what the agent can do.
- **S**ensors — what the agent can perceive.

The performance measure comes first and is the one people get wrong. It must be a measure of the *environment's* state, not of the agent's behaviour. An escalation adviser scored on “number of escalations made” will escalate everything. Scored on “breached service level agreements, plus a penalty for each escalation that turned out to be unnecessary”, it has to make a judgement — and now the problem is interesting.

Task	Performance measure	Environment	Actuators / Sensors
Escalation adviser (Ch. 13)	Breached service level agreements; penalty per unnecessary escalation; on-call hours consumed	Ticket queue, shift roster, the clock; other engineers acting independently	<i>Act</i> : escalate to a named engineer, defer, close as duplicate. <i>Sense</i> : ticket fields, roster, current loads
Sprint scheduler (Ch. 7)	Stories placed; zero constraint violations; developer load balance	The backlog, three sprint slots, capacity and precedence rules	<i>Act</i> : assign a story to a slot, unassign. <i>Sense</i> : the full backlog and all constraints
Release planner (Ch. 9)	Plan length; whether the goal state is actually reached	Code, review, test and deploy states of each component	<i>Act</i> : apply an operator (code, review, test, deploy, roll back). <i>Sense</i> : the current state of every component
Contract negotiator (Ch. 8)	Value of the agreed slot to us, given that they are maximising theirs	Two parties, alternating offers, a deadline	<i>Act</i> : offer, accept, walk away. <i>Sense</i> : their last offer and the history
Defect diagnoser (Ch. 15)	Probability mass placed on the true cause; cost of wrong remediation	A live system; causes not directly observable	<i>Act</i> : report a posterior over causes, order one test. <i>Sense</i> : symptoms, logs, test outcomes

Table 2.1: PEAS descriptions for five tasks from one project office. Each row names the chapter that builds the agent.

2.3 Six Properties of a Task Environment

Once the task is specified, classify the environment. The classification is not academic bookkeeping: each property rules some architectures in and others out, and getting it wrong is how projects fail in month four.

Property	The question	Where it bites
Fully <i>vs.</i> partially observable	Do the sensors give the complete state relevant to the choice?	Partially observable means the agent must <i>maintain</i> state — an internal model updated over time. This is the single most consequential property: it is the whole difference between the two reflex agents of Section 2.4.
Deterministic <i>vs.</i> stochastic	Does the current state plus the action fix the next state?	Deterministic lets you search for a plan once and execute it (Chapter 9). Stochastic forces probability (Chapter 14) or re-planning after every step.

Property	The question	Where it bites
Episodic <i>vs.</i> sequential	Is each decision independent, or does it constrain later ones?	Episodic problems can be handled one case at a time. Sequential ones need lookahead: escalating your only free engineer at 21:00 is what makes the 23:00 decision hard.
Static <i>vs.</i> dynamic	Can the world change while the agent is deliberating?	Dynamic environments put a deadline on thinking, which is why Chapter 8 spends as much effort on cutting search off as on searching.
Discrete <i>vs.</i> continuous	Are states, time and actions countable or real-valued?	Discrete gives you the graph of Chapter 4. Continuous needs the local search of Chapter 6.
Single- <i>vs.</i> multi-agent	Is anything else in the environment choosing its actions?	If another party's best move depends on yours, you need Chapter 8; treating a negotiator as weather is the classic error.

Table 2.2: The six properties, and the design decision each one forces.

⚠ Classify honestly, not conveniently

The escalation problem is *partially observable* (you cannot see which tickets are about to become urgent), *stochastic* (whether an escalation helps depends on the engineer), *sequential*, *dynamic* (tickets arrive while you think), *discrete* and *multi-agent* (engineers make their own choices). That is the hard corner of every one of the six.

The standard mistake is to notice this, find it discouraging, and quietly build for the easy corner anyway — a fully observable, deterministic, episodic model — because that is what the tutorial did. The resulting system works in demonstrations and fails on a Friday night. The honest response is to *simplify deliberately and write the simplification down*: this is exactly what Chapter 7 does when it treats a sprint as static for the duration of the solve, and says so.

2.4 Five Kinds of Agent Program

Figure 2.2 shows the five architectures in increasing order of what they keep inside. The professional skill is to choose the *simplest* one that can do the job, because each step up costs code, state and ways to be wrong.

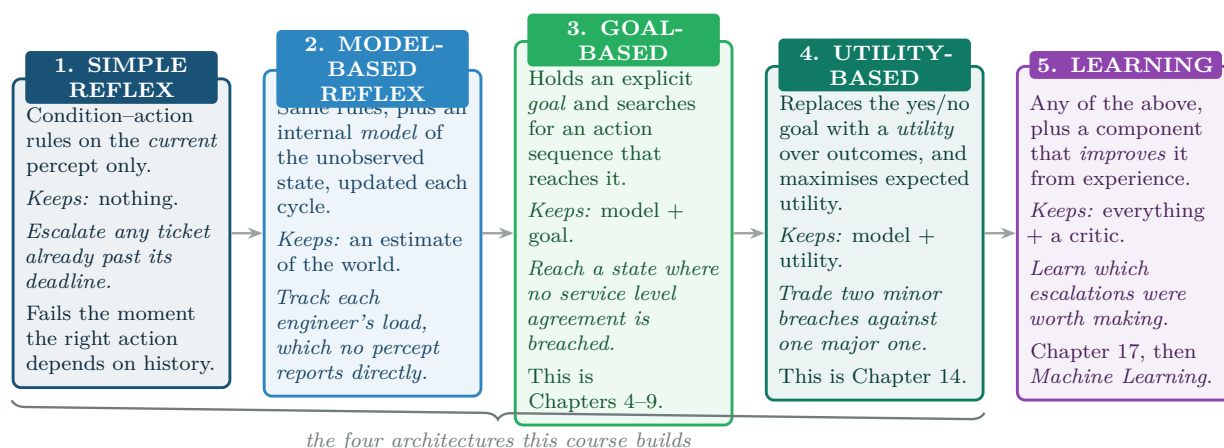


Figure 2.2: Five agent architectures. Each keeps more state than the one before it. Choose the leftmost one that can do the job.

2.5 Rationality Is Not Omniscience

A **rational agent**, for every percept sequence, selects the action expected to maximise its performance measure, given the evidence the percept sequence provides and whatever built-in knowledge it has.

Three things this definition deliberately excludes.

It does not require *omniscience*. An agent that escalates ticket 114 on excellent evidence, and turns out to have been wrong because the customer had already withdrawn the complaint by e-mail that nobody had read, was rational and unlucky. Rationality is a property of the decision procedure, judged against the information available, not a property of the outcome. This distinction is what lets you review a bad night fairly.

It does not require *perfection*. Perfection maximises actual performance; rationality maximises expected performance. Only the second is achievable, and demanding the first produces agents that stall.

And it does not forbid *information gathering*. If the agent can act to improve its own percepts — run a test, ask the customer, look at the log — then doing so is often the rational action. Chapter 14 puts a number on exactly how much such an act is worth.

Worked Example: PEAS and a full classification for one night's triage

The task. At 21:00 the queue holds three hundred open tickets. One engineer, Ayesha, is on call until 07:00. Tickets breach at different times. Escalating wakes Ayesha, which costs something, and there is a limit to how often it can be done in a night.

PEAS.

- *Performance measure:* −10 per breached major service level agreement, −3 per breached minor one, −1 per escalation that turned out unnecessary, −4 per hour of Ayesha's sleep lost. Note it is a measure of the world's state at 07:00, not of how busy the agent was.
- *Environment:* the queue, the roster, the clock, the customers (who are also acting), Ayesha (who is also acting).
- *Actuators:* escalate ticket t to Ayesha; defer t to the morning; merge t into another ticket.
- *Sensors:* every ticket's fields; Ayesha's current task; the clock. *Not sensed:* which quiet ticket is about to escalate itself into a major incident.

Classification. Partially observable (that last point), stochastic, sequential (waking Ayesha at 21:00 changes what is possible at 03:00), dynamic, discrete, multi-agent.

What the classification rules out. Partially observable kills agent 1 — a simple reflex agent cannot represent “Ayesha has already been woken twice”. Sequential kills the purely episodic treatment: you cannot score each ticket independently and take the top five. Stochastic and multi-agent together mean the plan will be wrong by 23:00, so any plan must be cheap enough to recompute.

The design that follows. A model-based agent that maintains Ayesha’s accumulated cost, using a goal-based search over the next few hours (Chapter 5), with the utility trade-off of Chapter 14 to weigh a major breach against lost sleep, recomputed every fifteen minutes. *Every one of those four choices was forced by a line of the classification*, which is why the classification is done before the code.

! Common Pitfalls

- **A performance measure that scores the agent instead of the world.** “Number of escalations” rewards noise; “tickets processed” rewards haste. Score the state of the environment at the end.
- **Confusing the agent function with the agent program.** The function can be an infinite table; the program has a deadline. Interview questions about “the optimal policy” are usually about the function, and implementation questions are always about the program.
- **Building a simple reflex agent for a partially observable world.** It will work on every test case you can construct by hand, because you can see the state that it cannot.
- **Treating another agent as weather.** If the other party’s choice depends on yours, a stochastic model of them is not merely inaccurate — it is exploitable. Chapter 8 is the fix.
- **Calling an agent irrational because the outcome was bad.** Judge the decision against the percepts available at the time. This is also how to run a blameless incident review.

🔧 Try It Yourself: A Reflex Agent on a Ticket Queue

Forty tickets arrive over one night. Two agents see exactly the same percepts: a simple reflex agent, and a model-based agent that remembers what it has already done. The performance measure is the one from the worked example. The point of the lab is that the second agent wins *on identical information*, purely because it keeps state.

```
"""Chapter 2 lab -- a reflex agent and a model-based agent.

Two agent programs, the same percepts, the same performance measure.
The simple reflex agent cannot represent how often it has already woken
the on-call engineer, because no percept reports it. That single
missing piece of state is the whole difference.
"""

import random
from collections import namedtuple

random.seed(0)

# PEAS, as code. A percept is one ticket as the agent sees it.
Ticket = namedtuple("Ticket", "tid hour severity hours_to_breach")

WAKE_LIMIT = 3          # more than this and Ayesha is useless tomorrow
COST = {"major": 10, "minor": 3}
UNNECESSARY = 1
SLEEP = 4
```

```

def night(n=40):
    """One night's percept sequence: n tickets, 21:00 to 07:00."""
    out = []
    for tid in range(1, n + 1):
        out.append(Ticket(
            tid=tid,
            hour=21 + (tid * 10) // n,
            severity=random.choice(["major", "minor", "minor"]),
            hours_to_breach=random.choice([1, 2, 4, 8, 12]),
        ))
    return out

def simple_reflex(t, state):
    """Condition-action rules on the current percept only."""
    return t.hours_to_breach <= 4 or t.severity == "major"

def model_based(t, state):
    """Same rules, plus a model of what has already been spent.

    state["wakes"] is not in any percept: the agent maintains it.
    """
    if state["wakes"] >= WAKE_LIMIT:
        # budget gone: escalate only what will certainly breach badly
        return t.severity == "major" and t.hours_to_breach <= 2
    if t.hours_to_breach > 10:
        return False
    return t.hours_to_breach <= 4 or t.severity == "major"

def run(agent, tickets):
    """One night. The environment, not the agent, decides what an
    escalation achieves: past WAKE_LIMIT the engineer is too tired to
    fix anything, but the sleep is lost all the same.
    """
    state = {"wakes": 0}
    breach = sleep = waste = woke = 0
    for t in tickets:
        act = agent(t, state)
        fixed = False
        if act:
            state["wakes"] += 1
            woke += 1
            sleep += SLEEP
            if state["wakes"] <= WAKE_LIMIT:
                fixed = True
                # still sharp enough to help
            if t.hours_to_breach > 10:
                waste += UNNECESSARY
                # would not have breached tonight
            if not fixed and t.hours_to_breach <= 10:
                breach += COST[t.severity]
    return breach + sleep + waste, breach, sleep + waste, woke

if __name__ == "__main__":
    tickets = night()
    print("tickets:", len(tickets),
          " major:", sum(1 for t in tickets if t.severity == "major"),
          " wake limit:", WAKE_LIMIT)
    print()

```

```

print("agent          total   breaches   on-call   escalations")
print("-" * 58)
rows = {}
for name, agent in (("simple reflex", simple_reflex),
                    ("model-based ", model_based)):
    rows[name.strip()] = run(agent, tickets)
    tot, br, oc, woke = rows[name.strip()]
    print("%-16s %6d %10d %9d %13d" % (name, tot, br, oc, woke))

t1, b1, o1, w1 = rows["simple reflex"]
t2, b2, o2, w2 = rows["model-based"]
assert t2 < t1, "the model-based agent should cost less"
assert w2 < w1, "and should wake the engineer less often"
print()
print("total cost  %d -> %d  (%.0f%% lower)" %
      (t1, t2, 100 * (t1 - t2) / t1))
print("escalations %d -> %d  (%.1fx fewer)" % (w1, w2, w1 / w2))
print()
print("The model-based agent saw no extra percepts. It kept one")
print("number -- how often it had already woken her -- that no")
print("percept reports and the simple agent cannot represent.")

```

The output is worth reading carefully, because the interesting column is the one that does *not* change:

Agent	Total	Breaches	On-call	Escalations
Simple reflex	259	136	123	30
Model-based	156	136	20	5

The breach cost is *identical*: 136 either way. The simple reflex agent's extra twenty-five escalations bought nothing at all, because past the third one Ayesha was too tired to fix anything — and the sleep was lost regardless. The total falls by 40% purely by not doing the useless work, and the escalations fall sixfold.

Now change `WAKE_LIMIT` to 40 and re-run. The two agents converge, because with an unlimited budget the number being tracked no longer constrains anything. That is the test of whether internal state is earning its keep: state is worth maintaining exactly when it represents a limit the environment imposes and the percepts do not report.

🔍 Checkpoint — Test Yourself

1. Write a PEAS description for an agent that decides which of a day's pull requests to review first. Make sure the performance measure describes the state of the repository, not the activity of the agent.
2. A vending machine dispenses a drink when the right coins are inserted. Classify its environment on all six properties, and say which of the five agent architectures it needs.
3. An agent escalates a ticket on strong evidence and the escalation turns out to have been unnecessary. Was it rational? Justify your answer in one sentence using the definition in this chapter.

📋 Chapter Summary

- An agent perceives through sensors and acts through actuators. The **agent function** maps every percept sequence to an action; the **agent program** is a bounded implementation of it. AI is the search for good programs, because the functions are too large to tabulate.

- Specify every task as **PEAS** before building anything. The performance measure must describe the state of the environment, and must be written so that two people would score the same run identically.
- Classify the environment on six properties: observable, deterministic, episodic, static, discrete, single-agent. Each one forces a design decision, and partial observability is the most consequential — it is what makes internal state necessary.
- There are five agent architectures: simple reflex, model-based reflex, goal-based, utility-based and learning. Use the simplest that can do the job. This course builds the middle three; Chapter 17 signposts the fifth.
- Rationality is maximising *expected* performance given the percepts so far. It is not omniscience, not perfection, and it positively recommends gathering information when information is cheap.

Review Questions

1. **(MCQ)** The agent function maps: (a) states to actions (b) percept sequences to actions (c) actions to utilities (d) percepts to goals.
2. **(MCQ)** An agent must keep internal state principally because the environment is: (a) stochastic (b) dynamic (c) partially observable (d) multi-agent.
3. **(MCQ)** Which is *not* part of a PEAS description? (a) performance measure (b) environment (c) algorithm (d) sensors.
4. **(MCQ)** Chess played against a clock is: (a) fully observable, deterministic, static (b) fully observable, deterministic, dynamic (c) partially observable, stochastic, dynamic (d) fully observable, stochastic, static.
5. **(Short)** Explain the difference between rationality and omniscience, and say why the distinction matters when reviewing an incident.
6. **(Short)** Give one task for which a simple reflex agent is the right choice, and one superficially similar task for which it is not.
7. **(Short)** Why must the performance measure describe the environment rather than the agent? Give a measure that fails this test and say how it would be exploited.
8. **(Applied)** Take the sprint scheduler of Table 2.1. Write its full PEAS description, classify the environment on all six properties, and state which agent architecture the classification forces. Then name the single simplification that Chapter 7 makes in order to solve it at all, and what is lost by making it.

Representations and Search in Python

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part I — Foundations of Intelligent Systems

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Represent** a state as an immutable, hashable value, and **explain** why every search algorithm in this book requires it.
- **Implement** the `Problem` interface — initial state, goal test, actions, result, step cost — that every later chapter subclasses.
- **Choose** the right frontier structure for a search strategy, and **justify** the choice by its cost: `deque` for first-in first-out, `list` for last-in first-out, `heapq` for best-first.
- **Encode** a project’s task dependencies as a graph and **separate** the problem’s data from the algorithm that searches it.
- **Use** generators for successor functions and `lru_cache` for repeated subproblems, and **measure** what each one saves.

🔗 Before You Start

Chapter 2 (agent, percept, action, goal-based agent). Object Oriented Programming: classes, inheritance, abstract methods, and the difference between a value and a reference. Appendix A is the reference for every standard-library call used here.

📖 Key Terms in This Chapter

state • immutability • hashability • `frozenset` • `namedtuple` • the `Problem` interface • successor function • node • frontier • explored set • `deque` • `heapq` • priority queue • adjacency dictionary • generator • memoisation • `lru_cache`

3.1 Why This Chapter Exists

The prerequisite for this course is Object Oriented Programming, so you can already write Python. This chapter is not a Python tutorial. It is the chapter that fixes the four data structures the rest of the book is written in, so that Chapters 4 to 9 can say “push the successor onto the frontier” and mean something precise.

Every search algorithm in Part II is about twenty lines long. They are short only because the representation carries the weight. Get the representation wrong and the algorithm triples in size and starts to produce wrong answers for reasons that are extremely hard to see. That is the whole content of this chapter, and it is worth a week.



Here is the judgement to take away, before the details. *A state is a value, not an object.* Two states that describe the same situation must be equal and must hash the same, whether or not they are the same object in memory. If you find yourself writing `if state1.tasks == state2.tasks and state1.hours == state2.hours`, you have already lost: the language will do that for you, correctly and once, if the state is a tuple or a `frozenset`.

3.2 A State Is a Value

Consider the state of a release: which components have been coded, reviewed, tested and deployed. The obvious first attempt is a class with mutable attributes. It is wrong for three reasons, and all three matter.

⚠ Three reasons a mutable state object breaks search

- **You cannot put it in a set.** Search must ask “have I seen this state before?” millions of times. That question is a set membership test, and Python can only put hashable objects in a set. A class with mutable attributes is hashable only by identity, so two identical situations look different and the search explores the same state again and again — usually forever.
- **Mutation corrupts the frontier.** If a successor is produced by changing the parent in place, every node already sitting in the frontier that shares that object silently changes too. The bug appears as a search that returns a path that does not work when executed.
- **You cannot reconstruct the path.** Solutions are recovered by walking parent pointers back to the root. If the states were mutated on the way down, the ancestors no longer hold the values they had.

So states are built from Python’s immutable, hashable types: `tuple`, `frozenset`, `str`, numbers, and `namedtuple`. Figure 3.1 shows the three representations this book uses and when each is right.

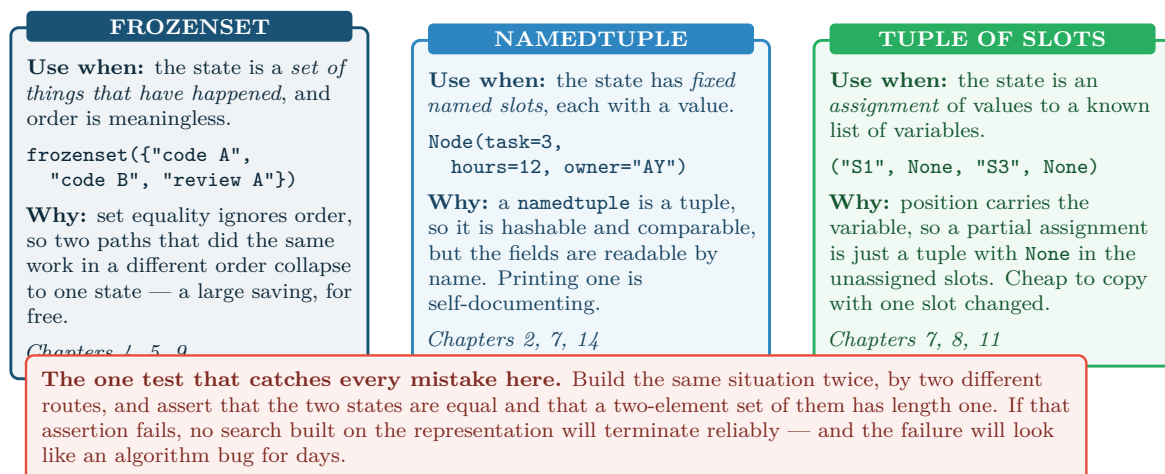


Figure 3.1: Three state representations, and the situation each one is for. All three are immutable and hashable; that is the property the algorithms depend on.

3.3 The Problem Interface

Every search problem in Chapters 4 to 9 is the same five things. Fixing them as one small class means the algorithms can be written once and reused unchanged.

☰ A search problem

A search problem is a five-tuple:

- an **initial state**;
- a **goal test**, a predicate on states — not necessarily a single goal state, because “every component deployed” describes many states;

- an **actions** function giving the actions legal in a state;
- a **result** function giving the state that follows from applying an action — the **successor function**, and the only place the rules of the domain live;
- a **step cost** for taking an action in a state.

Nothing else. Notably *not* part of the problem: how to search it. That separation is the point.

🔗 The interface, once, for the whole book

```
class Problem:
    """What every search algorithm in Chapters 4-9 is given."""

    def __init__(self, initial, goal=None):
        self.initial, self.goal = initial, goal

    def actions(self, state):          # -> iterable of actions
        raise NotImplementedError

    def result(self, state, action):  # -> new state (never mutated)
        raise NotImplementedError

    def goal_test(self, state):
        return state == self.goal

    def step_cost(self, state, action, nxt):
        return 1                      # override for weighted problems
```

A concrete problem overrides three methods and inherits the rest. Chapter 4's task-ordering problem, Chapter 5's sprint problem and Chapter 9's release planner are all subclasses of this, and the identical **breadth_first** function solves all three.

A **node** is not a state. A node is a state plus the bookkeeping the search needs: the parent it came from, the action that produced it, and the cost so far. States get compared and stored in sets; nodes get put on frontiers and walked backwards to build the solution. Confusing the two is the second most common bug in this material, after mutable states.

3.4 Frontiers: Three Structures, Three Strategies

The **frontier** holds the nodes that have been generated but not yet expanded. Remarkably, the choice of frontier structure *is* the choice of search strategy: the algorithm body does not change at all between breadth-first, depth-first and best-first search. Only the container does. Figure 3.2 shows the three, with the operation costs that decide the matter.

⚠️ Two frontier mistakes that cost real time

Using a list as a queue. `frontier.pop(0)` on a list moves every remaining element one place left. It is $O(n)$ per pop, so a breadth-first search generating n nodes does $O(n^2)$ pointless work. The cost is invisible on this chapter's nine tasks and decisive on anything the size of a real backlog. Use `deque.popleft()`.

Pushing bare tuples onto a heap. `heappush(h, (cost, node))` compares the second element whenever two costs are equal, and if `node` is a class without ordering, Python raises `TypeError` — intermittently, only on inputs that happen to tie. Always push `(priority, next(counter), node)`, with a monotonically increasing counter in the middle. The tie is then broken by insertion order and the node is never compared.

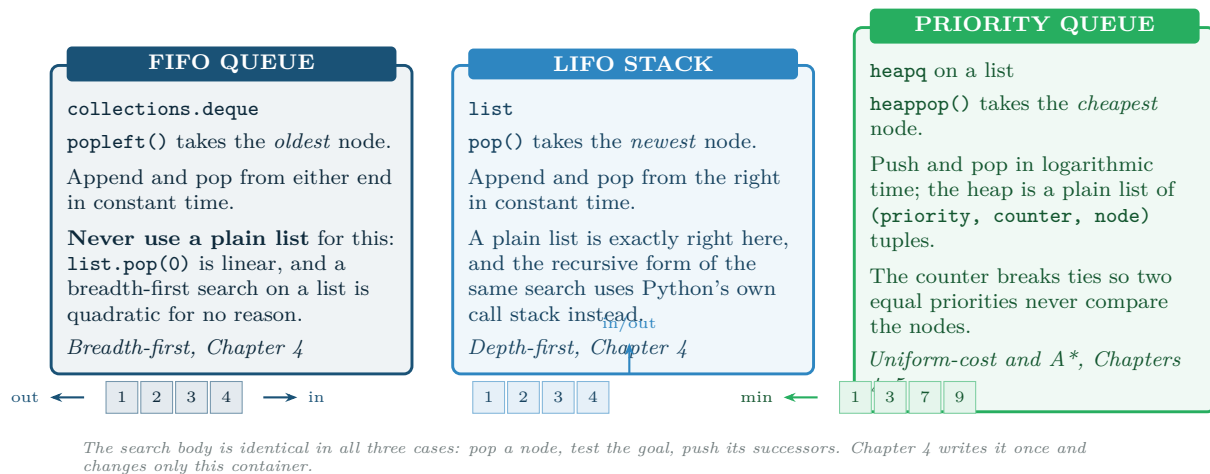


Figure 3.2: Three frontier structures. Choosing the container chooses the strategy, and the operation costs are the reason each one is matched to its search.

3.5 Representing the Project

All the domain data in this book is one small software project: nine tasks with a duration and a set of dependencies each. It lives in plain dictionaries, deliberately, so that every lab can be read without reference to a data file.

</> The project, as data

```
# task -> (duration in days, set of tasks that must finish first)
TASKS = {
    "spec": (3, set()),
    "schema": (2, {"spec"}),
    "api": (5, {"schema"}),
    "ui": (4, {"spec"}),
    "auth": (3, {"schema"}),
    "tests": (4, {"api", "auth"}),
    "docs": (2, {"api"}),
    "uat": (3, {"tests", "ui"}),
    "release": (1, {"uat", "docs"}),
}

def ready(done):
    """Tasks whose dependencies are all satisfied -- the successor
    function of Chapter 4, in one line."""
    return {t for t, (_, deps) in TASKS.items()
            if t not in done and deps <= done}
```

Two things to notice. `deps <= done` is subset containment, which reads as “every dependency is done” — this is why dependencies are sets. And `ready` returns the legal actions for a state, so it *is* the `actions` method of a `Problem`; Chapter 4 does no more than wrap it.

3.6 Generators and Memoisation

Two standard-library habits pay for themselves repeatedly in this course.

A **generator** successor function yields states one at a time instead of building a list. In search this matters because most successors are never used: a depth-first search that finds its goal in the first branch has generated one successor of each node on the path, not all of them. Writing `yield` instead of `result.append` is the whole change.

Memoisation caches a function’s result against its arguments. `functools.lru_cache` does it in one decorator, and it requires exactly the property Section 2 insisted on: the arguments must be hashable. Chapter 5’s heuristic is called many times on the same state, Chapter 8’s minimax re-reaches the same position by different move orders, and Chapter 11’s model counting revisits the same partial assignment. In all three, one decorator changes the running time by an order of magnitude and nothing else in the code.

Worked Example: Why the state representation decides whether search terminates

Two representations of “spec and schema are done”. Work out what each does to the search.

Representation A — a list, in completion order.

- Route 1: do `spec`, then `schema` $\Rightarrow$ `["spec", "schema"]`
- Route 2: not possible here, since `schema` depends on `spec` — so extend the example by one task. With `ui` also done: `["spec", "schema", "ui"]` and `["spec", "ui", "schema"]`.

These two lists are not equal. A list is also unhashable, so they cannot go in a set at all. Both routes are explored separately, and everything below them is explored twice.

Counting the damage. With n mutually independent tasks completed in any order, the number of distinct *orderings* is $n!$ while the number of distinct *situations* is 1. The dependencies in this project rule out most orderings, so the real gap is smaller than $n!$ but still large: the lab counts 184 reachable sequences against 22 reachable states, a factor of 8.

Representation B — a frozenset.

```
frozenset({"spec","schema","ui"}) = frozenset({"spec","ui","schema"})
```

Equal, and equal hashes. So the second route arrives at a state already in the explored set and is cut off immediately.

The measurement. This chapter’s lab reports both numbers on this project: 22 states reachable as sets against 184 paths as sequences. The search is the same search. *The representation bought a factor of eight, and not one line of algorithm changed.*

And the factor grows with the project. Add one task that depends on nothing and is required by nothing, and it can be inserted at any point in every existing sequence: the sequence count multiplies by roughly the plan length, while the state count merely doubles. On a realistic backlog of forty loosely coupled stories the ratio is no longer eight but astronomical, which is why this is a representation decision and not a micro-optimisation.

The lesson to carry forward. When a search is too slow, look at the state representation *before* looking at the algorithm. Ask: does my state record anything the goal test does not care about? Here, the order tasks were finished in is such a thing. Removing it is called *abstraction*, and it is the most effective single optimisation in this book.

Common Pitfalls

- **A state class with mutable attributes.** Unhashable in practice, hashable by identity at worst, and the resulting search never terminates. Use a tuple, `namedtuple` or `frozenset`.
- **Mutating a state to make a successor.** `done.add(t)` changes the parent that other frontier nodes point at. Write `done | {t}`, which builds a new set.
- **Recording in the state something the goal test ignores.** Completion order, a timestamp, a counter of loop iterations. Each one multiplies the state space.
- **`list.pop(0)` as a queue.** Quadratic. Use `collections.deque`.
- **Heap entries without a tie-breaker.** Works until two priorities are equal, then raises `TypeError` on unorderable nodes — intermittently, which is the worst way for a bug to behave.

- `lru_cache` on a function taking a list. Raises `TypeError: unhashable type`. This is the same discipline as the states, enforced by the standard library.

Try It Yourself: The Frontier and the Graph: The Course Toolkit

Everything later chapters import. Build it once, assert that it behaves as documented, and measure the claim of the worked example.

```

"""Chapter 3 lab -- the course toolkit.

The Problem interface, three frontiers, the project graph, and the
measurement that justifies using frozensets for states. Every later
chapter builds on exactly these shapes.
"""

import heapq
import itertools
from collections import deque, namedtuple

# --- the project, as data -----
TASKS = {
    "spec": (3, set()),
    "schema": (2, {"spec"}),
    "api": (5, {"schema"}),
    "ui": (4, {"spec"}),
    "auth": (3, {"schema"}),
    "tests": (4, {"api", "auth"}),
    "docs": (2, {"api"}),
    "uat": (3, {"tests", "ui"}),
    "release": (1, {"uat", "docs"}),
}
ALL = frozenset(TASKS)

# --- the interface every later chapter subclasses -----
class Problem:
    """A search problem: five things, and no search strategy."""

    def __init__(self, initial, goal=None):
        self.initial, self.goal = initial, goal

    def actions(self, state):
        raise NotImplementedError

    def result(self, state, action):
        raise NotImplementedError

    def goal_test(self, state):
        return state == self.goal

    def step_cost(self, state, action, nxt):
        return 1

class TaskOrder(Problem):
    """Finish every task, respecting dependencies.

    A state is the frozenset of finished tasks -- nothing else. The
    order they were finished in is deliberately not recorded.
    """

    def __init__(self):

```

```

    super().__init__(frozenset(), ALL)

    def actions(self, state):
        for t, (_, deps) in TASKS.items():
            if t not in state and deps <= state:
                yield t                # a generator, not a list

    def result(self, state, action):
        return state | {action}        # new value, never mutated

    def step_cost(self, state, action, nxt):
        return TASKS[action][0]        # duration in days

Node = namedtuple("Node", "state parent action cost")

def path(node):
    """Walk parent pointers back to the root."""
    out = []
    while node.parent is not None:
        out.append(node.action)
        node = node.parent
    return list(reversed(out))

# --- the three frontiers, behind one small interface -----
class FIFO:
    def __init__(self):
        self.q = deque()

    def push(self, node):
        self.q.append(node)

    def pop(self):
        return self.q.popleft()        # O(1); list.pop(0) is O(n)

    def __len__(self):
        return len(self.q)

class LIFO:
    def __init__(self):
        self.q = []

    def push(self, node):
        self.q.append(node)

    def pop(self):
        return self.q.pop()

    def __len__(self):
        return len(self.q)

class Cheapest:
    """A heap of (priority, tie, node). The counter keeps the heap
    from ever comparing two nodes."""
    def __init__(self):
        self.h, self.tie = [], itertools.count()

```

```

def push(self, node):
    heapq.heappush(self.h, (node.cost, next(self.tie), node))

def pop(self):
    return heapq.heappop(self.h)[2]

def __len__(self):
    return len(self.h)

def search(problem, Frontier):
    """One body; the container decides the strategy. Chapter 4 proves
    this by calling it three times."""
    start = Node(problem.initial, None, None, 0)
    frontier, seen = Frontier(), {problem.initial}
    frontier.push(start)
    expanded, peak = 0, 1
    while len(frontier):
        node = frontier.pop()
        if problem.goal_test(node.state):
            return path(node), node.cost, expanded, peak
        expanded += 1
        for a in problem.actions(node.state):
            nxt = problem.result(node.state, a)
            if nxt not in seen:
                seen.add(nxt)
                frontier.push(Node(nxt, node, a,
                                   node.cost + problem.step_cost(
                                       node.state, a, nxt)))
        peak = max(peak, len(frontier))
    return None, None, expanded, peak

# --- the measurement of the worked example -----
def count_states():
    """Reachable situations, when a state is a set of finished tasks."""
    seen, stack = {frozenset()}, [frozenset()]
    while stack:
        s = stack.pop()
        for t, (_, deps) in TASKS.items():
            if t not in s and deps <= s:
                n = s | {t}
                if n not in seen:
                    seen.add(n)
                    stack.append(n)
    return len(seen)

def count_orderings():
    """Reachable sequences, when order is (wrongly) part of the
    state."""
    total = 0
    stack = [frozenset()]
    while stack:
        s = stack.pop()
        total += 1
        for t, (_, deps) in TASKS.items():
            if t not in s and deps <= s:
                stack.append(s | {t})
    return total

```

```

if __name__ == "__main__":
    # 1. the representation test of Figure 3.1
    a = frozenset({"spec", "schema", "ui"})
    b = frozenset({"spec", "ui", "schema"})
    assert a == b and len({a, b}) == 1
    print("states equal by two routes:", a == b)

    # 2. the three frontiers on the same problem
    p = TaskOrder()
    print()
    print("frontier      plan len    days    expanded    peak")
    print("-" * 50)
    for name, F in ((("FIFO", FIFO), ("LIFO", LIFO),
                    ("Cheapest", Cheapest))):
        plan, cost, exp, peak = search(p, F)
        print("%s %9d %6d %10d %6d" % (name, len(plan), cost, exp, peak))

    # 3. the factor the representation buys
    s, o = count_states(), count_orderings()
    print()
    print("situations (frozenset states):", s)
    print("sequences (order in state) :", o)
    print("factor saved           : %.0fx" % (o / s))
    assert o > 5 * s, "sets should collapse many sequences into one"

    # 4. every plan is a legal ordering
    plan, _, _, _ = search(p, FIFO)
    done = set()
    for t in plan:
        assert TASKS[t][1] <= done, "dependency violated: " + t
        done.add(t)
    assert done == set(TASKS)
    print()
    print("plan is a legal topological order:", " -> ".join(plan))

```

All three frontiers find a legal ordering of all nine tasks, and all three report a cost of 27 days — every task has to be done, so the total duration does not depend on the order. That is precisely why cost only becomes interesting in Chapter 5, where not every action is taken. Where the three differ is work done: breadth-first and uniform-cost expand 21 states with a peak frontier of 6 and 5, depth-first expands 9 with a peak of 4. The representation measurement comes out at 22 states against 184 sequences.

Now break it on purpose: change `result` to return `state + (action,)` over a tuple state, and watch `expanded` climb towards the sequence count while the answer stays the same. That experiment is the chapter.

🔍 Checkpoint — Test Yourself

1. Why can a Python `set` not hold a state represented as a list, and what goes wrong in a search when the explored set cannot be built?
2. A colleague's best-first search raises a `TypeError` about comparing two `Node` objects, but only on about one input in twenty. Explain the bug and give the one-line fix.
3. A state records the set of finished tasks *and* the number of search steps taken so far. The goal test looks only at the finished tasks. What does the extra field do to the size of the state space, and why?

Chapter Summary

- A state is a *value*: immutable and hashable, so that two descriptions of the same situation are equal and hash alike. Use `frozenset` for sets of completed things, `namedtuple` for fixed named slots, and a tuple of slots for assignments.
- A **node** is a state plus parent, action and cost. States go in sets; nodes go on frontiers and are walked backwards to recover the solution.
- Every search problem is five things: initial state, goal test, actions, result and step cost. The strategy is not one of them — which is why one search body serves every chapter of Part II.
- The frontier structure *is* the strategy: `deque` for breadth-first, `list` for depth-first, `heapq` for best-first. Never use `list.pop(0)`, and always give heap entries a tie-breaking counter.
- Successor functions are generators, because most successors are never needed. `lru_cache` removes repeated subproblems for one line, and requires the same hashability the states already have.
- When a search is too slow, look first at what the state records that the goal test does not care about. Removing completion order collapsed 184 sequences into 22 states on this chapter's project, and the saving grows with the size of the backlog.

Review Questions

1. (MCQ) Which type is *not* usable as a search state? (a) `tuple` (b) `frozenset` (c) `list` (d) `namedtuple`.
2. (MCQ) Popping the oldest node from the frontier gives: (a) depth-first search (b) breadth-first search (c) uniform-cost search (d) best-first search.
3. (MCQ) `deps <= done` where both are sets tests that: (a) `done` has fewer elements (b) every dependency is finished (c) the sets are equal (d) `deps` is sorted before `done`.
4. (MCQ) The middle element of a heap entry (`cost`, `counter`, `node`) exists to: (a) speed up the heap (b) record insertion time for reporting (c) stop Python comparing two nodes when costs tie (d) allow duplicate states.
5. (Short) State the difference between a state and a node, and name one thing each is used for that the other cannot do.
6. (Short) Explain why making a successor by mutating the parent state produces a plan that fails when executed, even though the search reports success.
7. (Short) Why is a generator the right shape for a successor function? Name the search strategy that benefits most, and say why.
8. (Applied) Add a task `perf` of duration 2, depending on `api` and `ui`, and required before `release`. Write the new `TASKS` entry and the changed `release` entry, then predict — before running it — whether the number of reachable `frozenset` states goes up by more or less than a factor of two, and justify your prediction.

PART II

Problem Solving by Search

An agent that has a goal but cannot look ahead is stuck with the first action that seems sensible. Part II is how it looks ahead: systematically when it knows nothing, selectively when it can estimate distance to the goal, and — when the route does not matter and only the destination does — by improving one state until it cannot be improved.

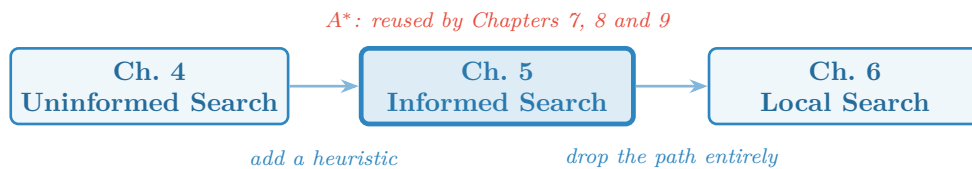


Figure P2.1: Reading path through Part II. Each chapter relaxes something the one before it insisted on: Chapter 5 drops blind expansion, Chapter 6 drops the path.

Problem Formulation and Uninformed Search

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part II — Problem Solving by Search

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Formulate** an informal task as a search problem, choosing a state representation and **justifying** what it leaves out.
- **Distinguish** the state space from the search tree, and **explain** why the tree can be infinite when the space is finite.
- **Trace** breadth-first, depth-first, depth-limited, iterative deepening and uniform-cost search by hand on a small graph.
- **Compare** the five strategies on completeness, optimality, time and space, and **choose** one for a given problem with a reason.
- **Explain** why uninformed search cannot scale, and **quantify** the growth that defeats it.

🔗 Before You Start

Chapter 3 (the `Problem` interface, nodes, the three frontiers, `frozenset` states) and Chapter 2 (the goal-based agent, whose deliberation is exactly the search of this chapter). Every algorithm here is the same twelve-line body with a different frontier.

📖 Key Terms in This Chapter

problem formulation • state space • search tree • node expansion • frontier • explored set • redundant path • branching factor • depth • completeness • optimality • breadth-first search • depth-first search • depth-limited search • iterative deepening • uniform-cost search • combinatorial explosion

4.1 From a Task to a State Space

A delivery manager says: *get the release out, and don't start anything before the things it depends on are finished*. That is a task, not a problem. Turning it into a problem means deciding five things — the five of Chapter 3 — and the decisions are not mechanical.

🔗 Formulating the release-ordering task

- *States*. The set of finished tasks. A `frozenset`, so that two routes that finished the same work collapse to one state.
- *Initial state*. The empty set.
- *Actions*. In state s , the tasks not yet done whose dependencies are all in s — the `ready` function of Chapter 3.
- *Result*. $s \cup \{t\}$. A new set; the old one is never touched.
- *Goal test*. Is every task in s ?
- *Step cost*. The task's duration in days.

Notice what the formulation *throws away*: who does each task, the calendar, holidays, the fact that two tasks by the same person cannot overlap. Each omission is a modelling decision that

makes the problem solvable and the answer less directly useful. Chapter 7 puts the people back in, and it costs a different algorithm to do it.



The single most valuable habit in this part of the course: *write the formulation down before writing the search*, and beside each state component write why the goal test needs it. Anything the goal test cannot see is a candidate for deletion, and every deletion shrinks the space multiplicatively.

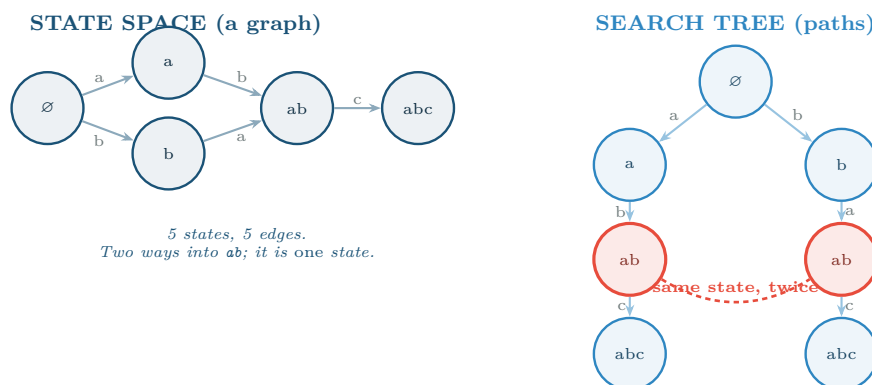
Beginners reach for a cleverer algorithm when a search is too slow. Professionals reach for the formulation first, because that is where the factors of a hundred are. Chapter 3's worked example measured one such factor; this chapter's lab measures another.

4.2 The State Space and the Search Tree

These are two different objects and confusing them causes both of the classic bugs.

The **state space** is a graph: nodes are states, edges are actions. It is a property of the problem. For this project it has 22 reachable states.

The **search tree** is what the algorithm builds as it explores. Its root is the initial state, and its branches are paths. The same state appears in it once for every distinct path that reaches it — so the tree has 184 nodes where the graph has 22. Figure 4.1 shows both for a three-task fragment.



The explored set is what turns the tree back into the graph. Keep a set of states already generated and refuse to add one twice, and the search visits 5 nodes instead of 7 — here a small saving, on the real project the difference between 22 and 184. Omit it and a state space with a cycle gives an *infinite* search tree, which is the first classic bug.

Figure 4.1: The state space and the search tree for a three-task fragment where c depends on both a and b. The tree repeats every state once per path that reaches it.

⚠ The two classic bugs, and their symptoms

No explored set. Symptom: the search never terminates, or returns a path that revisits a state. Cause: the search tree of a cyclic space is infinite. Fix: a **set** of states, tested before pushing.

Goal-testing on generation instead of on expansion. Symptom: breadth-first search returns a correct shallowest path (fine), but uniform-cost search returns a *sub-optimal* one. Cause: when a node is generated, a cheaper path to the same state may still be in the frontier. Breadth-first search may test on generation because all edges cost the same; uniform-cost search must test when the node is *popped*. This is the subtlest distinction in the chapter and Section 4.6 returns to it.

4.3 Breadth-First Search

Expand the shallowest unexpanded node: a first-in first-out frontier, so `deque.popleft()`.

Breadth-first search is **complete** — if a solution exists at finite depth it will be found — and it returns the *shallowest* solution. That is an optimal solution only when every action costs the same. With durations of 1 to 5 days, the shallowest plan is not the cheapest, and it is a mistake to call breadth-first search optimal without that qualification.

Its cost is the problem. With branching factor b and solution depth d , it generates $O(b^d)$ nodes and — fatally — must *store* $O(b^d)$ of them, because the whole frontier is one level wide.

4.4 Depth-First and Depth-Limited Search

Expand the deepest unexpanded node: a last-in first-out frontier, so `list.pop()`, or simple recursion.

The gain is memory: only the current path and its siblings need to be held, which is $O(bm)$ for maximum depth m rather than $O(b^d)$. The loss is both completeness and optimality — in a space with cycles and no explored set, depth-first search follows one branch forever.

Depth-limited search fixes the infinite descent by refusing to go below depth ℓ . Now it terminates, but it is incomplete in a new way: if the shallowest solution is deeper than ℓ it reports failure on a solvable problem, which is the worst possible answer.

4.5 Iterative Deepening

Run depth-limited search with $\ell = 0$, then 1, then 2, and so on until a solution is found. This sounds wasteful and is the standard answer for large uninformed problems, because it has breadth-first search's completeness and shallowest-solution guarantee with depth-first search's memory.

The apparent waste is small. The work is dominated by the deepest level, because the levels grow geometrically:

$$N(\text{IDS}) = (d+1)b^0 + db^1 + (d-1)b^2 + \cdots + 1 \cdot b^d$$

and for $b = 10$, $d = 5$ this is 123,456 nodes against breadth-first search's 111,111 — about 11% more work, in exchange for storing 50 nodes instead of 100,000. Figure 4.2 shows why the ratio improves as b grows.

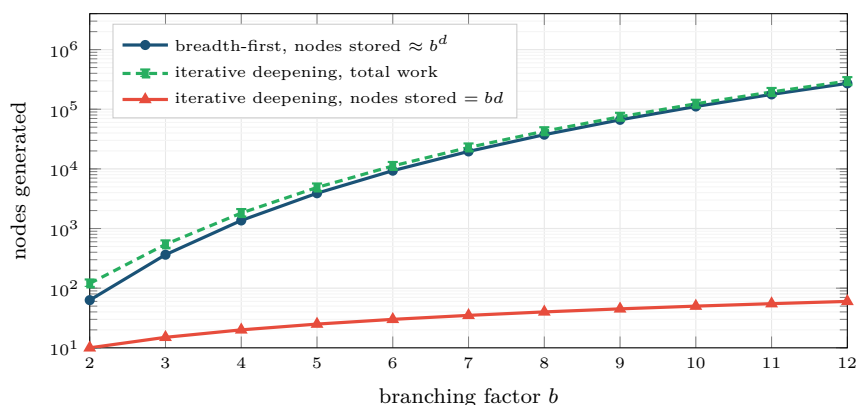


Figure 4.2: Iterative deepening against breadth-first search at solution depth $d = 5$. The repeated work is a modest overhead that shrinks as the tree widens — 91% extra at $b = 2$, 11% at $b = 10$, 9% at $b = 12$ — while the memory requirement falls from exponential to linear.

4.6 Uniform-Cost Search

Expand the node with the lowest *path cost* $g(n)$, not the shallowest: a priority-queue frontier, so **heapq**.

This is the first algorithm in the book that is genuinely optimal in cost, and it is optimal for a reason worth stating precisely: when a node is *popped*, every remaining path to it costs at least as much, because the frontier is ordered by cost and costs are non-negative. So the first time the goal is popped, it is popped by its cheapest path.

That argument is exactly why the goal test must happen on *popping* and not on generating, and why every edge cost must be non-negative. Break either condition and uniform-cost search quietly returns sub-optimal answers.

Uniform-cost search is breadth-first search when all costs are equal, and it is Dijkstra's algorithm. In Chapter 5 it becomes A^* by adding one term.

Strategy	Complete?	Cheapest?	Time	Space	Frontier
Breadth-first	Yes	Only if costs equal	$O(b^d)$	$O(b^d)$	deque
Depth-first	No (cycles)	No	$O(b^m)$	$O(bm)$	list
Depth-limited	No (if $\ell < d$)	No	$O(b^\ell)$	$O(b\ell)$	list
Iterative deepening	Yes	Only if costs equal	$O(b^d)$	$O(bd)$	list
Uniform-cost	Yes	Yes	$O(b^{1+C^*/\epsilon})$	same as time	heapq

Table 4.1: The five uninformed strategies. b is the branching factor, d the solution depth, m the maximum depth, C^* the optimal cost and ϵ the smallest edge cost. Note that only uniform-cost search is optimal in cost.

4.7 Why Uninformed Search Cannot Scale

Every entry in the time column of Table 4.1 is exponential, and no choice of frontier changes that. The reason is arithmetic: at branching factor 10, depth 10 is 10^{10} nodes. At a million nodes per second that is about three hours, and depth 12 is twelve days.

This is the **combinatorial explosion** that ended the first wave of AI research (Chapter 1), and it is not an implementation problem. These algorithms are *uninformed*: they know nothing about the problem except how to generate successors and recognise a goal, so they cannot prefer a promising branch. The only escape is to give the search a way to estimate how far a state is from the goal, which is Chapter 5.

Worked Example: Tracing all four strategies on one small graph

The three-task fragment of Figure 4.1, with costs: **a** costs 3, **b** costs 1, **c** costs 2, and **c** requires both. Goal: **abc**. Expansion order, with the frontier shown after each step.

Breadth-first (first-in first-out, explored set on).

- Pop $\emptyset$. Push **a**, **b**. Frontier: [**a**, **b**].
- Pop **a**. Push **ab**. Frontier: [**b**, **ab**].
- Pop **b**. Its only successor is **ab**, already in the explored set — *not* pushed. Frontier: [**ab**].
- Pop **ab**. Push **abc**. Frontier: [**abc**].
- Pop **abc**. Goal. Path **a,b,c**, depth 3, cost $3 + 1 + 2 = 6$.

Nodes expanded: 4. Without the explored set, step 3 would have pushed a second **ab** and everything below it would be built twice.

Depth-first (last-in first-out). Pop $\emptyset$, push **a**, **b**; pop **b** (the newest); push **ab**; pop **ab**; push **abc**; pop **abc**. Goal in 4 expansions, path **b,a,c**. Same cost here by coincidence; on the full project it is worse.

Iterative deepening. $\ell = 0$: expand $\emptyset$, fail (1 node). $\ell = 1$: $\emptyset$, **a**, **b**, fail (3). $\ell = 2$: $\emptyset$, **a**, **ab**, **b**, fail (4). $\ell = 3$: reaches **abc**. Total about 12 node generations against breadth-first search's 5 — but the deepest frontier ever held is 2 nodes, not 2 levels.

Uniform-cost (cheapest first). Frontier as (cost, state).

- Pop (0, $\emptyset$). Push (3, **a**), (1, **b**).
- Pop (1, **b**) — cheaper than **a**, so the order differs from breadth-first search. Push (4, **ab**).
- Pop (3, **a**). Successor **ab** at cost $3 + 1 = 4$, no better than the 4 already in the frontier, so nothing improves.
- Pop (4, **ab**). Push (6, **abc**).
- Pop (6, **abc**). Goal, cost 6, and now *provably* cheapest.

The point of the trace. All four found a solution; only uniform-cost search knows its answer is the cheapest. And notice step 3: had the goal been tested when **ab** was *generated* rather than when it was popped, the search would have committed to whichever path reached it first, which is the bug the alertbox warns about.

! Common Pitfalls

- **Omitting the explored set.** An infinite search tree on a finite space. The symptom is a hang, and the cause is two lines away.
- **Calling breadth-first search optimal.** It returns the *shallowest* solution. That is the cheapest only when every action costs the same — which, with task durations, it does not.
- **Goal-testing on generation in uniform-cost search.** Silently sub-optimal answers, and the tests pass because the answer is still *a* solution.
- **Negative step costs.** Uniform-cost search's optimality proof needs non-negative costs. “Refactoring saves two days” modelled as a negative cost breaks the algorithm, not just the estimate.
- **Depth-limited search reporting failure as impossibility.** It means “no solution within ℓ ”. Treating that as “no solution” is a reporting bug with real consequences.
- **Believing a faster language will fix b^d .** A hundredfold speedup buys two more levels of depth at $b = 10$. Change the formulation or add a heuristic.

🔧 Try It Yourself: Task Ordering Four Ways

One problem, four strategies, one table. The search body is the one from Chapter 3; only the frontier changes. The lab also measures the cost of dropping the explored set.

```
"""Chapter 4 lab -- uninformed search on the project graph.
```

```
Breadth-first, depth-first, iterative deepening and uniform-cost over
the same task-ordering problem. The search body never changes: the
frontier decides the strategy, exactly as Chapter 3 claimed.
"""
```

```
import heapq
import itertools
from collections import deque, namedtuple
```

```

# duration in days, and the tasks that must finish first
TASKS = {
    "spec": (3, set()),
    "schema": (2, {"spec"}),
    "api": (5, {"schema"}),
    "ui": (4, {"spec"}),
    "auth": (3, {"schema"}),
    "tests": (4, {"api", "auth"}),
    "docs": (2, {"api"}),
    "uat": (3, {"tests", "ui"}),
    "release": (1, {"uat", "docs"}),
}
ALL = frozenset(TASKS)
Node = namedtuple("Node", "state parent action cost depth")

def actions(state):
    for t, (_, deps) in TASKS.items():
        if t not in state and deps <= state:
            yield t

def result(state, action):
    return state | {action}

def cost_of(action):
    return TASKS[action][0]

def path(node):
    out = []
    while node.parent is not None:
        out.append(node.action)
        node = node.parent
    return list(reversed(out))

def graph_search(pop_which, explored=True):
    """One body. pop_which selects the strategy.

    'fifo' -> breadth-first      'lifo' -> depth-first
    'cheap' -> uniform-cost
    """
    start = Node(frozenset(), None, None, 0, 0)
    tie = itertools.count()
    if pop_which == "cheap":
        frontier = [(0, next(tie), start)]
    else:
        frontier = deque([start])
    seen = {start.state} if explored else None
    expanded = 0
    peak = 1
    while frontier:
        if pop_which == "cheap":
            node = heapq.heappop(frontier)[2]
        elif pop_which == "fifo":
            node = frontier.popleft()
        else:
            node = frontier.pop()

```

```

    if node.state == ALL:                # goal test on POPPING
        return path(node), node.cost, expanded, peak
    expanded += 1
    for a in actions(node.state):
        nxt = result(node.state, a)
        if explored:
            if nxt in seen:
                continue
            seen.add(nxt)
        child = Node(nxt, node, a, node.cost + cost_of(a),
                     node.depth + 1)
        if pop_which == "cheap":
            heapq.heappush(frontier, (child.cost, next(tie), child))
        else:
            frontier.append(child)
    peak = max(peak, len(frontier))
    return None, None, expanded, peak

def depth_limited(node, limit, seen, stats):
    """Recursive depth-limited search. Returns a node or None."""
    if node.state == ALL:
        return node
    if node.depth >= limit:
        return None
    stats["expanded"] += 1
    for a in actions(node.state):
        nxt = result(node.state, a)
        if nxt in seen:
            continue
        seen.add(nxt)
        child = Node(nxt, node, a, node.cost + cost_of(a),
                     node.depth + 1)
        found = depth_limited(child, limit, seen, stats)
        if found is not None:
            return found
        seen.discard(nxt)                # allow other branches to reuse it
    return None

def iterative_deepening():
    stats = {"expanded": 0}
    for limit in range(1, len(TASKS) + 2):
        start = Node(frozenset(), None, None, 0, 0)
        found = depth_limited(start, limit, {start.state}, stats)
        if found is not None:
            return path(found), found.cost, stats["expanded"], limit
    return None, None, stats["expanded"], None

def legal(plan):
    done = set()
    for t in plan:
        if not TASKS[t][1] <= done:
            return False
        done.add(t)
    return done == set(TASKS)

if __name__ == "__main__":
    print("tasks: %d    total duration: %d days"
          % (len(TASKS), sum(d for d, _ in TASKS.values())))

```

```

print()
print("strategy          len days expanded peak")
print("-" * 49)
rows = {}
for name, key in (("breadth-first ", "fifo"),
                  ("depth-first   ", "lifo"),
                  ("uniform-cost  ", "cheap")):
    plan, cost, exp, peak = graph_search(key)
    rows[name.strip()] = (plan, cost, exp, peak)
    assert legal(plan), name
    print("%s %6d %5d %9d %5d"
          % (name, len(plan), cost, exp, peak))
plan, cost, exp, depth = iterative_deepening()
assert legal(plan)
print("%s %6d %5d %9d %5s"
      % ("iter. deepening", len(plan), cost, exp, "-"))

# every task must be done, so every complete plan costs the same
costs = {c for (_, c, _, _) in rows.values()}
assert costs == {sum(d for d, _ in TASKS.values())}
print()
print("Every plan costs %d days: all nine tasks are mandatory, so"
      % costs.pop())
print("cost cannot discriminate. Chapter 5 gives it something to")
print("discriminate on.")

# what the explored set is worth
_, _, with_set, _ = graph_search("fifo", explored=True)
_, _, without, _ = graph_search("fifo", explored=False)
print()
print("breadth-first expansions with explored set:    %d" % with_set)
print("breadth-first expansions without explored set: %d" % without)
print("factor: %.1fx more work" % (without / with_set))
assert without > with_set

```

Read the output against Table 4.1: depth-first search holds the smallest frontier and breadth-first search the largest, every complete plan costs the same 27 days, and removing the explored set multiplies the work. The last number is the one to remember — it is the whole argument of Figure 4.1, measured.

🔍 Checkpoint — Test Yourself

1. A state space has 22 states and its search tree has 184 nodes. Explain in one sentence where the other 162 come from, and what single data structure removes them.
2. Why must uniform-cost search test for the goal when a node is popped rather than when it is generated? Give the smallest graph on which the difference shows.
3. Iterative deepening re-expands the top of the tree on every iteration. Why is the total work still the same order as breadth-first search, and what is bought with the repetition?

📋 Chapter Summary

- Formulating a problem means choosing states, initial state, actions, result, goal test and step cost — and deciding what to leave out. The omissions are where the factors of a hundred live.
- The **state space** is a graph and belongs to the problem; the **search tree** is built by the algorithm and repeats each state once per path. An **explored set** turns the tree back into the graph and is what makes search on a cyclic space terminate.

- Breadth-first search is complete and returns the shallowest solution — cheapest only when all actions cost the same. It needs $O(b^d)$ memory, which is what defeats it.
- Depth-first search needs only $O(bm)$ memory but is neither complete nor optimal. Depth-limited search terminates but reports failure on solvable problems.
- Iterative deepening has breadth-first search’s guarantees with depth-first search’s memory, for about 11% more work at $b = 10$, $d = 5$.
- Uniform-cost search expands the cheapest path first and is genuinely cost-optimal, provided the goal test happens on popping and every step cost is non-negative. It is Dijkstra’s algorithm, and Chapter 5 turns it into A^* .
- All five are exponential in time, because none of them knows anything about the problem beyond generating successors. That is what “uninformed” means, and why the next chapter exists.

Review Questions

1. **(MCQ)** Which strategy is guaranteed to return the cheapest solution when action costs differ? (a) breadth-first (b) depth-first (c) iterative deepening (d) uniform-cost.
2. **(MCQ)** Depth-first search’s space complexity is: (a) $O(b^d)$ (b) $O(bm)$ (c) $O(d)$ (d) $O(b^m)$.
3. **(MCQ)** Omitting the explored set on a state space with cycles makes the search: (a) slower but still correct (b) non-optimal only (c) potentially non-terminating (d) unable to find shallow solutions.
4. **(MCQ)** Uniform-cost search is equivalent to breadth-first search when: (a) the graph is a tree (b) all step costs are equal (c) the heuristic is zero (d) the goal is unique.
5. **(Short)** State the difference between the state space and the search tree, and give the size of each for the three-task fragment of Figure 4.1.
6. **(Short)** Explain why iterative deepening is preferred to breadth-first search on large problems despite doing more work.
7. **(Short)** Why does uniform-cost search require non-negative step costs? Describe what goes wrong with a negative edge.
8. **(Applied)** On the project graph, add a task `perf` of duration 2 depending on `api` and `ui`, required before `release`. By hand, give the depth of the shallowest complete plan and the total cost of every complete plan. Then say which of the five strategies would now be the right choice, and why cost-based search is still not useful on this problem.

Informed Search and Heuristics

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part II — Problem Solving by Search

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Define** a heuristic function and **distinguish** greedy best-first search from A^* by the term each one minimises.
- **State** admissibility and consistency precisely, and **explain** which one A^* needs for tree search and which for graph search.
- **Prove** that A^* with an admissible heuristic returns an optimal solution, and **identify** the step of the proof that fails when the heuristic overestimates.
- **Invent** an admissible heuristic for a new problem by relaxing a constraint, and **compare** two heuristics by dominance.
- **Measure** the trade-off between nodes expanded and solution quality, and **choose** greedy or A^* with a justification.

🔗 Before You Start

Chapter 4 (uniform-cost search, the explored set, goal-testing on popping, and why every uninformed strategy is exponential) and Chapter 3 (the `heapq` frontier with a tie-breaking counter). A^* is uniform-cost search with one term added, so the previous chapter's code is the starting point.

📖 Key Terms in This Chapter

heuristic function • greedy best-first search • A^* • evaluation function • path cost g • heuristic estimate h • admissibility • consistency • monotonicity • dominance • relaxed problem • critical path • effective branching factor • optimally efficient

5.1 A Sprint Where the Cost Matters

Chapter 4 ended on an awkward note: every complete plan for the release cost the same twenty-seven days, because every task was mandatory. Cost-based search had nothing to choose between. Real sprints are not like that — there is almost always more than one way to get a thing done. So this chapter uses a slightly larger network with one genuine choice in it. The team must get a release out, and the testing can be done either way:

- **Manual testing** — six days, needs only the API and authentication finished.
- **Automated testing** — two days, but it needs a test harness built first, which is three days. Five days in total, and the harness needs the schema.

Automation wins by one day, and the whole point of this chapter is that finding out *which* route is cheaper is a search problem, and that a good heuristic finds it while doing a third less work. Figure 5.1 is the network.

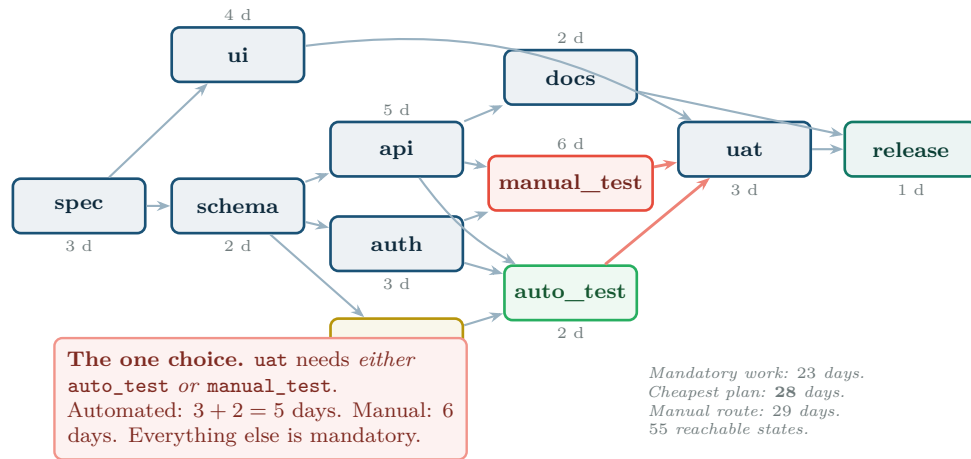


Figure 5.1: The sprint network. Ten of the eleven tasks are unavoidable; the testing step can be reached two ways, and the cheaper route is not the shorter one in number of tasks.

5.2 Heuristics: Telling the Search Where the Goal Is

Uniform-cost search spreads outwards in every direction equally, because it has no idea which direction the goal is in. A **heuristic** gives it one.

Heuristic function

A **heuristic** $h(n)$ estimates the cheapest cost from the state at node n to a goal state. By convention $h(n) = 0$ for every goal state.

h is **admissible** if it never overestimates:

$$0 \leq h(n) \leq h^*(n) \quad \text{for every } n,$$

where $h^*(n)$ is the true cheapest completion cost from n .

h is **consistent** (or **monotonic**) if for every node n and every successor n' reached by an action of cost $c(n, n')$:

$$h(n) \leq c(n, n') + h(n').$$

Consistency implies admissibility. The reverse does not hold.

Two heuristics for the sprint, both of which a delivery manager would recognise.

Mandatory work remaining. Eight of the eleven tasks appear in every plan that reaches a release: `spec`, `schema`, `api`, `auth`, `ui`, `docs`, `uat` and `release`. Sum the durations of those not yet done. At the start this is 23 days. It is admissible because the tasks it counts genuinely must be done and it ignores the testing work entirely, so it can only under-count.

Critical path remaining. The longest chain of dependencies still to be traversed, taking the cheaper branch at the choice. At the start this is 16 days. It is admissible because every task on a chain must be done in order, so the chain length is a lower bound on the total.

The mandatory-work estimate is much the larger of the two at the start — 23 against 16 — but, as Section 5.4.2 shows by measurement, it is *not* larger everywhere, and that turns out to matter.

5.3 Greedy Best-First Search, and Why It Is Not Enough

Order the frontier by h alone: always expand whatever *looks* closest to the goal. This is **greedy best-first search**, and it is fast — on the sprint it finds a plan after expanding ten or eleven states, against uniform-cost search’s forty-six.

It is also not optimal, and the sprint shows exactly how it fails. With the mandatory-work heuristic, greedy search returns the *manual testing* plan at 29 days instead of the automated one at 28. The reason is instructive: the harness is three days of work that reduces h by nothing at all, because the harness is not mandatory and the heuristic only counts mandatory work. Greedy search therefore sees the harness as pure delay and refuses to build it — which is a recognisable human failure mode as well as an algorithmic one.

⚠ Greedy search ignores the road already travelled

The defect is structural, not a matter of a better heuristic. Greedy search minimises h , the estimated cost *remaining*. It never consults g , the cost already *incurred*. So it cannot trade a more expensive first step against a cheaper remainder — which is precisely what investing in a test harness is.

Any decision of the form “spend more now to spend less later” is invisible to greedy search. That covers tooling, automation, refactoring and nearly every technical-debt decision in software delivery.

5.4 A* Search

Add the two terms:

$$f(n) = g(n) + h(n)$$

where $g(n)$ is the cost of the path from the start to n and $h(n)$ the estimate from n to a goal. Expand the node of lowest f . That is A^* , and it is the single most useful algorithm in this book. Read $f(n)$ as *the estimated cost of the cheapest complete solution that goes through n* . Uniform-cost search is A^* with $h = 0$, and greedy search is the same frontier with $g = 0$. Figure 5.2 shows the two routes competing on f .

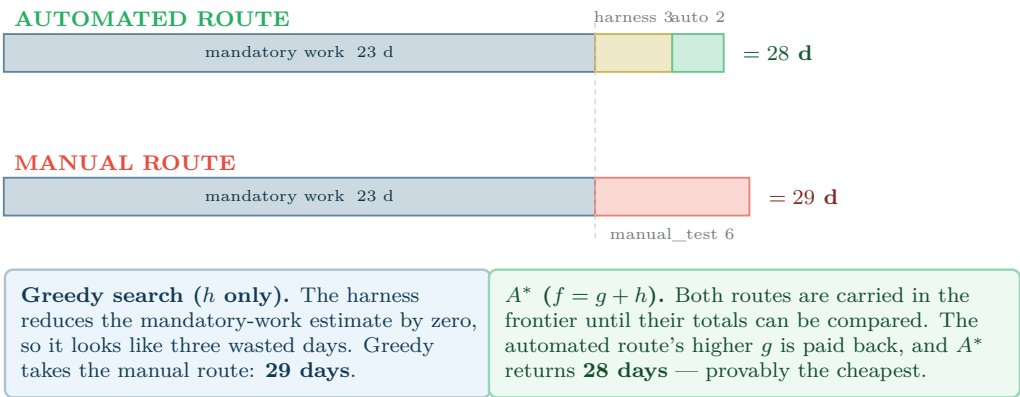


Figure 5.2: The two routes as cost. Only an algorithm that adds the cost already spent to the cost still expected can see that three days of harness buys four days of testing.

5.4.1 Why A* Is Optimal

Worked Example: The optimality argument, and where it breaks

Claim. With an admissible h , the first goal node A^* pops is on an optimal path.

Proof. Let G be the goal node popped, and suppose for contradiction that its cost $g(G)$ exceeds the optimal cost C^* . Since $h(G) = 0$ for a goal,

$$f(G) = g(G) > C^*.$$

Now consider any optimal solution path. At the moment G was popped, some node n on that optimal path was still in the frontier (the start is on it, and the path has not been fully expanded, so an unexpanded node on it exists). For that n , admissibility gives

$$f(n) = g(n) + h(n) \leq g(n) + h^*(n) = C^*.$$

So $f(n) \leq C^* < f(G)$, and n had a strictly smaller f than G . But A^* pops the smallest f in the frontier, so it would have popped n , not G . Contradiction, and therefore $g(G) = C^*$. ■

The step that fails when h overestimates. It is the inequality $h(n) \leq h^*(n)$. If h overestimates at n , then $f(n)$ can exceed C^* ; the node on the optimal path is then buried behind G in the frontier, and A^* returns the worse answer without noticing. Concretely: give the harness a pessimistic estimate of 10 days remaining and A^* reproduces greedy search's 29-day plan.

Why graph search needs consistency as well. The argument above assumes a node on the optimal path is available in the frontier. With an explored set, a state can be reached first by a dearer path, be marked explored, and then be reached again more cheaply — and refused. If h is merely admissible, that discards the optimal path. Consistency, $h(n) \leq c(n, n') + h(n')$, guarantees f never decreases along a path, so the first time a state is popped it is already by its cheapest path. Both heuristics in this chapter are consistent; the lab checks the weaker property, admissibility, over all 55 reachable states.

5.4.2 Dominance

If $h_2(n) \geq h_1(n)$ for every n , and both are admissible, then h_2 **dominates** h_1 , and A^* with h_2 expands no more nodes than with h_1 . The reason is direct: A^* must expand every node with $f(n) < C^*$, and a larger h pushes more nodes over that threshold.

So among admissible heuristics, bigger is better — as long as it stays admissible and stays cheap to compute. Those are competing pressures, and the balance is a judgement call: a heuristic that is perfect but takes as long to evaluate as solving the subproblem has bought nothing.

Now the measurement, which is more interesting than the tidy version of this story. At the root, mandatory work is worth 23 and the critical path 16, so it is tempting to conclude that mandatory work dominates. It does not. The lab checks all 55 reachable states and finds 6 on which the critical path is the *larger* of the two — states late in the sprint where almost all the mandatory work is finished but the testing chain still has to be walked. For instance with `spec`, `schema`, `api`, `auth`, `docs` and `ui` done, mandatory work is 4 while the critical path is 9.

So *neither heuristic dominates the other*, which is the ordinary situation and not a defect. It is also exactly the situation in which the maximum earns its keep: $\max(h_1, h_2)$ is admissible, dominates both by construction, and costs one extra comparison.



Dominance is a claim about *every* state, and the root is a sample of size one. Two heuristics almost always cross somewhere, because they are usually relaxations of different constraints and each is strongest where the constraint it kept is the binding one.

The practical consequence: do not choose between admissible heuristics, take their maximum.

The only reason not to is evaluation cost, and a comparison is free.

5.5 Inventing a Heuristic

The standard technique is **relaxation**: delete a constraint from the problem, solve the easier problem exactly, and use that as the estimate. It is admissible automatically, because any solution to the real problem is also a solution to the relaxed one, so the relaxed optimum cannot be larger.

Both heuristics here are relaxations:

- *Mandatory work* relaxes away the testing requirement entirely. The relaxed problem — do the eight unavoidable tasks — is solved by adding up durations.
- *Critical path* relaxes away the fact that only one task happens at a time, i.e. it assumes unlimited parallelism. The relaxed problem is the longest path in a directed acyclic graph.

And as Section 5.4.2 argued, their maximum is admissible too and dominates both. Taking the maximum of several admissible heuristics is always safe and is the cheapest route to a stronger one.

Algorithm	Cost	Optimal?	Expanded	Route chosen
Uniform-cost ($h = 0$)	28	Yes	46	automated
A^* , critical path	28	Yes	35	automated
A^* , mandatory work	28	Yes	33	automated
A^* , max of the two	28	Yes	33	automated
Greedy, critical path	28	Not guaranteed	11	automated
Greedy, mandatory work	29	No	10	manual

Table 5.1: Measured on the network of Figure 5.1. A^* keeps uniform-cost search's guarantee while doing 28% less work; greedy search does a quarter of the work and, with one of the two heuristics, returns the wrong plan.

The fourth row is worth a remark. Taking the maximum guarantees no *more* expansions, and here it delivers no fewer either — 33 both ways. The six states where the critical path is the stronger estimate are simply not states that A^* had to decide about. That is the normal return on a dominance improvement: a guarantee that you will not do worse, and an empirical gain that depends on the instance.

Read the last two rows together. Greedy search with the critical-path heuristic happens to find the optimal plan — and that is luck, not a property. Nothing in the algorithm tells you which of the two rows you are in on a problem whose answer you do not already know. That asymmetry is the whole case for A^* .

! Common Pitfalls

- **Forgetting the g term.** Ordering the frontier by h alone is greedy search, and it cannot recognise an investment. If your search refuses to build tooling, this is why.
- **An inadmissible heuristic.** Anything “tuned” upward to speed the search up — multiplying a lower bound by 1.5, say — forfeits optimality. That may be a reasonable trade, but make it deliberately and say so.
- **Assuming admissibility is enough for graph search.** With an explored set you want consistency. Otherwise a state reached cheaply on the second visit is silently discarded.
- **A heuristic that costs more than it saves.** Expanding 30% fewer nodes is a loss if evaluating h takes twice as long. Measure time as well as expansions.
- **$h > 0$ at a goal state.** Breaks the optimality proof at its first line. Always check $h(\text{goal}) = 0$.
- **Averaging two admissible heuristics.** The average is admissible but is dominated by the maximum. Take max, never the mean.

🔧 Try It Yourself: A* Over Sprint States, Two Heuristics

Uniform-cost, A* and greedy search on the network of Figure 5.1, with both heuristics, plus an exhaustive admissibility check over every reachable state.

```

"""Chapter 5 lab -- informed search on the sprint network.

Reproduces Table 5.1. The headline result is the last row: greedy
search with the mandatory-work heuristic returns a 29-day plan when a
28-day plan exists, because a test harness reduces its estimate by
nothing at all.
"""

import heapq
import itertools
from collections import deque

# duration, then a conjunction of disjunctions: a task is ready when
# every group has at least one member already finished.
TASKS = {
    "spec": (3, []),
    "schema": (2, [{"spec"}]),
    "api": (5, [{"schema"}]),
    "ui": (4, [{"spec"}]),
    "auth": (3, [{"schema"}]),
    "harness": (3, [{"schema"}]),
    "auto_test": (2, [{"api"}, {"auth"}, {"harness"}]),
    "manual_test": (6, [{"api"}, {"auth"}]),
    "docs": (2, [{"api"}]),
    "uat": (3, [{"ui"}, {"auto_test"}, {"manual_test"}]),
    "release": (1, [{"uat"}, {"docs"}]),
}
GOAL = "release"

# The eight tasks that appear in every plan. Everything except the two
# testing routes and the harness that one of them needs.
MANDATORY = ("spec", "schema", "api", "auth", "ui", "docs", "uat",
             "release")

def dur(t):
    return TASKS[t][0]

```

```

def ready(state):
    """Legal actions: the successor function, as a generator."""
    for t, (_, groups) in TASKS.items():
        if t not in state and all(g & state for g in groups):
            yield t

# --- the two heuristics, both relaxations -----
def h_zero(state):
    return 0

def h_mandatory(state):
    """Relax away the testing requirement: just add up the work that
    cannot be avoided. 23 days at the root."""
    return sum(dur(t) for t in MANDATORY if t not in state)

def h_chain(state):
    """Relax away the one-task-at-a-time constraint: the longest
    remaining dependency chain. 16 days at the root."""
    memo = {}

    def chain(t):
        if t in state:
            return 0
        if t in memo:
            return memo[t]
        need = 0
        for g in TASKS[t][1]:
            need = max(need, min(chain(x) for x in g))
        memo[t] = need + dur(t)
        return memo[t]

    return chain(GOAL)

def h_max(state):
    """Admissible, and dominates both by construction."""
    return max(h_mandatory(state), h_chain(state))

# --- best-first search; the priority decides the algorithm -----
def best_first(h, use_g=True):
    """use_g=True -> A*      (f = g + h)
       use_g=False -> greedy (f = h)

    The heap entry is (priority, counter, g, state, plan). The counter
    sits immediately after the priority so that ties are broken by
    insertion order alone. Putting g there instead would quietly let
    the cost leak into greedy search's ordering -- and greedy search
    would then find the optimal plan and the point of the lab would
    be lost.
    """
    start = frozenset()
    tie = itertools.count()
    frontier = [(h(start), next(tie), 0, start, ())]
    best = {start: 0}
    expanded = 0
    while frontier:
        _, _, g, state, plan = heapq.heappop(frontier)

```

```

    if GOAL in state:
        return g, list(plan), expanded
    if use_g and g > best.get(state, 1 << 30):
        continue # a cheaper path already went
    expanded += 1
    for t in ready(state):
        nxt, ng = state | {t}, g + dur(t)
        if use_g:
            if ng >= best.get(nxt, 1 << 30):
                continue
            best[nxt] = ng
            pri = ng + h(nxt)
        else:
            if nxt in best:
                continue
            best[nxt] = ng
            pri = h(nxt)
        heapq.heappush(frontier,
                       (pri, next(tie), ng, nxt, plan + (t,)))
    return None, None, expanded

def reachable():
    seen, q = {frozenset()}, deque([frozenset()])
    while q:
        s = q.popleft()
        for t in ready(s):
            n = s | {t}
            if n not in seen:
                seen.add(n)
                q.append(n)
    return seen

def cheapest_from(state):
    """True remaining cost h*, by uniform-cost search from state."""
    tie = itertools.count()
    fr = [(0, next(tie), state)]
    best = {state: 0}
    while fr:
        g, _, st = heapq.heappop(fr)
        if GOAL in st:
            return g
        if g > best.get(st, 1 << 30):
            continue
        for t in ready(st):
            n, ng = st | {t}, g + dur(t)
            if ng < best.get(n, 1 << 30):
                best[n] = ng
                heapq.heappush(fr, (ng, next(tie), n))
    return None

def route(plan):
    return "automated" if "auto_test" in plan else "manual"

if __name__ == "__main__":
    print("h at the root: zero=%d chain=%d mandatory=%d"
          % (h_zero(frozenset()), h_chain(frozenset()),
             h_mandatory(frozenset())))
    print()

```

```

print("algorithm          cost  expanded  route")
print("-" * 52)
runs = [
    ("uniform-cost (h=0)  ", h_zero, True),
    ("A* critical path    ", h_chain, True),
    ("A* mandatory work    ", h_mandatory, True),
    ("A* max of the two    ", h_max, True),
    ("greedy critical path ", h_chain, False),
    ("greedy mandatory work", h_mandatory, False),
]
results = {}
for name, h, use_g in runs:
    c, plan, exp = best_first(h, use_g)
    results[name.strip()] = (c, plan, exp)
    print("%s %5d %9d %s" % (name, c, exp, route(plan)))

opt = results["uniform-cost (h=0)"][0]
print()
print("optimal cost:", opt)

# 1. A* is optimal with every heuristic here
astars = ("A* critical path", "A* mandatory work",
          "A* max of the two")
for n in astars:
    assert results[n][0] == opt, n
# 2. and expands strictly fewer nodes than uniform-cost search
ucs_exp = results["uniform-cost (h=0)"][2]
for n in astars:
    assert results[n][2] < ucs_exp, n

# 3. NEITHER heuristic dominates the other -- the interesting part
states = reachable()
mand_bigger = [s for s in states if h_mandatory(s) > h_chain(s)]
chain_bigger = [s for s in states if h_chain(s) > h_mandatory(s)]
print()
print("reachable states: %d" % len(states))
print("  mandatory work is the larger estimate on %d"
      % len(mand_bigger))
print("  critical path  is the larger estimate on %d"
      % len(chain_bigger))
assert mand_bigger and chain_bigger, "expected them to cross"
# so max dominates both, by construction
assert all(h_max(s) >= h_mandatory(s) for s in states)
assert all(h_max(s) >= h_chain(s) for s in states)
assert results["A* max of the two"][2] <= min(
    results["A* critical path"][2],
    results["A* mandatory work"][2])
ex = min(chain_bigger, key=len)
print("  e.g. %s: mandatory=%d, critical path=%d"
      % (" ".join(sorted(ex)), h_mandatory(ex), h_chain(ex)))

# 4. greedy with the mandatory heuristic is NOT optimal
assert results["greedy mandatory work"][0] > opt
print()
print("greedy with the mandatory heuristic returns %d, not %d"
      % (results["greedy mandatory work"][0], opt))

# 5. all three heuristics are admissible on every reachable state
bad = []
for s in states:
    if GOAL in s:
        continue

```

```

true = cheapest_from(s)
if true is None:
    continue
for nm, h in (("mandatory", h_mandatory), ("chain", h_chain),
              ("max", h_max)):
    if h(s) > true:
        bad.append((nm, sorted(s), h(s), true))
print("admissibility violations:", len(bad))
assert not bad
print()
print("A* kept the guarantee and did %d%% less work than"
      % round(100 * (ucs_exp - results["A* mandatory work"][2])
              / ucs_exp))
print("uniform-cost search. Greedy did a quarter of the work and")
print("got the answer wrong.")

```

The admissibility check is the part worth keeping. It is easy to invent a heuristic that is plausible, fast and wrong, and the symptom — a sub-optimal answer on problems whose optimum you do not know — is invisible in production. Checking $h(s) \leq h^*(s)$ exhaustively is only possible on a small instance, which is exactly why you should do it while the instance is still small.

🔍 Checkpoint — Test Yourself

1. The mandatory-work heuristic values the root at 23 and the true optimum is 28. Is it admissible? Would a heuristic returning 30 at the root be admissible, and what would A^* do with it?
2. Explain in one sentence why greedy best-first search will never choose to build a test harness, however good the harness is.
3. You have two admissible heuristics, h_1 and h_2 , neither dominating the other. Give a third admissible heuristic that dominates both, and say why the average of the two would be a worse choice.

📋 Chapter Summary

- A **heuristic** $h(n)$ estimates the remaining cost. It is **admissible** if it never overestimates and **consistent** if $h(n) \leq c(n, n') + h(n')$; consistency implies admissibility and is what graph search with an explored set needs.
- **Greedy best-first search** orders the frontier by h alone. It is fast and not optimal, and its structural blind spot is any investment — it cannot trade a dearer step now against a cheaper remainder.
- A^* orders by $f(n) = g(n) + h(n)$ and returns an optimal solution whenever h is admissible. Uniform-cost search is A^* with $h = 0$.
- The optimality proof turns on $h(n) \leq h^*(n)$: an overestimate buries the optimal path behind a worse goal node, and A^* returns the worse answer without any sign that it has done so.
- Among admissible heuristics, larger is better — $h_2 \geq h_1$ everywhere means no more expansions — provided it stays cheap to compute. The maximum of several admissible heuristics is admissible and dominates each.
- **Relaxation** is the way to invent one: drop a constraint, solve the easier problem exactly. Dropping the testing requirement gives mandatory work; dropping the one-task-at-a-time rule gives the critical path.

- Measured on the sprint: A^* matched uniform-cost search's 28-day optimum while expanding 33 states instead of 46; greedy search expanded 10 and returned a 29-day plan.

Review Questions

1. (MCQ) A^* orders its frontier by: (a) $h(n)$ (b) $g(n)$ (c) $g(n) + h(n)$ (d) depth.
2. (MCQ) A heuristic is admissible if it: (a) never underestimates (b) never overestimates (c) is always zero at the root (d) is consistent.
3. (MCQ) Uniform-cost search is A^* with: (a) $h = 0$ (b) $g = 0$ (c) $h = h^*$ (d) a depth limit.
4. (MCQ) Given admissible h_1 and h_2 , which is guaranteed admissible and at least as strong as both? (a) $h_1 + h_2$ (b) $(h_1 + h_2)/2$ (c) $\max(h_1, h_2)$ (d) $\min(h_1, h_2)$.
5. (Short) State the step of the A^* optimality proof that fails when h overestimates, and describe the observable symptom.
6. (Short) Why does graph search with an explored set want consistency rather than mere admissibility?
7. (Short) Explain relaxation as a technique, and use it to invent an admissible heuristic for the problem of assigning nine developers to four workstreams.
8. (Applied) Suppose the harness took 6 days instead of 3. By hand, compute the cost of both routes, say which is now optimal, and predict whether greedy search with the mandatory-work heuristic would now be *right* by accident. Then state what that tells you about relying on greedy search.

Local Search and Optimisation

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part II — Problem Solving by Search

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Recognise** the problems for which local search is appropriate — those where the goal is a good final state and the path to it is of no interest.
- **Implement** steepest-ascent hill climbing, and **diagnose** the three landscape features that defeat it: local maxima, plateaus and ridges.
- **Explain** simulated annealing's acceptance rule, and **say** what the temperature schedule controls at each end.
- **Describe** a genetic algorithm's four operators, and **identify** the assumption about the problem that crossover relies on.
- **Compare** local search methods at an *equal evaluation budget*, and **explain** why any other comparison is meaningless.

🔗 Before You Start

Chapter 4 and Chapter 5 (the systematic searches this chapter abandons, and the objective-function thinking it keeps). Chapter 3 for representing a state as a tuple. This chapter needs no explored set and no frontier, and that is the point.

🔑 Key Terms in This Chapter

local search • objective function • state-space landscape • global maximum • local maximum • plateau • shoulder • ridge • hill climbing • steepest ascent • sideways move • random-restart hill climbing • simulated annealing • temperature schedule • local beam search • genetic algorithm • selection • crossover • mutation • evaluation budget

6.1 When the Path Does Not Matter

Every search so far returned a *path*: do this, then this, then this. For the release plan that was exactly right — the order is the deliverable.

Now a different question. Nine developers must be assigned to four workstreams. Each developer is better at some streams than others, and each stream wants a particular headcount. There is no sequence here; there is only an assignment, and it is either good or bad. Nobody cares in what order you decided it.

For problems of that shape, keeping a frontier and an explored set is pure waste. **Local search** keeps a single current state and tries to improve it, using no memory of how it got there. It gives up completeness and optimality, and in exchange it handles spaces that systematic search cannot approach at all.

🔧 The staffing problem

Nine developers, four streams (`api`, `ui`, `data`, `infra`). Each developer has a skill from 1 to 5 on each stream. The target headcount is (3, 2, 2, 2).

A state is a tuple of nine stream indices, e.g. (0, 0, 1, 2, 3, 2, 1, 3, 0). The objective to maximise is

$$\text{score}(s) = \underbrace{\sum_i \text{skill}(d_i, s_i)}_{\text{skill match}} - 4 \sum_j |\text{count}_j(s) - \text{target}_j|$$

The space has $4^9 = 262,144$ states, small enough that the lab can find the true optimum by exhaustive enumeration — it is **42** — and then check how close each method gets. That is the only reason the problem is this small. Twelve developers and six streams would be $6^{12} \approx 2.2$ billion.

6.2 The State-Space Landscape

Picture the objective as a surface over the space of states: every state is a point on the floor and its score is the height. Local search is a climber on that surface who can see only the immediately adjacent ground. Figure 6.1 names the features that decide the outcome.

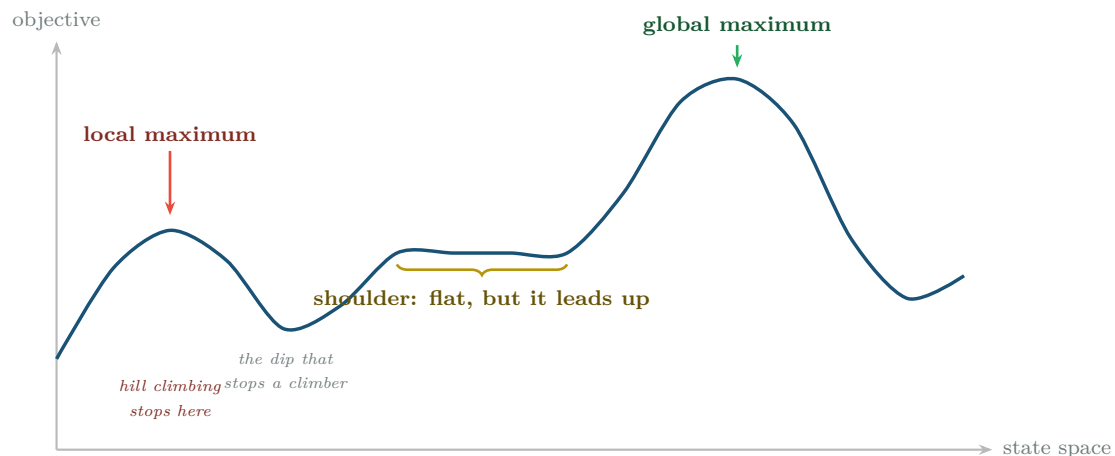


Figure 6.1: The state-space landscape. Local search sees only the ground next to it. A *local maximum* has no better neighbour but is not the best state; a *plateau* is flat, so there is no gradient to follow, and a *shoulder* is a plateau that does lead upward — which is why forbidding flat moves loses solutions and allowing them without limit loops forever. A *ridge* is a run of local maxima along which no single move goes up, though a combination of two would.

6.3 Hill Climbing

Look at every neighbour, move to the best one, stop when no neighbour is better. That is **steepest-ascent hill climbing**, and it is about eight lines long.

It is also, on this problem, close to useless: from a random start it reaches the optimum of 42 in roughly one run in two hundred, and its average final score is 30.3. The landscape is full of local maxima because the headcount penalty and the skill match pull against each other — moving a developer to the stream she is best at often unbalances two streams at once, and no *single* move repairs it.

That is a ridge, and it is the characteristic failure. Hill climbing is not a bad implementation of a good idea; it is a correct implementation of an idea that does not work unaided.

⚠ The plateau dilemma

If hill climbing stops when no neighbour is *strictly* better, it halts on every shoulder — flat ground that would have led upward. If it allows sideways moves onto equal-scoring neighbours, it can circle a genuine plateau forever.

The standard compromise is to allow sideways moves up to a fixed limit (a hundred, say) and then stop. It is a hack, it works, and it is worth knowing that this is what the literature does rather than assuming there is something more elegant.

6.4 Three Ways Out

6.4.1 Random-restart hill climbing

Run hill climbing from a random start; keep the best result; repeat until the budget is gone. It is three extra lines, it has no parameters, it is trivially parallel, and it is a far stronger baseline than its reputation suggests — on this problem it lifts the mean from 30.3 to 39.5 and finds the optimum in 37 runs out of 200.

Remember that figure while reading the rest of the chapter. Any more sophisticated method must beat random restart *at the same budget*, or it is not earning its complexity.

6.4.2 Simulated annealing

Instead of always climbing, pick a random neighbour and accept it if it is an improvement — but if it is worse, accept it anyway with probability

$$p = \exp\left(\frac{\Delta}{T}\right), \quad \Delta = \text{score}(\text{new}) - \text{score}(\text{current}) < 0,$$

where T is a **temperature** that starts high and is lowered to zero on a schedule. The name is from metallurgy: cool a metal slowly and its atoms settle into a low-energy crystal; cool it fast and they freeze into whatever disordered state they happened to be in.

Read the formula at its two extremes. When T is large, Δ/T is near zero and p is near 1 — almost anything is accepted, and the search wanders freely, ignoring the landscape. When T approaches zero, p approaches zero for any negative Δ , and the method becomes hill climbing. The schedule therefore interpolates between a random walk and a greedy climb, and the art is all in how fast. Figure 6.2 plots the acceptance probability.

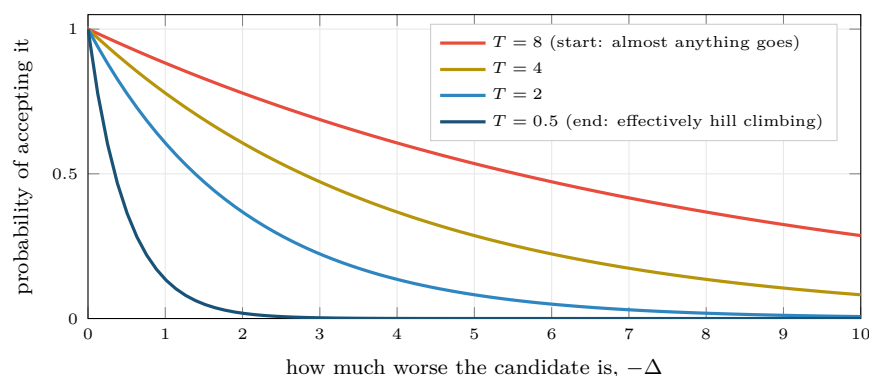


Figure 6.2: Simulated annealing’s acceptance probability $\exp(\Delta/T)$. High temperature accepts almost any move; as T falls the method converges on hill climbing. Note that an *improving* move is always accepted, at every temperature.

There is a theoretical result worth knowing and not over-trusting: if the temperature is lowered slowly enough, simulated annealing finds the global optimum with probability approaching one.

“Slowly enough” is slower than enumerating the space, so the guarantee is of no operational use. What matters in practice is that the method is robust, needs one parameter, and does well.

6.4.3 Local beam search

Keep k states rather than one; generate all their successors; keep the best k . This is not the same as k parallel hill climbs, because the k slots are reallocated globally — useful information passes between the threads, and the population tends to concentrate in the promising region. With randomised selection of which successors to keep, it is **stochastic beam search**, and it is the direct ancestor of the next idea.

6.5 Genetic Algorithms

A genetic algorithm is stochastic beam search with one extra operator: states are strings, and new states are made by *combining two parents* rather than mutating one. Four operators:

1. **Selection** — choose parents with a bias toward high scores. The lab uses tournament selection: sample three at random, keep the best.
2. **Crossover** — cut both parents at the same random point and splice. With nine developers, cut after developer 4 and take the first four assignments from one parent and the last five from the other.
3. **Mutation** — with small probability, reassign a developer at random. This is what stops the population collapsing to one state.
4. **Elitism** — carry the best few unchanged into the next generation, so a good solution is never lost.



Crossover is the whole claim of a genetic algorithm, and it rests on an assumption: that the string decomposes into *blocks that are independently good*. Splicing the first four developers' assignments from one parent onto the last five from another helps only if those are meaningful units that can be judged apart from each other.

On the staffing problem the assumption holds reasonably well, and it is worth seeing why. Most of a developer's contribution is her own skill score, which depends on her assignment alone — that part decomposes perfectly. The headcount penalty is the only coupling, and it is a mild global correction rather than a pairwise tangle. So inherited blocks mostly keep their value, and the measurement below shows the genetic algorithm winning clearly.

That is a happy outcome and not a general law. On a problem where the objective depends on *pairs* of positions — who is rostered alongside whom, which two services must not share a host — crossover routinely destroys exactly the structure it inherits, and the same code performs badly. Crossover is a claim about your problem. Check it; do not assume it.

6.6 Comparing Them Honestly

Almost every published comparison of these methods is worthless, for one reason: the methods are given different amounts of work. A genetic algorithm with a population of 100 over 500 generations has evaluated the objective 50,000 times; hill climbing from one start evaluates it a few hundred times. Declaring the former better is not a finding.

The only fair comparison fixes the **evaluation budget**: every method may call the objective function the same number of times. Figure 6.3 and Table 6.1 do that at 4,000 calls, over 200 independent runs with seeds 0 to 199.

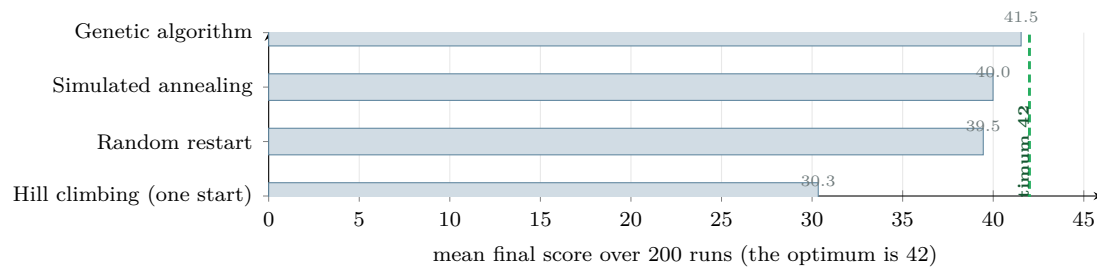


Figure 6.3: Mean final score at an equal budget of 4,000 objective evaluations per run. Hill climbing alone is far behind; the three escape strategies are close together, with the genetic algorithm ahead.

Method	Found optimum	Mean score	Worst run	Parameters
Hill climbing, one start	1 / 200	30.34	22	none
Random-restart hill climbing	37 / 200	39.45	36	none
Simulated annealing	51 / 200	39.99	37	initial temperature
Genetic algorithm	160 / 200	41.54	37	population, mutation rate, tournament size

Table 6.1: Local search on the staffing problem at 4,000 evaluations per run, 200 runs, seeds 0–199. The exhaustive optimum is 42.

Four things to take from the table.

Escaping local maxima is what matters, and almost any escape works. The gap from hill climbing to the *weakest* escape strategy is 9.1 points of mean score; the gap from the weakest escape to the best is 2.1. Nearly all the available improvement comes from not being a pure hill climber, and very little from which alternative you pick.

Random restart is a serious baseline. No parameters, three lines, perfect parallelism, and it comes within 2.1 points of the winner. Measure it before reaching for anything more interesting; if your sophisticated method cannot beat it, you have learned something useful cheaply.

The genetic algorithm wins here, on a problem whose structure suits it. It reached the optimum in four runs out of five. Section 6.6.1 shows how easily that result can be reversed by an apparently innocent change to how the budget is counted — which is the real lesson of this section.

None of them is reliable on a single run. Even the winner missed the optimum in 40 of 200 runs. If you need *the* optimum rather than a good answer, local search is the wrong tool and Chapter 7 is the right one.

6.6.1 How you count an evaluation can reverse the ranking

⚠ The same four methods, two defensible accountings

A genetic algorithm evaluates fitness in two places: once per individual per generation, and again inside each tournament when it compares candidate parents. The standard — and correct — practice is to evaluate each individual *once* per generation and cache the value for every selection decision, so a generation costs exactly `pop` evaluations. That is what Table 6.1 does, and the genetic algorithm comes first with a mean of 41.54.

Charge each tournament comparison as a fresh evaluation instead — two tournaments of three per child, so roughly six times as many — and the same code, same seeds, same budget, produces a mean of 38.25 and 29/200 optima. *The genetic algorithm moves from first place to last.* Nothing about the algorithm changed; only the bookkeeping did.

Which accounting is right? Caching is right, because in any real application the objective is the expensive part and no competent implementation would recompute a fitness it already holds. But notice how little the question announces itself, and how completely the answer changes the published conclusion. When you read a comparison of metaheuristics, the first question is not which method won — it is what the authors counted.

📊 Worked Example: Why one developer cannot be moved to fix a ridge

Take the state in which every developer is assigned to the stream she is individually best at. Reading the skill table: Ayesha → `api` (5), Bilal → `api` (4), Chen → `ui` (5), Dina → `data` (5), Emre → `infra` (5), Farah → `data` (5), Gul → `ui` (5), Hassan → `infra` (5), Iqra → tied at 3, say `api`.

Skill match: $5 + 4 + 5 + 5 + 5 + 5 + 5 + 5 + 3 = 42$. **Headcount:** `api` 3, `ui` 2, `data` 2, `infra` 2 — exactly the target (3, 2, 2, 2), so the imbalance penalty is zero. **Score:** $42 - 0 = 42$. This is the global optimum, and the lab confirms it by enumeration.

Now start one move away from it — swap Iqra to `ui`. Headcount becomes (2, 3, 2, 2): two streams are off target by one each, so the imbalance is 2 and the penalty is 8. Skill match becomes $42 - 3 + 3 = 42$. Score 34.

The trap. From that state, consider every single move. Moving Iqra back to `api` recovers 42 — so this particular state is *not* a local maximum, and hill climbing escapes it. The genuine traps are states where two developers are each in the wrong stream in a way that balances the headcount. Move either one alone and the headcount goes off target, costing 8, while the skill gain is at most 4. So both single moves go *down*, and only the simultaneous pair goes up.

That is a ridge, and it is why hill climbing reaches 42 only once in two hundred starts. It also explains why simulated annealing helps: accepting the first of the two moves, at a cost of 4, is exactly the kind of temporary worsening that a positive temperature permits and a hill climber forbids.

⚠ Common Pitfalls

- **Comparing methods at unequal budgets.** The most common error in this area, and it always flatters the method with the largest population. Fix the number of objective evaluations.
- **Using local search when you need a guarantee.** It gives a good state, never a proof. “No engineer is double-booked” is not something to establish by annealing — use Chapter 7.
- **Allowing unlimited sideways moves.** An infinite loop on a plateau. Cap them.

- **Reaching for a genetic algorithm first.** It has the most parameters and the strongest assumption — that crossover recombines meaningful blocks. It won here, but only 2.1 mean points ahead of parameterless random restart, and on a problem with pairwise structure it would lose. Establish the baseline first.
- **Not stating what an “evaluation” means in a comparison.** Charging a genetic algorithm per tournament comparison instead of per individual per generation moved it from first place to last on this problem, with no change to the code (Section 6.6.1).
- **An annealing schedule that cools too fast.** At $T \approx 0$ from the second iteration it *is* hill climbing, with extra code. Print the acceptance rate over time; if it is near zero early, the schedule is wrong.
- **Reporting the best of many runs as the method’s performance.** Report the mean and the worst as well, or you are describing your luck.

Try It Yourself: Staffing by Annealing and by Evolution

Four methods, one objective, one budget. The lab finds the true optimum by enumeration first, so every method can be scored against the truth rather than against each other.

```
"""Chapter 6 lab -- local search on the staffing problem.

Nine developers, four workstreams. Reproduces Table 6.1. Every method
gets exactly the same number of objective-function evaluations, which
is the only comparison that means anything.
"""

import itertools
import math
import random

DEVS = ("Ayesha", "Bilal", "Chen", "Dina", "Emre",
        "Farah", "Gul", "Hassan", "Iqra")
STREAMS = ("api", "ui", "data", "infra")

# skill of each developer on each stream, 1 (weak) to 5 (strong)
SKILL = {
    "Ayesha": (5, 2, 3, 1),
    "Bilal": (4, 3, 2, 2),
    "Chen": (2, 5, 1, 2),
    "Dina": (1, 4, 5, 2),
    "Emre": (3, 1, 4, 5),
    "Farah": (2, 2, 5, 3),
    "Gul": (1, 5, 2, 3),
    "Hassan": (4, 1, 2, 5),
    "Iqra": (3, 3, 3, 3),
}

TARGET = (3, 2, 2, 2)      # headcount wanted per stream
PENALTY = 4                # cost per developer away from target
NS = len(STREAMS)
BUDGET = 4000              # objective evaluations allowed per run

def raw_score(state):
    fit = sum(SKILL[DEVS[i]][s] for i, s in enumerate(state))
    counts = [0] * NS
    for s in state:
        counts[s] += 1
    off = sum(abs(counts[j] - TARGET[j]) for j in range(NS))
    return fit - PENALTY * off
```

```

class Budget:
    """Counts objective evaluations, so no method can cheat."""

    def __init__(self, cap):
        self.cap, self.n = cap, 0

    def score(self, state):
        self.n += 1
        return raw_score(state)

    def left(self):
        return self.n < self.cap

def neighbours(state):
    for i in range(len(state)):
        for s in range(NS):
            if s != state[i]:
                yield state[:i] + (s,) + state[i + 1:]

def random_state(rng):
    return tuple(rng.randrange(NS) for _ in DEVS)

def one_neighbour(state, rng):
    i = rng.randrange(len(state))
    s = rng.randrange(NS)
    while s == state[i]:
        s = rng.randrange(NS)
    return state[:i] + (s,) + state[i + 1:]

# --- 1. steepest-ascent hill climbing -----
def hill_climb(b, rng, start=None):
    st = start if start is not None else random_state(rng)
    while b.left():
        cur = b.score(st)
        best, bestv = None, cur
        for n in neighbours(st):
            if not b.left():
                break
            v = b.score(n)
            if v > bestv:
                best, bestv = n, v
        if best is None:
            return st, cur          # no better neighbour: stuck
        st = best
    return st, raw_score(st)

# --- 2. random-restart hill climbing -----
def random_restart(b, rng):
    best, bestv = None, None
    while b.left():
        st, v = hill_climb(b, rng)
        if bestv is None or v > bestv:
            best, bestv = st, v
    return best, bestv

```

```

# --- 3. simulated annealing -----
def anneal(b, rng, t0=8.0):
    st = random_state(rng)
    cur = b.score(st)
    best, bestv = st, cur
    while b.left():
        T = t0 * (1 - b.n / b.cap) + 1e-9      # linear cooling
        cand = one_neighbour(st, rng)
        v = b.score(cand)
        d = v - cur
        if d > 0 or rng.random() < math.exp(d / T):
            st, cur = cand, v
            if cur > bestv:
                best, bestv = st, cur
    return best, bestv

# --- 4. genetic algorithm -----
def genetic(b, rng, pop=30, mut=0.12):
    """Standard fitness accounting: every individual is evaluated once
    per generation and the value is then cached for every selection
    decision in that generation. So one generation costs exactly 'pop'
    evaluations -- no selection is ever free.

    This matters more than it looks. Charging each tournament
    comparison separately instead costs about six times as much per
    generation, and the method drops from first place to last. See the
    alertbox in Section 6.6.
    """
    P = [random_state(rng) for _ in range(pop)]
    bestx, bestv = None, None
    while b.n + pop <= b.cap:
        fit = [b.score(x) for x in P]           # pop evaluations, once
        order = sorted(range(pop), key=lambda i: -fit[i])
        if bestv is None or fit[order[0]] > bestv:
            bestv, bestx = fit[order[0]], P[order[0]]
        elite = [P[i] for i in order[: pop // 5]]
        nxt = list(elite)

        def tournament():
            """Best of three, judged on the cached fitness."""
            picked = rng.sample(range(pop), 3)
            return P[max(picked, key=lambda i: fit[i])]

        while len(nxt) < pop:
            a, c = tournament(), tournament()
            cut = rng.randrange(1, len(DEVS))
            child = a[:cut] + c[cut:]
            child = tuple(rng.randrange(NS) if rng.random() < mut else g
                          for g in child)
            nxt.append(child)
        P = nxt
    return bestx, bestv

def exhaustive():
    """4^9 = 262144 states: small enough to know the truth."""
    best, arg = None, None
    for st in itertools.product(range(NS), repeat=len(DEVS)):
        v = raw_score(st)
        if best is None or v > best:
            best, arg = v, st

```

```

    return best, arg

def show(state):
    by = {j: [] for j in range(NS)}
    for i, s in enumerate(state):
        by[s].append(DEVS[i])
    return " ".join("%s: %s" % (STREAMS[j], " ".join(by[j]))
                    for j in range(NS))

if __name__ == "__main__":
    opt, arg = exhaustive()
    print("states: %d      exhaustive optimum: %d" % (NS ** len(DEVS), opt))
    print(show(arg))
    print()
    print("budget: %d evaluations per run, 200 runs, seeds 0-199" % BUDGET)
    print()
    print("method              optimum      mean      worst")
    print("-" * 56)

    methods = (("hill climbing, one start", hill_climb),
               ("random restart", random_restart),
               ("simulated annealing", anneal),
               ("genetic algorithm", genetic))

    table = {}
    for name, fn in methods:
        hits, total, worst = 0, 0, None
        for seed in range(200):
            rng = random.Random(seed)
            _, v = fn(Budget(BUDGET), rng)
            total += v
            worst = v if worst is None else min(worst, v)
            if v == opt:
                hits += 1
        table[name.strip()] = (hits, total / 200, worst)
        print("%s %4d/200 %7.2f %6d"
              % (name, hits, total / 200, worst))

    hc = table["hill climbing, one start"]
    rr = table["random restart"]
    sa = table["simulated annealing"]
    ga = table["genetic algorithm"]
    escapes = (rr, sa, ga)

    # 1. plain hill climbing is hopeless here: the landscape is a ridge
    assert hc[0] <= 5
    # 2. every escape strategy is far better than no escape
    for m in escapes:
        assert m[1] > hc[1] + 5
    # 3. the gap to no-escape dwarfs the gap between the escapes
    spread = max(m[1] for m in escapes) - min(m[1] for m in escapes)
    gain = min(m[1] for m in escapes) - hc[1]
    assert spread < gain
    # 4. even the winner is unreliable on any single run
    assert max(m[0] for m in escapes) < 200

    print()
    print("escaping local maxima is worth %.1f points of mean score;"
          % gain)
    print("choosing the best escape is worth only %.1f more." % spread)
    print()

```

```

print("random restart has no parameters at all and reaches %.2f."
      % rr[1])
print("measure it before reaching for anything more interesting.")
print()
best = max(escapes, key=lambda m: m[1])
print("the best method still missed the optimum in %d of 200 runs."
      % (200 - best[0]))

```

Two experiments to run afterwards. Set the annealing temperature `t0` to 0.01 and confirm that it degenerates into hill climbing — the mean drops back towards 30. Then raise `PENALTY` from 4 to 20 and watch every method get worse together: a harder penalty makes the ridges steeper, and no amount of cleverness in the search compensates for a landscape that has been made hostile by the modelling.

🔍 Checkpoint — Test Yourself

1. Explain the difference between a plateau and a shoulder, and say what goes wrong if hill climbing (a) forbids sideways moves and (b) allows unlimited sideways moves.
2. Simulated annealing is offered a neighbour that is 6 points worse. Compute the acceptance probability at $T = 8$ and at $T = 0.5$, and say what the two numbers mean for the method's behaviour early and late in a run.
3. Name the assumption crossover makes about a problem, say why the staffing objective largely satisfies it, and give one change to the objective that would break it.

📋 Chapter Summary

- **Local search** keeps one current state and no path. Use it when the answer is a good final state and the route to it is of no interest — and never when you need a guarantee.
- The **state-space landscape** explains every failure: **local maxima**, **plateaus** (flat, possibly a **shoulder** that leads up) and **ridges** (no single move improves, though a combination would).
- **Hill climbing** moves to the best neighbour and stops when there is none. On the staffing problem it reaches the optimum once in 200 starts, because the skill match and the headcount penalty form a ridge.
- **Random restart** costs three lines, has no parameters, parallelises perfectly and lifted the mean from 30.3 to 39.5. It is the baseline every other method must beat.
- **Simulated annealing** accepts a worsening move with probability $\exp(\Delta/T)$, interpolating from a random walk at high T to hill climbing at $T \rightarrow 0$. One parameter, mean 40.0, 51/200 optima.
- A **genetic algorithm** adds **crossover** to stochastic beam search. Crossover assumes the state decomposes into independently good blocks; the staffing problem largely satisfies that, and the method led the table with 160/200 optima and a mean of 41.5.
- Compare methods only at an *equal evaluation budget*, say exactly what an evaluation is, and report the mean and the worst run rather than the best. Charging the genetic algorithm per tournament comparison rather than per individual per generation moves it from first place to last on this problem without touching its code.

🔧 Review Questions

1. (MCQ) Local search is appropriate when: (a) the path to the goal is the answer (b) only the final state matters (c) optimality must be proved (d) the space is small enough to enumerate.

2. **(MCQ)** A *ridge* defeats hill climbing because: (a) all neighbours are equal (b) no single move improves, though a combination would (c) the objective is undefined (d) the space is continuous.
3. **(MCQ)** In simulated annealing, as $T \rightarrow 0$ the method becomes: (a) a random walk (b) breadth-first search (c) hill climbing (d) a genetic algorithm.
4. **(MCQ)** The assumption behind crossover is that: (a) the objective is linear (b) the state decomposes into independently good blocks (c) mutation is unnecessary (d) the population is large.
5. **(Short)** Why is comparing a genetic algorithm to hill climbing without fixing the evaluation budget meaningless? Describe the comparison you would run instead.
6. **(Short)** Give the acceptance rule of simulated annealing and explain what each of its two extremes reduces to.
7. **(Short)** The genetic algorithm led Table 6.1 on the standard accounting and came last when each tournament comparison was charged separately. Say which accounting is correct and why, and what the episode implies about published comparisons of metaheuristics.
8. **(Applied)** Iqra has skill 3 on all four streams. Show that the optimal assignment puts her on `api`, by computing the score of moving her to each of the other three streams from the optimum. Then explain what her uniform skill row contributes to the difficulty of the landscape.

PART III

Constraints, Games and Plans

Three problems that are all search, once you look at them properly. A schedule that must satisfy every rule at once. A negotiation in which the other party is optimising against you. A plan whose steps must each make the next one possible. Part III takes the engine of Part II and points it at each in turn.

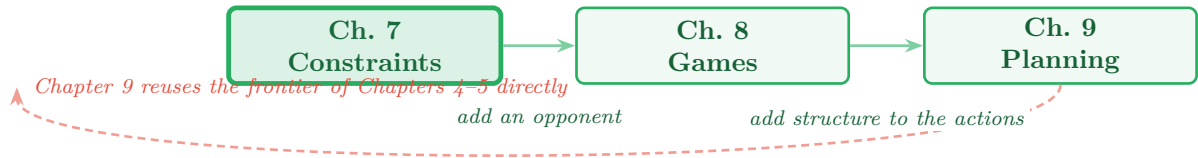


Figure P3.1: Reading path through Part III. Chapter 7 (highlighted) is the one that gives a *guarantee* — no constraint violated — which no learned model can offer.

Constraint Satisfaction

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part III — Constraints, Games and Plans

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Formulate** a problem as variables, domains and constraints, and **say** when that formulation is better than a search over states.
- **Apply** backtracking search with the minimum-remaining-values and degree heuristics, and **explain** what each one orders.
- **Propagate** constraints by forward checking and by arc consistency (AC-3), and **measure** what each removes.
- **Explain** why ordering heuristics and propagation are a package, and why either alone can be worthless.
- **Recognise** when a constraint problem is over-constrained, and **report** which constraint is responsible.

🔗 Before You Start

Chapter 3 (states as values) and Chapter 4 (backtracking is depth-first search with a commitment at each level). Chapter 5 is not needed: this chapter gets its leverage from the *structure* of the problem rather than from a heuristic estimate of distance.

📖 Key Terms in This Chapter

constraint satisfaction problem • variable • domain • constraint • constraint graph • assignment • partial assignment • consistency • backtracking • minimum remaining values • degree heuristic • least constraining value • forward checking • arc consistency • AC-3 • over-constrained

7.1 A Sprint Plan as a Constraint Problem

Twelve stories must be placed into four sprints. Each sprint has a capacity of twelve story points. Some stories cannot start until others have finished. Some pairs belong to the same developer and are too large to sit in one sprint together. Find a placement that breaks nothing.

Chapter 4 could search this: a state is a partial placement, an action adds one story. But that formulation throws away the thing that makes the problem tractable. A general search algorithm sees an opaque state and an opaque goal test. A **constraint satisfaction problem** exposes the structure — which variable each constraint touches — and the algorithms in this chapter exploit exactly that.

📖 Constraint satisfaction problem

A **CSP** is a triple:

- a set of **variables** $X_1, \dots, X_n$;
- a **domain** D_i of possible values for each variable;

- a set of **constraints**, each restricting the values that some subset of the variables may take simultaneously.

A **solution** is a complete assignment that violates no constraint. A constraint over two variables is **binary**; over more than two, n -ary.

The crucial property is that the goal test is not opaque: it is a conjunction of constraints, each of which can be checked against a *partial* assignment. That is what allows a search to fail early instead of at the leaves.

✎ The sprint problem, formulated

- *Variables*: the twelve stories S1 ... S12.
- *Domains*: {1, 2, 3, 4}, the sprint each is placed in.
- *Precedence* (binary): S1 < S4, S2 < S5, S3 < S7, S6 < S8, S9 < S12, S10 < S11, S4 < S12.
- *Same developer, too big together* (binary, “not equal”): {S1, S7}, {S2, S8}, {S4, S6}, {S9, S10}, {S7, S12}.
- *Capacity* (n -ary): the points in each sprint total at most 12. The twelve stories total 45 points against a capacity of 48, so there is almost no slack.

There are $4^{12} = 16,777,216$ assignments and exactly 14 solutions — about one in a million.

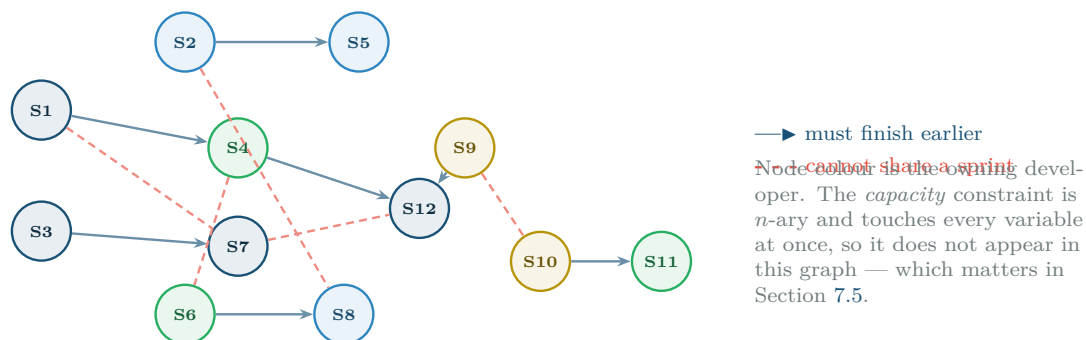


Figure 7.1: The constraint graph. Twelve variables, twelve binary constraints, and one n -ary capacity constraint that no edge can represent.

7.2 Backtracking Search

Assign one variable at a time; after each assignment check it against everything already assigned; if nothing is consistent, undo the last choice and try the next value. This is **backtracking search**, and it is depth-first search with the goal test applied incrementally.

The incremental check is the whole gain. A general search must place all twelve stories before it can test anything; backtracking discovers that S4 cannot go in sprint 1 the moment S1 is placed there, and never builds any of the 4^{10} assignments below that choice.

On the sprint problem, plain backtracking finds a solution after 123 nodes and 3,784 consistency checks — against 16,777,216 complete assignments. The structure was worth a factor of a hundred thousand before any heuristic was applied.

7.3 Which Variable Next, and Which Value

Backtracking leaves two choices open, and they are where the remaining leverage is.

Minimum remaining values (MRV). Assign next the variable with the fewest legal values left. The slogan is *fail first*: if some variable is going to run out of options, discovering it now costs one dead end, and discovering it ten levels later costs a subtree.

Degree heuristic. When several variables tie on MRV, pick the one involved in the most constraints on *unassigned* variables. It is a tie-breaker, not a strategy in its own right.

Least constraining value. Having chosen a variable, try first the value that rules out fewest options for its neighbours. Note that this pulls the opposite way from MRV — fail first on variables, succeed first on values — and both are right, because you must try every variable but only need one value to work.

⚠ MRV on its own does nothing at all

Here is a result from the lab that is worth more than the heuristics themselves. Adding MRV to plain backtracking on this problem changes the cost by *nothing*: 3,784 consistency checks and 123 nodes, both with it and without.

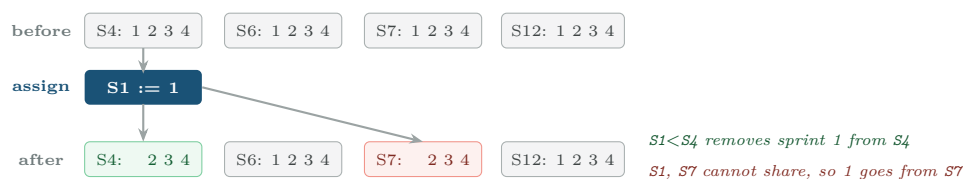
The reason is not that MRV is a bad idea. It is that MRV has no information to act on. Every domain starts at four values and *nothing ever shrinks them*, so every unassigned variable ties forever and MRV falls back on the first variable in the list — which is what plain backtracking already did.

Domains shrink only when something propagates the consequences of an assignment. So MRV and propagation are a package: neither is worth much without the other, and together they are worth a factor of nine here. If you take one thing from this chapter, take that.

7.4 Propagation: Forward Checking

Forward checking is the smallest useful propagation. When a variable is assigned, remove from every neighbour's domain the values now inconsistent with it. If any domain becomes empty, the assignment has already failed — backtrack at once rather than discovering it later.

Two things happen at the same time. The obvious one is that dead ends are found sooner. The important one is that domains now have different sizes, so MRV finally has something to compare.



And now MRV has something to say. Before the assignment every domain had four values and MRV could not choose. After it, S4 and S7 have three and the others four — so MRV picks a constrained variable, fails faster, and the two techniques begin to compound. Neither does this alone.

Figure 7.2: Forward checking after one assignment. The pruning is the visible effect; giving the ordering heuristic something to measure is the valuable one.

7.5 Propagation: Arc Consistency

Forward checking only looks one step from the variable just assigned. **Arc consistency** propagates further.

Arc consistency and AC-3

An arc $X_i \rightarrow X_j$ is **consistent** if for *every* value in D_i there is *some* value in D_j satisfying the binary constraint between them. A value in D_i with no partner can never appear in a solution and is removed.

AC-3 enforces this everywhere: keep a queue of arcs; take one, prune D_i against D_j ; if anything was removed, re-queue every arc *into* X_i , because those may now have lost partners. Stop when the queue empties, or fail if a domain empties.

Run as a preprocessing step on the sprint problem, before any search, AC-3 narrows every one of the twelve domains: stories at the head of a precedence chain lose the last sprint, stories at the tail lose the first. That is real information obtained without a single assignment.

What AC-3 cannot see, and it is the important constraint

Look again at Figure 7.1: the capacity constraint has no edge, because it is n -ary and a constraint graph draws binary constraints. AC-3 works on arcs, so it never considers capacity at all.

This matters, because capacity is the constraint that actually makes this problem hard — 45 points into 48 of capacity. Arc consistency prunes the precedence and same-developer structure beautifully and leaves the binding constraint untouched. The measurement in Table 7.1 shows the consequence: AC-3 preprocessing costs 224 extra consistency checks and removes *no* search nodes at all.

The general lesson is not that AC-3 is bad. It is that *propagation only propagates what you have expressed in a form it can read*. If the binding constraint is n -ary, you need a generalised consistency algorithm, or you must re-express the constraint — and knowing which situation you are in is the engineering judgement here.

7.6 The Measurement

All five configurations solve the same instance and find a solution. What differs is how much work it costs.

Configuration	Checks	Nodes	Comment
Plain backtracking	3,784	123	The baseline. Already 10^5 better than enumeration
+ MRV	3,784	123	No change. Nothing shrinks the domains, so MRV never has a preference
+ MRV + degree	4,333	436	Worse. The tie-break imposes an ordering that happens to be unlucky here

Configuration	Checks	Nodes	Comment
+ MRV + degree + forward checking	415	13	9× fewer checks. Propagation gives MRV information, and the two compound
+ ... + AC-3 preprocessing	639	13	Same nodes, more checks: AC-3 cannot see the capacity constraint

Table 7.1: Twelve stories into four sprints, counting every constraint check.
A solution exists (there are 14) and every configuration finds one.

Three conclusions, and the second is the one students get wrong in examinations.

Structure beats search. Before any heuristic, formulating the problem as a CSP turned 16.8 million assignments into 123 nodes.

An ordering heuristic can make things worse. The degree tie-breaker tripled the node count. Heuristics are informed guesses about a specific problem, not improvements in general, and the only way to know is to measure on your instance.

Propagation is what pays. Forward checking cut the work ninefold — and about half of that was the pruning and half was making MRV functional.

7.7 When There Is No Solution

An over-constrained problem is the common case in practice: the backlog does not fit. Backtracking then explores the whole space and reports failure, which is true and useless. A delivery manager needs to know *which* constraint to relax.

Two cheap and honest answers. Record the constraint that emptied a domain most often during propagation — that is usually the binding one. Or re-run with each constraint removed in turn and report which removals make the problem solvable. For this instance, raising the capacity from 12 to 13 turns 14 solutions into hundreds, which tells you at once that capacity, not precedence, is what binds.

Worked Example: Tracing forward checking through the first three assignments

Start with every domain $\{1, 2, 3, 4\}$. Take the variables in order.

Assign S1 := 1. Forward checking examines S1's neighbours in Figure 7.1:

- $S1 < S4$: S4 must be later than 1, so $D_{S4} = \{2, 3, 4\}$.
- $S1 \neq S7$: $D_{S7} = \{2, 3, 4\}$.

Capacity: sprint 1 now holds 3 points of 12. Fine.

MRV now chooses. S4 and S7 have three values; everything else has four. It picks one of those two — and note that this is the first moment in the entire search at which MRV has expressed a preference. Say S4.

Assign S4 := 2. Propagate:

- $S4 < S12$: $D_{S12} = \{3, 4\}$.
- $S4 \neq S6$: $D_{S6} = \{1, 3, 4\}$.

Capacity: sprint 2 holds 8 of 12; only 4 points remain there, which will matter shortly.

MRV chooses S12, now down to two values — the most constrained variable in the problem, and it was reached in two steps rather than by luck.

Assign S12 := 3. Propagate: $S9 < S12$ gives $D_{S9} = \{1, 2\}$; $S7 \neq S12$ gives $D_{S7} = \{2, 4\}$.

What has been bought. Three assignments, and four other variables have already lost values. Plain backtracking at this depth has pruned nothing and will discover the same facts one dead end at a time, hundreds of nodes later. The lab counts the difference: 13 nodes against 123.

! Common Pitfalls

- **Expecting MRV to help on its own.** Without propagation the domains never shrink, so MRV has nothing to order by. Measured here: exactly zero improvement.
- **Assuming a heuristic cannot hurt.** The degree tie-breaker tripled the node count on this instance.
- **Expecting AC-3 to enforce an n -ary constraint.** It works on arcs. Capacity is invisible to it, and capacity is what binds here.
- **Re-running AC-3 from scratch at every node.** Propagate incrementally from the variable just assigned; a full re-run usually costs more than it saves, as the last row of Table 7.1 shows even for a single preprocessing pass.
- **Reporting “no solution” and stopping.** True and useless. Report which constraint is binding.
- **Counting nodes and calling it performance.** A node with propagation costs more than a node without. Count consistency checks, or time, or both — the same mistake as Chapter 6’s evaluation budget.

🔧 Try It Yourself: Sprint Scheduling as a CSP

The instance of Figure 7.1, solved five ways, counting every constraint check. Reproduces Table 7.1.

```
"""Chapter 7 lab -- sprint scheduling as a constraint problem.

Twelve stories into four sprints under precedence, same-developer and
capacity constraints. The headline result is the second row: adding
MRV to plain backtracking changes nothing at all, because nothing has
shrunk the domains for it to compare.
"""

import itertools

# story -> (points, owner)
STORIES = {
    "S1": (3, "Ayesha"), "S2": (5, "Bilal"), "S3": (2, "Ayesha"),
    "S4": (8, "Chen"), "S5": (3, "Bilal"), "S6": (2, "Chen"),
    "S7": (5, "Ayesha"), "S8": (3, "Bilal"), "S9": (5, "Dina"),
    "S10": (3, "Dina"), "S11": (2, "Chen"), "S12": (4, "Ayesha"),
}

NAMES = list(STORIES)
SLOTS = (1, 2, 3, 4)
CAP = 12

# binary: a must finish in an earlier sprint than b
BEFORE = [("S1", "S4"), ("S2", "S5"), ("S3", "S7"), ("S6", "S8"),
          ("S9", "S12"), ("S10", "S11"), ("S4", "S12")]

# binary: same developer, too much combined work for one sprint
APART = [("S1", "S7"), ("S2", "S8"), ("S4", "S6"), ("S9", "S10"),
          ("S7", "S12")]

class Counter:
```

```

def __init__(self):
    self.checks = 0
    self.nodes = 0

def binary_ok(a, va, b, vb, c):
    """Is the pair (a=va, b=vb) consistent? One consistency check."""
    c.checks += 1
    if (a, b) in BEFORE and not va < vb:
        return False
    if (b, a) in BEFORE and not vb < va:
        return False
    if ((a, b) in APART or (b, a) in APART) and va == vb:
        return False
    return True

def capacity_ok(assign, c):
    """The n-ary constraint. No arc can express this."""
    c.checks += 1
    load = dict.fromkeys(SLOTS, 0)
    for n, v in assign.items():
        load[v] += STORIES[n][0]
    return all(load[s] <= CAP for s in SLOTS)

def consistent(assign, var, val, c):
    for other, oval in assign.items():
        if not binary_ok(var, val, other, oval, c):
            return False
    trial = dict(assign)
    trial[var] = val
    return capacity_ok(trial, c)

def select_var(assign, domains, mrv, degree):
    un = [v for v in NAMES if v not in assign]
    if not mrv:
        return un[0]
    fewest = min(len(domains[v]) for v in un)
    tied = [v for v in un if len(domains[v]) == fewest]
    if len(tied) == 1 or not degree:
        return tied[0]

    def deg(v):
        return sum(1 for (a, b) in BEFORE + APART
                    if (a == v and b not in assign)
                    or (b == v and a not in assign))

    return max(tied, key=deg)

def forward_check(var, val, domains, c):
    """Prune the neighbours. None means a domain emptied."""
    nd = {k: list(v) for k, v in domains.items()}
    nd[var] = [val]
    for other in NAMES:
        if other == var:
            continue
        keep = [x for x in nd[other] if binary_ok(var, val, other, x, c)]
        if not keep:
            return None

```

```

        nd[other] = keep
    return nd

def ac3(domains, c):
    """Arc consistency over the BINARY constraints only."""
    nd = {k: list(v) for k, v in domains.items()}
    arcs = []
    for (a, b) in BEFORE + APART:
        arcs += [(a, b), (b, a)]
    queue = list(arcs)
    while queue:
        xi, xj = queue.pop(0)
        keep = [vi for vi in nd[xi]
                 if any(binary_ok(xi, vi, xj, vj, c) for vj in nd[xj])]
        if len(keep) != len(nd[xi]):
            nd[xi] = keep
            if not nd[xi]:
                return None
            queue += [(a, b) for (a, b) in arcs if b == xi]
    return nd

def solve(mrv=False, degree=False, fc=False, use_ac3=False):
    c = Counter()
    domains = {v: list(SLOTS) for v in NAMES}
    if use_ac3:
        domains = ac3(domains, c)
        if domains is None:
            return None, c
    found = []

    def bt(assign, domains):
        c.nodes += 1
        if len(assign) == len(NAMES):
            found.append(dict(assign))
            return True
        var = select_var(assign, domains, mrv, degree)
        for val in domains[var]:
            if consistent(assign, var, val, c):
                assign[var] = val
                if fc:
                    nd = forward_check(var, val, domains, c)
                    if nd is not None and bt(assign, nd):
                        return True
                elif bt(assign, domains):
                    return True
                del assign[var]
        return False

    bt({}, domains)
    return (found[0] if found else None), c

def count_solutions():
    """Brute force over all 412 assignments, for the record."""
    c, n = Counter(), 0
    for combo in itertools.product(SLOTS, repeat=len(NAMES)):
        a = dict(zip(NAMES, combo))
        if all(a[x] < a[y] for x, y in BEFORE) \
            and all(a[x] != a[y] for x, y in APART) \
            and capacity_ok(a, c):

```

```

        n += 1
    return n

def show(sol):
    by = {s: [] for s in SLOTS}
    for n, v in sol.items():
        by[v].append(n)
    out = []
    for s in SLOTS:
        pts = sum(STORIES[n][0] for n in by[s])
        out.append("sprint %d: %-18s %2d pts"
                   % (s, ",".join(sorted(by[s], key=lambda x: int(x[1:])))
                      , pts))
    return "\n".join(out)

if __name__ == "__main__":
    total = sum(p for p, _ in STORIES.values())
    print("stories %d sprints %d assignments %d"
          % (len(NAMES), len(SLOTS), len(SLOTS) * len(NAMES)))
    print("points %d against capacity %d -- almost no slack"
          % (total, CAP * len(SLOTS)))
    print("solutions:", count_solutions())
    print()
    print("configuration                      checks  nodes")
    print("-" * 50)
    runs = [
        ("plain backtracking", dict()),
        ("+ MRV", dict(mrv=True)),
        ("+ MRV + degree", dict(mrv=True, degree=True)),
        ("+ MRV + degree + FC", dict(mrv=True, degree=True,
                                     fc=True)),
        ("+ MRV + degree + FC + AC-3", dict(mrv=True, degree=True,
                                             fc=True, use_ac3=True)),
    ]
    res = {}
    for name, kw in runs:
        sol, c = solve(**kw)
        assert sol is not None, name
        res[name.strip()] = (c.checks, c.nodes)
        print("%s %7d %6d" % (name, c.checks, c.nodes))

    plain = res["plain backtracking"]
    mrv_only = res["+ MRV"]
    fc = res["+ MRV + degree + FC"]
    ac = res["+ MRV + degree + FC + AC-3"]

    # 1. MRV alone changes NOTHING: no propagation, no information
    assert mrv_only == plain
    print()
    print("MRV alone: %d checks, exactly as without it. Nothing has"
          % mrv_only[0])
    print("shrunk a domain, so every variable ties and MRV cannot")
    print("express a preference.")

    # 2. forward checking is the win, and it is large
    assert fc[0] * 5 < plain[0]
    print()
    print("forward checking: %d -> %d checks (%.0fx), %d -> %d nodes"
          % (plain[0], fc[0], plain[0] / fc[0], plain[1], fc[1]))

```

```
# 3. AC-3 cannot see the capacity constraint, so it buys no nodes
assert ac[1] == fc[1] and ac[0] > fc[0]
print()
print("AC-3 preprocessing: %d extra checks, %d fewer nodes."
      % (ac[0] - fc[0], fc[1] - ac[1]))
print("It prunes the binary structure and never sees capacity,")
print("which is the constraint that actually binds.")

# what AC-3 removes before any search
c = Counter()
d0 = {v: list(SLOTS) for v in NAMES}
d1 = ac3(d0, c)
narrowed = [v for v in NAMES if d1[v] != d0[v]]
print()
print("AC-3 narrowed %d of %d domains before any assignment:"
      % (len(narrowed), len(NAMES)))
for v in narrowed[:4]:
    print("    %-4s %s -> %s" % (v, d0[v], d1[v]))
print("    ...")

print()
sol, _ = solve(mrv=True, degree=True, fc=True)
print(show(sol))
```

Two experiments afterwards. Raise CAP to 13 and count the solutions again — the problem stops being interesting, which is what “the binding constraint” means. Then lower it to 11 and confirm that the search reports failure after exploring the whole space; that is the over-constrained case of the last section, and it is the one a delivery manager will actually bring you.

🔍 Checkpoint — Test Yourself

1. Adding MRV to plain backtracking on this problem changed the number of consistency checks by exactly zero. Explain why, and state the one change that makes MRV worth having.
2. AC-3 narrowed every domain in the problem and removed no search nodes at all. Explain how both of those can be true at once.
3. A delivery manager reports that the backlog does not fit and asks what to do. Your solver returns “no solution”. Describe two ways to turn that into a useful answer, and say which constraint you would expect to be binding here.

📋 Chapter Summary

- A **CSP** is variables, domains and constraints. Its advantage over a general search is that the goal test is not opaque: constraints can be checked against a *partial* assignment, so the search fails early. That alone turned 16.8 million assignments into 123 nodes.
- **Backtracking search** assigns one variable at a time and undoes the last choice on failure.
- **MRV** assigns the most constrained variable next — fail first. The **degree heuristic** breaks ties; **least constraining value** orders values the other way, and both are right.
- **Forward checking** prunes neighbours’ domains after each assignment. It cut the work ninefold here — half by pruning, half by giving MRV something to measure. *Ordering and propagation are a package.*
- **Arc consistency** (AC-3) propagates further, over binary constraints only. On this problem it narrowed all twelve domains and removed no nodes, because the binding constraint is *n*-ary and invisible to it.

- A heuristic is a guess about a particular problem, not a general improvement: the degree tie-breaker tripled the node count. Measure on your instance, counting consistency checks rather than nodes.
- For an over-constrained problem, report which constraint binds. “No solution” is true and useless.

Review Questions

1. **(MCQ)** The minimum-remaining-values heuristic chooses: (a) the value ruling out fewest options (b) the variable with the fewest legal values left (c) the variable in the most constraints (d) the cheapest variable.
2. **(MCQ)** Forward checking prunes: (a) all variables (b) the neighbours of the variable just assigned (c) only the variable just assigned (d) nothing until a domain empties.
3. **(MCQ)** AC-3 operates on: (a) n -ary constraints (b) binary constraints (c) the objective function (d) the search tree.
4. **(MCQ)** An arc $X_i \rightarrow X_j$ is consistent when: (a) every value of X_i has some consistent partner in X_j (b) some value of X_i has a partner (c) $D_i = D_j$ (d) X_i is assigned.
5. **(Short)** Why does MRV need propagation to be useful? Use the measured result from this chapter in your answer.
6. **(Short)** Explain why MRV picks the most constrained variable while least-constraining-value picks the least constraining value, and why that is not a contradiction.
7. **(Short)** Give an n -ary constraint from a project you know, and say what goes wrong if you expect arc consistency to enforce it.
8. **(Applied)** Take the constraint graph of Figure 7.1. By hand, assign $S10 := 3$ and apply forward checking. Which domains change, and to what? Then say which variable MRV would select next, and whether the degree heuristic would be consulted.

Adversarial Search and Game Playing

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part III — Constraints, Games and Plans

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Model** a two-party interaction as a game tree with a utility at each leaf, and **say** what the zero-sum assumption commits you to.
- **Compute** the minimax value of a position by hand and in code.
- **Apply** alpha–beta pruning, **trace** its cutoffs, and **state** the bound it achieves under perfect move ordering.
- **Explain** why move ordering determines how much is pruned, and **judge** when the cost of ordering exceeds what it saves.
- **Cut off** search with an evaluation function, and **recognise** the horizon effect.

🔗 Before You Start

Chapter 4 (depth-first search, which is what a game tree search is) and Chapter 2 — specifically the *multi-agent* property of a task environment. This chapter is what you do when the sixth property of Table 2.2 comes out multi-agent.

📖 Key Terms in This Chapter

game tree • ply • terminal state • utility • zero-sum • minimax value • MAX and MIN • alpha–beta pruning • cutoff • move ordering • principal variation • cutoff test • evaluation function • horizon effect

8.1 When the Other Party Is Choosing Too

The delivery team and the client are agreeing a release slot. We propose, they respond, we respond, and so on until somebody accepts or the deadline forces a decision. Every option we might take is worth something to us — and what we eventually get depends on what they do next, which depends on what we do now.

Chapter 6 would treat their behaviour as part of the landscape. Chapter 7 would treat it as a constraint. Both are wrong in the same way: the other party is not weather and not a rule. They are optimising, and if we model them as noise they will exploit us systematically. That is the *multi-agent* property of Chapter 2, and this chapter is what it costs to take it seriously.

📋 Game tree and minimax value

A **game tree** has the current position at its root; the children of a node are the positions reachable by one legal move; and each **terminal** position has a **utility** for us. Levels alternate between **MAX** (our move — we choose the largest value) and **MIN** (their move — they choose the smallest). One level is a **ply**.

The **minimax value** of a node is what the position is worth under optimal play by both sides:

$$\text{minimax}(n) = \begin{cases} \text{utility}(n) & n \text{ terminal} \\ \max_s \text{minimax}(s) & n \text{ is a MAX node} \\ \min_s \text{minimax}(s) & n \text{ is a MIN node} \end{cases}$$

⚠ What MIN assumes, and when it is wrong

Minimax assumes the opponent plays the move that is worst for us — that the game is **zero-sum**, our gain being exactly their loss. A negotiation usually is not: both sides can lose by failing to agree, and both can gain from a slot that suits each.

Assuming an optimal adversary is nevertheless the right default, because it is the *safe* assumption. The minimax value is a guarantee: we can obtain at least this much whatever they do. Model them as cooperative and you get a better number and no guarantee at all. Know which one you are quoting.

8.2 Minimax by Hand

Figure 8.1 is a three-ply negotiation, small enough to compute in your head and used for the rest of this chapter. Leaves are days of delay avoided, from our point of view.

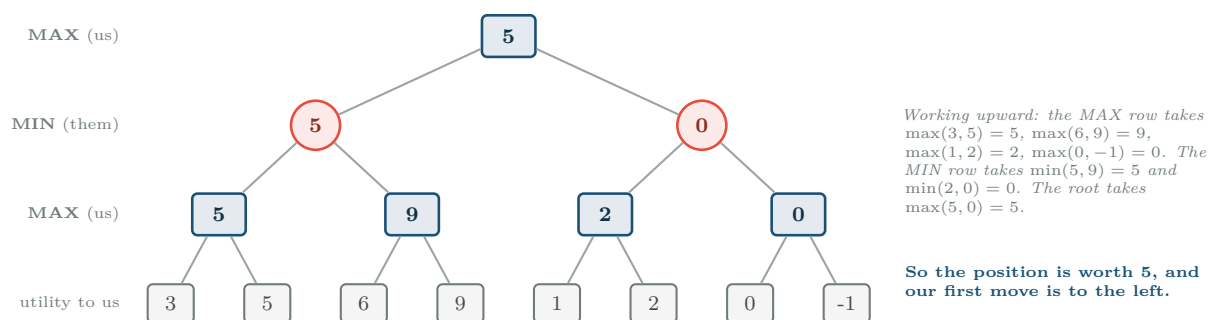


Figure 8.1: A three-ply negotiation. Squares are our choices, circles theirs. Minimax evaluates all eight leaves to establish that the position is worth 5.

8.3 Alpha–Beta Pruning

Minimax examined every leaf. Most of them could not have changed the answer.

📖 Alpha and beta

Carry two bounds down the tree:

- α — the best value MAX can already guarantee somewhere above.
- β — the best value MIN can already guarantee somewhere above.

At any node, if $\alpha \geq \beta$, stop examining the remaining children: the player above already has a better alternative and will never choose this branch, so its exact value cannot matter.

Alpha–beta pruning returns exactly the minimax value. It is not an approximation — it computes the same answer while declining to evaluate leaves that provably cannot affect it.

Figure 8.2 traces it on the same tree. Five of eight leaves are evaluated.

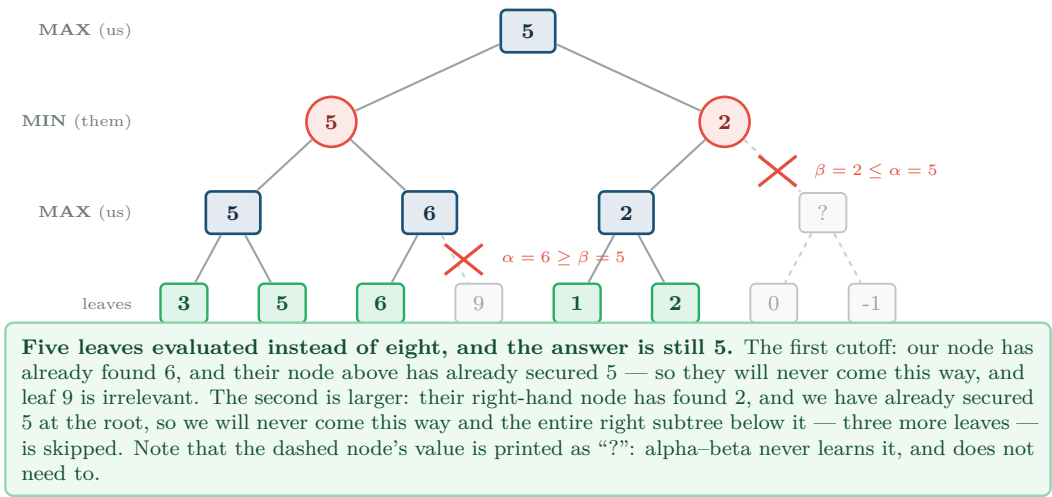


Figure 8.2: Alpha-beta on the tree of Figure 8.1. Green leaves are evaluated, grey are never looked at, and the crosses mark the two cutoffs.

8.4 Move Ordering Is Everything

Alpha-beta prunes when it has already found something good. So the order in which moves are tried decides how much is pruned: try the best move first and the bounds tighten immediately; try it last and there is nothing to prune against.

With *perfect* ordering — best move first at every node — alpha-beta evaluates

$$b^{\lceil d/2 \rceil} + b^{\lfloor d/2 \rfloor} - 1$$

leaves instead of b^d . That is roughly $b^{d/2}$: the same time buys *twice the depth*, which for a game-playing program is the whole ball game.

The chapter’s lab measures all of this on a five-ply, three-branch negotiation — 243 leaves — and the perfect-ordering bound is reached exactly: $3^3 + 3^2 - 1 = 27 + 9 - 1 = 35$.

Algorithm	Leaves seen	Saving	Note
Minimax, no pruning	243	—	Every leaf, always
Alpha-beta, worst ordering	217	1.1×	Pruning still helps a little, even at its worst
Alpha-beta, natural ordering	75	3.2×	The realistic case
Alpha-beta, perfect ordering	35	6.9×	Exactly $b^{\lceil d/2 \rceil} + b^{\lfloor d/2 \rfloor} - 1$

Table 8.1: Leaf evaluations on a five-ply, three-branch negotiation. All four return the same minimax value of 14 — pruning is exact, not approximate.

Perfect ordering requires knowing the best move, which is what the search is for. So the practical question is how good an ordering you can buy cheaply. Real engines use a shallow search first and order the deep search by its results, plus remembered “killer” moves that caused cutoffs elsewhere.

But cheaply is doing real work in that sentence. The lab measures a shallow-probe ordering on this tree and it costs *506* leaf evaluations against natural ordering’s 75 — nearly seven times worse, because the probes themselves are evaluations and the tree is not deep enough

to amortise them. Ordering pays in chess, at twenty ply with a transposition table caching the probes. At five ply it does not. Measure before you believe.

8.5 Cutting Off the Search

Real games do not fit. A negotiation with ten rounds and five options each is $5^{10} \approx 10$ million leaves, and chess is unthinkable. So replace the terminal test with a **cutoff test** — stop at a fixed depth — and replace the utility with an **evaluation function** that estimates the position's worth.

The evaluation function is where domain knowledge enters, and the search is no better than it. For the negotiation: how far the current offer is from our target, how much slack remains in the schedule, whether the remaining rounds favour us.

The horizon effect is the price. A cutoff at depth d cannot see anything at depth $d + 1$, so the search happily chooses a line that pushes an unavoidable loss just past the horizon — conceding a small point now to delay a large concession into invisibility. The standard mitigations are to search *quiescent* positions deeper (do not stop in the middle of an exchange) and to extend the search along forcing lines.

Worked Example: Tracing alpha–beta through the tree of Figure 8.1

Left to right, carrying (α, β) .

Root (MAX), $\alpha = -\infty$, $\beta = +\infty$. Descend left.

Left MIN node, $(-\infty, +\infty)$. Descend to its left child.

MAX node a, $(-\infty, +\infty)$: evaluate leaf 3, so $v = 3$, $\alpha = 3$; evaluate leaf 5, so $v = 5$, $\alpha = 5$. Return 5. (*2 leaves*)

Back at the left MIN node: $v = 5$, so $\beta = 5$. Descend to its right child with $(-\infty, 5)$.

MAX node b, $(-\infty, 5)$: evaluate leaf 6, so $v = 6$ and $\alpha = 6$. Now $\alpha \geq \beta$ ($6 \geq 5$) — **cutoff**. Leaf 9 is never evaluated. Return 6. (*1 leaf*)

Why it is safe: node b is a MAX node that can already secure 6. The MIN node above it will never accept 6 when it already has 5 available. Whatever leaf 9 holds, b can only get *better* than 6, so MIN's choice is settled.

Left MIN node returns $\min(5, 6) = 5$. Root: $v = 5$, $\alpha = 5$.

Right MIN node, $(5, +\infty)$. Descend left.

MAX node c, $(5, +\infty)$: leaf 1 gives $v = 1$; leaf 2 gives $v = 2$. Return 2. (*2 leaves*)

Right MIN node: $v = 2$, so $\beta = 2$. Now $\beta \leq \alpha$ ($2 \leq 5$) — **cutoff**. Node d and both its leaves are skipped entirely. Return 2.

Why it is safe: this MIN node can already hold us to 2, and we have already secured 5 at the root. We will never play into this branch, so its exact value is irrelevant — which is why Figure 8.2 prints it as “?”.

Root returns $\max(5, 2) = 5$. **Leaves evaluated:** 3, 5, 6, 1, 2 — **five of eight**, and the value is identical to minimax's.

Now reverse the leaves to $[-1, 0, 2, 1, 9, 6, 5, 3]$ and trace again. The minimax value is still 5, but the cutoffs fall differently — that is move ordering, and it is the entire content of Table 8.1.

Common Pitfalls

- **Thinking alpha–beta is an approximation.** It returns exactly the minimax value. What it approximates is nothing; what it saves is leaves.

- **Modelling a negotiator as a stochastic process.** If their choice depends on yours, an average-case model is not merely inaccurate, it is exploitable.
- **Quoting a cooperative-model number as a guarantee.** The minimax value is what you can secure *whatever* they do. Any friendlier assumption gives a bigger number and no floor.
- **Assuming move ordering always pays.** Measured here: a shallow-probe ordering cost 506 leaf evaluations against natural ordering's 75. It pays at depth twenty, not at depth five.
- **Stopping the search in a volatile position.** The horizon effect makes the program push bad news just out of sight. Search quiescent positions deeper.
- **Forgetting that the evaluation function is the ceiling.** Beyond a point, deeper search of a bad evaluation function just finds worse moves more confidently.

Try It Yourself: Minimax and Alpha-Beta on a Slot Negotiation

The five-ply negotiation of Table 8.1, with four move orderings, counting leaf evaluations — the operation that costs.

```

"""Chapter 8 lab -- minimax and alpha-beta on a release negotiation.

Five plies, three options each, 243 leaves. All four configurations
return the same minimax value; they differ only in how many leaves
they have to look at. The perfect-ordering row hits the theoretical
bound  $b^{\lceil d/2 \rceil} + b^{\lfloor d/2 \rfloor} - 1$  exactly.
"""

import random

DEPTH = 5          # plies of negotiation
BRANCH = 3         # hold, concede, swap
OPTIONS = ("hold", "concede", "swap")

def build(seed=7):
    """Leaf utilities to us, in days of delay avoided."""
    rng = random.Random(seed)
    leaves = {}

    def rec(path):
        if len(path) == DEPTH:
            leaves[path] = rng.randrange(-20, 21)
            return
        for m in range(BRANCH):
            rec(path + (m,))

    rec(())
    return leaves

LEAVES = build()

class Counter:
    def __init__(self):
        self.evals = 0          # leaf evaluations: the expensive op
        self.nodes = 0

    def minimax(path, maximising, c):

```

```

c.nodes += 1
if len(path) == DEPTH:
    c.evals += 1
    return LEAVES[path]
vals = [minimax(path + (m,), not maximising, c)
         for m in range(BRANCH)]
return max(vals) if maximising else min(vals)

def true_value(path, maximising):
    """An oracle, used only to build the best/worst orderings."""
    return minimax(path, maximising, Counter())

def ordered(path, maximising, how):
    ms = list(range(BRANCH))
    if how == "natural":
        return ms
    # 'best' puts the strongest move first, 'worst' puts it last
    rev = maximising if how == "best" else not maximising
    return sorted(ms, key=lambda m: true_value(path + (m,), not maximising),
                  reverse=rev)

def alphabeta(path, maximising, alpha, beta, c, how="natural"):
    c.nodes += 1
    if len(path) == DEPTH:
        c.evals += 1
        return LEAVES[path]
    if maximising:
        v = -10 ** 9
        for m in ordered(path, maximising, how):
            v = max(v, alphabeta(path + (m,), False, alpha, beta, c, how))
            alpha = max(alpha, v)
            if alpha >= beta:
                break
        return v
    v = 10 ** 9
    for m in ordered(path, maximising, how):
        v = min(v, alphabeta(path + (m,), True, alpha, beta, c, how))
        beta = min(beta, v)
        if beta <= alpha:
            break
    return v

def shallow(path, maximising, limit, c):
    """A depth-limited probe, used to ORDER a deeper search.

    Every leaf it touches is charged, because in a real engine the
    evaluation function is what costs.
    """
    if len(path) == DEPTH:
        c.evals += 1
        return LEAVES[path]
    if limit == 0:
        p = path
        while len(p) < DEPTH:
            p = p + (0,)
            c.evals += 1
        return LEAVES[p]
    vals = [shallow(path + (m,), not maximising, limit - 1, c)

```

```

        for m in range(BRANCH)]
    return max(vals) if maximising else min(vals)

def ab_probe_ordered(path, maximising, alpha, beta, c, probe=1):
    """Alpha-beta ordering its moves by a shallow probe."""
    c.nodes += 1
    if len(path) == DEPTH:
        c.evals += 1
        return LEAVES[path]
    ms = sorted(range(BRANCH),
                 key=lambda m: shallow(path + (m,), not maximising,
                                       probe, c),
                 reverse=maximising)
    if maximising:
        v = -10 ** 9
        for m in ms:
            v = max(v, ab_probe_ordered(path + (m,), False, alpha, beta,
                                       c, probe))
            alpha = max(alpha, v)
            if alpha >= beta:
                break
        return v
    v = 10 ** 9
    for m in ms:
        v = min(v, ab_probe_ordered(path + (m,), True, alpha, beta,
                                   c, probe))
        beta = min(beta, v)
        if beta <= alpha:
            break
    return v

if __name__ == "__main__":
    print("depth %d, branching %d, leaves %d"
          % (DEPTH, BRANCH, BRANCH ** DEPTH))

    c0 = Counter()
    value = minimax(), True, c0)
    print("minimax value: %d" % value)
    print()
    print("algorithm                leaf evals   nodes   value")
    print("-" * 60)
    print("%-30s %10d %7d %6d"
          % ("minimax, no pruning", c0.evals, c0.nodes, value))

    results = {"minimax": c0.evals}
    for how, label in (("worst", "alpha-beta, worst ordering "),
                      ("natural", "alpha-beta, natural ordering"),
                      ("best", "alpha-beta, perfect ordering")):
        c = Counter()
        v = alphabeta(), True, -10 ** 9, 10 ** 9, c, how)
        assert v == value, how          # pruning is EXACT
        results[how] = c.evals
        print("%-30s %10d %7d %6d" % (label, c.evals, c.nodes, v))

    # the theoretical bound, reached exactly by perfect ordering
    bound = BRANCH ** ((DEPTH + 1) // 2) + BRANCH ** (DEPTH // 2) - 1
    print()
    print("b^ceil(d/2) + b^floor(d/2) - 1 = %d^%d + %d^%d - 1 = %d"
          % (BRANCH, (DEPTH + 1) // 2, BRANCH, DEPTH // 2, bound))
    assert results["best"] == bound

```

```

print("perfect ordering evaluated %d leaves: the bound, exactly."
      % results["best"])

# pruning never loses, and ordering decides how much it wins
assert results["worst"] < results["minimax"]
assert results["best"] < results["natural"] < results["worst"]
print()
print("saving against minimax:  worst %.1fx   natural %.1fx   "
      "perfect %.1fx"
      % (results["minimax"] / results["worst"],
         results["minimax"] / results["natural"],
         results["minimax"] / results["best"]))

# does a shallow probe pay for the ordering it buys? Not here.
c = Counter()
v = ab_probe_ordered((), True, -10 ** 9, 10 ** 9, c, probe=1)
assert v == value
print()
print("ordering by a depth-1 probe: %d leaf evals against natural"
      % c.eval)
print("ordering's %d. The probes cost more than they save at this"
      % results["natural"])
print("depth. Ordering pays in chess, at twenty ply with a cache.")
assert c.eval > results["natural"]

# how much does the instance matter?
totals = []
for seed in range(20):
    globals()["LEAVES"] = build(seed)
    c = Counter()
    alphabeta((), True, -10 ** 9, 10 ** 9, c, "natural")
    totals.append(c.eval)
print()
print("across 20 different negotiations, natural ordering:")
print("  leaf evals min %d mean %.1f max %d"
      % (min(totals), sum(totals) / len(totals), max(totals)))

```

Then try the experiment from the worked example: set `DEPTH = 6` and watch both the minimax count and the perfect-ordering bound move — 729 leaves against $3^3 + 3^3 - 1 = 53$. The gap is what buys the extra depth.

🔍 Checkpoint — Test Yourself

1. Alpha-beta evaluated five leaves where minimax evaluated eight, and both returned 5. Explain in one sentence why that is not a coincidence and not an approximation.
2. In Figure 8.2 the right-hand MIN node's right child is printed as "?". State the two facts that together make its value irrelevant.
3. A colleague adds shallow-probe move ordering to your alpha-beta search and reports that it is slower. Give the most likely explanation, and say what would have to change for the ordering to pay.

📋 Chapter Summary

- A **game tree** alternates **MAX** levels (ours) and **MIN** levels (theirs). The **minimax value** is the position's worth under optimal play by both sides, and it is a *guarantee*: at least this much, whatever they do.
- Minimax assumes a zero-sum optimal opponent. That is usually false and still the right default, because it is the only assumption that yields a floor.

- **Alpha–beta pruning** carries the best values each player can already secure and stops examining a branch when $\alpha \geq \beta$. It returns *exactly* the minimax value.
- **Move ordering** decides how much is pruned. With perfect ordering alpha–beta sees $b^{\lceil d/2 \rceil} + b^{\lfloor d/2 \rfloor} - 1$ leaves — measured here as exactly 35 against minimax’s 243 — which is roughly $b^{d/2}$, so the same time buys twice the depth.
- Buying a good ordering is not free. A shallow-probe ordering cost 506 leaf evaluations here against natural ordering’s 75: it pays at great depth with a cache, and not at five ply.
- Real games need a **cutoff test** and an **evaluation function**, which is where domain knowledge enters and where the search’s ceiling is set. The **horizon effect** is the cost: the program pushes unavoidable bad news just past the cutoff.

Review Questions

1. (MCQ) Alpha–beta pruning returns: (a) an approximation of the minimax value (b) exactly the minimax value (c) the greedy value (d) a lower bound only.
2. (MCQ) A cutoff occurs when: (a) $\alpha \geq \beta$ (b) $\alpha < \beta$ (c) the depth limit is reached (d) a leaf is negative.
3. (MCQ) Under perfect move ordering alpha–beta examines about: (a) b^d (b) $b^{d/2}$ (c) d^b (d) bd leaves.
4. (MCQ) The horizon effect is: (a) running out of memory (b) pushing an unavoidable loss beyond the depth limit (c) pruning too aggressively (d) a bad evaluation function.
5. (Short) Explain why the minimax value is a guarantee, and what you give up by modelling the other party as cooperative instead.
6. (Short) Why does move ordering affect alpha–beta but not minimax?
7. (Short) Using the measured numbers in this chapter, explain the circumstances in which investing in move ordering pays and the circumstances in which it does not.
8. (Applied) Take the tree of Figure 8.1 with its leaves reversed to $[-1, 0, 2, 1, 9, 6, 5, 3]$. Compute the minimax value, then trace alpha–beta left to right and list which leaves are evaluated and where each cutoff occurs. Compare the count with the five of the worked example and say what that demonstrates.

Planning

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part III — Constraints, Games and Plans

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Represent** actions as STRIPS operators with preconditions, an add list and a delete list, and **explain** what the delete list is for.
- **Solve** a planning problem by progression search, reusing the frontier of Chapter 4 unchanged.
- **Apply** means–ends analysis, and **relate** it to the General Problem Solver that HEC names as a required case study.
- **Identify** goal clobbering, and **explain** why it makes means–ends analysis sensitive to the order in which goals are taken.
- **Say** what the operator representation buys that an opaque successor function does not.

🔗 Before You Start

Chapter 4 and Chapter 5 (the frontier, breadth-first search and A^* — this chapter reuses them without modification) and Chapter 3 (states as sets). Chapter 7 is useful background: like a CSP, a planning problem exposes structure that a general search cannot see.

📖 Key Terms in This Chapter

planning • STRIPS • operator • precondition • add list • delete list • ground operator • progression search • regression • means–ends analysis • difference • General Problem Solver • goal clobbering • goal ordering • partial-order planning • frame problem

9.1 From a Task List to Operators

Chapter 4 planned a release by searching over sets of finished tasks. The successor function was a Python function: opaque, and the only thing the search could do with it was call it.

Now a harder version of the same problem. Two components must each be coded, reviewed, tested and deployed. Testing requires the staging environment, and there is only one — so testing one component *takes staging away* from the other until it is released.

That last clause is what makes this a planning problem rather than a search problem. An action no longer just adds things to the world; it can take things away, and an action taken to achieve one goal can destroy the conditions another goal depended on.

📖 STRIPS operator

A **STRIPS** operator is a named action described by three sets of literals:

- **preconditions** — what must be true for the action to be applicable;
- the **add list** — what it makes true;
- the **delete list** — what it makes false.

Applying operator o in state s gives

$$s' = (s \setminus \text{delete}(o)) \cup \text{add}(o)$$

and o is applicable only when $\text{pre}(o) \subseteq s$.

Everything not mentioned in the add or delete list is unchanged. That convention is the **STRIPS assumption**, and it is what makes the representation usable: you describe what an action changes, not the vastly larger set of things it leaves alone. Stating the latter explicitly is the **frame problem**.

After R. E. Fikes and N. J. Nilsson, “STRIPS: A New Approach to the Application of Theorem Proving to Problem Solving”, *Artificial Intelligence* 2, 1971.

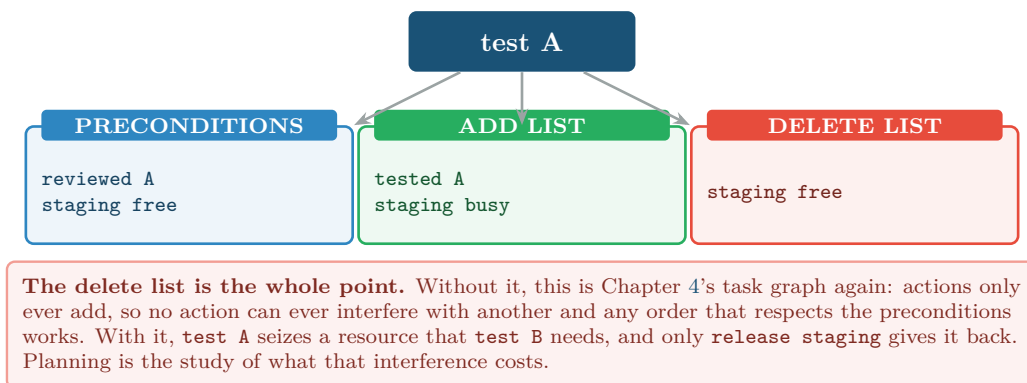


Figure 9.1: One STRIPS operator. Applying it removes the delete list and then adds the add list; everything else in the state is untouched.

✎ The release domain

Four operator schemas, instantiated once per component, plus one for the shared environment. With two components that is nine ground operators.

Operator	Preconditions	Add	Delete
code c	—	coded c	—
review c	coded c	reviewed c	—
test c	reviewed c , staging free	tested c , staging busy	staging free
deploy c	tested c	deployed c	—
release staging	staging busy	staging free	staging busy

Initial state: {staging free}. Goal: {deployed A, deployed B}.

9.2 Planning as Progression Search

A plan is a sequence of operators leading from the initial state to a state satisfying the goal. So a planning problem *is* a search problem, and Chapter 3’s **Problem** interface fits it without modification:

- *State* — a **frozenset** of literals currently true.
- *Actions* — the operators whose preconditions are a subset of the state.
- *Result* — $(s \setminus \text{delete}) \cup \text{add}$.
- *Goal test* — is the goal a subset of the state?

This is **progression** (or forward) search: start at the initial state and move towards the goal. Running Chapter 4's breadth-first search over it finds a nine-step plan after expanding 37 of the 41 reachable states:

```
code A → review A → test A → deploy A → code B
review B → release staging → test B → deploy B
```

Note step seven. Nothing in the goal mentions staging; **release staging** is there purely to undo the interference that **test A** caused. That step is the signature of a planning problem, and no ordering of Chapter 4's tasks could have produced it.

⚠ Why a heuristic barely helps here, and when that happens

Chapter 5 would suggest adding a heuristic. A good admissible one is available: for each component count the actions it still needs — four if untouched, three if coded, two if reviewed, one if tested, none if deployed — and sum. Those actions are specific to their component, so they cannot be shared and the sum is a lower bound. At the initial state with four components it estimates 16 against a true optimum of 19: a strong heuristic.

It buys almost nothing. A^* expands 1,152 states where breadth-first search expands 1,163 — under one per cent.

The reason is worth understanding, because it generalises. A^* must expand every node with $f(n) < C^*$, and it saves work only where f is large enough to rule states out. In this domain almost every reachable state lies on *some* optimal plan — nearly all the actions are necessary, and their order is largely free — so f is close to C^* nearly everywhere and there is nothing to prune.

Heuristics pay when most of the state space is wasted effort. Here very little of it is. The lesson is not that A^* is weak; it is that measuring before optimising applies to search as much as to code.

9.3 Means–Ends Analysis and the General Problem Solver

The 1961 General Problem Solver of Newell and Simon did not search the state space at all. It worked on **differences**.

📖 Means–ends analysis

To achieve a goal from a state:

1. If the goal already holds, stop.
2. Otherwise pick a **difference** — a goal literal not yet true.
3. Choose an operator whose add list contains that literal.
4. *Recursively* achieve that operator's preconditions.
5. Apply the operator, and continue with the remaining goals.

The recursion at step 4 is the idea: the means to an end become ends in their own right.

The difference in cost is large. On this domain, means–ends analysis considers 187 operators at four components where progression search expands 1,163 states — and the gap widens with size, because it never enumerates states at all.

Components	States	Plan length	BFS expanded	A* expanded	GPS ops
2	41	9	37	36	45
3	223	14	218	214	104
4	1,169	19	1,163	1,152	187

Table 9.1: Progression search against means–ends analysis. All three find an optimal plan on every instance; the state space grows about fivefold per component while the work means–ends analysis does grows roughly linearly.

9.4 Goal Clobbering

Means–ends analysis takes the goals in some order, and that order matters. Achieving one goal can destroy a condition that a later goal needs — **goal clobbering** — and a means–ends planner has no machinery for noticing in advance.

Figure 9.2 is the smallest example, and it is two operators long.

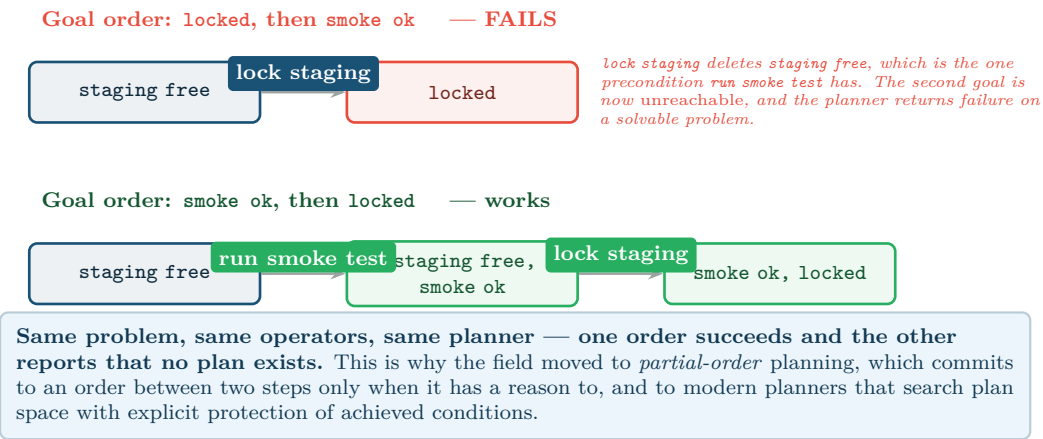


Figure 9.2: Goal clobbering, in two operators. Means–ends analysis is complete only if it happens to take the goals in a workable order.

The historical response was **partial-order planning**: build a set of steps with ordering constraints only where needed, detect when one step threatens a condition another step established, and resolve the threat by adding an ordering constraint. This course names that idea and does not develop it; it is the natural next reading in Russell and Norvig, Chapter 10.

9.5 What the Representation Buys

Progression search over STRIPS operators expands the same states a general search would. So what was gained by describing actions instead of writing a successor function?

The planner can read the actions. A successor function can only be called. An operator can be *inspected* — which is what lets means–ends analysis ask “which action adds **deployed B**?” and what lets a heuristic be derived automatically by, for example, ignoring every delete list.

Domain and planner separate. The operator table is data. A delivery lead can add a deployment gate without touching the search, which is the same argument Chapter 7 made for constraints and Chapter 13 will make for rules. It is the third test of Chapter 1’s checklist, and it is most of what distinguishes an AI system from a program that happens to solve the problem.

Plans are explainable. Every step names an operator with stated preconditions, so the answer to “why is **release staging** in my plan?” is mechanical: because **test B** requires **staging free**, and **test A** deleted it.

📅 Worked Example: Means–ends analysis on the release domain, traced

Goal {**deployed A**, **deployed B**}, initial state {**staging free**}. Take the goals in order.

Difference: deployed A. The only operator adding it is **deploy A**, whose precondition is **tested A**. Recurse.

Difference: tested A. Added by **test A**, which needs **reviewed A** and **staging free**. Recurse on each.

reviewed A needs **coded A**, which needs nothing. Apply **code A**, then **review A**.

staging free already holds. Nothing to do.

Apply **test A**. State becomes {**coded A**, **reviewed A**, **tested A**, **staging busy**} — note that **staging free** is *gone*.

Apply **deploy A**. First goal achieved, in four steps.

Difference: deployed B. By the same recursion this needs **tested B**, which needs **reviewed B** and **staging free**.

reviewed B: apply **code B**, **review B**.

staging free is *no longer true* — **test A** deleted it. But it is a goal like any other, and **release staging** adds it, with precondition **staging busy**, which does hold. Apply **release staging**.

Apply **test B**, then **deploy B**.

The plan, nine steps: **code A**, **review A**, **test A**, **deploy A**, **code B**, **review B**, **release staging**, **test B**, **deploy B**. It is optimal, and it was found by considering 45 operators rather than by expanding 37 states.

Where the luck was. Re-achieving **staging free** worked because an operator exists that restores it. Had there been none — had locking staging been irreversible — means–ends analysis would have failed here exactly as it fails in Figure 9.2, and the failure would have been reported as “no plan exists”.

! Common Pitfalls

- **Omitting the delete list.** Without it every action only adds, no action can interfere, and the problem collapses to Chapter 4’s topological sort. If your planning problem seems too easy, check whether you modelled the interference.
- **Assuming a plan step must appear in the goal.** **release staging** appears in no goal and is indispensable.
- **Trusting means–ends analysis to be complete.** It is sensitive to goal order and can report failure on a solvable problem (Figure 9.2).
- **Adding a heuristic before measuring.** A strong admissible heuristic removed under one per cent of the expansions here, because almost every state lies on an optimal plan.
- **Confusing progression with regression.** Progression searches forward from the initial state over world states; regression searches backward from the goal over *sets* of states, and the two have very different branching factors.
- **Forgetting the STRIPS assumption.** Everything unmentioned is unchanged. If an action really does change something you did not list, the plan will be correct on paper and wrong in the world.

🔧 Try It Yourself: A Release Plan from Operators

STRIPS operators, progression search reusing Chapter 4’s frontier, means–ends analysis, and the clobbering example. Reproduces Table 9.1.

```

"""Chapter 9 lab -- planning a release from STRIPS operators.

Components go through code, review, test and deploy, sharing a single
staging environment. Testing seizes staging and deletes it -- that one
delete effect is what makes this planning rather than search.
"""

import heapq
import itertools
from collections import deque

COMPONENTS = ("A", "B")

def operators(components):
    """(name, preconditions, add, delete), all frozensets."""
    ops = []
    for c in components:
        ops.append(("code %s" % c, frozenset(),
                    frozenset({"coded %s" % c}), frozenset()))
        ops.append(("review %s" % c, frozenset({"coded %s" % c}),
                    frozenset({"reviewed %s" % c}), frozenset()))
        ops.append(("test %s" % c,
                    frozenset({"reviewed %s" % c, "staging free"}),
                    frozenset({"tested %s" % c, "staging busy"}),
                    frozenset({"staging free"})))
        ops.append(("deploy %s" % c, frozenset({"tested %s" % c}),
                    frozenset({"deployed %s" % c}), frozenset()))
    ops.append(("release staging", frozenset({"staging busy"}),
                frozenset({"staging free"}),
                frozenset({"staging busy"})))
    return ops

def applicable(state, ops):
    for op in ops:
        if op[1] <= state:
            yield op

def apply_op(state, op):
    _, _, add, dele = op
    return (state - dele) | add

def reachable(init, ops):
    seen, q = {init}, deque([init])
    while q:
        s = q.popleft()
        for op in applicable(s, ops):
            n = apply_op(s, op)
            if n not in seen:
                seen.add(n)
                q.append(n)
    return seen

# --- 1. progression search: Chapter 4's frontier, unchanged -----
def progression_bfs(init, goal, ops):
    seen = {init}
    q = deque([(init, ())])
    expanded = 0

```

```

while q:
    state, plan = q.popleft()
    if goal <= state:
        return list(plan), expanded
    expanded += 1
    for op in applicable(state, ops):
        nxt = apply_op(state, op)
        if nxt not in seen:
            seen.add(nxt)
            q.append((nxt, plan + (op[0],)))
return None, expanded

def h_remaining(state, components):
    """Actions each component still needs. They are specific to that
    component, so the sum is a lower bound: admissible."""
    total = 0
    for c in components:
        if "deployed %s" % c in state:
            total += 0
        elif "tested %s" % c in state:
            total += 1
        elif "reviewed %s" % c in state:
            total += 2
        elif "coded %s" % c in state:
            total += 3
        else:
            total += 4
    return total

def progression_astar(init, goal, ops, components):
    tie = itertools.count()
    frontier = [(h_remaining(init, components), next(tie), 0, init, ())]
    best = {init: 0}
    expanded = 0
    while frontier:
        _, _, g, state, plan = heapq.heappop(frontier)
        if goal <= state:
            return list(plan), expanded
        if g > best.get(state, 1 << 30):
            continue
        expanded += 1
        for op in applicable(state, ops):
            nxt, ng = apply_op(state, op), g + 1
            if ng < best.get(nxt, 1 << 30):
                best[nxt] = ng
                heapq.heappush(frontier,
                               (ng + h_remaining(nxt, components),
                                next(tie), ng, nxt, plan + (op[0],)))
    return None, expanded

# --- 2. means-ends analysis: the General Problem Solver -----
class Work:
    def __init__(self):
        self.ops = 0

def means_ends(state, goals, ops, work, depth=0):
    """Achieve each goal in turn, recursing on preconditions.

```

```

Returns (state, plan) or None. Note it never enumerates states.
"""
if depth > 40:
    return None
plan = []
for g in goals:
    if g in state:
        continue
    achieved = False
    for op in ops:
        work.ops += 1
        name, pre, add, dele = op
        if g not in add:
            continue
        got = means_ends(state, sorted(pre), ops, work, depth + 1)
        if got is None:
            continue
        state, sub = got
        if not pre <= state:
            continue
        state = apply_op(state, op)
        plan += sub + [name]
        achieved = True
        break
    if not achieved:
        return None                # clobbered, or simply impossible
return state, plan

def legal(plan, init, goal, ops):
    by_name = {o[0]: o for o in ops}
    s = init
    for step in plan:
        op = by_name.get(step)
        if op is None or not op[1] <= s:
            return False
        s = apply_op(s, op)
    return goal <= s

if __name__ == "__main__":
    print("components states plan BFS exp A* exp GPS ops")
    print("-" * 54)
    rows = {}
    for n in (2, 3, 4):
        comps = ("A", "B", "C", "D")[:n]
        ops = operators(comps)
        init = frozenset({"staging free"})
        goal = frozenset({"deployed %s" % c for c in comps})

        states = len(reachable(init, ops))
        bplan, bexp = progression_bfs(init, goal, ops)
        aplan, aexp = progression_astar(init, goal, ops, comps)
        work = Work()
        got = means_ends(init, sorted(goal), ops, work)
        gplan = got[1]

        for p in (bplan, aplan, gplan):
            assert legal(p, init, goal, ops)
            # all three find an OPTIMAL plan on this domain
            assert len(aplan) == len(bplan) == len(gplan)

```

```

rows[n] = (states, len(bplan), bexp, aexp, work.ops)
print("%10d %7d %5d %8d %7d %8d"
      % (n, states, len(bplan), bexp, aexp, work.ops))

# the heuristic is strong and still buys almost nothing
st, ln, bexp, aexp, gops = rows[4]
comps = ("A", "B", "C", "D")
print()
print("h at the initial state, 4 components: %d against a true"
      % h_remaining(frozenset({"staging free"}), comps))
print("optimum of %d -- a strong admissible heuristic." % ln)
print("A* expanded %d where BFS expanded %d: %.1f%% saved."
      % (aexp, bexp, 100 * (bexp - aexp) / bexp))
print("Almost every reachable state lies on some optimal plan,")
print("so there is nothing for f < C* to rule out.")
assert (bexp - aexp) / bexp < 0.05

# means-ends scales quite differently
print()
print("means-ends considered %d operators where BFS expanded %d"
      % (gops, bexp))
assert gops * 5 < bexp

# the nine-step plan, and why release staging is in it
ops = operators(("A", "B"))
init = frozenset({"staging free"})
goal = frozenset({"deployed A", "deployed B"})
plan, _ = progression_bfs(init, goal, ops)
print()
print("an optimal two-component plan:")
for i, s in enumerate(plan, 1):
    print("    %d. %s" % (i, s))
assert "release staging" in plan
print("'release staging' appears in no goal and is indispensable.")

# --- 3. goal clobbering, in two operators -----
print()
print("goal clobbering:")
clob = [
    ("lock staging", frozenset({"staging free"}),
     frozenset({"locked"}), frozenset({"staging free"})),
    ("run smoke test", frozenset({"staging free"}),
     frozenset({"smoke ok"}), frozenset()),
]
c_init = frozenset({"staging free"})
c_goal = frozenset({"locked", "smoke ok"})
for order in ([["locked", "smoke ok"], ["smoke ok", "locked"]]):
    got = means_ends(c_init, order, clob, Work())
    if got is None:
        print("    order %-22s -> FAILS" % (" then ".join(order)))
    else:
        print("    order %-22s -> %s"
              % (" then ".join(order), " then ".join(got[1])))
# one order fails, the other succeeds, on the same solvable problem
assert means_ends(c_init, ["locked", "smoke ok"], clob, Work()) is None
assert means_ends(c_init, ["smoke ok", "locked"], clob,
                  Work()) is not None
plan, _ = progression_bfs(c_init, c_goal, clob)
print("    progression search finds it either way:", plan)

```

Then delete the **staging free** literal from **test**'s delete list and re-run. The plan loses **release staging**, the state space shrinks, and the problem becomes the topological sort of Chapter 4 — which is the clearest possible demonstration of what the delete list is for.

🔍 Checkpoint — Test Yourself

1. The nine-step plan contains **release staging**, which appears in neither the initial state nor the goal. Explain why it is there, and what would have to be deleted from the domain for it to disappear.
2. A strong admissible heuristic reduced A^* 's expansions by under one per cent on this problem. Give the reason, and describe the kind of planning problem on which the same heuristic would pay well.
3. Means–ends analysis found an optimal plan considering 187 operators where breadth-first search expanded 1,163 states. Give one property of means–ends analysis that breadth-first search has and it does not.

📋 Chapter Summary

- A **STRIPS** operator is preconditions, an add list and a delete list. Applying it gives $(s \setminus \text{delete}) \cup \text{add}$; everything unmentioned is unchanged, which is the **STRIPS assumption** and the practical answer to the **frame problem**.
- The **delete list** is what distinguishes planning from search. Without it, actions cannot interfere and the problem is a topological sort.
- **Progression search** runs Chapter 4's frontier over states of literals, unchanged. A plan step may appear in no goal: **release staging** is indispensable and mentioned nowhere.
- A strong admissible heuristic saved under one per cent here, because almost every reachable state lies on an optimal plan. Heuristics pay when most of the space is wasted effort; measure before optimising.
- **Means–ends analysis** works on differences rather than states, recursing from a missing goal literal to the preconditions of an operator that supplies it. It is the General Problem Solver, and it did the job here for a sixth of the work.
- It is not complete: **goal clobbering** makes it sensitive to goal order, and it can report failure on a solvable problem. **Partial-order planning** is the response.
- The representation is the gain: a planner can *read* operators rather than merely call a successor function, the domain is data separate from the algorithm, and every plan step is explainable by its preconditions.

🔧 Review Questions

1. (MCQ) Applying a STRIPS operator produces: (a) $s \cup \text{add}$ (b) $(s \setminus \text{delete}) \cup \text{add}$ (c) $(s \cup \text{add}) \setminus \text{pre}$ (d) $\text{add} \setminus \text{delete}$.
2. (MCQ) The STRIPS assumption says that: (a) all actions are reversible (b) anything not in the add or delete list is unchanged (c) preconditions are always satisfiable (d) plans are optimal.
3. (MCQ) Goal clobbering means: (a) two goals are contradictory (b) achieving one goal destroys a condition another needs (c) the goal is unreachable (d) the heuristic overestimates.
4. (MCQ) Progression search proceeds: (a) backward from the goal (b) forward from the initial state (c) over plan space (d) over constraint arcs.
5. (Short) Explain what the delete list is for, using **test A** and **test B** as your example.

6. **(Short)** Give two things a planner can do with an operator table that it cannot do with an opaque successor function.
7. **(Short)** Means–ends analysis is much cheaper than progression search on this domain and is not complete. State the trade precisely, and say which you would deploy for a release planner used by a delivery team.
8. **(Applied)** Add a component **C** and an operator **sign off c** requiring **deployed c** and adding **signed c**, with the goal extended to include **signed C**. Give the optimal plan length by hand, then say whether means–ends analysis taking the goals in the order **signed C**, **deployed A**, **deployed B** would succeed, and why.

PART IV

Knowledge, Logic and Reasoning

The question changes here. Parts II and III asked what should I do; Part IV asks what follows from what I have been told. That shift is what lets a system be corrected by a domain expert who cannot program, and audited by someone who was not there — and it ends in a rule engine that prints the chain of reasoning behind every conclusion.

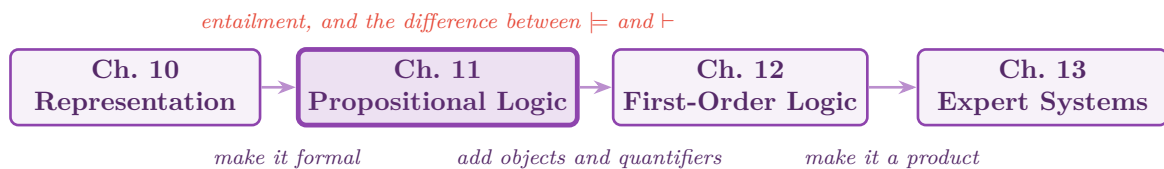


Figure P4.1: Reading path through Part IV. Chapter 11 (highlighted) is the hinge: once entailment and inference are separated, everything after it is engineering.

Knowledge Representation

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part IV — Knowledge, Logic and Reasoning

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **State** what a knowledge representation must support, and **judge** a candidate representation against those criteria.
- **Build** a semantic network and **answer** queries by inheritance.
- **Distinguish** frames, slots and defaults, and **explain** what default override buys and what it costs.
- **Identify** a multiple-inheritance conflict and **say** why no purely local rule resolves it.
- **Describe** what an ontology adds, and **say** where this course hands over to *Knowledge Representation and Reasoning*.

🔗 Before You Start

Chapter 3 (dictionaries, sets and recursion) and Chapter 1 (the third test of the checklist: an explicit representation a domain expert could read and correct). No logic is assumed — Chapter 11 makes formal what this chapter introduces informally.

📖 Key Terms in This Chapter

knowledge representation • ontological commitment • epistemological adequacy • semantic network • node and link • **is-a** • **instance-of** • inheritance • frame • slot • filler • default • override • non-monotonic reasoning • multiple inheritance • ontology • taxonomy

10.1 The Change of Question

Parts II and III asked *what should I do*. Every algorithm took a goal and searched for a sequence of actions. The knowledge was implicit: buried in a successor function, a constraint predicate, an operator table.

Part IV asks a different question: *what follows from what I have been told*. The delivery team knows a great many things — that a hotfix is a kind of release, that every release needs a sign-off, that Ayesha owns the payments service, that releases normally go out on Thursdays except this one. None of that is a goal. It is what the organisation knows, and the useful question is what else must therefore be true.

That change is why this part exists, and it has a practical consequence worth stating at the start: *a representation can be corrected by someone who cannot program*. An operations lead can tell you that a hotfix does not need a full regression pass. If that fact lives in a data structure, they can point at it. If it lives in an **if** statement, they cannot.

📖 What a knowledge representation must support

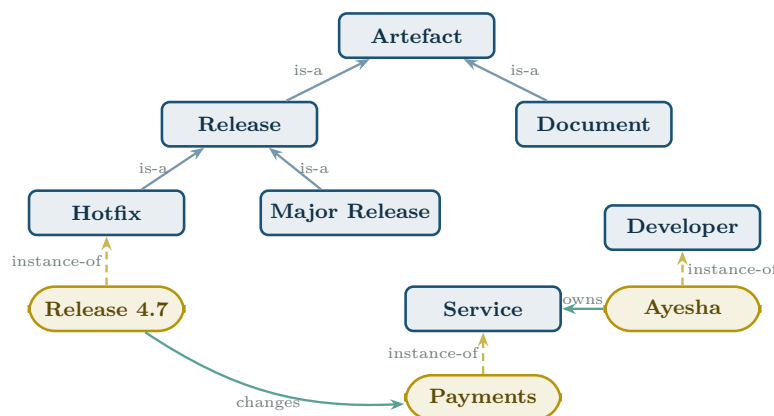
A representation is a scheme for writing down what is known so that a program can reason with it. Four requirements, from Russell and Norvig:

- **Representational adequacy** — it can express the things you need to say, including the awkward ones: defaults, exceptions, uncertainty.
- **Inferential adequacy** — new facts can be derived from stored ones.
- **Inferential efficiency** — the derivation happens in acceptable time.
- **Acquisitional efficiency** — new knowledge can be added easily, and ideally by a domain expert rather than by an engineer.

These pull against each other. A representation expressive enough to say everything is generally one in which inference is intractable, and that tension is the whole subject.

10.2 Semantic Networks

The oldest and simplest representation: a graph. Nodes are concepts or individuals; labelled edges are relations between them. Two link types carry most of the weight: **is-a**, relating a class to a more general class, and **instance-of**, relating an individual to its class.



The query that matters. Nothing in this graph says *Release 4.7 is an Artefact*. It is derived, by walking **instance-of** once and then **is-a** upwards until the walk stops: Release 4.7 → Hotfix → Release → Artefact. That walk is **inheritance**, and it is the entire inference mechanism of a semantic network.

Figure 10.1: A fragment of a delivery ontology. Rounded boxes are individuals, square boxes are classes; solid arrows are **is-a**, dashed are **instance-of**.

Inheritance is cheap, and it is the whole reason the representation is worth having: a property stated once at **Release** is available to every hotfix and every major release, without being repeated and — more importantly — without being repeated *inconsistently*.

10.3 Frames, Slots and Defaults

A semantic network drawn as a graph becomes unreadable at a few hundred nodes. The same information organised by concept is a **frame**: a named bundle of **slots**, each with a **filler** or a **default**.

Three frames from the delivery domain

Frame	Slot	Value
Release	is-a	Artefact
	needs-signoff	yes (<i>default</i>)
	regression-pass	full (<i>default</i>)
	window	Thursday (<i>default</i>)
Hotfix	is-a	Release
	regression-pass	smoke (overrides)
	window	any (overrides)
Release 4.7	instance-of	Hotfix
	changes	Payments

Ask for Release 4.7’s regression-pass. It has none of its own, so the search goes to Hotfix, which says smoke, and stops. Ask for needs-signoff: Release 4.7 has none, Hotfix has none, Release says yes. The rule is *the most specific value wins*, and it falls out of searching upward and stopping at the first hit.



Defaults are what make this representation fit an organisation, and they are also where it stops being logic.

“Releases go out on Thursdays” is not a universally quantified statement — it is false of hotfixes — and yet it is exactly what the delivery lead would tell you, and exactly what you want the system to assume when it knows nothing more. Reasoning that can be *withdrawn* when a more specific fact arrives is **non-monotonic**: adding knowledge can remove a conclusion.

Classical logic, the subject of Chapter 11 and Chapter 12, is monotonic: once entailed, always entailed. The two chapters therefore buy soundness and lose the ability to say “normally”. That trade is real, and neither side wins outright.

10.4 When Inheritance Has No Answer

Defaults are convenient until a concept inherits from two places that disagree.

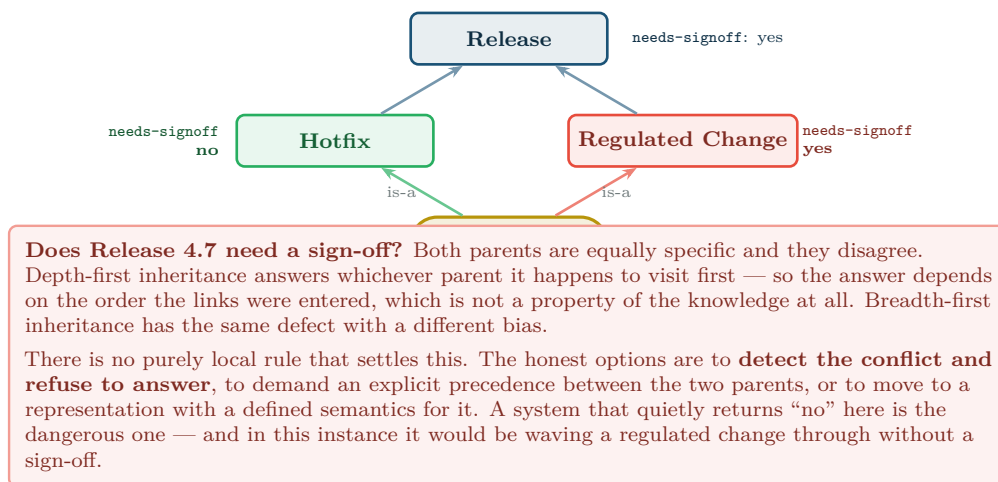


Figure 10.2: A multiple-inheritance conflict — the “Nixon diamond” of the literature, in a delivery setting. The lab detects it rather than guessing.

10.5 Ontologies

An **ontology** is a semantic network taken seriously: a shared, formal vocabulary of the types in a domain, their relations, and the constraints on them, agreed across an organisation rather than invented per system.

The value is not the graph; it is the *agreement*. When the delivery tool, the incident tool and the finance system all mean the same thing by “release”, a query can cross them. When they do not, the integration work is endless and the reports disagree. Most of the practical benefit of ontologies in industry comes from that, and not from the inference.

Formally, an ontology commits to answers for questions a semantic network leaves open: is **is-a** transitive (yes), can an individual belong to two classes (usually yes), can a class be an individual, what makes two individuals identical. Those are **ontological commitments**, and writing them down is most of the work.

10.6 Where This Course Stops

This chapter is a first pass, deliberately. Everything after it in this handout uses semantic networks and frames as the *thing reasoned over*: Chapter 11 and Chapter 12 give inference a formal footing, and Chapter 13 turns a frame base plus rules into a working system.

What this course does not cover is the formal side of representation itself: description logic, reasoning over description logic, ontology languages such as OWL, logical agents and commonsense reasoning. Those are the content of the HEC elective *Knowledge Representation and Reasoning*, for which this course is the prerequisite; Appendix F sets out the line.

Worked Example: Answering four queries by inheritance

Using the frames of the examplebox and the conflict of Figure 10.2.

Query 1: Release 4.7 . regression-pass

- Release 4.7: no local slot. Follow instance-of to Hotfix.
- Hotfix: regression-pass = smoke. **Stop.**

Answer: smoke, inherited from Hotfix. Note that Release says full and is never consulted — the search stops at the first hit, which is what “most specific wins” means operationally.

Query 2: Release 4.7 . needs-signoff (ignoring the second parent for the moment)

- Release 4.7: nothing. Hotfix: nothing. Release: yes. **Stop.**

Answer: yes, inherited from two levels up. The same walk answered both queries; only the depth at which it stopped differed.

Query 3: Release 4.7 . window

- Hotfix overrides with any. **Answer: any** — the default Thursday at Release is correctly suppressed.

Now add the fact that Release 4.7 is also a Regulated Change whose window is Thursday. The answer changes to a conflict. *Adding knowledge removed a conclusion:* that is non-monotonicity, in three lines.

Query 4: Release 4.7 . needs-signoff, with both parents.

- Hotfix says no; Regulated Change says yes. Both are at distance 1 from the individual.
- Depth-first inheritance returns whichever was entered first. That is not an answer; it is an artefact of the file.

Answer: conflict, and the system should say so. The lab returns the set {yes,no} with its provenance, and leaves the decision to a person — which in this case is the only defensible behaviour, because the two sources are a delivery convention and a regulatory obligation and they are not equally negotiable.

! Common Pitfalls

- **Letting inheritance guess on a multiple-inheritance conflict.** The answer then depends on insertion order. Detect it and refuse.
- **Treating a default as a universal.** “Releases go out on Thursdays” is false of hotfixes. If you write it as $\forall$ in Chapter 12, the knowledge base becomes inconsistent the first time a hotfix appears.
- **Putting knowledge in code.** A rule inside an if statement cannot be reviewed by the person who knows whether it is right.
- **Cycles in is-a.** An inheritance walk that does not keep a visited set will not terminate. The lab keeps one.
- **Confusing is-a with instance-of.** A hotfix *is a kind of* release; Release 4.7 *is a* hotfix. Conflating them lets a class inherit properties of an individual, and the errors are very hard to see.
- **Expecting the ontology to pay for itself through inference.** In practice most of the value is the shared vocabulary.

Try It Yourself: A Semantic Network That Answers Queries

Frames in a dictionary, inheritance by walking upward, defaults with override, and a conflict that is detected rather than guessed.

```
"""Chapter 10 lab -- a semantic network with frames and defaults.
```

```
Inheritance is one upward walk. The interesting cases are the third
query, where a default is correctly suppressed by a more specific one,
and the fourth, where two equally specific parents disagree and the
only defensible answer is to say so.
"""
```

```
# frame -> {slot: value}. 'is-a' and 'instance-of' are the links the
# inheritance walk follows; everything else is ordinary knowledge.
```

```
FRAMES = {
    "Artefact": {
        "owned-by": "Delivery",
    },
```

```

"Release": {
    "is-a": ["Artefact"],
    "needs-signoff": "yes",          # default
    "regression-pass": "full",      # default
    "window": "Thursday",           # default
},
"Hotfix": {
    "is-a": ["Release"],
    "regression-pass": "smoke",     # overrides Release
    "window": "any",                # overrides Release
},
"Regulated Change": {
    "is-a": ["Release"],
    "needs-signoff": "yes",
    "window": "Thursday",
},
"Release 4.7": {
    "instance-of": ["Hotfix"],
    "changes": "Payments",
},
"Developer": {},
"Ayesha": {"instance-of": ["Developer"], "owns": "Payments"},
}

LINKS = ("instance-of", "is-a")

def parents(frame):
    out = []
    for link in LINKS:
        out += FRAMES.get(frame, {}).get(link, [])
    return out

def ancestors(frame):
    """Every class above this one, nearest first. The seen set is what
    stops an is-a cycle from looping forever."""
    out, seen, queue = [], {frame}, list(parents(frame))
    while queue:
        f = queue.pop(0)
        if f in seen:
            continue
        seen.add(f)
        out.append(f)
        queue += parents(f)
    return out

def get(frame, slot):
    """Most specific value wins. Returns (value, where it came from),
    or a conflict if two equally distant sources disagree."""
    if slot in FRAMES.get(frame, {}):
        return FRAMES[frame][slot], frame

    # breadth-first by distance, so 'equally specific' is well defined
    level, seen = list(parents(frame)), {frame}
    distance = 1
    while level:
        hits = []
        for f in level:
            if f in seen:
                continue

```

```

        if slot in FRAMES.get(f, {}):
            hits.append((FRAMES[f][slot], f))
        values = {v for v, _ in hits}
        if len(values) > 1:
            return ("CONFLICT", sorted(hits)), distance
        if hits:
            return hits[0][0], hits[0][1]
        seen |= set(level)
        nxt = []
        for f in level:
            nxt += parents(f)
        level = [f for f in nxt if f not in seen]
        distance += 1
    return None, None

def isa_chain(frame):
    return [frame] + ancestors(frame)

if __name__ == "__main__":
    print("inheritance chain for Release 4.7:")
    print("    " + " -> ".join(isa_chain("Release 4.7")))
    assert "Artefact" in ancestors("Release 4.7")
    print()

    print("queries, with the frame each answer came from")
    print("-" * 56)
    for slot in ("regression-pass", "needs-signoff", "window",
                "changes", "owned-by"):
        val, src = get("Release 4.7", slot)
        print("    %-16s = %-10s (from %s)" % (slot, val, src))

    # the default was correctly suppressed
    assert get("Release 4.7", "regression-pass") == ("smoke", "Hotfix")
    assert get("Release", "regression-pass") == ("full", "Release")
    # and a default two levels up is still found
    assert get("Release 4.7", "needs-signoff") == ("yes", "Release")
    print()
    print("'smoke' overrode 'full'; 'yes' was still reached two levels up.")

    # --- now add a second, equally specific parent -----
    print()
    print("adding 'Release 4.7 is-a Regulated Change' ...")
    FRAMES["Hotfix"]["needs-signoff"] = "no" # delivery convention
    FRAMES["Release 4.7"]["is-a"] = ["Regulated Change"]

    val, dist = get("Release 4.7", "needs-signoff")
    print("    needs-signoff =", val[0] if isinstance(val, tuple) else val)
    assert isinstance(val, tuple) and val[0] == "CONFLICT"
    for v, src in val[1]:
        print("        %-4s from %s" % (v, src))
    print()
    print("Two parents at the same distance disagree. Depth-first")
    print("inheritance would answer whichever link was entered first,")
    print("which is a property of the file and not of the knowledge.")
    print("Here that would wave a regulated change through with no")
    print("sign-off, so the only defensible answer is to report the")
    print("conflict and its provenance.")

    # non-monotonicity, demonstrated: adding knowledge REMOVED an answer
    print()

```

```
print("Note what just happened: adding a fact removed a")
print("conclusion. Classical logic cannot do that, which is")
print("exactly the trade Chapters 11 and 12 make.")
```

Then add an explicit precedence — a slot **overrides** on **Regulated Change** naming **Hotfix** — and make **get** consult it before reporting a conflict. That is the smallest honest fix, and it is worth noticing that it works by adding *knowledge about the knowledge*, which is the direction description logic goes in.

? Checkpoint — Test Yourself

1. **Release** says **regression-pass** = **full** and **Hotfix** says **smoke**. Explain, in terms of the inheritance walk, why **Release 4.7** gets **smoke**, and why no rule about “specificity” has to be stated separately.
2. Adding the fact that **Release 4.7** is a regulated change *removed* the system’s answer about sign-off. Name the property of this kind of reasoning that this demonstrates, and say whether classical logic has it.
3. Give one thing a semantic network can express that a set of universally quantified logical statements cannot, and one thing logic can do that the network cannot.

✓ Chapter Summary

- A knowledge representation must be **representationally** and **inferentially adequate**, **efficient**, and easy to add to — and those requirements pull against each other. Expressive representations tend to have intractable inference.
- A **semantic network** is nodes and labelled links. **is-a** relates a class to a more general class, **instance-of** relates an individual to a class, and **inheritance** — one upward walk — is the whole inference mechanism.
- A **frame** organises the same knowledge by concept: slots with fillers or **defaults**. The most specific value wins, which falls out of stopping the walk at the first hit.
- Defaults make the representation fit an organisation and make the reasoning **non-monotonic**: adding knowledge can remove a conclusion. Classical logic is monotonic and cannot say “normally”.
- **Multiple inheritance** from disagreeing parents has no purely local resolution. Detect the conflict, report its provenance, and refuse to guess — the dangerous system is the one that quietly answers.
- An **ontology** is a shared, formal vocabulary with explicit **ontological commitments**. In practice most of its value is the agreement, not the inference.
- This is a first pass. Description logic, ontology languages and commonsense reasoning belong to *Knowledge Representation and Reasoning* (Appendix F).

✍ Review Questions

1. (MCQ) In a semantic network, **is-a** relates: (a) an individual to a class (b) a class to a more general class (c) two individuals (d) a slot to a filler.
2. (MCQ) “The most specific value wins” is implemented by: (a) sorting all values (b) stopping the upward walk at the first hit (c) preferring the root (d) taking the majority.
3. (MCQ) Reasoning in which adding a fact can remove a conclusion is: (a) monotonic (b) non-monotonic (c) unsound (d) incomplete.

4. **(MCQ)** A multiple-inheritance conflict between two equally specific parents should be: (a) resolved depth-first (b) resolved breadth-first (c) reported, with its provenance (d) resolved by the alphabetically first parent.
5. **(Short)** State the four requirements on a knowledge representation and give one pair that pull against each other, with an example.
6. **(Short)** Explain the difference between **is-a** and **instance-of**, and describe a concrete error caused by conflating them.
7. **(Short)** Why is “most of the value of an ontology is the agreement, not the inference” a reasonable claim? Give a delivery example.
8. **(Applied)** Extend the frames of this chapter with a class **Emergency Hotfix**, a kind of **Hotfix**, whose **needs-signoff** is **retrospective**. Give the value of **needs-signoff**, **regression-pass** and **window** for an instance of it, naming the frame each answer comes from. Then say what happens if that instance is also a **Regulated Change**, and what the system should do.

Propositional Logic and Inference

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part IV — Knowledge, Logic and Reasoning

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Write** statements about a domain in propositional logic and **evaluate** them in a model.
- **Distinguish** entailment from derivation, and **explain** what soundness and completeness each guarantee.
- **Decide** entailment by model checking, by DPLL and by resolution refutation, and **compare** their cost.
- **Convert** a sentence to conjunctive normal form.
- **Produce** a countermodel when a conclusion does *not* follow, and **explain** why that is the more useful answer in practice.

🔗 Before You Start

Chapter 10 (what a representation is for, and the price defaults charge) and Chapter 3 (recursion and sets). No previous logic is assumed. Chapter 7 is useful: deciding satisfiability is a constraint problem, and DPLL is backtracking with propagation under another name.

📖 Key Terms in This Chapter

proposition • connective • model • truth table • satisfiable • valid • entailment ($\models$) • derivation ($\vdash$) • soundness • completeness • literal • clause • conjunctive normal form • model checking • DPLL • unit propagation • pure literal • resolution • refutation • countermodel

11.1 The Release Gate, Written Down

The delivery team has a rule about when a release may go out. Everyone can state it and nobody has written it down precisely, so every release involves an argument. Here it is in logic.

🔗 The release gate

Six propositions — each either true or false — and seven sentences.

	Sentence	In English
R1	$Reviewed \wedge Tested \Rightarrow Staged$	Code that is reviewed and tested reaches staging
R2	$Staged \wedge SignedOff \Rightarrow Deploy$	Staged and signed off means it may deploy
R3	$Blocker \Rightarrow \neg Deploy$	An open blocker forbids deployment
F1	$Reviewed$	The code was reviewed
F2	$Tested$	The tests passed
F3	$SignedOff$	The product owner signed off
F4	$\neg Blocker$	There is no open blocker

The question the team actually argues about: *may we deploy?* In the notation of this chapter, is $KB \models Deploy$?

11.2 Syntax, Semantics and Models

Model, satisfiability, validity

A **model** is an assignment of true or false to every proposition symbol. With n symbols there are 2^n models; the gate above has six symbols and therefore 64.

A sentence is **satisfiable** if some model makes it true, **valid** if every model does, and **unsatisfiable** if none does. The connectives have their usual truth tables; the only one that regularly surprises people is implication:

a	b	$a \Rightarrow b$
F	F	T
F	T	T
T	F	F
T	T	T

$a \Rightarrow b$ is false in exactly one case: a true and b false. So “if there is a blocker then we do not deploy” says nothing whatever about the case where there is no blocker — which is correct, and is why F4 has to be stated separately.

⚠ Implication is not causation, and not relevance

$Blocker \Rightarrow \neg Deploy$ does not say the blocker *causes* the decision, and $a \Rightarrow b$ is true whenever a is false regardless of what b is about. “If the build number is prime then Ayesha is on call” is a true sentence on any day the build number is composite.

This bothers everyone at first and it is not a defect: it is the price of a connective whose truth depends only on the truth of its parts, which is exactly the property that makes mechanical inference possible.

11.3 Entailment and Derivation

Two ideas that look alike, are constantly confused, and are the foundation of the whole part.

Entailment and derivation

$KB \models \alpha$ — **entailment** — means α is true in *every* model in which KB is true. It is a statement about *meaning*, and it makes no reference to any algorithm.

$KB \vdash_i \alpha$ — **derivation** — means the inference procedure i can produce α from KB . It is a statement about *that procedure*.

A procedure is **sound** if everything it derives is entailed ($\vdash$ implies $\models$): it never lies. It is **complete** if everything entailed can be derived ($\models$ implies $\vdash$): it never misses anything.

An inference procedure is worth having exactly when it is both. Soundness without completeness gives a system that is right when it speaks and sometimes silent; completeness without soundness is worthless, because a procedure that derives everything is complete.

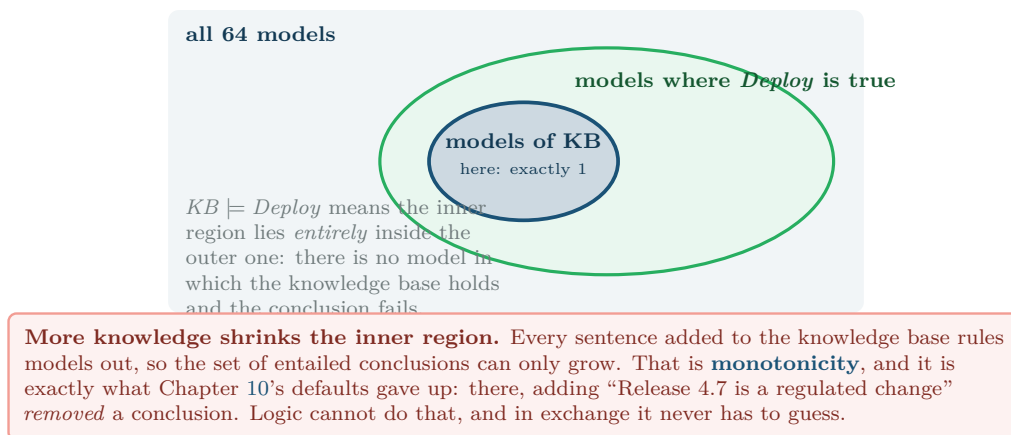


Figure 11.1: Entailment as containment between sets of models. The picture is the definition; the algorithms in the rest of the chapter are three ways of checking it.

11.4 Deciding It: Model Checking

The definition is directly executable: enumerate all 2^n models, keep those satisfying every sentence of the knowledge base, and check whether the query is true in all of them.

On the release gate this is decisive. Of 64 models, exactly **one** satisfies the knowledge base:

$$Reviewed = Tested = SignedOff = Staged = Deploy = T, \quad Blocker = F$$

and $Deploy$ is true in it. So $KB \models Deploy$: the release may go out, and the argument is over.

Model checking is sound, complete, and trivially correct — and it is $O(2^n)$. Six symbols is 64; a realistic gate with forty conditions is 10^{12} . It is the definition made executable, not a usable algorithm.

11.5 Deciding It Better: DPLL

DPLL is a backtracking search over assignments — exactly Chapter 7's backtracking, with propagation — plus two rules that do most of the work.

DPLL

Work on the knowledge base in **conjunctive normal form**: a conjunction of **clauses**, each a disjunction of **literals**. To test satisfiability:

- If there are no clauses left, the assignment satisfies everything: report *satisfiable*.
- If any clause is empty, it cannot be satisfied: backtrack.

- **Unit propagation:** if a clause has one literal left, that literal *must* be true. Assign it and simplify.
- **Pure literal:** if a symbol appears with only one sign anywhere, assign it that way; it can never help to do otherwise.
- Otherwise pick a symbol and try both values.

To decide entailment with a satisfiability test, use the equivalence that underlies every refutation proof in this part:

$$KB \models \alpha \quad \text{if and only if} \quad (KB \wedge \neg\alpha) \text{ is unsatisfiable}$$

If assuming the opposite of the conclusion leads to a contradiction, the conclusion follows.

On the release gate, DPLL settles it in **6 recursive calls with 5 unit propagations**, against model checking's 64 enumerations — and it never branches once, because the facts F1–F4 are unit clauses that cascade.

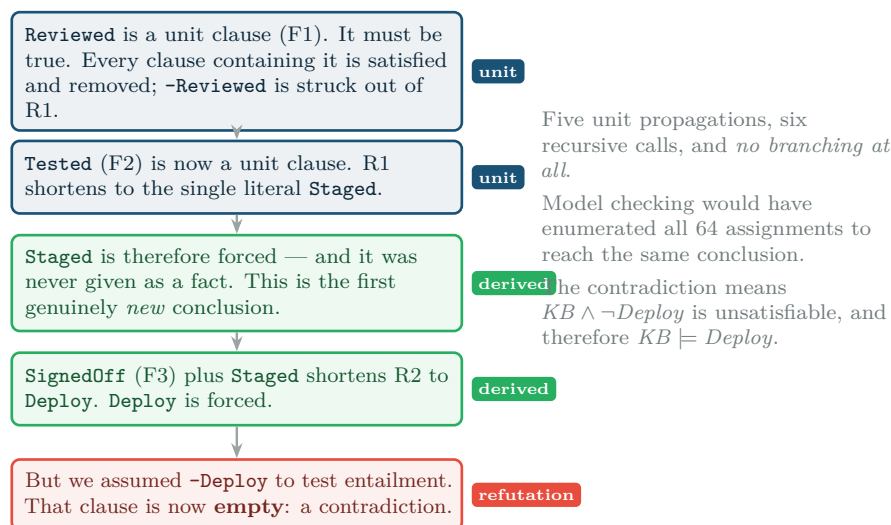


Figure 11.2: DPLL deciding the release gate. Unit propagation cascades from the four given facts and closes the question without a single branch.

11.6 Conjunctive Normal Form

DPLL and resolution both need CNF. Every propositional sentence has an equivalent one, obtained mechanically:

1. Eliminate $\Leftrightarrow$: replace $a \Leftrightarrow b$ with $(a \Rightarrow b) \wedge (b \Rightarrow a)$.
2. Eliminate $\Rightarrow$: replace $a \Rightarrow b$ with $\neg a \vee b$.
3. Drive $\neg$ inwards by De Morgan: $\neg(a \wedge b) \equiv \neg a \vee \neg b$, and $\neg(a \vee b) \equiv \neg a \wedge \neg b$; remove double negations.
4. Distribute $\vee$ over $\wedge$.

Applying step 2 to the gate's rules gives the clauses the lab uses:

	Sentence	Clause
R1	$Reviewed \wedge Tested \Rightarrow Staged$	$\neg Reviewed \vee \neg Tested \vee Staged$
R2	$Staged \wedge SignedOff \Rightarrow Deploy$	$\neg Staged \vee \neg SignedOff \vee Deploy$
R3	$Blocker \Rightarrow \neg Deploy$	$\neg Blocker \vee \neg Deploy$

11.7 Resolution

The resolution rule

From two clauses containing complementary literals, infer their combination with that pair removed:

$$\frac{\alpha \vee \ell \quad \beta \vee \neg \ell}{\alpha \vee \beta}$$

Resolution is sound, and — applied to CNF with the negated query added — it is **refutation complete**: if the set is unsatisfiable, repeated resolution will derive the **empty clause**, which is a disjunction of nothing and therefore false.

The proof for the release gate is five steps. Set as a table, because a proof is a numbered list of clauses with a provenance column:

#	Clause	Derived from
1	<i>Reviewed</i>	given (F1)
2	<i>Tested</i>	given (F2)
3	<i>SignedOff</i>	given (F3)
4	$\neg \textit{Reviewed} \vee \neg \textit{Tested} \vee \textit{Staged}$	given (R1)
5	$\neg \textit{Staged} \vee \neg \textit{SignedOff} \vee \textit{Deploy}$	given (R2)
6	$\neg \textit{Deploy}$	assumed : the negation of the query
7	$\neg \textit{Tested} \vee \textit{Staged}$	1, 4
8	<i>Staged</i>	2, 7
9	$\neg \textit{Staged} \vee \textit{Deploy}$	3, 5
10	<i>Deploy</i>	8, 9
11	$\square$ (the empty clause)	6, 10

Table 11.1: A resolution refutation of $\neg \textit{Deploy}$. Deriving the empty clause shows the assumption is inconsistent with the knowledge base, so $KB \models \textit{Deploy}$.

Resolution is not the fast one

That proof is five steps, and a naive implementation does not find it in five. The lab’s breadth-first version — resolve every pair, add everything new, repeat — takes **105 resolution steps** to reach the empty clause, which is more work than enumerating all 64 models.

So resolution is not here because it is efficient on propositional problems; DPLL is, and modern SAT solvers are DPLL with better bookkeeping. Resolution is here for two reasons. It produces an *auditable proof* — Table 11.1 can be read by a person and checked line by line, which no model count can. And it *lifts to first-order logic* (Chapter 12), where there are infinitely many models and enumerating them is not merely slow but impossible.

11.8 When the Answer Is No

Drop F3 — suppose the product owner has not signed off. Now three models satisfy the knowledge base, and *Deploy* is false in one of them:

$$\textit{Reviewed} = \textit{Tested} = \textit{Staged} = \text{T}, \quad \textit{SignedOff} = \textit{Blocker} = \textit{Deploy} = \text{F}$$

That is a **countermodel**: a concrete situation consistent with everything known in which the conclusion fails. Resolution correctly fails to derive the empty clause, and model checking hands you the countermodel directly.

In practice the countermodel is the more valuable output. “It does not follow” is an assertion a team can argue with; “here is a situation where everything you told me holds and the release still must not go out — nobody signed off” ends the argument. When you build one of these systems, return the countermodel.

Worked Example: A conclusion the knowledge base implies without being told

Drop F4 instead — delete $\neg \text{Blocker}$, so nobody has checked the blocker list. Does *Deploy* still follow?

The intuition says no. Nothing now rules out a blocker, and R3 says a blocker forbids deployment.

The models say yes. Work it through:

- F1, F2, F3 force *Reviewed*, *Tested*, *SignedOff* all true.
- R1 then forces *Staged* true.
- R2 then forces *Deploy* true.
- R3 says $\text{Blocker} \Rightarrow \neg \text{Deploy}$. Since *Deploy* is true, $\neg \text{Deploy}$ is false, so *Blocker* must be false — otherwise R3 is violated.

Exactly one model survives, and it is the same one as before. So $KB \models \text{Deploy}$, and $KB \models \neg \text{Blocker}$: the knowledge base entails that there is no blocker, without anyone having said so.

Is that right? Logically, yes, and it is worth sitting with the discomfort. The knowledge base asserts as facts that the code is reviewed, tested and signed off, and asserts as a rule that these permit deployment. Taken together those *are inconsistent with* an open blocker. The system has not been careless; it has drawn a consequence of what it was told.

And it is a modelling error. The team’s intent was surely “deployment is permitted *provided* no blocker”, which is $\text{Staged} \wedge \text{SignedOff} \wedge \neg \text{Blocker} \Rightarrow \text{Deploy}$. Write R2 that way and the entailment disappears: without F4, *Deploy* no longer follows, which is what everyone expected.

The lesson. Inference is only as good as the axioms, and the failure mode is not a wrong answer — it is a *correct answer to a question you did not mean to ask*. Contrast Chapter 10: a default would have said “normally no blocker” and withdrawn it on evidence. Logic will not guess, so everything you mean must be written down, including the things that seem too obvious to state.

Common Pitfalls

- **Confusing $\models$ with $\vdash$.** The first is about meaning, the second about a procedure. Soundness and completeness are precisely the two claims that connect them.
- **Expecting $a \Rightarrow b$ to mean a causes b .** It is false in exactly one row of the truth table and is true whenever a is false.
- **Omitting a condition from a rule because it is obvious.** The worked example above turns on exactly this, and the result is a system that concludes there is no blocker.
- **Using model checking beyond about twenty symbols.** 2^n is the definition, not an algorithm.
- **Assuming resolution is the efficient method.** It took 105 steps where DPLL took 6 calls. Its value is auditable proofs and the lift to first-order logic.
- **Reporting only “does not follow”.** Return the countermodel; it is what turns a refusal into a diagnosis.

Try It Yourself: Three Ways to Decide a Release Gate

Model checking, DPLL and resolution refutation on the same knowledge base, plus the two variants of the worked example.

```

"""Chapter 11 lab -- deciding a release gate three ways.

A clause is a frozenset of literals; a literal is "X" or "-X". The
three procedures agree on every question, and differ enormously in
what they cost and in what they hand back.
"""

import itertools

SYMBOLS = ["Reviewed", "Tested", "SignedOff", "Blocker", "Staged",
           "Deploy"]

# The knowledge base, already in conjunctive normal form.
KB = [
    frozenset({"-Reviewed", "-Tested", "Staged"}),      # R1
    frozenset({"-Staged", "-SignedOff", "Deploy"}),     # R2
    frozenset({"-Blocker", "-Deploy"}),                 # R3
    frozenset({"Reviewed"}),                             # F1
    frozenset({"Tested"}),                               # F2
    frozenset({"SignedOff"}),                            # F3
    frozenset({"-Blocker"}),                             # F4
]
LABEL = {KB[0]: "R1", KB[1]: "R2", KB[2]: "R3", KB[3]: "F1",
         KB[4]: "F2", KB[5]: "F3", KB[6]: "F4"}

def neg(lit):
    return lit[1:] if lit.startswith("-") else "-" + lit

def holds(lit, model):
    return (not model[lit[1:]]) if lit.startswith("-") else model[lit]

def clause_true(clause, model):
    return any(holds(l, model) for l in clause)

# --- 1. model checking: the definition, made executable -----
def model_check(kb, query, symbols):
    """Returns (entailed, models of kb, models enumerated)."""
    models, seen = [], 0
    for combo in itertools.product([False, True], repeat=len(symbols)):
        m = dict(zip(symbols, combo))
        seen += 1
        if all(clause_true(c, m) for c in kb):
            models.append(m)
    entailed = bool(models) and all(m[query] for m in models)
    return entailed, models, seen

# --- 2. DPLL: backtracking with propagation -----
class Work:
    def __init__(self):
        self.calls = 0
        self.units = 0
        self.branches = 0

```

```

def simplify(kb, lit):
    out = []
    for c in kb:
        if lit in c:
            continue          # clause already satisfied
        if neg(lit) in c:
            out.append(c - {neg(lit)})  # clause shortens
        else:
            out.append(c)
    return out

def dpll(kb, assign, w):
    w.calls += 1
    if not kb:
        return True, assign
    if any(len(c) == 0 for c in kb):
        return False, None
    for c in kb:              # unit propagation
        if len(c) == 1:
            lit = next(iter(c))
            w.units += 1
            a = dict(assign)
            a[lit.lstrip("-")] = not lit.startswith("-")
            return dpll(simplify(kb, lit), a, w)
    lits = {l for c in kb for l in c}
    for l in lits:            # pure literal
        if neg(l) not in lits:
            a = dict(assign)
            a[l.lstrip("-")] = not l.startswith("-")
            return dpll(simplify(kb, l), a, w)
    l = next(iter(next(iter(kb))))  # branch
    w.branches += 1
    for choice in (l, neg(l)):
        a = dict(assign)
        a[choice.lstrip("-")] = not choice.startswith("-")
        ok, res = dpll(simplify(kb, choice), a, w)
        if ok:
            return True, res
    return False, None

def entails_dpll(kb, query):
    """KB |= q iff KB and not-q is unsatisfiable."""
    w = Work()
    sat, _ = dpll(list(kb) + [frozenset({neg(query)})], {}, w)
    return (not sat), w

# --- 3. resolution refutation -----
def resolve(ci, cj):
    out = []
    for l in ci:
        if neg(l) in cj:
            out.append((ci - {l}) | (cj - {neg(l)}))
    return out

def resolution(kb, query):
    """Breadth-first saturation. Honest, complete, and slow."""
    clauses = list(kb) + [frozenset({neg(query)})]

```

```

steps = 0
while True:
    new = set()
    for i in range(len(clauses)):
        for j in range(i + 1, len(clauses)):
            for r in resolve(clauses[i], clauses[j]):
                steps += 1
                if not r:
                    return True, steps
                if r not in clauses:
                    new.add(r)
    if not new - set(clauses):
        return False, steps
    clauses += sorted(new, key=lambda c: (len(c), sorted(c)))

def show(model):
    return " ".join("%s=%d" % (s, model[s]) for s in SYMBOLS)

if __name__ == "__main__":
    print("symbols %d, so %d models to enumerate"
          % (len(SYMBOLS), 2 ** len(SYMBOLS)))
    print()

    ent, models, seen = model_check(KB, "Deploy", SYMBOLS)
    print("MODEL CHECKING")
    print("  %d of %d models satisfy the KB" % (len(models), seen))
    for m in models:
        print("    " + show(m))
    print("  KB entails Deploy:", ent)
    assert ent and len(models) == 1

    ent2, w = entails_dp11(KB, "Deploy")
    print()
    print("DPLL")
    print("  %d calls, %d unit propagations, %d branches"
          % (w.calls, w.units, w.branches))
    print("  KB entails Deploy:", ent2)
    assert ent2 and ent2 == ent
    assert w.branches == 0, "the facts cascade; no branching needed"

    ent3, steps = resolution(KB, "Deploy")
    print()
    print("RESOLUTION")
    print("  empty clause derived:", ent3, " in %d steps" % steps)
    assert ent3 == ent
    print("  ... which is more work than enumerating all %d models."
          % seen)
    assert steps > seen

    # --- drop the sign-off: the answer becomes no -----
    print()
    print("=== drop F3 (SignedOff) ===")
    KB2 = [c for c in KB if c != frozenset({"SignedOff"})]
    ent4, models4, _ = model_check(KB2, "Deploy", SYMBOLS)
    print("  %d models satisfy the KB; entails Deploy: %s"
          % (len(models4), ent4))
    counter = [m for m in models4 if not m["Deploy"]]
    print("  countermodel:")
    print("    " + show(counter[0]))
    assert not ent4 and counter

```

```

ok4, _ = resolution(KB2, "Deploy")
assert not ok4
print("    resolution correctly fails to derive the empty clause.")
print("    The countermodel is the useful output: everything you")
print("    told me holds, and the release still must not go out.")

# --- drop the blocker fact: the surprise -----
print()
print("=== drop F4 (not Blocker) ===")
KB3 = [c for c in KB if c != frozenset({"-Blocker"})]
ent5, models5, _ = model_check(KB3, "Deploy", SYMBOLS)
print("    %d models satisfy the KB; entails Deploy: %s"
      % (len(models5), ent5))
assert ent5 and len(models5) == 1
print("    Blocker is %d in the only model: the KB entails that"
      % models5[0]["Blocker"])
print("    there is no blocker, without anyone saying so.")
assert not models5[0]["Blocker"]

# --- and the fix: put the condition in the rule -----
print()
print("=== the fix: R2 requires no blocker ===")
KB4 = [c for c in KB3
       if c != frozenset({"-Staged", "-SignedOff", "Deploy"})]
KB4.append(frozenset({"-Staged", "-SignedOff", "Blocker", "Deploy"}))
ent6, models6, _ = model_check(KB4, "Deploy", SYMBOLS)
print("    %d models satisfy the KB; entails Deploy: %s"
      % (len(models6), ent6))
assert not ent6
print("    Now Deploy does NOT follow without knowing about")
print("    blockers -- which is what the team meant all along.")

```

Then add a fortieth condition to the gate and re-run. Model checking stops being possible (2^{40} models); DPLL does not notice. That is the moment the difference between a definition and an algorithm becomes concrete.

🔍 Checkpoint — Test Yourself

1. State the difference between $KB \models \alpha$ and $KB \vdash \alpha$, and say which of soundness and completeness would be violated by a procedure that derives every sentence.
2. With F4 removed, the knowledge base still entails *Deploy* and entails $\neg \text{Blocker}$. Explain how, and say whether this is a bug in the inference procedure.
3. Resolution took 105 steps where DPLL took 6 calls on the same problem. Give the two reasons resolution is nevertheless worth teaching.

📋 Chapter Summary

- A **model** assigns true or false to every symbol; n symbols give 2^n models. A sentence is **satisfiable** if some model makes it true and **valid** if all do.
- $KB \models \alpha$ means α holds in every model of KB — a statement about meaning. $KB \vdash \alpha$ means a procedure derives it. **Soundness** and **completeness** are the two claims linking them.
- Logic is **monotonic**: adding knowledge only ever rules models out, so conclusions are never withdrawn. That is precisely what Chapter 10's defaults traded away.
- **Model checking** is the definition made executable and is $O(2^n)$. **DPLL** is backtracking with **unit propagation** and the **pure literal** rule — 6 calls against 64 enumerations here, with no branching at all.

- $KB \models \alpha$ if and only if $KB \wedge \neg\alpha$ is unsatisfiable. Every refutation method rests on this.
- **Resolution** on CNF is refutation complete. It is slower than DPLL (105 steps against 6 calls) and is worth having because it yields an auditable proof and because it lifts to first-order logic.
- When a conclusion does not follow, return the **countermodel**. And remember that inference is only as good as the axioms: an omitted condition produces a correct answer to a question you did not mean to ask.

Review Questions

1. (MCQ) $KB \models \alpha$ means: (a) a procedure derives α (b) α is true in every model of KB (c) α is valid (d) KB is satisfiable.
2. (MCQ) A sound inference procedure: (a) derives everything entailed (b) derives only things that are entailed (c) always terminates (d) is complete.
3. (MCQ) $KB \models \alpha$ if and only if: (a) $KB \wedge \alpha$ is satisfiable (b) $KB \wedge \neg\alpha$ is unsatisfiable (c) $KB \vee \alpha$ is valid (d) α is a unit clause.
4. (MCQ) Unit propagation applies when a clause has: (a) no literals (b) exactly one literal left (c) only positive literals (d) duplicate literals.
5. (Short) Convert $Staged \wedge SignedOff \wedge \neg Blocker \Rightarrow Deploy$ to a clause, showing each step.
6. (Short) Why is a countermodel a more useful answer than “it does not follow”? Give the countermodel from this chapter and the sentence you would say to the delivery team.
7. (Short) Resolution is slower than DPLL on propositional problems. Give the two reasons it is still the method developed in Chapter 12.
8. (Applied) Add the rule “a release with an open blocker may still deploy if the incident commander approves” to the gate, introducing one new symbol. Write it as a sentence, convert it to a clause, and determine — by hand, using unit propagation — whether $Deploy$ follows from F1–F3 together with $Blocker$ and the commander’s approval.

First-Order Logic and Resolution

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part IV — Knowledge, Logic and Reasoning

🕒 Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** why propositional logic cannot state a general rule, and **quantify** what it costs to try.
- **Express** statements about objects and relations using quantifiers.
- **Unify** two expressions, **apply** the occurs check, and **say** what goes wrong without it.
- **Convert** a first-order sentence to clause form, including skolemisation.
- **Prove** a conclusion by resolution refutation and **present** the proof as an auditable table.

🔗 Before You Start

Chapter 11 — entailment and derivation, conjunctive normal form, the resolution rule, and the equivalence $KB \models \alpha \Leftrightarrow (KB \wedge \neg\alpha)$ unsatisfiable. Everything here is that machinery with objects added, and Chapter 3 supplies the recursion that unification needs.

📖 Key Terms in This Chapter

first-order logic • constant • variable • function symbol • term • predicate • atom • universal quantifier • existential quantifier • substitution • unification • most general unifier • occurs check • standardising apart • skolemisation • clause form • resolution refutation • set of support • subsumption

12.1 Where Propositional Logic Runs Out

Chapter 11 settled the release gate. Now a rule the delivery team uses constantly: *if something you depend on is delayed, you are delayed*.

Propositional logic cannot say that. It has no way to talk about “something” — only about specific, named facts. So the rule has to be written out once per pair:

$$\begin{aligned} & \text{DependsRelease47Payments} \wedge \text{DelayedPayments} \Rightarrow \text{DelayedRelease47} \\ & \text{DependsPaymentsAuth} \wedge \text{DelayedAuth} \Rightarrow \text{DelayedPayments} \\ & \dots \text{and so on} \end{aligned}$$

With n services that is n^2 sentences for one rule, each of which must be maintained by hand, and a new service means writing $2n$ more. At forty services the single rule above becomes 1,600 sentences. The knowledge is not large; the *encoding* is, and it is an encoding no domain expert would agree to maintain.

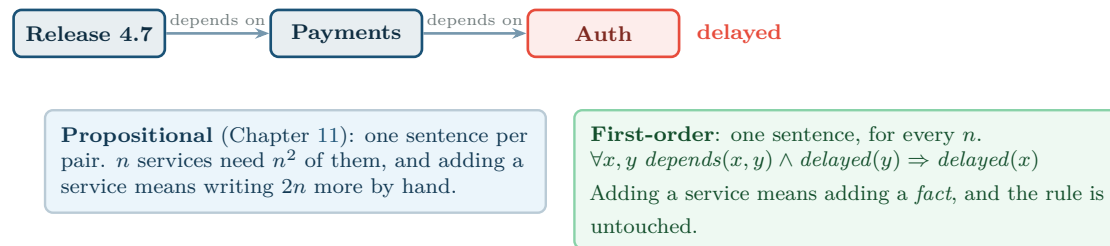


Figure 12.1: The delay chain. Quantifiers buy the ability to state a rule once, which is the difference between a knowledge base a team can maintain and one it cannot.

12.2 Syntax and Semantics

First-order logic adds three things to the propositional language.

- **Terms** denote objects: **constants** (`Auth`), **variables** (x), and **function symbols** applied to terms ($\text{owner}(\text{Payments})$).
- **Predicates** denote relations: $\text{depends}(x, y)$ is true or false of a pair of objects.
- **Quantifiers**: $\forall x$ (for all) and $\exists x$ (there exists).

The two quantifiers pair with different connectives, and getting this wrong is the most common beginner's error:

$$\forall x \text{ service}(x) \Rightarrow \text{monitored}(x) \quad \text{but} \quad \exists x \text{ service}(x) \wedge \text{unmonitored}(x)$$

Universal with implication; existential with conjunction. Writing $\forall x \text{ service}(x) \wedge \text{monitored}(x)$ claims that *everything in the universe* is a monitored service — including Ayesha and the number seven. Writing $\exists x \text{ service}(x) \Rightarrow \text{unmonitored}(x)$ is satisfied by any object that is not a service, and so says essentially nothing.

12.3 Unification

Resolution needs to know when two atoms can be made identical. That is **unification**.

Unification and the most general unifier

A **substitution** θ maps variables to terms. $\text{Unify}(p, q)$ returns a substitution with $p\theta = q\theta$, or failure.

Among all unifiers there is a **most general** one, which commits to as little as possible. That matters: a unifier that binds x to `Auth` when it only needed to bind x to y throws away conclusions.

The **occurs check** refuses to bind a variable to a term containing it.

SUCCEEDS

```
unify( blocks(x, Auth), blocks(Release47, y) )
```

Match argument by argument: x is a variable, so bind $x \mapsto \text{Release47}$; then Auth is a constant and y a variable, so bind $y \mapsto \text{Auth}$.

Result: $\theta = \{x/\text{Release47}, y/\text{Auth}\}$

FAILS

```
unify( blocks(x, x), blocks(Auth, Payments) )
```

Bind $x \mapsto \text{Auth}$ from the first argument. The second argument then requires Auth to equal Payments , and two distinct constants never unify.

Result: failure. The same variable twice means *the same object* twice.

FAILS — OCCURS CHECK

```
unify( x, f(x) )
```

Tempting to bind $x \mapsto f(x)$ — and that substitution describes no finite term. Applying it gives $f(f(f(\dots)))$, and a prover that accepts it will loop or crash.

Result: failure, by the occurs check. This is the one case people leave out, and it is the one that produces a hang rather than a wrong answer.

Figure 12.2: Three unification problems. The third is why every implementation needs an occurs check, and why many production Prolog systems omit it for speed and document the risk.

12.4 Clause Form

Resolution works on clauses, so first-order sentences must be converted. The procedure extends Chapter 11's with two steps for quantifiers.

1. Eliminate $\Leftrightarrow$ and $\Rightarrow$ as before.
2. Drive $\neg$ inwards. Over quantifiers: $\neg\forall x P \equiv \exists x \neg P$ and $\neg\exists x P \equiv \forall x \neg P$.
3. **Standardise apart:** rename so no two quantifiers bind the same variable name.
4. **Skolemise:** replace each existentially quantified variable by a new constant, or — if it lies inside any universal quantifiers — by a new *function* of those universally quantified variables.
5. Drop the universal quantifiers (everything left is implicitly universal) and distribute $\vee$ over $\wedge$.

Skolemisation, and why the function is necessary

“Every service has an owner” is $\forall x \text{ service}(x) \Rightarrow \exists y \text{ owns}(y, x)$.

Skolemising y to a *constant* Sk gives $\neg\text{service}(x) \vee \text{owns}(\text{Sk}, x)$, which claims one person owns every service. That is a different and much stronger statement, and it is wrong.

y depends on x , so it must become a *function* of x :

$$\neg\text{service}(x) \vee \text{owns}(\text{owner}(x), x)$$

where *owner* is a new function symbol meaning “the owner of”. Now each service has its own. The rule is mechanical: *skolemise to a function of every universal quantifier you are inside*.

The delay rule needs no skolemisation — it is purely universal:

$$\forall x, y \text{ depends}(x, y) \wedge \text{delayed}(y) \Rightarrow \text{delayed}(x)$$

becomes the single clause $\neg\text{depends}(x, y) \vee \neg\text{delayed}(y) \vee \text{delayed}(x)$.

12.5 Resolution Refutation

The rule is Chapter 11's, with unification doing the matching:

$$\frac{\alpha \vee \ell \quad \beta \vee \neg m \quad \theta = \text{Unify}(\ell, m)}{(\alpha \vee \beta)\theta}$$

Before each resolution the two clauses are **standardised apart**, so that an x in one is not accidentally identified with an x in the other.

✎ The knowledge base

- R1 $\neg \text{depends}(x, y) \vee \neg \text{delayed}(y) \vee \text{delayed}(x)$
- F1 $\text{depends}(\text{Release47}, \text{Payments})$
- F2 $\text{depends}(\text{Payments}, \text{Auth})$
- F3 $\text{delayed}(\text{Auth})$

Query: $\text{delayed}(\text{Release47})$ — is the release late?

#	Clause	Derived from
1	$\neg \text{depends}(x, y) \vee \neg \text{delayed}(y) \vee \text{delayed}(x)$	given (R1)
2	$\text{depends}(\text{Release47}, \text{Payments})$	given (F1)
3	$\text{depends}(\text{Payments}, \text{Auth})$	given (F2)
4	$\text{delayed}(\text{Auth})$	given (F3)
5	$\neg \text{delayed}(\text{Release47})$	assumed: negated query
6	$\neg \text{depends}(x, \text{Auth}) \vee \text{delayed}(x)$	1, 4 with $\theta = \{y/\text{Auth}\}$
7	$\text{delayed}(\text{Payments})$	3, 6 with $\theta = \{x/\text{Payments}\}$
8	$\neg \text{depends}(x, \text{Payments}) \vee \text{delayed}(x)$	1, 7 with $\theta = \{y/\text{Payments}\}$
9	$\text{delayed}(\text{Release47})$	2, 8 with $\theta = \{x/\text{Release47}\}$
10	$\square$ (the empty clause)	5, 9

Table 12.1: A resolution refutation in five derived steps. Each line names its parents and its unifier, so the proof can be checked by hand — which is what makes it an audit artefact rather than an assertion.

⚠ Finding that proof is much harder than checking it

Table 12.1 is five derived lines. A naive implementation — resolve every pair, keep everything new, repeat — reaches the empty clause in **705 resolution steps**.

The standard first remedy is **set of support**: require every resolution to involve a clause descended from the negated query. Nothing else is relevant to proving *this* conclusion, and the restriction preserves refutation completeness. It cuts the search to **202 steps**, a factor of 3.5.

Even that is far from five, and the reason is visible in the lab's trace: the same clause is derived repeatedly with different variable names, because standardising apart creates fresh names each time and a set of clauses does not recognise alpha-variants as equal. Real provers add **subsumption** — discard a clause if a more general one is already present — along with unit preference and indexing. That is the subject of automated theorem proving, and it is where this course stops.

12.6 What This Buys, and What It Costs

First-order logic is strictly more expressive than propositional logic, and the price is paid in decidability.

Propositional entailment is *decidable*: model checking always terminates, if slowly. First-order entailment is only *semi-decidable*. If the conclusion follows, resolution will eventually find a proof. If it does not, the procedure may run forever without ever being able to report that fact — there are infinitely many models and it cannot enumerate them to find a countermodel.

So Chapter 11’s most practically useful output, the countermodel, is in general unavailable here. That is a real loss, and it is one reason production systems often restrict themselves to a decidable fragment — Horn clauses, which is what Chapter 13 uses and what makes Prolog work.

Worked Example: Following the unifiers through the proof

The point of this trace is that the general rule R1 is used *three times* with three different substitutions, which is exactly what propositional logic could not do.

Step 6: resolve R1 with F3. R1 contains $\neg \text{delayed}(y)$ and F3 is $\text{delayed}(\text{Auth})$. Unifying $\text{delayed}(y)$ with $\text{delayed}(\text{Auth})$ gives $\theta = \{y/\text{Auth}\}$. Remove the complementary pair and apply θ to what remains:

$$\neg \text{depends}(x, \text{Auth}) \vee \text{delayed}(x)$$

Read it in English: *anything that depends on Auth is delayed*. A new general rule, derived from a general rule and one fact.

Step 7: resolve that with F2. $\text{depends}(\text{Payments}, \text{Auth})$ unifies with $\text{depends}(x, \text{Auth})$ under $\theta = \{x/\text{Payments}\}$, leaving $\text{delayed}(\text{Payments})$. The chain has advanced one link.

Step 8: resolve R1 with that. A *fresh* copy of R1 — standardised apart, so its x and y are new variables — resolves against $\text{delayed}(\text{Payments})$ under $\theta = \{y/\text{Payments}\}$, giving $\neg \text{depends}(x, \text{Payments}) \vee \text{delayed}(x)$.

Step 9: resolve with F1, $\theta = \{x/\text{Release47}\}$, giving $\text{delayed}(\text{Release47})$.

Step 10: that contradicts the assumed $\neg \text{delayed}(\text{Release47})$, so the empty clause follows and the query is proved.

Why standardising apart matters. At step 8, R1’s variables must be renamed. If the x in the fresh copy were identified with the x already bound to **Payments** at step 7, the copy would be unusable for **Release47** and the proof would fail. The variables in a clause are local to it, and forgetting that is a bug whose symptom is proofs that mysteriously do not close.

Common Pitfalls

- **$\forall$ with $\wedge$.** $\forall x \text{ service}(x) \wedge \text{monitored}(x)$ claims everything in the universe is a monitored service. Use $\Rightarrow$ with $\forall$ and $\wedge$ with $\exists$.
- **Omitting the occurs check.** Unify($x, f(x)$) then succeeds with an infinite term, and the symptom is a hang rather than a wrong answer.
- **Skolemising to a constant inside a universal.** “Every service has an owner” becomes “one person owns every service”.
- **Forgetting to standardise apart.** Two clauses sharing a variable name become accidentally linked, and proofs fail for reasons invisible in the output.
- **Expecting a countermodel when the proof fails.** First-order entailment is only semi-decidable: failing to find a proof is not evidence that none exists.
- **Assuming naive resolution will scale.** 705 steps for a five-line proof. Set of support, subsumption and indexing are not optional extras.

Try It Yourself: Unification and a Resolution Proof

A real unifier with the occurs check, first-order resolution, and the set-of-support strategy measured against naive saturation.

```

"""Chapter 12 lab -- first-order resolution on a delay chain.

Representation: a constant is a string; a variable is ("?", name);
a compound term is (functor, arg, ...); a literal is
(sign, predicate, args).
"""

def var(n):
    return ("?", n)

def is_var(t):
    return isinstance(t, tuple) and len(t) == 2 and t[0] == "?"

def is_compound(t):
    return isinstance(t, tuple) and not is_var(t)

def walk(t, s):
    while is_var(t) and t in s:
        t = s[t]
    return t

def occurs(v, t, s):
    """The occurs check: is v inside t? Omit this and unify(x, f(x))
    succeeds with a term that does not exist."""
    t = walk(t, s)
    if t == v:
        return True
    if is_compound(t):
        return any(occurs(v, a, s) for a in t[1:])
    return False

def unify(x, y, s):
    """Most general unifier, or None."""
    if s is None:
        return None
    x, y = walk(x, s), walk(y, s)
    if x == y:
        return s
    if is_var(x):
        if occurs(x, y, s):
            return None
        s = dict(s)
        s[x] = y
        return s
    if is_var(y):
        return unify(y, x, s)
    if is_compound(x) and is_compound(y):
        if x[0] != y[0] or len(x) != len(y):
            return None
        for a, b in zip(x[1:], y[1:]):
            s = unify(a, b, s)
            if s is None:

```

```

        return None
    return s
return None

def subst(t, s):
    t = walk(t, s)
    if is_compound(t):
        return (t[0],) + tuple(subst(a, s) for a in t[1:])
    return t

def lit_subst(lit, s):
    sign, pred, args = lit
    return (sign, pred, tuple(subst(a, s) for a in args))

def standardise(clause, n):
    """Rename every variable, so two clauses never share one."""
    out = []
    for (sign, pred, args) in clause:
        out.append((sign, pred,
                    tuple(("?", "%s#%d" % (a[1], n)) if is_var(a) else a
                        for a in args)))
    return frozenset(out)

def resolve(ci, cj, n):
    ci, cj = standardise(ci, n), standardise(cj, n + 500000)
    out = []
    for li in ci:
        for lj in cj:
            if li[0] == lj[0] or li[1] != lj[1]:
                continue
            if len(li[2]) != len(lj[2]):
                continue
            s, ok = {}, True
            for a, b in zip(li[2], lj[2]):
                s = unify(a, b, s)
                if s is None:
                    ok = False
                    break
            if ok:
                out.append(frozenset(
                    lit_subst(l, s) for l in (ci - {li}) | (cj - {lj})))
    return out

def show_term(a):
    if is_var(a):
        return a[1]
    if is_compound(a):
        return "%s(%s)" % (a[0], ",".join(show_term(x) for x in a[1:]))
    return a

def show(clause):
    if not clause:
        return "[] (the empty clause)"
    parts = []
    for (sign, pred, args) in clause:
        parts.append(("" if sign else "-") +

```

```

        "%s(%s)" % (pred,
                    ", ".join(show_term(a) for a in args)))
    return " OR ".join(sorted(parts))

# --- the knowledge base -----
x, y = var("x"), var("y")
R1 = frozenset({(False, "depends", (x, y)),
               (False, "delayed", (y,)),
               (True, "delayed", (x,))})
F1 = frozenset({(True, "depends", ("Release47", "Payments"))})
F2 = frozenset({(True, "depends", ("Payments", "Auth"))})
F3 = frozenset({(True, "delayed", ("Auth",))})
KB = [R1, F1, F2, F3]
QUERY = (True, "delayed", ("Release47",))

def refute_naive(kb, query, rounds=6):
    """Resolve every pair, keep everything new, repeat."""
    clauses = list(kb) + [frozenset({(not query[0], query[1], query[2])})]
    steps = 0
    for _ in range(rounds):
        new = []
        for i in range(len(clauses)):
            for j in range(len(clauses)):
                if i == j:
                    continue
                for r in resolve(clauses[i], clauses[j], steps):
                    steps += 1
                    if not r:
                        return True, steps
                    if r not in clauses and r not in new:
                        new.append(r)
        if not new:
            return False, steps
        clauses += new
    return False, steps

def refute_sos(kb, query, rounds=8):
    """Set of support: every resolution must involve a clause
    descended from the negated query. Nothing else is relevant to
    proving THIS conclusion, and completeness is preserved."""
    neg_q = frozenset({(not query[0], query[1], query[2])})
    base, sos = list(kb), [neg_q]
    seen = set(base) | {neg_q}
    steps = 0
    for _ in range(rounds):
        new = []
        for s in sos:
            for b in base + sos:
                if b is s:
                    continue
                for r in resolve(s, b, steps):
                    steps += 1
                    if not r:
                        return True, steps
                    if r not in seen:
                        seen.add(r)
                        new.append(r)
        if not new:
            return False, steps

```

```

        sos += new
    return False, steps

if __name__ == "__main__":
    print("=== unification ===")
    cases = [
        ("blocks(x, Auth) vs blocks(Release47, y)",
         ("blocks", x, "Auth"), ("blocks", "Release47", y)),
        ("blocks(x, x) vs blocks(Auth, Payments)",
         ("blocks", x, x), ("blocks", "Auth", "Payments")),
        ("x vs f(x) [occurs check]",
         x, ("f", x)),
    ]
    for label, a, b in cases:
        s = unify(a, b, {})
        if s is None:
            print(" %-42s FAIL" % label)
        else:
            binding = ", ".join("%s/%s" % (k[1], show_term(v))
                                for k, v in sorted(s.items()))
            print(" %-42s {%s}" % (label, binding))

    # the three results the chapter claims
    assert unify(("blocks", x, "Auth"),
                 ("blocks", "Release47", y), {}) is not None
    assert unify(("blocks", x, x),
                 ("blocks", "Auth", "Payments"), {}) is None
    assert unify(x, ("f", x), {}) is None

    print()
    print("=== the proof, by hand ===")
    # step 6: R1 with F3
    s6 = unify(("delayed", y), ("delayed", "Auth"), {})
    c6 = frozenset(lit_subst(1, s6)
                  for l in R1 - {(False, "delayed", (y,))})
    print(" 6.", show(c6), " theta = {y/Auth}")
    # step 7: that with F2
    got = resolve(c6, F2, 1)
    c7 = min(got, key=len)
    print(" 7.", show(c7), " theta = {x/Payments}")
    assert c7 == frozenset({(True, "delayed", ("Payments",))})
    # step 8: R1 with step 7
    c8 = min((c for c in resolve(R1, c7, 2) if len(c) == 2), key=len)
    print(" 8.", show(c8), " theta = {y/Payments}")
    # step 9: that with F1
    c9 = min(resolve(c8, F1, 3), key=len)
    print(" 9.", show(c9), " theta = {x/Release47}")
    assert c9 == frozenset({(True, "delayed", ("Release47",))})
    print(" 10. [] -- contradicts the assumed -delayed(Release47)")
    print()
    print(" R1 was used three times, with three different unifiers.")
    print(" That is exactly what propositional logic cannot do.")

    print()
    print("=== finding it automatically ===")
    ok1, n1 = refute_naive(KB, QUERY)
    ok2, n2 = refute_sos(KB, QUERY)
    print(" naive saturation : proved %s in %4d resolution steps"
          % (ok1, n1))
    print(" set of support : proved %s in %4d resolution steps"
          % (ok2, n2))

```

```

print("    set of support is %.1fx cheaper" % (n1 / n2))
assert ok1 and ok2 and n2 < n1
print()
print("    Both find a five-line proof. Checking a proof is easy;")
print("    finding one is the hard part, and real provers add")
print("    subsumption and indexing on top of this.")

# and a query that does NOT follow
print()
print("=== a query that does not follow ===")
ok3, n3 = refute_sos(KB, (True, "delayed", ("Billing",)), rounds=4)
print("    delayed(Billing): proved %s after %d steps" % (ok3, n3))
assert not ok3
print("    No proof was found -- but first-order entailment is only")
print("    semi-decidable, so 'no proof yet' is not the same as")
print("    'no proof exists'. Chapter 11 could hand back a")
print("    countermodel here; in general this chapter cannot.")

```

Then remove the occurs check from `unify` — delete the two lines that call `occurs` — and re-run. The first two cases are unchanged and the third now “succeeds” with a binding that denotes no finite term. That is the bug, and the reason it is worth a name.

🔍 Checkpoint — Test Yourself

1. Write “every service has an owner” in first-order logic and convert it to clause form. Explain why the existential variable must become a function of x rather than a constant.
2. In the proof of Table 12.1, rule R1 is used three times. State how many propositional sentences would be needed to say the same thing over n services, and what happens when a service is added.
3. Chapter 11 could return a countermodel when a conclusion did not follow. Explain why this chapter generally cannot, and name the property of first-order entailment responsible.

📋 Chapter Summary

- First-order logic adds **terms** (constants, variables, functions), **predicates** and **quantifiers**. One quantified sentence replaces the n^2 propositional ones a general rule would otherwise need.
- Pair $\forall$ with $\Rightarrow$ and $\exists$ with $\wedge$. The other combinations say something much stronger, or nothing at all.
- **Unification** finds a substitution making two atoms identical, and the **most general** one commits to as little as possible. The **occurs check** refuses to bind a variable inside a term containing it; omitting it produces a hang, not a wrong answer.
- Clause form adds **standardising apart** and **skolemisation** to Chapter 11’s procedure. An existential inside a universal must become a *function* of it, or a rule about every service collapses into a claim about one.
- **Resolution** lifts to first-order logic by unifying the complementary literals. The proof is an auditable table: each line names its parents and its unifier.
- Checking a proof is easy; finding one is not. Naive saturation took 705 steps for a five-line proof; **set of support** reduced it to 202, and real provers add **subsumption** and indexing.
- First-order entailment is only *semi-decidable*. Failing to find a proof is not evidence that none exists, and the countermodel of Chapter 11 is generally unavailable.

 Review Questions

1. **(MCQ)** The occurs check prevents: (a) two constants unifying (b) binding a variable to a term containing it (c) skolemisation (d) variable capture between clauses.
2. **(MCQ)** $\forall x \text{ service}(x) \wedge \text{monitored}(x)$ claims that: (a) every service is monitored (b) everything is a monitored service (c) some service is monitored (d) nothing is monitored.
3. **(MCQ)** An existential variable inside a universal quantifier skolemises to: (a) a new constant (b) a new function of the universally quantified variables (c) the same variable (d) a predicate.
4. **(MCQ)** First-order entailment is: (a) decidable (b) semi-decidable (c) undecidable in both directions (d) decidable only for finite domains.
5. **(Short)** Explain why clauses must be standardised apart before resolution, and describe the symptom when they are not.
6. **(Short)** Give the most general unifier of $\text{owns}(x, \text{svc}(y))$ and $\text{owns}(\text{Ayesha}, \text{svc}(\text{Auth}))$, and say what a less general unifier would throw away.
7. **(Short)** Set of support cut the search from 705 steps to 202. State the restriction it imposes and why it does not lose completeness.
8. **(Applied)** Add the fact $\text{depends}(\text{Billing}, \text{Payments})$ to the knowledge base and prove $\text{delayed}(\text{Billing})$ by resolution. Present the proof as a table with parents and unifiers, and say which earlier derived clause you were able to reuse.

Rule-Based Expert Systems

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part IV — Knowledge, Logic and Reasoning

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Build** a production system: working memory, a rule base and an inference engine running the match–resolve–act cycle.
- **Apply** forward and backward chaining, and **choose** between them for a given task.
- **Resolve** conflicts between applicable rules, and **justify** the strategy chosen.
- **Produce** an explanation of a conclusion, and **argue** why it is the artefact an auditor accepts.
- **Describe** the knowledge acquisition bottleneck and brittleness, and **say** what they did to the expert-system industry.

🔗 Before You Start

Chapter 11 and Chapter 12 (inference, Horn clauses, chaining as resolution under a restriction) and Chapter 10 (the representation the rules act on). This chapter is where Part IV becomes a product.

📖 Key Terms in This Chapter

production rule • working memory • rule base • inference engine • match–resolve–act • forward chaining • backward chaining • conflict set • conflict resolution • specificity • recency • refraction • explanation • knowledge acquisition bottleneck • brittleness • graceful degradation

13.1 The Escalation Adviser

Chapter 2 introduced it and deferred it: at 21:00 the queue holds three hundred open tickets, one engineer is on call, and something must decide what to do with each one. The decision depends on severity, customer tier, whether there is a workaround, how much of the service level agreement is left, whether the service owner is reachable, and how often the on-call engineer has already been woken.

Nobody can write that as a formula, and everybody in the operations team can state the individual rules. That combination — many small, separately intelligible conditions and no closed form — is exactly what a **rule-based expert system** is for, and it is still the commonest deployed form of symbolic AI.

📋 Production system

A **production system** has three parts:

- a **working memory** of facts currently believed;
- a **rule base** of **production rules**, each a condition–action pair, conventionally written IF conditions THEN conclusion;

- an **inference engine** that repeatedly runs the **match–resolve–act** cycle: find every rule whose conditions hold (the **conflict set**), choose one, and apply its conclusion to working memory.

The rules do not call each other. Each fires when the facts happen to satisfy it, so a rule can be added or corrected without anyone tracing control flow — which is the property that lets a domain expert maintain the system.

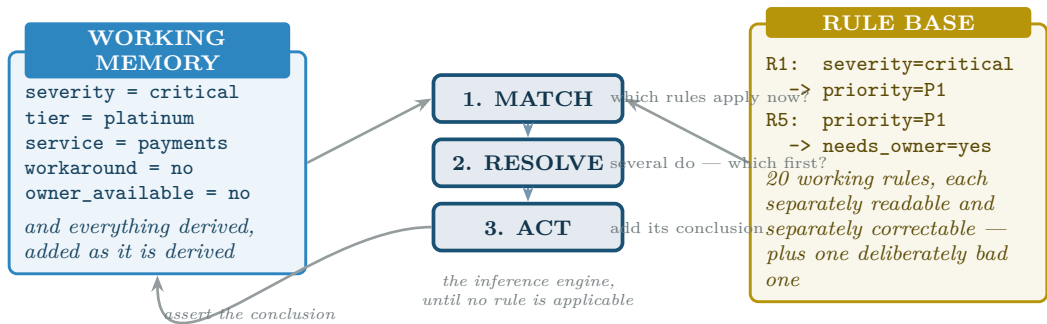


Figure 13.1: The architecture. Working memory grows as conclusions are asserted, which makes further rules applicable, and the cycle repeats until nothing new can be derived.

13.2 Forward Chaining

Start from the facts and derive everything that follows. This is **forward chaining**, and it is *data-driven*: the arrival of a new fact triggers whatever it triggers.

On ticket T-4471 — critical, platinum, payments, under two hours of service level agreement left, owner unreachable, on-call budget available — the engine fires nine rules in ten cycles, testing 188 conditions, and reaches **action = wake_on_call**. Along the way it also derives that the account team must be notified, that finance must be notified because the service is payments, that the payments incident runbook applies, and that the page is urgent — none of which anybody asked about.

That is the character of forward chaining. It is the right choice for monitoring and alerting, where you want every consequence of an arriving fact. It is the wrong choice when you have one question, because it derives a great deal you did not ask for.

13.3 Backward Chaining

Start from the question and work backwards to the facts. To establish a goal, find a rule concluding it and recursively establish that rule’s conditions.

Asked only *why is T-4471 escalated?*, backward chaining visits six goals against forward chaining’s nine firings, and — far more importantly — it produces a proof tree rather than a set of conclusions.

	Forward chaining	Backward chaining
Driven by	the data: a new fact triggers rules	the goal: a question triggers subgoals
Derives	everything that follows	only what bears on the question
On T-4471	9 rules fired, 188 condition tests	6 goals visited

	Forward chaining	Backward chaining
Natural output	a set of conclusions	a proof tree
Good for	monitoring, alerting, updating a dashboard	diagnosis, advice, answering “why?”
Wasteful when	you have one specific question	you want every consequence

Table 13.1: The two directions. Most deployed systems use both: forward chaining to maintain state, backward chaining to answer questions about it.

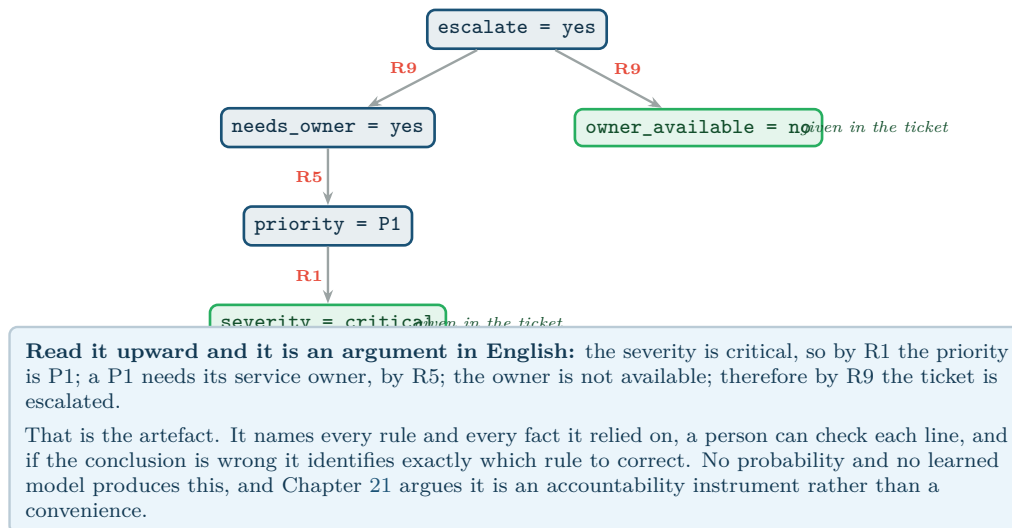


Figure 13.2: The proof tree for *why is T-4471 escalated?* Rounded green boxes are facts from the ticket; blue boxes are derived, each labelled with the rule that derived it.

13.4 Conflict Resolution

At most cycles several rules apply at once, and the order changes the outcome — sometimes only in efficiency, sometimes in the conclusion. Choosing is **conflict resolution**, and the standard strategies are:

- **Specificity** — prefer the rule with more conditions. A rule that mentions *major severity and platinum tier* is about a narrower situation than one mentioning *major severity* alone, and the narrower rule is the one the expert meant to apply. This is the workhorse.
- **Recency** — prefer the rule matching the most recently added facts, which keeps the system on one line of reasoning instead of drifting.
- **Refraction** — never fire the same rule on the same facts twice. Not an optimisation: without it the engine loops forever.
- **Explicit priority** — a numeric salience on each rule. Always available, and a warning sign: if a rule base needs many priorities, the rules are probably overlapping in ways nobody has thought through.

⚠ Specificity is worth a night's sleep

The rule base contains one deliberately over-general rule, R0: *a major ticket is P1*. It is listed first.

Ticket T-4472 is major severity on a *standard*-tier customer with a workaround. Under plain rule-order resolution the engine reaches R0 first, makes the ticket P1, and ends at `wake_on_call` — an engineer woken at three in the morning for a ticket that has a workaround. Under specificity the two-condition R3 is preferred, the ticket is P2, and the action is `leave_in_queue`.

Same rules, same facts, different conflict-resolution strategy, and the difference is somebody's night. That is why a rule base is not a list of IF statements: a list has the order whoever typed it chose, while specificity derives the order from how specific each rule's claim is — which is a property of the knowledge rather than of the file.

13.5 What Conflict Resolution Does Not Fix

Now run the two directions against each other on that same ticket, and something uncomfortable happens.

Forward chaining, with specificity, concludes `escalate = no`. Backward chaining, asked to prove `escalate = yes`, *succeeds* — via R5 from *priority = P1*, which it obtains from R0. The two directions disagree about the same ticket in the same rule base.



Backward chaining has no conflict set. It asks “is there *any* rule concluding this goal?”, and R0 is one. Forward chaining never used R0 only because specificity reached R3 first. So specificity was not repairing the rule base; it was *masking* a bad rule. R0 asserts that every major ticket is P1, which is simply false, and no conflict-resolution strategy makes a false rule true — it merely arranges for something else to be consulted first, on the inputs you happened to test.

The fix is to delete R0, whose work is already done correctly by R2 and R3. With it gone the two directions agree, and the lab checks that they do.

The general lesson is worth carrying beyond rule engines: *when two correct procedures give different answers from the same knowledge, the knowledge is inconsistent*. Do not tune the procedure. And note how the inconsistency was found — not by inspection, but by asking the same question two ways and comparing. That is a cheap and underused test for any rule base.

13.6 Why the Industry Collapsed

Rule engines work. MYCIN diagnosed bacterial infections in the 1970s at the level of faculty specialists; XCON configured VAX systems and saved Digital tens of millions of dollars a year. By the mid-1980s there was a substantial commercial industry, and by 1993 it had collapsed. Two causes, and both are still live.

The knowledge acquisition bottleneck. Every rule has to be elicited from an expert, by an engineer, one at a time. Experts are busy, and — worse — they are frequently unable to state what they do: the knowledge is compiled into judgement. A thousand-rule system represents years of interviews and cannot be assembled any faster. Meanwhile, nothing in the system learns; it fails Chapter 1’s fourth test and it will still be failing it next year.

Brittleness. A rule base is confident inside its competence and gives no signal at the edge. Present a ticket whose situation nobody anticipated and the system does not degrade gracefully — it either fires nothing, or fires a rule whose conditions technically match and whose conclusion is absurd. A human specialist says “I have not seen this before”; a rule base has no representation of its own boundary.



The historical lesson is the one from Chapter 1: what failed was a claim about *reach*, not the method. Rule engines are running in production today in every bank, insurer and operations centre, generally under a name like “business rules engine” or “decision tables”, and generally with exactly the architecture of Figure 13.1.

And the two failures are precisely the two strengths of what came next. Machine learning acquires its knowledge from data instead of from interviews, which dissolves the bottleneck; and it degrades smoothly rather than falling off a cliff. It gives up the explanation tree of Figure 13.2 to do so. That trade — explicability for scale — is the central tension of Chapter 21, and it is worth noticing that this course has now built both sides of the argument.

📅 Worked Example: One ticket, two budgets, two different actions

Tickets T-4471 and T-4473 differ in only one fact, and the difference is not about the ticket at all.

T-4471: critical, platinum, payments, under two hours left, owner unavailable, `wake_budget = available`.

The engine fires, in order: R17 (critical and platinum, so notify the account team), R1 (critical, so P1), R18 (payments and P1, so notify finance), R5 (P1, so needs the owner), R9 (needs the owner, who is unavailable, so escalate), R15 (escalate with budget available, so *wake the on-call engineer*), R19 (escalate on payments, so the payments runbook applies), R20 (waking someone for a platinum customer, so the page is urgent), R12 (under two hours, so the risk is high). Nine rules, and the answer is `wake_on_call`.

T-4473: major, platinum, auth, over two hours left, owner unavailable, `wake_budget = spent`.

R2 (major and platinum, so P1 — note the two-condition rule, chosen by specificity over the one-condition major rule), R5 (P1, so needs the owner), R9 (escalate), R14 (escalate but *the budget is spent*, so `queue_for_morning`), R13 (over two hours, so risk is low). Five rules, and the answer is `queue_for_morning`.

What to notice. Both tickets are escalated — `escalate = yes` in both working memories. They receive different *actions* because R14 and R15 consult a fact about the *engineer*, not about the ticket. That is exactly the internal state Chapter 2 showed a model-based agent must maintain, and here it is one more fact in working memory and one more pair of rules.

And notice what it costs to change. Suppose the operations lead decides that platinum customers may wake the engineer even past the budget. That is one new rule, with three conditions so specificity puts it ahead of R14, and *no existing rule is touched*. Compare making the same change inside nested `if` statements, where the modification is entangled with everything around it and nobody can be sure what else moved.

⚠️ Common Pitfalls

- **Omitting refraction.** The engine re-fires the same rule on the same facts forever. The symptom is a hang on the first realistic input.
- **Relying on rule order.** If the answer changes when the rules are rearranged, the rule base is under-specified; add the missing condition rather than reordering.
- **Forward chaining to answer one question.** It derived nine conclusions where six goals would have done, and none of the extras was asked for.
- **Treating explanation as a feature to add later.** The proof tree is a by-product of backward chaining and nearly impossible to bolt on afterwards. It is also the reason the system will be allowed into production.
- **Many explicit priorities.** A salience number is a patch over overlapping rules. More than a handful means the rule base needs restructuring, not more numbers.
- **Expecting graceful degradation.** At the edge of its competence a rule base is confidently wrong rather than uncertain, and it will not tell you which.

🔧 Try It Yourself: A Twenty-Rule Escalation Shell That Explains Itself

Forward chaining with conflict resolution, backward chaining with an explanation tree, and the measurement of what each costs.

```
"""Chapter 13 lab -- a rule-based escalation adviser.
```

```
Twenty rules an operations lead could read and correct. The engine
```

```

fires until nothing new follows; the explanation tree is what makes
the result auditable, and it is a by-product of backward chaining
rather than something added afterwards.
"""

# (name, [conditions], conclusion); each is an (attribute, value) pair
RULES = [
    # R0 is a deliberately general fallback, and it is listed FIRST.
    # Plain rule-order resolution therefore reaches it before the
    # narrower R2 and R3, and gets the wrong answer. Specificity does
    # not, which is the point of the comparison below.
    ("R0", [("severity", "major")], ("priority", "P1")),
    ("R1", [("severity", "critical")], ("priority", "P1")),
    ("R2", [("severity", "major"), ("tier", "platinum")],
     ("priority", "P1")),
    ("R3", [("severity", "major"), ("tier", "standard")],
     ("priority", "P2")),
    ("R4", [("severity", "minor")], ("priority", "P3")),
    ("R5", [("priority", "P1")], ("needs_owner", "yes")),
    ("R6", [("priority", "P2"), ("workaround", "no")],
     ("needs_owner", "yes")),
    ("R7", [("priority", "P2"), ("workaround", "yes")],
     ("needs_owner", "no")),
    ("R8", [("priority", "P3")], ("needs_owner", "no")),
    ("R9", [("needs_owner", "yes"), ("owner_available", "no")],
     ("escalate", "yes")),
    ("R10", [("priority", "P1"), ("sla_risk", "high")],
     ("escalate", "yes")),
    ("R11", [("needs_owner", "no")], ("escalate", "no")),
    ("R12", [("sla_hours_left", "under2")], ("sla_risk", "high")),
    ("R13", [("sla_hours_left", "over2")], ("sla_risk", "low")),
    ("R14", [("escalate", "yes"), ("wake_budget", "spent")],
     ("action", "queue_for_morning")),
    ("R15", [("escalate", "yes"), ("wake_budget", "available")],
     ("action", "wake_on_call")),
    ("R16", [("escalate", "no")], ("action", "leave_in_queue")),
    ("R17", [("severity", "critical"), ("tier", "platinum")],
     ("notify_account_team", "yes")),
    ("R18", [("service", "payments"), ("priority", "P1")],
     ("notify_finance", "yes")),
    ("R19", [("escalate", "yes"), ("service", "payments")],
     ("runbook", "payments_incident")),
    ("R20", [("action", "wake_on_call"), ("tier", "platinum")],
     ("page_severity", "urgent")),
]

TICKETS = {
    "T-4471": {"severity": "critical", "tier": "platinum",
               "service": "payments", "workaround": "no",
               "sla_hours_left": "under2", "owner_available": "no",
               "wake_budget": "available"},
    "T-4472": {"severity": "major", "tier": "standard",
               "service": "search", "workaround": "yes",
               "sla_hours_left": "over2", "owner_available": "no",
               "wake_budget": "available"},
    "T-4473": {"severity": "major", "tier": "platinum",
               "service": "auth", "workaround": "no",
               "sla_hours_left": "over2", "owner_available": "no",
               "wake_budget": "spent"},
}

```

```

class Trace:
    def __init__(self):
        self.fired = []
        self.tests = 0
        self.cycles = 0

def matches(rule, wm, t):
    for (attr, val) in rule[1]:
        t.tests += 1
        if wm.get(attr) != val:
            return False
    return True

def forward_chain(facts, resolve="specificity", rules=None):
    """Match, resolve, act -- until nothing new follows.

    The 'fired' set is REFRACTION: without it the engine re-fires the
    same rule on the same facts forever.

    'r[2][0] not in wm' means each attribute is DECIDED ONCE, by the
    rule conflict resolution picks. Without it a less specific rule
    that fires later would quietly overwrite a more specific rule's
    conclusion, and conflict resolution would be pointless.
    """
    rules = RULES if rules is None else rules
    wm, t, fired = dict(facts), Trace(), set()
    while True:
        t.cycles += 1
        conflict = [r for r in rules
                    if r[0] not in fired
                    and matches(r, wm, t)
                    and r[2][0] not in wm]

        if not conflict:
            return wm, t
        if resolve == "specificity":
            # more conditions first; ties broken by rule order
            conflict.sort(key=lambda r: (-len(r[1]), rules.index(r)))
        else:
            conflict.sort(key=lambda r: rules.index(r))
        r = conflict[0]
        wm[r[2][0]] = r[2][1]
        fired.add(r[0])
        t.fired.append(r[0])

def backward_chain(goal, facts, seen=None):
    """Prove an (attribute, value) goal. Returns a proof tree."""
    if seen is None:
        seen = set()
    if facts.get(goal[0]) == goal[1]:
        return ("fact", goal)
    if goal in seen:
        return None  # a cycle in the rule base
    seen = seen | {goal}
    for r in RULES:
        if r[2] != goal:
            continue
        subs = []
        for c in r[1]:
            got = backward_chain(c, facts, seen)

```

```

        if got is None:
            subs = None
            break
        subs.append(got)
    if subs is not None:
        return ("rule", r[0], goal, subs)
    return None

def explain(tree, indent=0):
    pad = " " * indent
    if tree[0] == "fact":
        return ["%s%s = %s (given in the ticket)"
                % (pad, tree[1][0], tree[1][1])]
    _, name, goal, subs = tree
    out = ["%s%s = %s (by %s, because)" % (pad, goal[0], goal[1], name)]
    for s in subs:
        out += explain(s, indent + 1)
    return out

def goals_visited(goal, facts, seen=None, count=None):
    """How much work backward chaining does for ONE question."""
    if count is None:
        count = [0]
    count[0] += 1
    if seen is None:
        seen = set()
    if facts.get(goal[0]) == goal[1]:
        return True, count[0]
    if goal in seen:
        return False, count[0]
    seen = seen | {goal}
    for r in RULES:
        if r[2] != goal:
            continue
        if all(goals_visited(c, facts, seen, count)[0] for c in r[1]):
            return True, count[0]
    return False, count[0]

if __name__ == "__main__":
    print("rule base: %d rules" % len(RULES))
    print()
    print("ticket    action                rules  tests  cycles")
    print("-" * 52)
    results = {}
    for tid, facts in TICKETS.items():
        wm, t = forward_chain(facts)
        results[tid] = (wm, t)
        print("%-9s %-19s %5d %6d %7d"
              % (tid, wm.get("action"), len(t.fired), t.tests, t.cycles))

    # the three actions differ, and only one fact separates the last two
    assert results["T-4471"][0]["action"] == "wake_on_call"
    assert results["T-4472"][0]["action"] == "leave_in_queue"
    assert results["T-4473"][0]["action"] == "queue_for_morning"
    print()
    print("T-4471 firing order:", " ".join(results["T-4471"][1].fired))
    print("T-4473 firing order:", " ".join(results["T-4473"][1].fired))
    print()
    print("Both are escalated; they differ only in wake_budget, and")

```

```

print("that is a fact about the ENGINEER, not about the ticket.")
assert results["T-4471"][0]["escalate"] == "yes"
assert results["T-4473"][0]["escalate"] == "yes"

# --- specificity is doing real work -----
print()
print("=== specificity against plain rule order ===")
print("ticket      specificity      rule order")
print("-" * 46)
for tid, facts in TICKETS.items():
    a, _ = forward_chain(facts, "specificity")
    b, _ = forward_chain(facts, "order")
    flag = " <-- DIFFER" if a["action"] != b["action"] else ""
    print("%-9s %-19s %-19s%s"
          % (tid, a["action"], b["action"], flag))

spec, _ = forward_chain(TICKETS["T-4472"], "specificity")
order, _ = forward_chain(TICKETS["T-4472"], "order")
assert spec["priority"] == "P2" and spec["action"] == "leave_in_queue"
assert order["priority"] == "P1" and order["action"] == "wake_on_call"
print()
print("T-4472 is major severity on a STANDARD tier customer.")
print("R0 ('major -> P1') is general and listed first, so plain")
print("rule order reaches it and wakes an engineer at 3 a.m. for")
print("a standard-tier ticket with a workaround. Specificity")
print("prefers the two-condition R3, gets P2, and leaves it in")
print("the queue. That is a real engineer's night, decided by the")
print("conflict-resolution strategy alone.")

# --- the explanation -----
print()
print("=== why is T-4471 escalated? ===")
tree = backward_chain(("escalate", "yes"), TICKETS["T-4471"])
assert tree is not None
for line in explain(tree):
    print("    " + line)

# --- the two directions DISAGREE, and that is a real bug -----
print()
print("=== and is T-4472 escalated? ===")
fwd_wm, _ = forward_chain(TICKETS["T-4472"])
up = backward_chain(("escalate", "yes"), TICKETS["T-4472"])
print("    forward chaining says escalate =", fwd_wm["escalate"])
print("    backward chaining proves escalate = yes:", up is not None)
assert fwd_wm["escalate"] == "no" and up is not None
print()
print("    They disagree. Backward chaining's proof:")
for line in explain(up):
    print("        " + line)
print()
print("    Backward chaining has no conflict resolution: it asks")
print("    'is there ANY rule concluding this?' and R0 is one.")
print("    Forward chaining never used R0 because specificity")
print("    preferred R3 -- so specificity was MASKING a bad rule,")
print("    not fixing it. R0 says every major ticket is P1, which")
print("    is simply false, and no resolution strategy makes a")
print("    false rule true.")

# the fix is to delete the over-general rule, not to tune the
# conflict-resolution strategy
print()
print("=== delete R0 and ask both directions again ===")

```

```

fixed = [r for r in RULES if r[0] != "R0"]
fwd2, _ = forward_chain(TICKETS["T-4472"], rules=fixed)
globals()["RULES"], saved = fixed, RULES
up2 = backward_chain(("escalate", "yes"), TICKETS["T-4472"])
down2 = backward_chain(("escalate", "no"), TICKETS["T-4472"])
globals()["RULES"] = saved
print("    forward says escalate =", fwd2["escalate"])
print("    backward proves escalate = yes:", up2 is not None)
assert fwd2["escalate"] == "no" and up2 is None and down2 is not None
print("    They now agree. The explanation:")
for line in explain(down2):
    print("        " + line)

# --- what each direction costs -----
fwd = len(results["T-4471"][1].fired)
_, bwd = goals_visited(("escalate", "yes"), TICKETS["T-4471"])
print()
print("forward chaining : %d rules fired to derive everything" % fwd)
print("backward chaining: %d goals visited to answer one question"
      % bwd)
assert bwd < fwd
print()
print("Forward chaining also concluded notify_finance, runbook and")
print("page_severity -- useful for a dashboard, wasted for a")
print("single question.")

# --- adding a rule changes nothing else -----
print()
print("=== the operations lead changes the policy ===")
extended = RULES + [("R21", [("escalate", "yes"),
                             ("wake_budget", "spent"),
                             ("tier", "platinum")],
                    ("action", "wake_on_call"))]
wm_new, t_new = forward_chain(TICKETS["T-4473"], rules=extended)
print("    'platinum may wake the engineer past the budget'")
print("    T-4473 action was queue_for_morning, now:",
      wm_new["action"])
print("    firing order:", " ".join(t_new.fired))
assert wm_new["action"] == "wake_on_call"
print("    One rule added. No existing rule was touched, and")
print("    specificity put it ahead of R14 automatically.")

```

Then remove the `fired` set from `forward_chain` — that is refraction — and re-run. The engine never terminates, because R1 is applicable forever once the severity is critical. It is worth seeing once.

🔍 Checkpoint — Test Yourself

1. T-4471 and T-4473 are both escalated and receive different actions. Name the single fact that separates them, say whether it is a fact about the ticket, and relate it to the agent architectures of Chapter 2.
2. Forward chaining fired nine rules on T-4471; backward chaining answered “why escalated?” in six goals. Give one task for which the nine is the right answer and the five is the wrong one.
3. An auditor asks why a particular customer’s ticket was left in the queue overnight. Explain what your system hands them, and why a model that merely output a probability would not satisfy the request.

Chapter Summary

- A **production system** is working memory, a rule base and an inference engine running **match–resolve–act**. Rules do not call each other, so one can be added or corrected without tracing control flow — which is what lets a domain expert maintain it.
- **Forward chaining** is data-driven and derives everything that follows: nine rules and 188 condition tests on one ticket. Right for monitoring; wasteful for a single question.
- **Backward chaining** is goal-driven, visits only what bears on the question — six goals here — and produces a **proof tree**.
- **Conflict resolution** chooses among applicable rules. **Specificity** is the workhorse and encodes a real property of the knowledge; **refraction** is not an optimisation but the thing that makes the engine terminate.
- The **explanation** is the deliverable. It names every rule and fact the conclusion rests on, a person can check it line by line, and it identifies which rule to correct when the answer is wrong.
- Conflict resolution can *mask* a bad rule without fixing it. Asking the same question forwards and backwards made the two directions disagree on T-4472 and exposed an over-general rule that specificity had been hiding. When two correct procedures differ on the same knowledge, the knowledge is inconsistent — fix the rules, not the strategy.
- The industry collapsed on the **knowledge acquisition bottleneck** — rules elicited one at a time from busy experts, and nothing learns — and on **brittleness**, confident wrongness at the edge of competence.
- Both failures are the strengths of machine learning, which acquires knowledge from data and degrades smoothly, and which gives up the explanation tree to do it. That trade is the subject of Chapter 21.

Review Questions

1. (MCQ) The conflict set is: (a) rules that contradict each other (b) rules whose conditions currently hold (c) facts that disagree (d) goals not yet proved.
2. (MCQ) Refraction prevents: (a) two rules firing at once (b) a rule firing twice on the same facts (c) backward chaining looping (d) conflicting conclusions.
3. (MCQ) Backward chaining is preferable when: (a) a new fact has arrived and you want its consequences (b) you have one specific question (c) working memory is large (d) the rules have many conditions.
4. (MCQ) The knowledge acquisition bottleneck is: (a) slow inference (b) too many rules to store (c) rules must be elicited from experts one at a time (d) working memory overflow.
5. (Short) Explain specificity as a conflict-resolution strategy and why it encodes something real about the knowledge rather than being an arbitrary tie-break.
6. (Short) Why is an explanation tree difficult to add to a system afterwards, and easy to obtain from backward chaining?
7. (Short) State the two causes of the expert-system collapse and, for each, say how machine learning addresses it and what is given up.
8. (Applied) Write a rule implementing “a P1 on a platinum customer with no workaround must notify the account team even if the owner is available”. Give its conditions and conclusion, say where specificity places it relative to R17, and trace what T-4473 would do with it added.

PART V

Reasoning Under Uncertainty

Part IV assumed everything it was told was true. Almost nothing a delivery team knows is true in that sense: the estimate is a guess, the cause of the defect is unobserved, and “high risk” has no crisp boundary. Part V is three different and incompatible ways of being precise about not knowing — and the chapter on making a decision anyway.

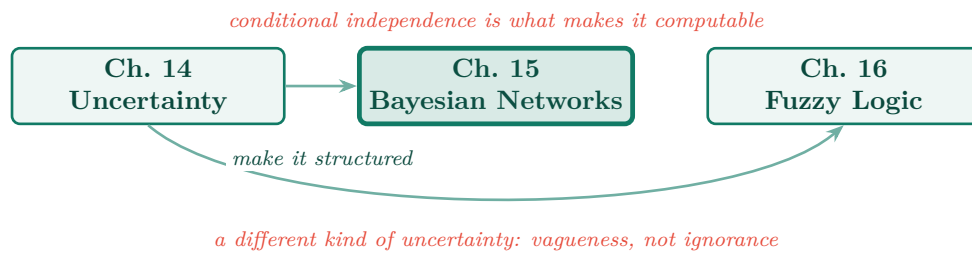


Figure P5.1: Reading path through Part V. Chapter 16 branches from Chapter 14 rather than following Chapter 15: fuzzy logic answers a different question, and confusing the two is the standard error.

Uncertainty and Decision Making

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part V — Reasoning Under Uncertainty

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** why logic alone cannot represent what a delivery team knows, and **state** probability as a degree of belief.
- **Apply** Bayes' rule to update a belief on evidence, and **avoid** the base-rate fallacy.
- **Use** conditional independence to reduce the size of a joint distribution.
- **Choose** an action by maximising expected utility, and **distinguish** a change of belief from a change of decision.
- **Compute** the value of perfect information, and **say** when gathering evidence is worth paying for.

🔗 Before You Start

Chapter 11 (what a truth value can and cannot express) and Chapter 2 (the utility-based agent, and rationality as maximising *expected* performance — this chapter is where that phrase acquires a number). School probability; nothing beyond it is assumed.

📖 Key Terms in This Chapter

degree of belief • prior • posterior • likelihood • base rate • joint distribution • marginalisation • Bayes' rule • conditional independence • utility • expected utility • maximum expected utility • decision table • value of perfect information

14.1 What Logic Cannot Say

Chapter 11 settled the release gate because everything it was told was either true or false. Almost nothing a delivery team knows is like that.

This release probably has a defect. The smoke test usually catches this kind of problem. Auth is normally the culprit when payments is slow. Every one is actionable, none is a truth value, and writing any of them as a logical sentence forces a choice between two wrong options: assert it and be committed to something false, or omit it and reason without what the team knows.

Chapter 10's defaults were one answer — “normally”, withdrawn on evidence. They work, and they cannot be combined: two defaults pointing different ways gave Figure 10.2's conflict, with no principled resolution. Probability is the other answer, and it is the one that composes.

📖 Probability as degree of belief

$P(a) \in [0, 1]$ is an agent's **degree of belief** in proposition a given what it knows — not a claim about long-run frequency, and not a property of the world. $P(\text{defect}) = 0.1$ says this agent, with this evidence, would accept either side of a bet at those odds.

A **prior** $P(a)$ is the belief before evidence; a **posterior** $P(a | b)$ the belief after. The definition of conditional probability,

$$P(a | b) = \frac{P(a \wedge b)}{P(b)},$$

is the whole apparatus, and everything else in this chapter follows from it.

Unlike defaults, degrees of belief *combine*. Two pieces of evidence pointing different ways produce a number between them rather than a conflict, and the combination rule is forced — not chosen by whoever wrote the system.

14.2 Bayes' Rule

Rearranging the definition gives the tool:

$$P(a | b) = \frac{P(b | a) P(a)}{P(b)}$$

It matters because the two conditionals are not interchangeable and the available one is usually the wrong way round. A test report says *how often the test fires when there is a defect* — $P(\text{alarm} | \text{defect})$ — and what you need is *how likely a defect is, given that it fired* — $P(\text{defect} | \text{alarm})$. Confusing the two is the **base-rate fallacy**, and it is the single most consequential probabilistic error made in engineering organisations.

✎ The smoke test

The team's smoke test is a good test:

- It fires on 90% of releases that really do have a serious defect: $P(\text{alarm} | \text{defect}) = 0.9$.
- It fires on 20% of releases that do not: $P(\text{alarm} | \neg \text{defect}) = 0.2$.
- Historically one release in ten has a serious defect: $P(\text{defect}) = 0.1$.

The test has just fired. How worried should you be?

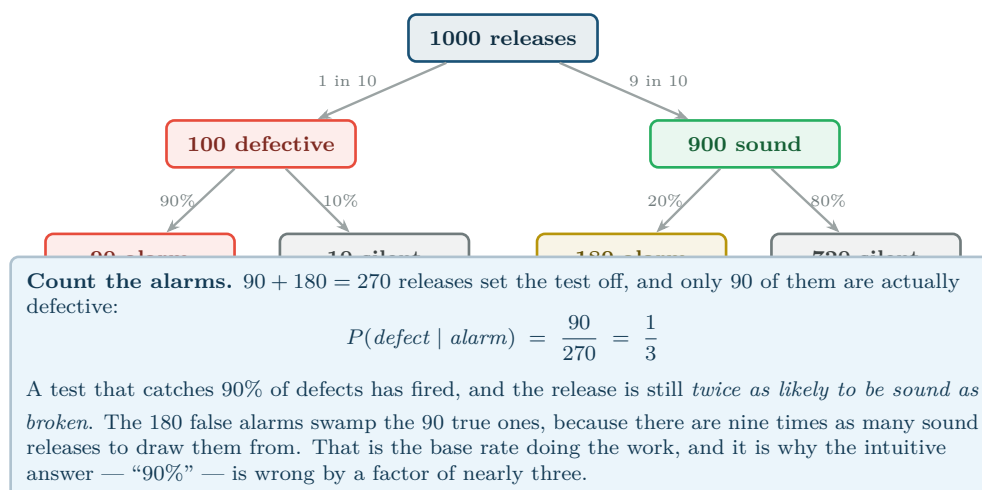


Figure 14.1: The same calculation as Bayes' rule, in whole releases rather than fractions. Natural frequencies make the base-rate effect visible, which is why this is the form to use when explaining a result to a team.

Silence is far more informative than an alarm here: $P(\text{defect} | \neg \text{alarm}) = 10/730 = 1/73 \approx 0.014$. A test with a high false-alarm rate tells you much more by staying quiet.

14.3 Conditional Independence

A **joint distribution** over n binary variables has 2^n entries, which is Chapter 11's problem again. The escape is the same idea that made semantic networks tractable: most things are irrelevant to most other things.

Conditional independence

a and b are **conditionally independent given** c when

$$P(a \wedge b \mid c) = P(a \mid c) P(b \mid c)$$

Equivalently $P(a \mid b \wedge c) = P(a \mid c)$: once c is known, b says nothing further about a .

The smoke test and a customer complaint are both caused by the defect, and both are evidence for it — but once you *know* whether there is a defect, the complaint tells you nothing more about whether the test will fire. That is a conditional independence, and asserting it replaces a table of $2^3 = 8$ numbers with three small ones. Chapter 15 makes this the organising principle of a whole representation.

14.4 From Belief to Decision

A belief is not yet an action. The team does not need to know how likely the defect is; they need to know whether to ship.

Expected utility and the principle of maximum expected utility

A **utility** $U(s)$ is a number expressing how much outcome s is worth. The **expected utility** of action a is

$$EU(a) = \sum_s P(s \mid a) U(s)$$

and a rational agent chooses $a^* = \arg \max_a EU(a)$. This is the precise form of the definition of rationality given informally in Chapter 2.

Ship now or delay a week

Four outcomes, with the team's own numbers:

Action	If there is a defect	If there is not
Ship now	−100 incident, rollback, reputation	+40 a week of revenue earned early
Delay a week	−5 caught it; a small slip	−10 a week lost for nothing

Note that delaying is never *good* — both its outcomes are negative. It is insurance, and insurance always costs something.

With $p = P(\text{defect})$:

$$EU(\text{ship}) = -100p + 40(1 - p) = 40 - 140p$$

$$EU(\text{delay}) = -5p - 10(1 - p) = 5p - 10$$

They cross where $40 - 140p = 5p - 10$, that is at

$$p^* = \frac{50}{145} = \frac{10}{29} \approx 0.345$$

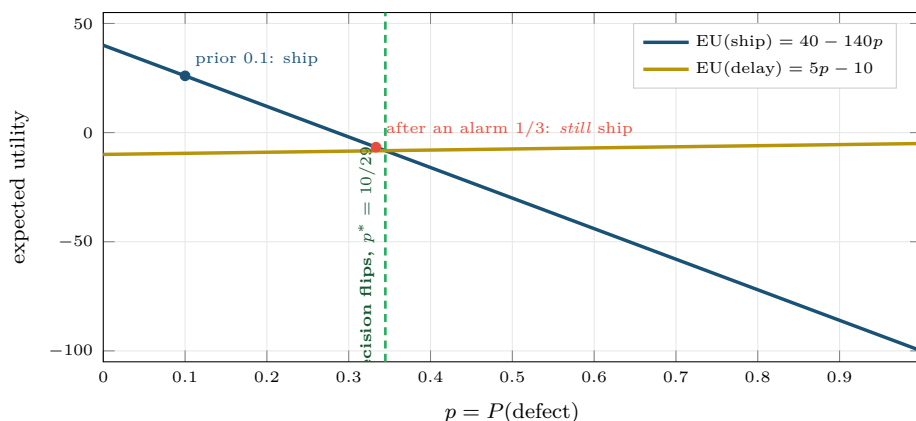


Figure 14.2: The two expected utilities as the probability of a defect varies. The alarm moves the belief a long way and leaves the decision alone, because $1/3$ falls just short of the crossing at $10/29$.

14.5 A Change of Belief Is Not a Change of Decision

This is the chapter’s most useful idea, and Figure 14.2 contains it.

The alarm fires. Belief in a defect more than triples, from 0.1 to $1/3$. And the recommendation does not change: at $p = 1/3$, $EU(\text{ship}) = -6.67$ against $EU(\text{delay}) = -8.33$, so shipping is still — barely — the better bet. The flip comes at $10/29 \approx 0.345$, and $1/3 \approx 0.333$ falls just short.

Two consequences worth taking seriously.

Do not report beliefs to decision makers as though they were decisions. An engineer who says “the probability of a defect has tripled” has said something true and nearly useless. The question is which side of p^* the number falls on.

Attend to the margin. At $p = 1/3$ the two options differ by 1.67 utility units, which is noise. The honest report is not “ship” but “ship, marginally, and this is now close enough that better evidence would be worth having” — which is exactly what the next section makes precise.

14.6 The Value of Information

If the decision is nearly balanced, finding out is worth a lot. If it is not, it is worth nothing at all, however interesting the answer.

Value of perfect information

The **value of perfect information** about a variable is the increase in expected utility from learning it before deciding:

$$\text{VPI} = \left[\sum_s P(s) \max_a EU(a \mid s) \right] - \max_a EU(a)$$

the first term being what you could achieve choosing the best action for each case once you know which case you are in. It is never negative, and it is an upper bound on what the evidence is worth paying for.

For the release decision, $\text{VPI}(p) = 95p$ below the flip point and $50 - 50p$ above it. Evaluated at the two beliefs of interest:

Situation	$P(\text{defect})$	Decision	Value of perfect information
Before any test	0.1	ship	9.50
After the test fires	1/3	ship (marginally)	31.67
After the test is silent	1/73	ship	1.30
At the flip point	10/29	indifferent	32.76 (the maximum)

Table 14.1: Information is worth most exactly where the decision is hardest, and nothing at all where it is easy.

That shape is the point. Before any evidence, certainty would be worth 9.50. After an ambiguous alarm — which changed nothing about what to do — it is worth 31.67, more than three times as much. Information is valuable in proportion to how close the decision is to flipping, not in proportion to how interesting it is.



This is a usable rule for engineering organisations, and it cuts both ways.

Stop gathering evidence that cannot change the decision. If the answer is “ship” at every plausible value of p , another day of investigation has a value of zero however much everyone would like to know.

Gather evidence hard when the decision is balanced. The correct response to “it is basically a coin flip” is not to flip the coin but to notice that this is precisely the circumstance in which finding out pays best.

And notice that both follow from arithmetic rather than from temperament. This is what it means for an agent to be rational about its own information gathering — the third thing Chapter 2 said rationality does not forbid.

Worked Example: Working the alarm through, end to end

Step 1: the posterior. With $P(d) = 0.1$, $P(al | d) = 0.9$, $P(al | \neg d) = 0.2$:

$$P(al) = 0.9 \times 0.1 + 0.2 \times 0.9 = 0.09 + 0.18 = 0.27$$

$$P(d | al) = \frac{0.9 \times 0.1}{0.27} = \frac{0.09}{0.27} = \frac{1}{3}$$

Check against Figure 14.1: 90 true alarms out of 270. The same number, and the frequency version is the one to put in front of the team.

Step 2: the decision.

$$EU(\text{ship}) = 40 - 140 \times \frac{1}{3} = 40 - 46.67 = -6.67$$

$$EU(\text{delay}) = 5 \times \frac{1}{3} - 10 = -8.33$$

Ship, by 1.67. Both options are now bad — the alarm has cost the team something whatever they do — but shipping is less bad.

Step 3: is more evidence worth buying? With perfect information we would delay when there is a defect (−5 rather than −100) and ship when there is not (+40 rather than −10):

$$\text{with information} = \frac{1}{3}(-5) + \frac{2}{3}(40) = -1.67 + 26.67 = 25.00$$

$$VPI = 25.00 - (-6.67) = 31.67$$

So certainty is worth 31.67 units — against the 10 units that delaying a week costs when the release turns out to be sound. *An investigation costing less than a week's delay is clearly worth doing*, and that conclusion came from the numbers rather than from anybody's instinct.

Step 4: the sensitivity that matters. Suppose the team disputes the -100 . Redo the flip point with shipping a defect costing -60 : $EU(\text{ship}) = 40 - 100p$, which meets $5p - 10$ at $p^* = 50/105 \approx 0.476$. The alarm's $1/3$ is now far from flipping anything, and the value of information falls accordingly. *The utilities matter as much as the probabilities*, and they are the part nobody measures — which is the honest weakness of this whole apparatus and the reason to write them down where they can be argued with.

! Common Pitfalls

- **The base-rate fallacy.** $P(\text{alarm} \mid \text{defect}) = 0.9$ does not make $P(\text{defect} \mid \text{alarm})$ anything like 0.9. Here it is $1/3$. Draw the natural-frequency tree.
- **Reporting a belief as though it were a decision.** “The probability tripled” is true and does not say what to do. Say which side of p^* it falls.
- **Ignoring the margin.** At $p = 1/3$ the options differ by 1.67 units. Report the closeness, not only the winner.
- **Gathering evidence that cannot change the decision.** If the action is the same across every plausible probability, the value of information is zero.
- **Leaving utilities implicit.** They decide the answer as much as the probabilities do, and unlike the probabilities nobody measures them. Write them down so they can be disputed.
- **Assuming independence to make the arithmetic easy.** Two symptoms of the same cause are *not* independent, and treating them as such double-counts the evidence. They may be conditionally independent *given* the cause, which is a different claim — and the one Chapter 15 is built on.

🔧 Try It Yourself: Go/No-Go Under Uncertainty

Bayes' rule with exact arithmetic, the decision, the flip point and the value of information. Every number the chapter quotes is computed here.

```
"""Chapter 14 lab -- ship now or delay a week.

Fractions throughout, so every printed number can be checked by hand
against the natural-frequency tree of Figure 14.1.
"""

from fractions import Fraction as F

# --- the smoke test -----
P_DEFECT = F(1, 10)          # one release in ten has a serious defect
SENS = F(9, 10)              # P(alarm | defect)
FPR = F(2, 10)                # P(alarm | no defect)

# --- what each outcome is worth -----
UTIL = {
    ("ship", "defect"): -100,  # incident, rollback, reputation
    ("ship", "ok"): 40,        # a week of revenue earned early
    ("delay", "defect"): -5,    # caught it; a small slip
    ("delay", "ok"): -10,      # a week lost for nothing
}
ACTIONS = ("ship", "delay")
```

```

def posterior(prior, sens, fpr, alarm=True):
    """Bayes' rule. P(defect | alarm) or P(defect | no alarm)."""
    like_d = sens if alarm else 1 - sens
    like_ok = fpr if alarm else 1 - fpr
    return (like_d * prior) / (like_d * prior + like_ok * (1 - prior))

def eu(action, p):
    return UTIL[(action, "defect")] * p + UTIL[(action, "ok")] * (1 - p)

def best(p):
    a = max(ACTIONS, key=lambda x: eu(x, p))
    return a, eu(a, p)

def vpi(p):
    """Value of perfect information about the defect."""
    informed = (p * max(UTIL[(a, "defect")] for a in ACTIONS)
                + (1 - p) * max(UTIL[(a, "ok")] for a in ACTIONS))
    return informed - best(p)[1]

def flip_point():
    """Solve EU(ship) = EU(delay) for p, exactly.

    EU(ship) = Ud*p + Uo*(1-p) = Uo + (Ud-Uo)p
    EU(delay) = Vd*p + Vo*(1-p) = Vo + (Vd-Vo)p
    """
    ud, uo = UTIL[("ship", "defect")], UTIL[("ship", "ok")]
    vd, vo = UTIL[("delay", "defect")], UTIL[("delay", "ok")]
    return F(vo - uo, (ud - uo) - (vd - vo))

if __name__ == "__main__":
    print("=== the smoke test ===")
    print("P(defect) = ", P_DEFECT)
    print("P(alarm | defect) = ", SENS)
    print("P(alarm | no defect) = ", FPR)

    post = posterior(P_DEFECT, SENS, FPR, alarm=True)
    quiet = posterior(P_DEFECT, SENS, FPR, alarm=False)
    print()
    print("P(defect | alarm) = ", post, "=", round(float(post), 4))
    print("P(defect | silence) = ", quiet, "=", round(float(quiet), 4))
    assert post == F(1, 3)

    # the natural-frequency check of Figure 14.1
    n = 1000
    defective = n * P_DEFECT # 100
    true_alarms = defective * SENS # 90
    false_alarms = (n - defective) * FPR # 180
    print()
    print("in %d releases: %d defective, %d true alarms, %d false"
          % (n, defective, true_alarms, false_alarms))
    print(" %d of %d alarms are real = %s"
          % (true_alarms, true_alarms + false_alarms,
             F(int(true_alarms), int(true_alarms + false_alarms))))
    assert F(int(true_alarms), int(true_alarms + false_alarms)) == post
    print(" a 90%-sensitive test fired, and the release is still")
    print(" TWICE as likely sound as broken.")

```

```

# --- the decision -----
print()
print("=== the decision ===")
print("situation      P(defect)   EU(ship)  EU(delay)  choose")
print("-" * 60)
rows = (("no test yet   ", P_DEFECT),
        ("alarm fired   ", post),
        ("test is silent", quiet))
for label, p in rows:
    a, _ = best(p)
    print("%s %10s %10.2f %10.2f  %s"
          % (label, round(float(p), 4), float(eu("ship", p)),
             float(eu("delay", p)), a))

star = flip_point()
print()
print("flip point p* =", star, "=", round(float(star), 4))
assert star == F(10, 29)
# just either side of it
assert best(star - F(1, 1000))[0] == "ship"
assert best(star + F(1, 1000))[0] == "delay"

# the headline: belief changed a lot, decision did not change
assert best(P_DEFECT)[0] == "ship"
assert best(post)[0] == "ship"
margin = eu("ship", post) - eu("delay", post)
print()
print("the alarm moved belief from %s to %s -- more than triple --"
      % (P_DEFECT, post))
print("and the decision did not change, because %s < %s."
      % (post, star))
print("the margin is now only %.2f utility units." % float(margin))
assert float(margin) < 2

# --- the value of information -----
print()
print("=== value of perfect information ===")
for label, p in rows + (("at the flip point", star),):
    print("    %-18s p=%-8s VPI = %6.2f"
          % (label.strip(), round(float(p), 4), float(vpi(p))))
assert vpi(post) > 3 * vpi(P_DEFECT)
print()
print("Information is worth most where the decision is hardest:")
print("%.2f before the test, %.2f after an ambiguous alarm."
      % (float(vpi(P_DEFECT)), float(vpi(post))))
print("Delaying a week costs 10 when the release is sound, so an")
print("investigation cheaper than that is clearly worth doing.")

# --- and the utilities matter as much as the probabilities ----
print()
print("=== if shipping a defect cost 60 rather than 100 ===")
UTIL["ship", "defect"] = -60
star2 = flip_point()
print("    flip point moves from %s to %s (%.3f)"
      % (star, star2, float(star2)))
print("    the alarm's 1/3 is now far from flipping anything:",
      best(post)[0])
print("    VPI after an alarm falls from %.2f to %.2f"
      % (31.67, float(vpi(post))))
assert star2 > star
print()
print("The utilities decide the answer as much as the")

```

```
print("probabilities do -- and they are the part nobody measures.")
```

Then change FPR to 0.02 — a test that almost never cries wolf — and re-run. The posterior after an alarm jumps to about 0.83, the decision flips to *delay*, and the value of information collapses. A test is worth having in proportion to how often its answer changes what you do.

? Checkpoint — Test Yourself

1. A test that fires on 90% of defective releases has fired, and the posterior probability of a defect is 1/3. Explain to a delivery manager, in two sentences and without algebra, why those numbers are consistent.
2. The alarm moved the belief from 0.1 to 1/3 and did not change the recommendation. State the general lesson, and say what an engineer should report instead of “the probability tripled”.
3. The value of perfect information was 9.50 before the test and 31.67 after an ambiguous alarm. Explain why more evidence became *more* valuable after evidence arrived.

✓ Chapter Summary

- Probability is a **degree of belief** given what is known. Unlike Chapter 10’s defaults it *combines*: two pieces of evidence pointing different ways give a number between them, by a rule nobody gets to choose.
- **Bayes’ rule** inverts a conditional. The one you have is usually the wrong way round, and assuming otherwise is the **base-rate fallacy**: a 90%-sensitive test fired and left $P(\text{defect}) = 1/3$.
- Natural frequencies — 90 true alarms among 270 — make the base rate visible. Use them when explaining a result to people.
- **Conditional independence** is what stops a joint distribution growing as 2^n , and it is the organising idea of Chapter 15.
- A rational agent maximises **expected utility**, $EU(a) = \sum_s P(s | a)U(s)$. This is the precise form of Chapter 2’ definition of rationality.
- *A change of belief is not a change of decision*. The alarm tripled the probability and left the recommendation alone, because 1/3 falls short of the flip point 10/29. Report which side of p^* you are on, and how close.
- The **value of perfect information** is greatest exactly where the decision is most finely balanced — 31.67 after the ambiguous alarm against 9.50 before it. Gather evidence when it could change the action, and stop when it could not.
- The utilities decide the outcome as much as the probabilities, and they are the part nobody measures. Write them down so they can be argued with.

✍ Review Questions

1. (MCQ) The base-rate fallacy is confusing: (a) $P(a)$ with $P(\neg a)$ (b) $P(a | b)$ with $P(b | a)$ (c) a prior with a utility (d) independence with conditional independence.
2. (MCQ) A rational agent chooses the action maximising: (a) probability of success (b) utility of the best outcome (c) expected utility (d) the value of information.
3. (MCQ) The value of perfect information is largest when: (a) one action is clearly best (b) the decision is finely balanced (c) the prior is zero (d) the utilities are equal.
4. (MCQ) a and b are conditionally independent given c when: (a) $P(a \wedge b) = P(a)P(b)$ (b) $P(a | b \wedge c) = P(a | c)$ (c) $P(a | c) = P(a)$ (d) $P(c | a) = P(c | b)$.

5. **(Short)** Draw the natural-frequency tree for a test with 80% sensitivity and a 10% false-alarm rate on a 5% base rate, and give $P(\text{defect} \mid \text{alarm})$.
6. **(Short)** Explain why the value of information rose after the alarm, even though the alarm did not change the recommended action.
7. **(Short)** Give one decision from your own experience where more investigation had a value of zero, and say how you would have known in advance.
8. **(Applied)** The team argues that delaying a week should cost -25 when the release turns out sound, not -10 . Recompute the flip point p^* , say whether the alarm at $p = 1/3$ now changes the decision, and compute the new value of perfect information at that point.

Bayesian Networks

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part V — Reasoning Under Uncertainty

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Read** a Bayesian network as a set of conditional independence claims, and **count** the numbers it needs against the full joint.
- **Compute** a posterior by enumeration and by variable elimination.
- **Estimate** a posterior by rejection sampling and by likelihood weighting, and **say** when each is preferable.
- **Recognise** *explaining away*, and **explain** why it cannot be produced by any rule-based system.
- **Judge** when variable elimination is worth its overhead.

🔗 Before You Start

Chapter 14 — Bayes' rule, marginalisation, and above all conditional independence, which is the single idea this whole representation is built on.

📖 Key Terms in This Chapter

Bayesian network • directed acyclic graph • conditional probability table • parent • chain rule • enumeration • factor • variable elimination • rejection sampling • likelihood weighting • explaining away • diagnostic and causal direction

15.1 Diagnosing a Defect

A release has been rolled back. Why?

The team believes three things make defects likely: a rushed specification, a junior developer working unsupervised, and weak test coverage. A defect in turn makes two things likely: an emergency hotfix, and a rollback. Nobody observes the defect directly — they observe the rollback and want to know what caused it.

That is a diagnosis problem, and its structure is what a **Bayesian network** represents.

📄 Bayesian network

A **Bayesian network** is a directed acyclic graph in which each node is a random variable and each node carries a **conditional probability table** giving $P(\text{node} \mid \text{its parents})$.

The graph asserts that *each variable is conditionally independent of its non-descendants given its parents*. That assertion licenses the chain rule

$$P(x_1, \dots, x_n) = \prod_{i=1}^n P(x_i \mid \text{parents}(x_i))$$

so the full joint distribution is recoverable from the tables — which is why the tables are all you have to store, and all anybody has to estimate.

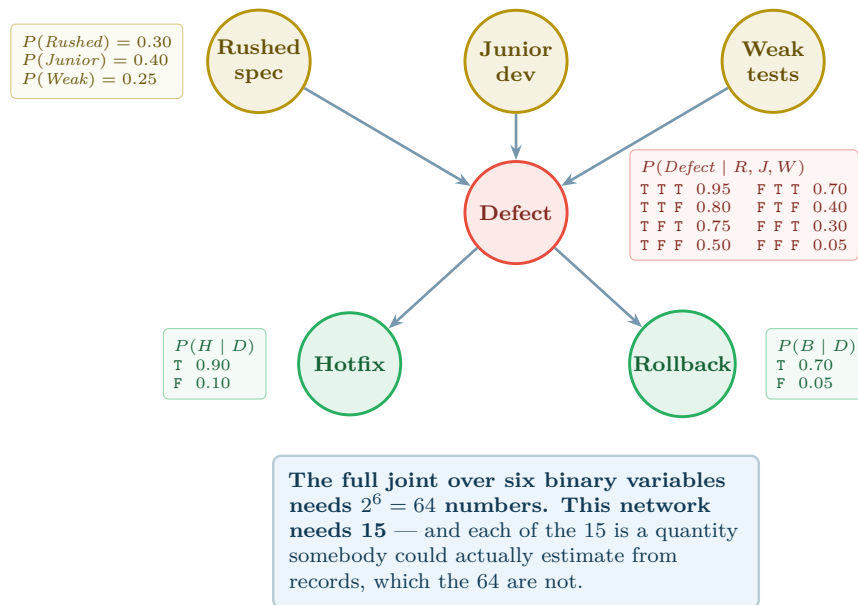


Figure 15.1: The diagnosis network. Causes point to effects, so the arrows run the way the world does; inference then runs against them, from rollback back to cause.

⚠ What the missing arrows say

The content of a Bayesian network is in the edges it *lacks*.

There is no arrow from *Hotfix* to *Rollback*, which asserts that once you know whether there is a defect, learning that a hotfix was issued tells you nothing further about whether a rollback happened. There is no arrow between *Rushed* and *Junior*, asserting they are independent before any evidence.

Those are substantive claims about the domain and they can be wrong. If rushed specifications are usually handed to junior developers, the second claim is false and the network will give confidently wrong answers. *Drawing the graph is the modelling; the numbers are the easy part.*

15.2 Exact Inference by Enumeration

$P(Rushed | Rollback)$ follows directly from the definition:

$$P(r | b) = \alpha \sum_{j,w,d,h} P(r)P(j)P(w)P(d | r, j, w)P(h | d)P(b | d)$$

Sum the joint over every hidden variable, for each value of the query, and normalise. On this network that is 32 terms, and the answer is

$$P(Rushed | Rollback) = 0.4899$$

against a prior of 0.30. A rollback makes a rushed specification substantially more likely — not because any rule says so, but because the arithmetic of the graph says so.

Enumeration is exact and it is $O(2^n)$. That is Chapter 11's model checking again, and it fails for the same reason.

15.3 Variable Elimination

Enumeration recomputes the same subexpressions over and over. **Variable elimination** computes each once, by pushing sums inward and caching the results as **factors**.

A factor is a table over some subset of the variables. The procedure is: build one factor per node; then for each hidden variable, multiply together every factor mentioning it and sum it out, leaving a new factor; finally multiply what remains and normalise.

⚠ On this network, variable elimination is slower

It uses 74 operations where enumeration uses 32 terms, and returns the same 0.4899. The bookkeeping costs more than the sharing saves, because the network has six nodes and there is almost nothing to share.

This is the same lesson as Chapter 7's AC-3 and Chapter 9's heuristic, and it is worth stating as a rule: *an algorithm whose advantage is asymptotic has no advantage at the size where you first test it*. Measure at the size you will deploy at.

Deploy it at scale and the picture inverts completely. Take a chain of services, each of which may be delayed by the one before it, and ask about the first given evidence about the last:

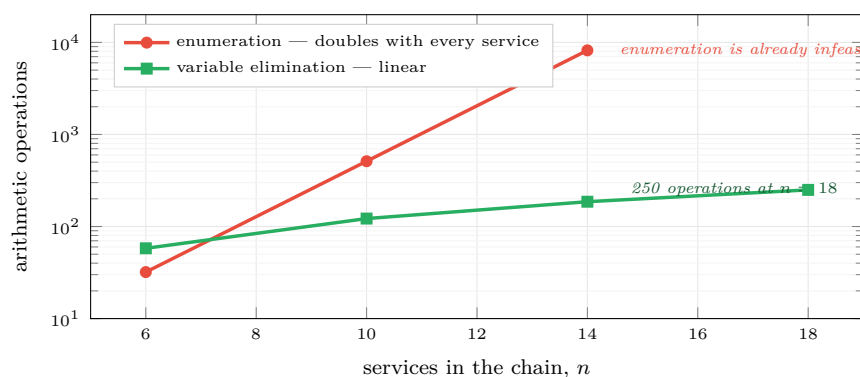


Figure 15.2: Enumeration against variable elimination on a chain. At six nodes enumeration wins; at fourteen it needs 8,192 terms against 186 operations, and beyond that it stops being an option.

Variable elimination is not always polynomial — its cost is governed by the largest intermediate factor, which depends on the graph's structure and on the elimination order. On a chain it is linear; on a densely connected graph it is exponential again, and exact inference in general Bayesian networks is NP-hard.

15.4 Approximate Inference by Sampling

When exact inference is out of reach, generate samples from the network and count.

Rejection sampling is the obvious method: sample every variable in topological order using its conditional probability table, discard any sample that disagrees with the evidence, and take the proportion among the survivors.

It is correct, and it wastes almost everything. Here the evidence *Rollback* = T has probability about 0.30, so roughly 70% of samples are thrown away: 20,000 samples leave about 6,000 usable. With rarer evidence — and diagnostic evidence is usually rare, since that is why it is interesting — the rejection rate approaches 100% and the method stops working.

Likelihood weighting fixes this. Do not sample the evidence variables at all: fix them to their observed values, sample only the rest, and weight each sample by the probability of the evidence given what was sampled. Every sample now counts.

Method	Answer	Error	Note
Enumeration (exact)	0.48990	—	32 terms
Variable elimination (exact)	0.48990	—	74 operations
Rejection, 20,000 samples	0.4952	0.0053	6,034 kept; 70% discarded
Likelihood weighting, 20,000	0.4913	0.0014	every sample used

Table 15.1: $P(\text{Rushed} \mid \text{Rollback})$ four ways. The two exact methods agree to every digit; the two approximate ones differ mainly in how much of their work they throw away.

15.5 Explaining Away

Here is the behaviour that justifies the whole apparatus, and that no rule base of Chapter 13 can produce.

Learn that there *is* a defect. Belief in a rushed specification rises from 0.30 to 0.5281 — the defect needs explaining, and a rushed specification is one of the explanations.

Now additionally learn that a junior developer was working unsupervised. Belief in a rushed specification *falls*, to 0.4304. The junior developer already accounts for the defect, so less of the explanatory work is left for the specification to do.



Two causes of a common effect are independent *until* the effect is observed, and become dependent once it is — negatively so. That is **explaining away**, and it is the reason to build a network rather than a rule base.

Notice what a production rule would have to say. IF defect AND junior developer THEN rushed specification is *less* likely — a rule whose conclusion is weakened by adding a condition, which no forward-chaining engine can express, and which Chapter 13's specificity strategy would actively get wrong. The behaviour falls out of the graph and the arithmetic without anybody writing it down, and it would have to be anticipated and hand-coded in any rule-based system.

It is also the formal version of something every incident reviewer does: on finding one sufficient cause, they become less interested in the others. The network says exactly how much less.

Worked Example: Computing $P(\text{Rushed} \mid \text{Rollback})$ by hand

The query has one evidence variable, and *Junior*, *Weak*, *Defect* and *Hotfix* are hidden.

Step 1: drop *Hotfix*. It is not in the query and not in the evidence, and nothing else depends on it. Summing it out gives $\sum_h P(h \mid d) = 1$ for either value of d , so it contributes a factor of one and can be deleted before any arithmetic. *A hidden variable with no observed descendants is irrelevant* — a useful check to run before computing anything.

Step 2: the remaining sum. With r for *Rushed*, b for *Rollback*:

$$P(r \mid b) = \alpha P(r) \sum_j P(j) \sum_w P(w) \sum_d P(d \mid r, j, w) P(b \mid d)$$

Step 3: the inner sum. For each combination of r, j, w let $q = P(\text{Defect} \mid r, j, w)$. Then

$$\sum_d P(d \mid r, j, w) P(b \mid d) = 0.70q + 0.05(1 - q) = 0.05 + 0.65q$$

so every row of the big table reduces to one number. With r true the four q values are 0.95, 0.80, 0.75, 0.50, giving 0.6675, 0.5700, 0.5375, 0.3750.

Step 4: weight by the priors. $P(j) = 0.40$, $P(w) = 0.25$:

$$\begin{aligned} &0.40 \times 0.25 \times 0.6675 + 0.40 \times 0.75 \times 0.5700 \\ &+ 0.60 \times 0.25 \times 0.5375 + 0.60 \times 0.75 \times 0.3750 \\ &= 0.06675 + 0.17100 + 0.08063 + 0.16875 = 0.48713 \end{aligned}$$

Multiply by $P(r) = 0.30$: the unnormalised weight for r true is 0.146138.

Step 5: the other branch. With r false the four q values are 0.70, 0.40, 0.30, 0.05, giving 0.5050, 0.3100, 0.2450, 0.0825; weighting by the same priors gives 0.21738, and multiplying by $P(\neg r) = 0.70$ gives 0.152163.

Step 6: normalise.

$$P(r \mid b) = \frac{0.146138}{0.146138 + 0.152163} = \frac{0.146138}{0.298301} = 0.4899$$

What the work shows. Steps 3 and 4 were done once for r true and once for r false, and the inner structure was identical both times. Variable elimination is exactly the observation that this shared structure need not be recomputed — and on six variables the saving is not worth the bookkeeping, while on Figure 15.2's chain it is the difference between possible and impossible.

! Common Pitfalls

- **Reading arrows as “causes” and inference as following them.** The arrows point from cause to effect; the query here runs backwards, from rollback to specification. Both directions are ordinary conditional probability.
- **Forgetting that the missing arrows are the claims.** No edge between *Rushed* and *Junior* asserts independence. If rushed work is usually given to junior developers, the network is simply wrong.
- **Using rejection sampling with rare evidence.** 70% of samples discarded here; with evidence of probability 0.01 it is 99%. Use likelihood weighting.
- **Assuming variable elimination is always faster.** It lost on this network, 74 operations against 32. Its advantage is asymptotic and depends on the elimination order.
- **Expecting exact inference to scale in general.** It is linear on a chain and NP-hard on general graphs.
- **Summing over hidden variables that cannot matter.** A hidden variable with no observed descendants sums to one. Delete *Hotfix* before computing, not after.

🔧 Try It Yourself: Diagnosing a Defect's Cause

The network of Figure 15.1, four inference methods, the explaining-away measurement, and the scaling comparison of Figure 15.2.

```
"""Chapter 15 lab -- inference in a Bayesian network.

Four ways to compute P(Rushed | Rollback): two exact, two by
sampling. Then explaining away, and the scaling comparison that
justifies variable elimination.
"""

import itertools
```

```

import random

# node -> (parents, table mapping parent values to P(node = True))
NET = {
    "Rushed": ([], {(): 0.30}),
    "Junior": ([], {(): 0.40}),
    "WeakTest": ([], {(): 0.25}),
    "Defect": ([ "Rushed", "Junior", "WeakTest"], {
        (True, True, True): 0.95, (True, True, False): 0.80,
        (True, False, True): 0.75, (True, False, False): 0.50,
        (False, True, True): 0.70, (False, True, False): 0.40,
        (False, False, True): 0.30, (False, False, False): 0.05,
    }),
    "Hotfix": ([ "Defect"], {(True,): 0.90, (False,): 0.10}),
    "Rollback": ([ "Defect"], {(True,): 0.70, (False,): 0.05}),
}
ORDER = [ "Rushed", "Junior", "WeakTest", "Defect", "Hotfix", "Rollback"]

def p_true(net, node, a):
    parents, table = net[node]
    return table[tuple(a[p] for p in parents)]

def prob(net, node, value, a):
    p = p_true(net, node, a)
    return p if value else 1 - p

# --- 1. exact: enumeration -----
def enumerate_query(net, order, query, evidence):
    hidden = [n for n in order if n != query and n not in evidence]
    totals, terms = {True: 0.0, False: 0.0}, 0
    for qv in (True, False):
        for combo in itertools.product([True, False],
                                       repeat=len(hidden)):
            a = dict(evidence)
            a[query] = qv
            a.update(dict(zip(hidden, combo)))
            pr = 1.0
            for n in order:
                pr *= prob(net, n, a[n], a)
            totals[qv] += pr
            terms += 1
    z = totals[True] + totals[False]
    return totals[True] / z, terms

# --- 2. exact: variable elimination -----
class Factor:
    def __init__(self, vars_, table):
        self.vars, self.table = list(vars_), table

def make_factor(net, node, evidence):
    allv = net[node][0] + [node]
    keep = [v for v in allv if v not in evidence]
    table = {}
    for combo in itertools.product([True, False], repeat=len(allv)):
        a = dict(zip(allv, combo))
        if any(v in evidence and a[v] != evidence[v] for v in allv):
            continue

```

```

        table[tuple(a[v] for v in keep)] = prob(net, node, a[node], a)
    return Factor(keep, table)

def multiply(f1, f2, ops):
    vars_ = f1.vars + [v for v in f2.vars if v not in f1.vars]
    table = {}
    for combo in itertools.product([True, False], repeat=len(vars_)):
        a = dict(zip(vars_, combo))
        table[combo] = (f1.table[tuple(a[v] for v in f1.vars)]
                        * f2.table[tuple(a[v] for v in f2.vars)])
    ops[0] += 1
    return Factor(vars_, table)

def sum_out(f, var, ops):
    i = f.vars.index(var)
    keep = [v for v in f.vars if v != var]
    table = {}
    for combo, val in f.table.items():
        k = combo[:i] + combo[i + 1:]
        table[k] = table.get(k, 0.0) + val
    ops[0] += 1
    return Factor(keep, table)

def var_elim(net, order, query, evidence):
    ops = [0]
    factors = [f for f in (make_factor(net, n, evidence) for n in order)
                if f.vars]
    for h in [n for n in order if n != query and n not in evidence]:
        involved = [f for f in factors if h in f.vars]
        if not involved:
            continue
        factors = [f for f in factors if h not in f.vars]
        prod = involved[0]
        for f in involved[1:]:
            prod = multiply(prod, f, ops)
        factors.append(sum_out(prod, h, ops))
    prod = factors[0]
    for f in factors[1:]:
        prod = multiply(prod, f, ops)
    i = prod.vars.index(query)
    totals = {True: 0.0, False: 0.0}
    for combo, val in prod.table.items():
        totals[combo[i]] += val
    z = totals[True] + totals[False]
    return totals[True] / z, ops[0]

# --- 3. rejection sampling -----
def rejection(net, order, query, evidence, n, rng):
    kept = hits = 0
    for _ in range(n):
        a = {}
        for node in order:
            a[node] = rng.random() < p_true(net, node, a)
        if all(a[k] == v for k, v in evidence.items()):
            kept += 1
            hits += a[query]
    return (hits / kept if kept else None), kept

```

```

# --- 4. likelihood weighting -----
def likelihood_weighting(net, order, query, evidence, n, rng):
    num = den = 0.0
    for _ in range(n):
        a, w = {}, 1.0
        for node in order:
            if node in evidence:
                a[node] = evidence[node]
                w *= prob(net, node, a[node], a)      # weight, not sample
            else:
                a[node] = rng.random() < p_true(net, node, a)
        den += w
        if a[query]:
            num += w
    return num / den, den

def make_chain(n):
    """X1 -> X2 -> ... -> Xn: a service dependency chain."""
    net = {"X1": ([], {(): 0.3})}
    for i in range(2, n + 1):
        net["X%d" % i] = (["X%d" % (i - 1)],
                           {(True,): 0.8, (False,): 0.1})
    return net, ["X%d" % i for i in range(1, n + 1)]

if __name__ == "__main__":
    print("variables: %d    full joint entries: %d"
          % (len(ORDER), 2 ** len(ORDER)))
    print("numbers the network stores: %d"
          % sum(len(NET[n][1]) for n in ORDER))
    print()

    ev = {"Rollback": True}
    exact, terms = enumerate_query(NET, ORDER, "Rushed", ev)
    ve, ops = var_elim(NET, ORDER, "Rushed", ev)
    print("P(Rushed)                = %.4f    (the prior)" % 0.30)
    print("P(Rushed | Rollback)      = %.5f" % exact)
    assert abs(exact - ve) < 1e-9
    print()
    print("method                        answer    work")
    print("-" * 48)
    print("enumeration (exact)                %9.5f  %4d terms" % (exact, terms))
    print("variable elimination (exact) %9.5f  %4d ops" % (ve, ops))

    rng = random.Random(0)
    rej, kept = rejection(NET, ORDER, "Rushed", ev, 20000, rng)
    rng = random.Random(0)
    lw, _ = likelihood_weighting(NET, ORDER, "Rushed", ev, 20000, rng)
    print("rejection, 20000 samples          %9.4f  %4d kept (%.0f%%)"
          % (rej, kept, 100.0 * kept / 20000))
    print("likelihood weighting, 20000    %9.4f  all samples used" % lw)
    print()
    print("    errors: rejection %.4f, weighting %.4f"
          % (abs(rej - exact), abs(lw - exact)))
    assert abs(lw - exact) < 0.01 and abs(rej - exact) < 0.02

    # variable elimination LOSES on a network this small
    assert ops > terms
    print()
    print("Note that variable elimination used MORE work than")

```

```

print("enumeration here (%d against %d). Its advantage is" % (ops, terms))
print("asymptotic, and six variables is not asymptotic.")

# --- explaining away -----
print()
print("=== explaining away ===")
prior = 0.30
given_d, _ = enumerate_query(NET, ORDER, "Rushed", {"Defect": True})
given_dj, _ = enumerate_query(NET, ORDER, "Rushed",
                              {"Defect": True, "Junior": True})

print("    P(Rushed)                        = %.4f" % prior)
print("    P(Rushed | Defect)                = %.4f    (rises)" % given_d)
print("    P(Rushed | Defect, Junior)         = %.4f    (FALLS)"
      % given_dj)
assert given_d > prior > 0 and given_dj < given_d
print()
print("    The junior developer already accounts for the defect,")
print("    so less explaining is left for the rushed spec to do.")
print("    Two causes of one effect are independent until the")
print("    effect is seen, and negatively dependent afterwards.")
print("    No production rule can express a conclusion that gets")
print("    WEAKER when a condition is added.")

# --- and where variable elimination earns its keep -----
print()
print("=== scaling, on a chain of services ===")
print("    n    enumeration    VE ops")
print("-" * 30)
for n in (6, 10, 14, 18):
    net, order = make_chain(n)
    e = {order[-1]: True}
    v, vops = var_elim(net, order, "X1", e)
    if n <= 14:
        ex, et = enumerate_query(net, order, "X1", e)
        assert abs(ex - v) < 1e-9
        print("%3d %13d %8d" % (n, et, vops))
    else:
        print("%3d %13s %8d" % (n, "infeasible", vops))
print()
print("Enumeration doubles with every service; variable")
print("elimination grows linearly on a chain. At n=14 that is")
print("8192 terms against 186 operations.")

```

Then add an arrow from *Junior* to *Rushed* — suppose rushed work really is given to junior developers — and re-run. The explaining-away effect changes size, because the two causes are no longer independent to begin with. That is the modelling decision of the alertbox, made concrete.

🔍 Checkpoint — Test Yourself

1. The full joint over these six variables needs 64 numbers and the network stores 15. State precisely what the network asserts in order to get away with the smaller number, and name one assertion in Figure 15.1 that could be false in a real organisation.
2. Learning that there is a defect raised belief in a rushed specification from 0.30 to 0.5281; additionally learning that a junior developer was involved *lowered* it to 0.4304. Explain why, and say why a forward-chaining rule base cannot reproduce this.
3. Rejection sampling discarded 70% of its samples here. Describe the circumstance in which it becomes useless, and say what likelihood weighting does differently.

Chapter Summary

- A **Bayesian network** is a directed acyclic graph with a conditional probability table at each node. It asserts that each variable is conditionally independent of its non-descendants given its parents, which licenses the chain rule and lets 15 numbers stand for a joint of 64.
- *The missing arrows are the claims.* They are substantive assertions about the domain and they can be wrong; drawing the graph is the modelling.
- **Enumeration** is exact and $O(2^n)$. **Variable elimination** is exact and computes shared subexpressions once — but it *lost* on this six-node network, 74 operations against 32 terms. Its advantage is asymptotic: on a fourteen-service chain it is 186 operations against 8,192 terms.
- Exact inference in general Bayesian networks is NP-hard; the cost is governed by the largest intermediate factor.
- **Rejection sampling** discards every sample disagreeing with the evidence — 70% here, and nearly everything when the evidence is rare. **Likelihood weighting** fixes the evidence and weights instead, so every sample counts.
- **Explaining away**: two causes of a common effect are independent until the effect is observed and negatively dependent afterwards. Belief in a rushed specification rose to 0.5281 on seeing a defect and fell to 0.4304 on also learning of a junior developer.
- That behaviour is why one builds a network rather than a rule base: it requires a conclusion to become *weaker* as a condition is added, which no production rule expresses and Chapter 13's specificity strategy would get backwards.

Review Questions

1. (MCQ) A Bayesian network asserts that each variable is conditionally independent of: (a) all other variables (b) its non-descendants, given its parents (c) its parents (d) its descendants, given the evidence.
2. (MCQ) Explaining away means that: (a) evidence for one cause raises belief in another (b) evidence for one cause lowers belief in another (c) causes are always independent (d) effects are independent of causes.
3. (MCQ) Likelihood weighting differs from rejection sampling by: (a) sampling more variables (b) fixing the evidence and weighting each sample (c) using exact arithmetic (d) eliminating variables.
4. (MCQ) A hidden variable with no observed descendants: (a) must be summed over (b) sums to one and can be deleted (c) makes the query intractable (d) must be given evidence.
5. (Short) Explain what it means to say “the missing arrows are the claims”, using one missing arrow from Figure 15.1.
6. (Short) Variable elimination was slower than enumeration on this network and far faster on a chain of fourteen. Explain both results with one principle.
7. (Short) Why can a production rule not express explaining away? State what such a rule would have to do.
8. (Applied) Compute $P(\text{Defect} \mid \text{Hotfix})$ by hand from the tables of Figure 15.1, showing the marginalisation over *Rushed*, *Junior* and *WeakTest*. Then say whether *Rollback* needs to be summed over, and why.

Fuzzy Logic

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part V — Reasoning Under Uncertainty

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Distinguish** vagueness from uncertainty, and **say** which of the two fuzzy logic addresses.
- **Define** membership functions and **apply** the fuzzy operators.
- **Run** a Mamdani inference by hand: fuzzify, evaluate the rules, clip, aggregate, defuzzify.
- **Explain** why a fuzzy controller's output is smooth where a threshold rule's is not, and why that matters.
- **State** the limitations of centroid defuzzification and of the method generally.

🔗 Before You Start

Chapter 14 — and specifically the idea of a degree of belief, which is what a degree of *membership* is repeatedly confused with. This chapter branches from Chapter 14 rather than following Chapter 15: it answers a different question.

📖 Key Terms in This Chapter

vagueness • fuzzy set • membership function • degree of membership • linguistic variable • fuzzy partition • fuzzy operators • fuzzification • firing strength • Mamdani inference • clipping • aggregation • defuzzification • centroid

16.1 Vagueness Is Not Uncertainty

Two sentences that sound alike and are not:

This project is probably high-risk.

This project is fairly high-risk.

The first is **uncertainty**. There is a fact of the matter — the project either is or is not high-risk — and the speaker does not know which. Chapter 14 is about that, and evidence resolves it: learn enough and the probability goes to 0 or 1.

The second is **vagueness**. The speaker knows everything there is to know about the project. The trouble is that “high-risk” has no sharp boundary, and no amount of evidence will produce one. A project at 70 effort-days with moderate uncertainty is high-risk to a degree, and that is the whole truth about it.

📖 Fuzzy set and membership

A **fuzzy set** A over a domain is given by a **membership function** $\mu_A(x) \in [0, 1]$ saying to what degree x belongs to A .

$\mu_{high}(70) = 0.4$ does *not* mean “a 40% chance the project is high-risk”. It means the project is high-risk to degree 0.4 — a statement about the *word*, not about the world, and it does not change when more evidence arrives.

⚠ The confusion this chapter exists to prevent

A probability and a membership are both numbers in $[0, 1]$ and they mean entirely different things.

$P(\text{defect}) = 0.3$ describes ignorance about a sharp fact; test the release and it resolves to 0 or 1. $\mu_{\text{high}}(70) = 0.4$ describes a sharp fact about a blurry word; measure the project more carefully and it stays 0.4.

Probabilities over a partition sum to one and memberships do not. Combining probabilities uses multiplication and Bayes' rule; combining memberships uses min and max. And Russell and Norvig are openly sceptical that the second apparatus is ever necessary — their position is that a well-specified probabilistic model can do this work. It is worth reading their argument, and worth knowing that fuzzy controllers run production systems from cement kilns to camera autofocus, which is an argument of a different kind.

16.2 Membership Functions and Linguistic Variables

A **linguistic variable** takes values that are words — *low*, *medium*, *high* — each defined by a membership function over the underlying numeric domain.

The usual construction is a **fuzzy partition**: overlapping triangles whose peaks sit at evenly spaced points and whose memberships sum to one everywhere. Two consequences: every input is described by at most two words, and the description changes continuously as the input moves.

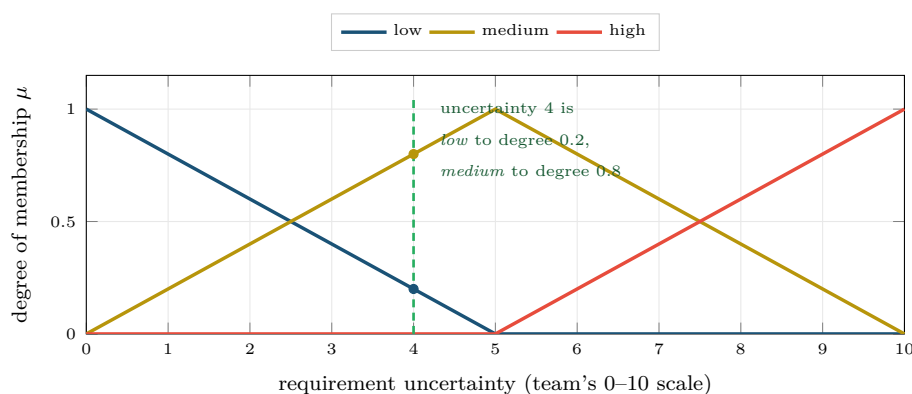


Figure 16.1: A fuzzy partition of the uncertainty scale. The memberships sum to one at every point, so each input is described by at most two words and the description moves smoothly as the input does.

16.3 The Fuzzy Operators

Conjunction, disjunction and negation extend to memberships in the standard way:

$$\mu_{A \cap B}(x) = \min(\mu_A(x), \mu_B(x)), \quad \mu_{A \cup B}(x) = \max(\mu_A(x), \mu_B(x)), \quad \mu_{\neg A}(x) = 1 - \mu_A(x)$$

Note what is *not* true. In classical logic $A \cap \neg A = \emptyset$; here $\min(\mu, 1 - \mu)$ is 0.5 when $\mu = 0.5$, so a borderline project is in “high-risk and not high-risk” to degree 0.5. That is a feature of the intended reading — a genuinely borderline case is half in and half out of a vague category — and it is also why fuzzy logic is not a logic in the sense of Chapter 11: there is no entailment relation and no soundness theorem.

16.4 Mamdani Inference

The classical fuzzy controller, in five steps. The example is the team’s risk model: inputs are *effort* (person-days) and *requirement uncertainty*, the output is *risk*, and there are nine rules — one for each combination of two linguistic variables with three values each.

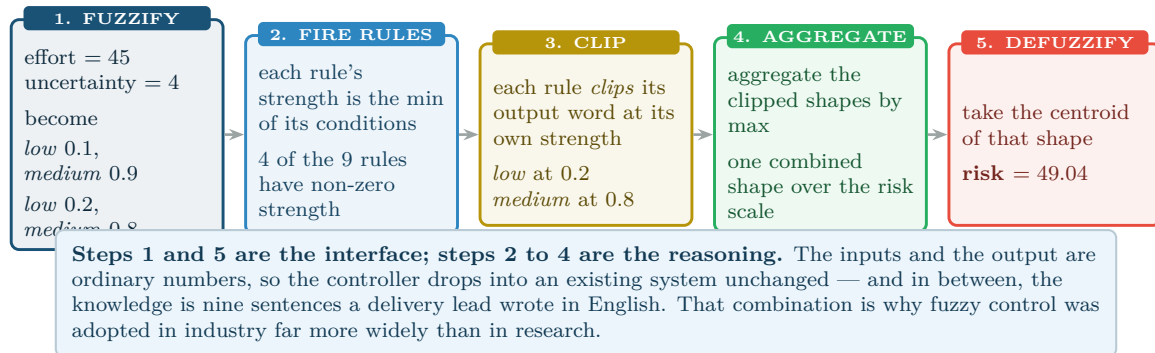


Figure 16.2: Mamdani inference on the project *Beacon*, effort 45 and uncertainty 4. Four of the nine rules fire, and the output is a single number.

</> The nine rules

Effort	Uncertainty	Risk
low	low	low
low	medium	low
low	high	medium
medium	low	low
medium	medium	medium
medium	high	high
high	low	medium
high	medium	high
high	high	high

Nine rules for two inputs with three values each. Note how this scales: three inputs would need 27, four would need 81. *Rule explosion* is the practical limit on fuzzy control, and it is why these systems typically have two or three inputs.

16.5 Defuzzification

Aggregation leaves a shape; a controller needs a number. The **centroid** method takes the shape’s centre of area:

$$x^* = \frac{\int x \mu(x) dx}{\int \mu(x) dx}$$

computed in practice by summing over a grid.

⚠ Centroid defuzzification cannot reach the extremes

Run the risk model across its whole input range and the output never leaves roughly [16.6, 83.4] — it never says 0 and never says 100, even for a project with minimum effort and minimum uncertainty.

The reason is geometric rather than conceptual: the centroid of a shape clipped from a triangle centred at 0 still has area to the right of 0, so its centre of area is positive. The output range is compressed towards the middle, always.

Three responses, all used in practice: rescale the output afterwards; use *mean of maxima* instead, which does reach the extremes but is discontinuous; or accept it, on the grounds that a risk score is compared against other risk scores rather than read as an absolute. Whichever you choose, know that the numbers a Mamdani controller emits are not on the scale you think they are, and do not label the output “percent”.

16.6 Why Smoothness Is the Point

Everything above could have been done with a threshold rule: IF effort > 70 AND uncertainty > 7 THEN risk is high. It would be shorter, and it has one fatal property.

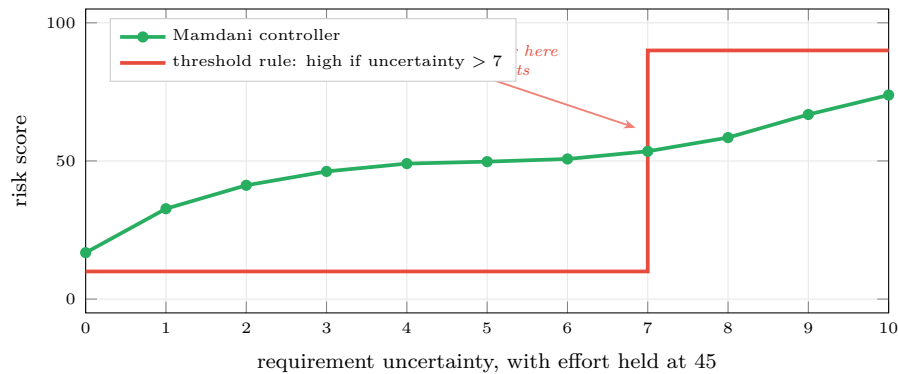


Figure 16.3: The same knowledge as a fuzzy controller and as a threshold rule. The fuzzy output moves continuously; the threshold output is flat, flat, and then an eighty-point jump between two adjacent inputs.

The threshold rule says the same thing at uncertainty 1 and at 6, and something completely different between 7 and 8. If the uncertainty score is a judgement with a unit of noise in it — and it is, because a person assigned it — then two assessors will disagree wildly about projects near the boundary and not at all about the rest. That is not a property anybody wants in a risk register.

The fuzzy controller’s output moves a little when the input moves a little, everywhere. *That is the engineering reason fuzzy control was adopted:* not because vagueness is philosophically interesting, but because interpolating between rules gives a smooth response surface from a handful of sentences that domain experts can actually write.

Worked Example: Beacon, by hand

Effort 45 person-days, requirement uncertainty 4.

Step 1 — fuzzify. With triangles peaking at 0, 50, 100 for effort:

$$\mu_{low}(45) = \frac{50 - 45}{50} = 0.1, \quad \mu_{medium}(45) = \frac{45 - 0}{50} = 0.9, \quad \mu_{high}(45) = 0$$

and with triangles peaking at 0, 5, 10 for uncertainty:

$$\mu_{low}(4) = \frac{5 - 4}{5} = 0.2, \quad \mu_{medium}(4) = \frac{4 - 0}{5} = 0.8, \quad \mu_{high}(4) = 0$$

Each input is described by exactly two words, as the partition guarantees.

Step 2 — rule strengths, by min. Five of the nine rules involve a word with membership zero and contribute nothing. The other four:

- low effort, low uncertainty → *low* risk: $\min(0.1, 0.2) = 0.1$
- low effort, medium uncertainty → *low* risk: $\min(0.1, 0.8) = 0.1$

- medium effort, low uncertainty $\rightarrow$ *low* risk: $\min(0.9, 0.2) = 0.2$
- medium effort, medium uncertainty $\rightarrow$ *medium* risk: $\min(0.9, 0.8) = 0.8$

Step 3 — clip. Three rules conclude *low* risk, with strengths 0.1, 0.1 and 0.2; aggregating by max means the *low* triangle is clipped at 0.2. One rule concludes *medium* risk, so that triangle is clipped at 0.8.

Step 4 — aggregate. The combined shape is the pointwise maximum: a plateau at height 0.2 over the low-risk region and a larger plateau at 0.8 over the middle.

Step 5 — defuzzify. The centre of area of that shape is **49.04**.

Sanity check. The dominant rule concluded *medium*, whose triangle peaks at 50; the weaker *low* contribution drags the answer slightly below it. 49.04 is exactly where it should be, and the arithmetic is worth doing once by hand because it is the only way to believe the pipeline.

Now move one input. At uncertainty 9 instead of 4 the answer is 66.81; at uncertainty 0 it is 16.79. Every intermediate value appears — no jumps, and no rule had to be written for the cases in between.

! Common Pitfalls

- **Reading a membership as a probability.** $\mu = 0.4$ is not “a 40% chance”. Memberships do not sum to one, do not update on evidence, and combine by min and max rather than by Bayes’ rule.
- **Membership functions that do not overlap.** Then only one rule ever fires, there is no interpolation, and you have written a slow lookup table. Build a partition whose memberships sum to one.
- **Believing the output range.** Centroid defuzzification cannot reach the extremes — here the output never leaves roughly [16.6, 83.4]. Never label it “percent”.
- **Adding inputs.** Three linguistic variables with three values each need 27 rules, four need 81. Fuzzy controllers stay small for a reason.
- **Expecting entailment.** There is no soundness theorem here and $A \cap \neg A \neq \emptyset$. This is a control technique, not a logic in Chapter 11’s sense.
- **Using it where probability belongs.** If the question is “will this release fail?” the answer is uncertain, not vague, and Chapter 14 is the right chapter.

🔧 Try It Yourself: Risk Rating by Mamdani Inference

The full pipeline on five projects, the hand-checkable trace for *Beacon*, the smoothness comparison of Figure 16.3, and the output-range limitation.

```
"""Chapter 16 lab -- rating project risk by Mamdani inference.

Two inputs, nine rules, one number out. The interesting outputs are
the smoothness table and the range check: a centroid defuzzifier
cannot reach the ends of its own scale.
"""

import numpy as np

def tri(x, a, b, c):
    """Triangular membership: 0 at a, 1 at b, 0 at c."""
    x = np.asarray(x, dtype=float)
    return np.clip(np.minimum((x - a) / (b - a), (c - x) / (c - b)), 0, 1)
```

```

# Fuzzy partitions: peaks evenly spaced, memberships summing to one,
# so every input is described by at most two words.
EFFORT = {
    "low": lambda x: tri(x, -50, 0, 50),
    "medium": lambda x: tri(x, 0, 50, 100),
    "high": lambda x: tri(x, 50, 100, 150),
}
UNCERTAINTY = {
    "low": lambda x: tri(x, -5, 0, 5),
    "medium": lambda x: tri(x, 0, 5, 10),
    "high": lambda x: tri(x, 5, 10, 15),
}
RISK = {
    "low": lambda x: tri(x, -50, 0, 50),
    "medium": lambda x: tri(x, 0, 50, 100),
    "high": lambda x: tri(x, 50, 100, 150),
}

# (effort word, uncertainty word) -> risk word
RULES = [
    ("low", "low", "low"), ("low", "medium", "low"),
    ("low", "high", "medium"),
    ("medium", "low", "low"), ("medium", "medium", "medium"),
    ("medium", "high", "high"),
    ("high", "low", "medium"), ("high", "medium", "high"),
    ("high", "high", "high"),
]

GRID = np.linspace(0, 100, 1001)

def mamdani(effort, uncertainty, trace=False):
    # 1. fuzzify
    me = {k: float(f(effort)) for k, f in EFFORT.items()}
    mu = {k: float(f(uncertainty)) for k, f in UNCERTAINTY.items()}
    if trace:
        print("    effort %g    -> %s" % (effort,
            {k: round(v, 3) for k, v in me.items() if v > 0}))
        print("    uncertainty %g -> %s" % (uncertainty,
            {k: round(v, 3) for k, v in mu.items() if v > 0}))

    # 2. firing strength = min of the conditions
    # 3. clip the consequent at that strength
    # 4. aggregate by max
    aggregate = np.zeros_like(GRID)
    fired = []
    for (e, u, r) in RULES:
        w = min(me[e], mu[u])
        if w > 1e-9:
            fired.append((e, u, r, w))
            aggregate = np.maximum(aggregate, np.minimum(w, RISK[r](GRID)))
    if trace:
        for (e, u, r, w) in fired:
            print("    %-6s x %-6s -> %-6s    strength %.3f"
                % (e, u, r, w))

    # 5. defuzzify by centroid
    area = aggregate.sum()
    crisp = float((GRID * aggregate).sum() / area) if area else 0.0
    return crisp, fired

```

```

def threshold_rule(effort, uncertainty):
    """The same knowledge as a crisp rule, for contrast."""
    return 90 if (effort > 40 and uncertainty > 7) else 10

PROJECTS = {
    "Atlas": (20, 2),
    "Beacon": (45, 4),
    "Cobalt": (85, 9),
    "Delta": (45, 9),
    "Echo": (90, 2),
}

if __name__ == "__main__":
    print("project effort uncert risk rules fired")
    print("-" * 50)
    for name, (e, u) in PROJECTS.items():
        risk, fired = mamdani(e, u)
        print("%-9s %6d %7d %7.2f %8d" % (name, e, u, risk, len(fired)))
        assert len(fired) == 4, "a partition should fire 2x2 rules"

    print()
    print("=== Beacon in full (effort 45, uncertainty 4) ===")
    risk, fired = mamdani(45, 4, trace=True)
    print(" crisp risk = %.2f" % risk)
    assert abs(risk - 49.04) < 0.01

    # the memberships are exactly the hand calculation
    assert abs(float(EFFORT["low"](45)) - 0.1) < 1e-9
    assert abs(float(EFFORT["medium"](45)) - 0.9) < 1e-9
    assert abs(float(UNCERTAINTY["low"](4)) - 0.2) < 1e-9
    assert abs(float(UNCERTAINTY["medium"](4)) - 0.8) < 1e-9

    # --- smoothness -----
    print()
    print("=== effort fixed at 45, uncertainty swept ===")
    print(" unc fuzzy threshold rule")
    print(" " + "-" * 34)
    curve = []
    for u in range(11):
        r, _ = mamdani(45, u)
        curve.append(r)
        print(" %3d %8.2f %12d" % (u, r, threshold_rule(45, u)))

    # the fuzzy output is monotone and moves in small steps
    steps = [curve[i + 1] - curve[i] for i in range(len(curve) - 1)]
    assert all(s > 0 for s in steps), "should be monotone increasing"
    print()
    print(" fuzzy: largest single step %.2f points" % max(steps))
    print(" threshold rule: one step of 80 points, at u = 7 -> 8")
    assert max(steps) < 20

    # --- the range limitation -----
    print()
    print("=== what range can the output actually reach? ===")
    lo, hi = 1e9, -1e9
    for e in range(0, 101, 5):
        for u in range(0, 11):
            r, _ = mamdani(e, u)
            lo, hi = min(lo, r), max(hi, r)
    print(" across the whole input space, risk lies in [%.2f, %.2f]"

```

```

    % (lo, hi))
print("    a minimum-effort, minimum-uncertainty project scores")
print("    %.2f, not 0. Centroid defuzzification cannot reach the"
      % mamdani(0, 0)[0])
print("    ends of its own scale, so never label this 'percent'.")
assert lo > 5 and hi < 95

# --- and the confusion this chapter exists to prevent -----
print()
print("=== membership is not probability ===")
print("    mu_high(effort=70) = %.3f" % float(EFFORT["high"](70)))
print("    This does NOT mean a 40% chance the project is high")
print("    effort. It is 70 person-days -- we are certain of that.")
print("    The number describes the word 'high', not our ignorance.")
print("    and no further measurement will change it.")
total = sum(float(f(70)) for f in EFFORT.values())
print("    memberships at 70 sum to %.3f (probabilities would sum"
      % total)
print("    to 1 only by coincidence; here it is by construction)")

```

Then widen the *medium* risk triangle to `tri(x, -25, 50, 125)` and re-run. Beacon's score barely moves, but the reachable range narrows further: the defuzzified output depends on the *shape* of the output sets as much as on the rules, which is the part of a fuzzy controller that is tuned rather than derived.

🔍 Checkpoint — Test Yourself

1. A project is measured at exactly 70 person-days and $\mu_{high}(70) = 0.4$. Explain why gathering more information about the project will not change that number, and what would.
2. Four of nine rules fired for *Beacon*, and the same four would fire for any input in a neighbourhood of it. Explain what property of the membership functions guarantees “at most four”, and what goes wrong if the membership functions do not overlap.
3. The controller's output never leaves roughly $[16.6, 83.4]$. Give the reason, and one consequence for how the score should be reported to a project board.

📋 Chapter Summary

- **Vagueness** is not **uncertainty**. A probability describes ignorance about a sharp fact and resolves with evidence; a **degree of membership** describes a blurry word and does not.
- A **fuzzy set** is a membership function $\mu_A(x) \in [0, 1]$. A **fuzzy partition** — overlapping triangles whose memberships sum to one — means each input is described by at most two words and the description moves continuously.
- The operators are min, max and $1 - \mu$. Note that $A \cap \neg A \neq \emptyset$: there is no entailment relation and no soundness theorem, and this is a control technique rather than a logic.
- **Mamdani inference**: fuzzify, take each rule's strength as the min of its conditions, clip its consequent at that strength, aggregate by max, and defuzzify by **centroid**. For *Beacon* four of nine rules fired and the answer was 49.04.
- Centroid defuzzification *cannot reach the extremes* — the output here lives in roughly $[16.6, 83.4]$ whatever the input. Do not call it a percentage.
- The engineering justification is *smoothness*: the output moves a little when the input moves a little, everywhere, where a threshold rule jumps eighty points between two adjacent inputs. A handful of sentences from a domain expert becomes a continuous response surface.

- Rules grow as v^n in the number of inputs — nine here, 81 for four inputs — which is why fuzzy controllers stay small.

Review Questions

1. **(MCQ)** A degree of membership $\mu_A(x) = 0.4$ means: (a) a 40% chance x is in A (b) x belongs to A to degree 0.4 (c) 40% of objects like x are in A (d) A is 40% certain.
2. **(MCQ)** In Mamdani inference a rule's firing strength is: (a) the product of its condition memberships (b) the min of them (c) the max of them (d) their average.
3. **(MCQ)** Aggregation across rules combines the clipped consequents by: (a) min (b) max (c) addition (d) multiplication.
4. **(MCQ)** Centroid defuzzification: (a) always reaches the extremes of the output range (b) compresses the output towards the middle (c) is discontinuous (d) requires probabilities.
5. **(Short)** Give one question about a software release that is a matter of uncertainty and one that is a matter of vagueness, and say which chapter handles each.
6. **(Short)** Explain why membership functions must overlap, and what a fuzzy controller degenerates into if they do not.
7. **(Short)** State the engineering argument for a fuzzy controller over a threshold rule, using Figure 16.3.
8. **(Applied)** Compute by hand the fuzzy output for a project with effort 75 and uncertainty 6: give the memberships, the firing strengths of every rule that fires, which output words are clipped and at what level, and say whether the crisp result should be above or below 50, with a reason.

PART VI

Learning, Language and Responsible AI

The course closes by placing itself. One chapter on learning — enough to know what it is and where this book stops. The symbolic half of language. The four classic systems, read through machinery you can now write yourself. What the current wave adds, what it does not change, and where the limits are. Then your own project.

the course stops here



*Machine Learning,
Semester 6, takes over*

*both read through Chap-
ters 9, 12, 13 and 18*

Figure P6.1: Reading path through Part VI. Chapter 17 (outlined in red) is a signpost, not a unit; Chapter 21 (highlighted) is where the limits of everything in the book are stated plainly.

Learning from Examples

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part VI — Learning, Language and Responsible AI

🕒 Learning Outcomes

By the end of this chapter you will be able to:

- **Name** the three learning paradigms and **give** a delivery example of each.
- **Train** a perceptron by hand on a small table, applying the learning rule step by step.
- **Show** that a single perceptron cannot represent exclusive-or, and **explain** what that result did to the field.
- **Recognise** non-convergence from a training trace, and **distinguish** it from slow convergence.
- **State** precisely where this course stops and *Machine Learning* begins.

🔗 Before You Start

Chapter 1 — and specifically its claim that learning is one route to intelligent behaviour rather than the definition of it. Chapter 14 for the probabilistic view of evidence. Nothing in Parts II to V depends on this chapter.

📖 Key Terms in This Chapter

supervised learning • unsupervised learning • reinforcement learning • training example • feature • label • hypothesis • generalisation • perceptron • weight • bias • learning rate • perceptron learning rule • epoch • decision boundary • linear separability • exclusive-or • artificial neural network

17.1 One Chapter, and Why

Everything so far has been *told* the rules. Constraints, operators, logical sentences, conditional probability tables — somebody wrote them down, and the agent's job was to reason with them.

This chapter is about the other case: the rules are not available, and only examples are. It exists for three reasons, and it is important to be clear which.

The HEC outlines require it. The 2017 outline names supervised, unsupervised and reinforcement learning; the 2023 outline names artificial neural networks and learning from examples. This chapter covers all of them, in named sections, and the Course Specification traces each clause here.

You cannot judge symbolic methods without it. Chapter 13 argued that the expert-system industry collapsed on the knowledge acquisition bottleneck, and that machine learning dissolves it. That argument is empty unless you have seen a program acquire a rule from data.

And it is a signpost. *Machine Learning* (Semester 6) has this course as its published prerequisite and owns the subject entirely. This chapter goes as far as the perceptron and the exclusive-or limit — the exact point at which the historical field stalled for fifteen years — and then stops. Appendix F states the boundary chapter by chapter.

17.2 Three Kinds of Learning

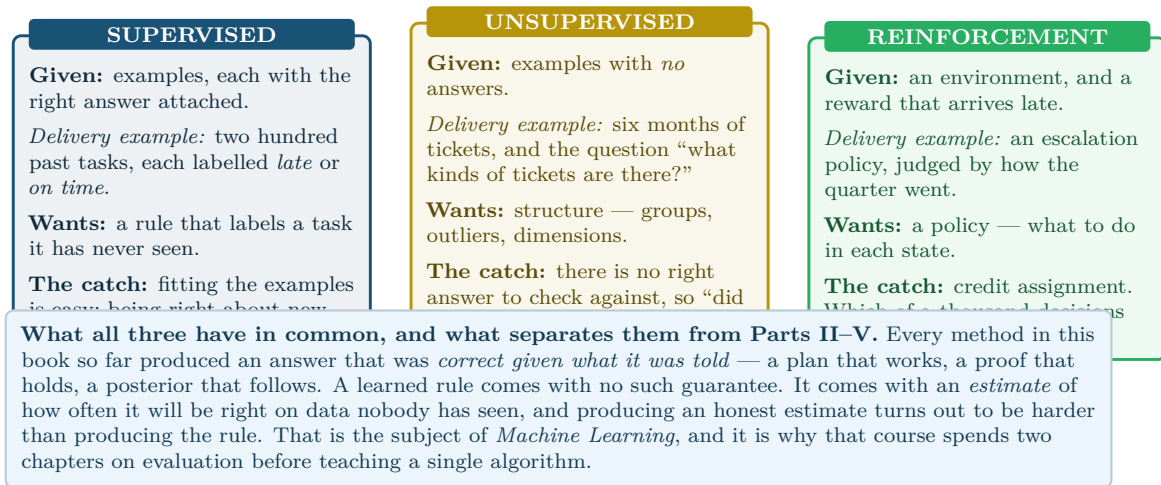


Figure 17.1: The three learning paradigms, with a delivery example of each. This section discharges the HEC 2017 outline’s *Learning* clause; the sections that follow take the first one further.

17.3 Supervised Learning

Two hundred tasks have been completed. For each, the team recorded the estimated effort in days and a clarity score for the requirement, and whether the task finished late. The question is whether a rule can be extracted that predicts lateness for the next task.

Here are twelve of them, enough to work by hand:

Task	Effort	Clarity	Late?	Task	Effort	Clarity	Late?
1	2	9	no	7	9	3	yes
2	3	8	no	8	8	2	yes
3	4	9	no	9	10	4	yes
4	3	7	no	10	9	1	yes
5	5	8	no	11	7	2	yes
6	2	6	no	12	8	4	yes

Nobody stated a rule. The rule — if there is one — has to come out of the table.

The perceptron

A **perceptron** computes a weighted sum of its inputs and fires if the sum exceeds a threshold:

$$\hat{y} = \begin{cases} 1 & \text{if } \sum_i w_i x_i + b > 0 \\ 0 & \text{otherwise} \end{cases}$$

The **weights** w_i say how much each feature counts and in which direction; the **bias** b shifts the threshold.

Learning means adjusting them. For each example in turn, apply the **perceptron learning rule**:

$$w_i \leftarrow w_i + \eta (y - \hat{y}) x_i, \quad b \leftarrow b + \eta (y - \hat{y})$$

where y is the true label and η the **learning rate**. If the prediction is right, $y - \hat{y} = 0$ and nothing changes. If it is wrong, every weight moves in the direction that would have helped, by an amount proportional to the input.

After F. Rosenblatt, “The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain”, *Psychological Review* 65, 1958.

Trained on the twelve tasks with $\eta = 0.1$, starting from all-zero weights, the perceptron converges after **three passes through the data and only two weight updates**, reaching 100% accuracy. The rule it found is

$$0.70 \times \text{effort} - 0.60 \times \text{clarity} > 0$$

which reads: *a task is predicted late when its effort outweighs its clarity*, in roughly a 6:7 ratio. Nobody wrote that down. It was extracted from twelve rows, and that is the thing Chapter 13’s knowledge engineer could not do.

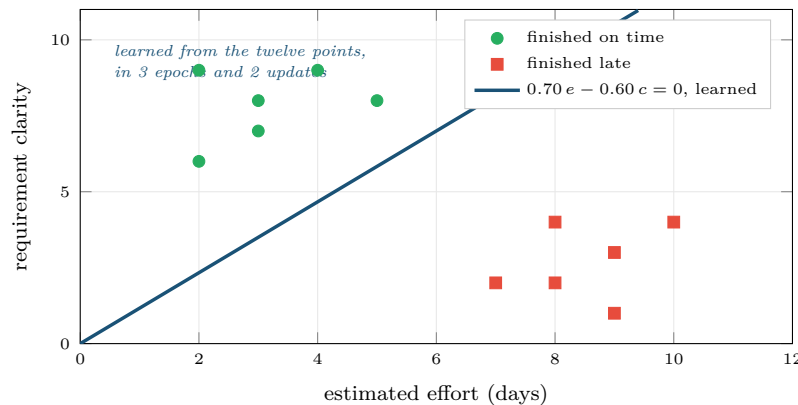


Figure 17.2: The twelve tasks and the boundary the perceptron learned. Everything above the line is predicted on time, everything below it late.

17.4 Artificial Neural Networks

A perceptron is a single artificial neuron: inputs, weights, a summation and a threshold, loosely modelled on a biological cell. An **artificial neural network** connects many of them in layers, so that the outputs of one layer are the inputs of the next.

The idea is old — McCulloch and Pitts proposed the artificial neuron in 1943 — and the reason it took until the 1980s to become useful is the subject of the next section. What a layered network buys is the ability to represent decision boundaries that are not straight lines, and what it costs is a learning rule: the perceptron rule works because we know the right answer at the output, and in a hidden layer we do not. Solving that is **backpropagation**, and it is Chapter 13 of *Machine Learning*.

17.5 Linear Separability and the Exclusive-Or Limit

Look again at Figure 17.2. The perceptron found *a straight line*. That is all a perceptron can ever find, because $\sum w_i x_i + b = 0$ is the equation of a line (a plane, a hyperplane). A dataset that some line can separate is **linearly separable**.

Now a small change to the delivery story. Suppose the team observes that tasks go late when *exactly one* of two things is true — the effort is large, or the requirement is unclear — and that when *both* are true everybody knows it is hard, plans accordingly, and it lands on time. Four representative tasks:

Effort	Clarity	Late?	Why
small (2)	high (9)	no	easy, and understood
large (9)	low (2)	no	obviously hard; planned for
large (9)	high (9)	yes	looked easy because it was clear
small (2)	low (2)	yes	looked easy because it was small

That is **exclusive-or**, and no straight line separates those four points.

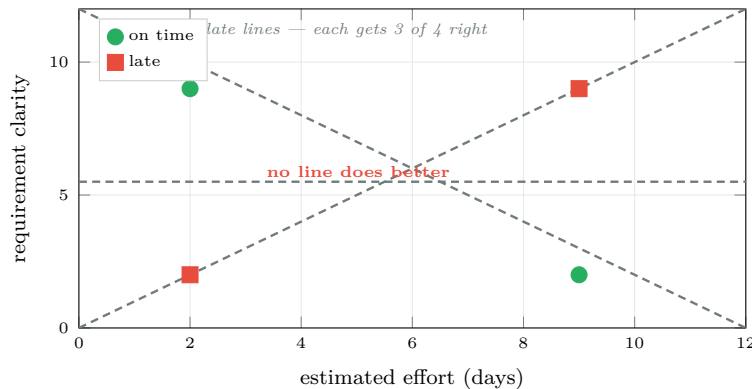


Figure 17.3: The exclusive-or arrangement. The two *late* tasks sit at opposite corners, and so do the two *on time* tasks. Any straight line has at least one point on the wrong side.

Run the perceptron on it and it never stops. The lab allows it 100, then 1,000, then 10,000 epochs and counts 383, 3,983 and 39,983 weight updates — the count simply grows with the budget, because the weights are still oscillating when the budget runs out. Nothing in the trace says “impossible”; it looks exactly like slow convergence.

⚠ Proving the limit, not merely observing it

Failing to converge is weak evidence. The learning rate might be wrong, the initialisation unlucky, the budget too small — any practitioner has seen all three.

So the lab settles it by brute force: it sweeps a grid of every line $w_0x + w_1y + b = 0$ over a wide range of coefficients and records the best accuracy any of them achieves. The answer is **75%** — three points out of four — and no line in the grid does better.

That is the difference between “my training run failed” and “this hypothesis class cannot represent this function”. The first is an engineering problem; the second is a mathematical fact, and no amount of training fixes it. Learning to tell them apart is worth more than the perceptron rule itself.



In 1969 Minsky and Papert published exactly this analysis — with more rigour and over a wider class of problems — in *Perceptrons*. The effect on the field was severe: funding for neural network research largely stopped, and the line stayed closed for roughly fifteen years. Two things are worth knowing about that episode, because it is routinely retold badly.

The result is correct. A single-layer perceptron genuinely cannot represent exclusive-or, and the lab reproduces the proof.

The conclusion drawn from it was not. A network with a hidden layer can represent exclusive-or easily; what was missing in 1969 was a way to *train* one, and backpropagation supplied it in the mid-1980s. So the field abandoned a line of work because a limitation of the simplest

case was read as a limitation of the approach — which is Chapter 1’s recurring pattern, a true result about reach mistaken for a verdict.

17.6 Unsupervised Learning

Six months of tickets, no labels, and the question *what kinds of ticket do we actually get?*

There is no right answer to check against, which is what makes this different in kind rather than in difficulty. A program can group tickets by similarity and the groups may be illuminating or useless; nothing in the data says which. Judging the result requires a person who knows the domain, and inventing measures that stand in for that judgement is an active research area.

That is as far as this course goes. *Machine Learning* Chapters 15 and 16 give *k*-means, hierarchical clustering, principal component analysis and self-organizing maps, with the evaluation machinery that makes them usable.

17.7 Reinforcement Learning

An escalation policy is not judged ticket by ticket. It is judged by how the quarter went — and a quarter contains a thousand decisions, of which perhaps three mattered.

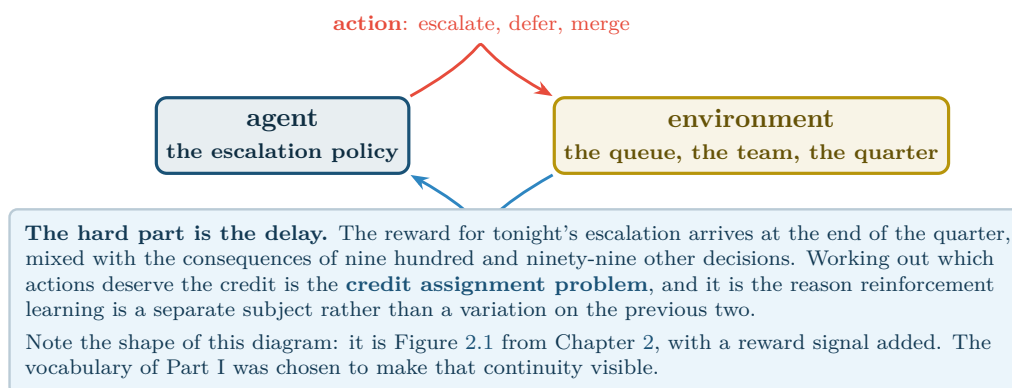


Figure 17.4: The reinforcement learning loop. One figure is all this course gives it; *Machine Learning* Chapters 18–20 develop Markov decision processes, Monte Carlo methods and Q-learning.

17.8 Where This Course Hands Over

⚠ The boundary, stated exactly

This chapter has taught: the three paradigms and what distinguishes them; a perceptron trained by hand; the fact that it finds a straight line; and a proof that some problems have no straight line.

This course does not teach, and *Machine Learning* does: multi-layer networks and backpropagation (its Chapter 13); how to tell whether a learned rule will work on data it has not seen — generalisation, overfitting, the bias–variance trade-off, cross-validation (Chapters 7 and 8, taught *before* the algorithms, deliberately); decision trees, naive Bayes, *k*-nearest neighbours, support vector machines, ensembles (Chapters 9–14); clustering and dimensionality reduction (15–17); and Markov decision processes and reinforcement learning (18–20).

Why not here. Two reasons, and neither is that the material is too hard. It would duplicate a course three semesters later that has the statistical prerequisites to do it properly — and the room it would take comes out of the symbolic material, which no other core course in the

degree covers at all. A graduate who has not met resolution or arc consistency will not meet them anywhere else; a graduate who has not met backpropagation meets it in Semester 6.

Worked Example: Training the perceptron by hand

Weights start at zero, $\eta = 0.1$, and the rule fires when $w_1e + w_2c + b > 0$. Take the examples in order.

Task 1 $(e, c) = (2, 9)$, not late. Sum = 0, which is not > 0 , so $\hat{y} = 0$. Correct, and *nothing changes* — the rule only learns from mistakes.

Tasks 2–6 are all *not late* and all still predicted 0. No change.

Task 7 $(9, 3)$, late. Sum = 0, so $\hat{y} = 0$ and $y - \hat{y} = 1$. Update:

$$w_1 = 0 + 0.1 \times 1 \times 9 = 0.9, \quad w_2 = 0 + 0.1 \times 1 \times 3 = 0.3, \quad b = 0 + 0.1 = 0.1$$

Now the rule is $0.9e + 0.3c + 0.1 > 0$ — which predicts *everything* late, since all inputs are positive. It has over-corrected, and that is normal.

Tasks 8–12 are all late and now all predicted late. No change. End of the first pass: **one error**.

Second pass, task 1 $(2, 9)$, not late. Sum = $0.9(2) + 0.3(9) + 0.1 = 1.8 + 2.7 + 0.1 = 4.6 > 0$, so $\hat{y} = 1$: wrong. Now $y - \hat{y} = -1$:

$$w_1 = 0.9 - 0.1(2) = 0.7, \quad w_2 = 0.3 - 0.1(9) = -0.6, \quad b = 0.1 - 0.1 = 0.0$$

The clarity weight has gone negative, which is the moment the rule becomes sensible: more clarity now argues *against* lateness. The learning rule was not told that; it followed from one mistake on one example.

Tasks 2–12 in the second pass are all now predicted correctly. End of pass two: **one error**.

Third pass: zero errors. Converged, after 3 epochs and 2 updates, with

$$0.70e - 0.60c > 0$$

What to take from it. Both updates came from a single misclassified example, and the second one produced the insight — a negative weight on clarity — that a person would have had to think of. That is the appeal of learning, in twelve rows and two updates.

And what to be careful about. The rule has 100% accuracy *on the data it was trained on*, which is the least interesting fact about it. Whether it is right about the next task is a different question, and this course does not have the machinery to answer it. That question is the first thing *Machine Learning* takes up.

Common Pitfalls

- **Reading training accuracy as performance.** 100% on the twelve rows it learned from says almost nothing about the thirteenth task.
- **Mistaking non-convergence for slow convergence.** The XOR trace looks identical to a run that needs more epochs. Test whether *any* hypothesis in the class can fit before tuning anything.
- **Concluding that neural networks cannot do exclusive-or.** A single perceptron cannot; a hidden layer can, easily. The 1969 result was correct and the inference drawn from it was not.

- **Expecting a learned rule to come with a guarantee.** Parts II to V produced plans that work and proofs that hold. A learned rule comes with an error estimate, and producing an honest one is harder than producing the rule.
- **Treating unsupervised learning as supervised learning without labels.** There is no right answer to check against, which changes what “it worked” can even mean.
- **Believing this chapter is a machine learning course.** It is four sections and a signpost. Appendix F lists what is missing.

Try It Yourself: The Perceptron and Its Limit

The learning rule from scratch on twelve tasks, then on the exclusive-or arrangement, then the brute-force proof that no line can do it.

```

"""Chapter 17 lab -- the perceptron, and where this course stops.

Two datasets and one proof. The first converges in three passes. The
second never converges, and the grid sweep at the end shows that this
is a fact about straight lines rather than about the training run.
"""

# twelve past tasks: (effort days, requirement clarity) -> late?
TASKS = [
    ((2, 9), 0), ((3, 8), 0), ((4, 9), 0), ((3, 7), 0),
    ((5, 8), 0), ((2, 6), 0),
    ((9, 3), 1), ((8, 2), 1), ((10, 4), 1), ((9, 1), 1),
    ((7, 2), 1), ((8, 4), 1),
]

# the exclusive-or arrangement: late when EXACTLY ONE of
# "large effort" and "low clarity" holds
XOR_TASKS = [
    ((2, 9), 0),    # small effort, high clarity -> on time
    ((9, 2), 0),    # large effort, low clarity -> on time; they planned
    ((9, 9), 1),    # large effort, high clarity -> late; looked easy
    ((2, 2), 1),    # small effort, low clarity -> late; looked small
]

def predict(x, w, b):
    return 1 if w[0] * x[0] + w[1] * x[1] + b > 0 else 0

def accuracy(data, w, b):
    return sum(predict(x, w, b) == y for x, y in data) / len(data)

def train(data, eta=0.1, epochs=100, trace=False):
    """The perceptron learning rule. Learns only from mistakes."""
    w, b, updates = [0.0, 0.0], 0.0, 0
    for ep in range(1, epochs + 1):
        errors = 0
        for (x, y) in data:
            err = y - predict(x, w, b)
            if err:
                errors += 1
                updates += 1
                w[0] += eta * err * x[0]
                w[1] += eta * err * x[1]
                b += eta * err
        if trace:
            print("    epoch %2d: %d error(s), w = (%.2f, %.2f), b = %.2f"

```

```

        % (ep, errors, w[0], w[1], b))
    if errors == 0:
        return w, b, ep, updates, True
    return w, b, epochs, updates, False

def best_line_accuracy(data, span=20, step=0.5):
    """Brute force over a grid of ALL lines  $w_0x + w_1y + b = 0$ .

    This is the difference between 'training failed' and 'no line
    exists'. It is only possible because the problem is tiny.
    """
    best, arg = 0.0, None
    vals = [i * step for i in range(-span * 2, span * 2 + 1)]
    for w0 in vals:
        for w1 in vals:
            for bb in vals:
                a = accuracy(data, [w0, w1], bb)
                if a > best:
                    best, arg = a, (w0, w1, bb)
    return best, arg

if __name__ == "__main__":
    print("=== twelve past tasks ===")
    w, b, epochs, updates, converged = train(TASKS, trace=True)
    print("    converged: %s, after %d epochs and %d weight updates"
          % (converged, epochs, updates))
    print("    learned rule:  %.2f*effort %+.2f*clarity %+.2f > 0"
          % (w[0], w[1], b))
    print("    accuracy on the training data: %.0f%%"
          % (100 * accuracy(TASKS, w, b)))
    assert converged and epochs == 3 and updates == 2
    assert accuracy(TASKS, w, b) == 1.0
    # the clarity weight went NEGATIVE: more clarity argues against late
    assert w[1] < 0
    print()
    print("    The weight on clarity is negative. Nobody said that;")
    print("    it came from one misclassified example in pass two.")
    print()
    print("    But 100% here is accuracy on the data it LEARNED from.")
    print("    Whether it is right about the next task is a different")
    print("    question, and this course cannot answer it.")

    print()
    print("=== the exclusive-or arrangement ===")
    for (x, y) in XOR_TASKS:
        print("    effort %2d, clarity %2d -> %s"
              % (x[0], x[1], "late" if y else "on time"))

    print()
    print("    epochs allowed    converged    weight updates")
    print("    " + "-" * 44)
    for budget in (100, 1000, 10000):
        _, _, _, ups, conv = train(XOR_TASKS, epochs=budget)
        print("    %14d    %9s    %13d" % (budget, conv, ups))
    _, _, _, _, conv = train(XOR_TASKS, epochs=10000)
    assert not conv
    print()
    print("    The update count just grows with the budget: the")
    print("    weights are still oscillating when time runs out.")
    print("    Nothing here says 'impossible' -- it looks exactly")

```

```

print("    like slow convergence.")

# so settle it by brute force
print()
print("=== can ANY straight line do it? ===")
best, arg = best_line_accuracy(XOR_TASKS)
print("    best accuracy over a grid of every line: %.0f%%"
      % (100 * best))
print("    achieved by %.1f*effort %+.1f*clarity %+.1f > 0"
      % arg)
assert best == 0.75
print()
print("    Three of four, and no line does better. This is not a")
print("    training failure -- it is a fact about the hypothesis")
print("    class. No learning rate, initialisation or patience")
print("    fixes it; only a richer class does, and a hidden layer")
print("    is exactly that.")

# and the same sweep on the separable data, for contrast
best2, _ = best_line_accuracy(TASKS)
assert best2 == 1.0
print()
print("    (The same sweep on the twelve tasks finds a line with")
print("    100% accuracy, which is why the perceptron found one.)")

print()
print("==" * 58)
print("Machine Learning Fall 2026, Chapter 13, takes over here.")
print("==" * 58)

```

Two things to try. Set `eta` to 10 and re-run the first dataset: it still converges, because the perceptron convergence theorem does not depend on the learning rate — only on separability. Then change one label in `TASKS` to make the data non-separable, and watch the first dataset start behaving like the second.

🔍 Checkpoint — Test Yourself

1. The perceptron made only two weight updates in three passes over twelve examples. Explain why so few, and what the second update changed about the meaning of the rule.
2. A colleague's perceptron has run for ten thousand epochs without converging. Give two possible explanations and describe the single cheapest experiment that distinguishes them.
3. State what a single perceptron cannot represent, what the 1969 conclusion from that result was, and why the conclusion was wrong.

📋 Chapter Summary

- **Supervised** learning is given labelled examples and wants a rule that generalises; **unsupervised** learning is given unlabelled data and wants structure, with no right answer to check against; **reinforcement** learning is given delayed reward and must solve **credit assignment**.
- A **perceptron** fires when $\sum w_i x_i + b > 0$. The **perceptron learning rule** adjusts weights only on mistakes, by $\eta(y - \hat{y})x_i$.
- On twelve separable tasks it converged in 3 epochs and 2 updates to $0.70e - 0.60c > 0$. The negative weight on clarity — the insight in the rule — came from one misclassified example.

- A perceptron can only ever find a *straight line*. The **exclusive-or** arrangement has no separating line, and the lab proves it by sweeping every line and finding that the best reaches 75%.
- Distinguish *training failed* from *no hypothesis in the class fits*. The traces look identical; only the second is a mathematical fact, and no tuning fixes it.
- Minsky and Papert's 1969 result was correct and the inference drawn from it was not: a hidden layer represents exclusive-or easily, and what was missing was a way to train one.
- This is a signpost, not a unit. Generalisation, evaluation, backpropagation, trees, kernels, ensembles, clustering and reinforcement learning belong to *Machine Learning* in Semester 6; Appendix F lists the boundary chapter by chapter.

Review Questions

1. **(MCQ)** The perceptron learning rule updates the weights: (a) after every example (b) only when the prediction is wrong (c) once per epoch (d) only at the end of training.
2. **(MCQ)** A single perceptron can represent exactly the functions that are: (a) monotone (b) linearly separable (c) continuous (d) boolean.
3. **(MCQ)** The best accuracy any straight line achieves on the exclusive-or arrangement is: (a) 50% (b) 75% (c) 90% (d) 100%.
4. **(MCQ)** Credit assignment is the central difficulty of: (a) supervised learning (b) unsupervised learning (c) reinforcement learning (d) concept learning.
5. **(Short)** Explain why a training run that has not converged is weak evidence that the problem is unlearnable, and what would be strong evidence.
6. **(Short)** The learned rule scores 100% on its training data. State why that is the least interesting fact about it, and name the course that addresses the interesting question.
7. **(Short)** Give one delivery question that is supervised, one that is unsupervised and one that is reinforcement learning, and say what makes each one that kind.
8. **(Applied)** Work the perceptron by hand on the four exclusive-or tasks for two full passes, starting from zero weights with $\eta = 0.1$. Show the weights after each update, and identify the point at which the sequence begins to repeat.

Natural Language Processing

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part VI — Learning, Language and Responsible AI

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Describe** the symbolic language pipeline from characters to meaning, and **say** what each stage contributes.
- **Write** a context-free grammar for a small sub-language and **parse** sentences with the CYK algorithm.
- **Identify** structural ambiguity, **count** the parses, and **explain** why the ambiguity is operationally consequential.
- **Implement** pattern substitution in the style of Eliza, and **say** precisely what it demonstrates and what it does not.
- **Build** an n-gram model and **state** the limitation that no amount of data removes.

🔗 Before You Start

Chapter 3 (dictionaries, recursion) and Chapter 12 — pattern matching with variables is unification in another guise, and recognising that saves learning it twice. Chapter 14 for the n-gram section.

📖 Key Terms in This Chapter

tokenisation • morphology • stemming • part of speech • context-free grammar • terminal and non-terminal • Chomsky normal form • parse tree • chart parsing • CYK • structural ambiguity • attachment • pattern substitution • Eliza • n-gram • Markov assumption • sparsity

18.1 Ticket Subject Lines

The service desk receives several hundred tickets a day, each with a one-line subject written by a person in a hurry:

```
restart the server after the timeout
check the payments gateway
fix the login error on the gateway
```

The routing system needs to know what is being asked for and what it is being asked about. That is a language problem, and this chapter takes the *symbolic* approach to it: grammars, parsing and pattern matching, with no learned component anywhere.

That approach is not how a modern system would do it, and the reasons to study it are specific. Grammars are the clearest case of a representation whose structure *is* the knowledge. Ambiguity is easiest to see when a parser hands you two trees rather than one confident answer. And Eliza, in Section 18.6, is the most instructive failure in the history of the field — it is impossible to understand what modern language systems are and are not doing without it.

18.2 The Symbolic Pipeline

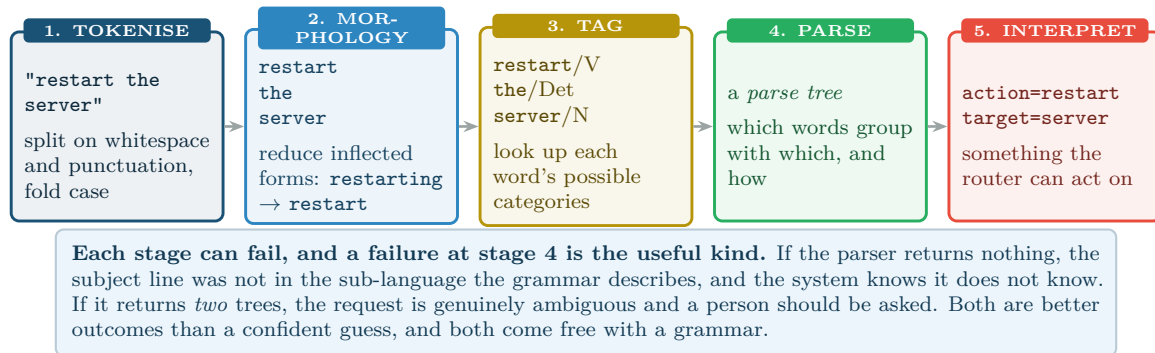


Figure 18.1: The symbolic pipeline. Stages 1 to 3 are bookkeeping; stage 4 is where the knowledge lives.

18.3 Context-Free Grammars

Context-free grammar

A **context-free grammar** is a set of rules $A \rightarrow \beta$, where A is a **non-terminal** (a category such as NP) and β a sequence of non-terminals and **terminals** (words). A string is **in the language** if the start symbol can be rewritten to it.

“Context-free” means a rule may be applied wherever its left side appears, regardless of surroundings — which is what makes efficient parsing possible and what makes the formalism too weak for some natural-language phenomena.

The grammar for ticket subjects is seven rules:

Rule	Reading
$S \rightarrow VP$	an imperative: “restart the server”
$S \rightarrow NP VP$	a statement: “the server crashed”
$VP \rightarrow V NP$	a verb and its object
$VP \rightarrow VP PP$	a verb phrase modified by a phrase
$NP \rightarrow Det N$	a determiner and a noun
$NP \rightarrow NP PP$	a noun phrase modified by a phrase
$PP \rightarrow P NP$	a preposition and its object
$N \rightarrow N N$	a noun compound: “payments gateway”

Every rule has either two non-terminals or one terminal on its right, which is **Chomsky normal form** — the shape the CYK algorithm needs.

18.4 Parsing with CYK

CYK is dynamic programming over spans. Fill a triangular chart: cell (i, j) holds every category that could cover words i to j . Single words come from the lexicon; longer spans are built by trying every split point k and every rule $A \rightarrow BC$, entering A whenever B covers (i, k) and C covers (k, j) . The sentence parses if the start symbol reaches the top cell.

Its cost is $O(n^3|G|)$ — cubic in the sentence length — which for a subject line of seven words means a chart of 28 cells, of which 18 are non-empty. The important property is that it finds *all* parses, and it finds them without backtracking.

18.5 Ambiguity Is the Point

</> Five subject lines, parsed

Subject	Parses?	How many
restart the server	yes	1
check the payments gateway	yes	1
restart the server after the timeout	yes	2
fix the login error on the gateway	yes	2
restart the gateway after the payments error	yes	2
the the server restart	no	0

Three of the five are ambiguous, and the ambiguity is not a quirk of the grammar. Consider:

restart the server after the timeout

Does *after the timeout* attach to the verb phrase — restart it, and do so after the timeout occurs — or to the noun phrase, meaning *the server that handles timeouts*? The two readings give different instructions, and the grammar correctly reports that the sentence supports both.

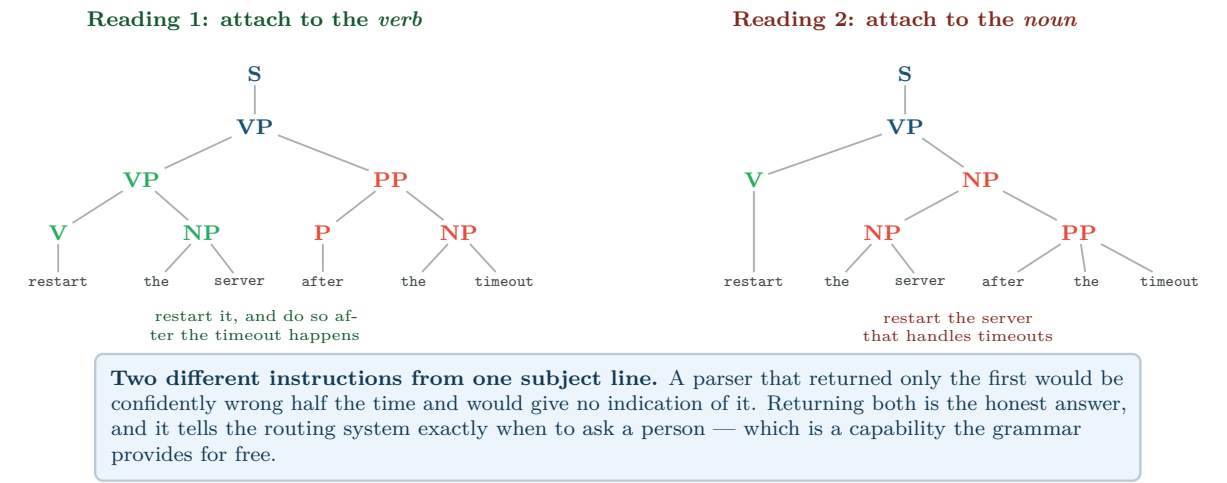


Figure 18.2: Prepositional-phrase attachment ambiguity. The words and their order are identical; only the structure differs, and the structure is the meaning.

Ambiguity multiplies. Each additional prepositional phrase roughly doubles the number of parses — the counts follow the Catalan numbers — so a sentence with six of them has hundreds of readings. Resolving them requires knowledge the grammar does not contain, which is where statistical and, later, neural methods enter.

18.6 Pattern Substitution: Eliza

In 1966 Joseph Weizenbaum wrote a program of a few hundred lines that conducted conversations in the manner of a Rogerian psychotherapist. It had no grammar, no parser, no representation of the conversation and no model of anything. It matched patterns and rearranged pronouns.

⌘ A service-desk Eliza

User says	Rule that matched, and the reply
I am worried about the release	I am * → How long have you been *? “How long have you been worried about the release?”
my deploy keeps failing	my * keeps * → Tell me more about your *. “Tell me more about your deploy.”
because the config is wrong	because * → Is that the real reason? “Is that the real reason?”
no idea what is happening	(no pattern) → fallback “Can you tell me more about that?”

The mechanism is exactly Chapter 12’s unification, restricted to strings: a pattern containing a variable is matched against an input, the variable binds to whatever it covered, and the binding is substituted into a template. Weizenbaum added one more trick — swap *my* for *your* and *I* for *you* in the captured text — and that is the whole program.

⚠ The Eliza effect

Weizenbaum’s own account is that people who knew exactly how the program worked — his secretary among them — nonetheless asked to be left alone with it, and disclosed things they would not have told a colleague. He found this so disturbing that he spent much of the rest of his career arguing against the uses being proposed for such systems.

The lesson named after the episode — the **Eliza effect** — is that *people attribute understanding to any system that produces fluent, contextually apt responses*, and that the attribution is nearly impossible to suppress even with full knowledge of the mechanism.

That is worth holding onto when you meet Chapter 20. The question is not whether a system’s output feels like understanding; Eliza’s did, in two hundred lines, in 1966. The question is what the system *represents* and what it can be held to — which is what Chapter 1’s five-point checklist asks, and why the checklist is the right instrument rather than your impression of the transcript.

18.7 N-Grams and Their Limit

The alternative to grammar is counting. An **n-gram** model estimates the probability of a word from the $n - 1$ words before it:

$$P(w_i \mid w_1 \dots w_{i-1}) \approx P(w_i \mid w_{i-n+1} \dots w_{i-1})$$

This is the **Markov assumption**: everything before the window is irrelevant. Estimating the right-hand side is just counting, which is why n-gram models were the backbone of speech recognition and machine translation for decades and why they still make excellent autocompletions for a constrained domain such as ticket subjects.

Two limitations, and only one of them is fixable.

Sparsity is fixable. Most possible trigrams never appear, even in a large corpus, so unseen ones get probability zero and the model assigns zero to any sentence containing one. Smoothing — reserving probability mass for the unseen — handles it, and *Machine Learning* Chapter 10 covers the standard methods.

The fixed window is not. In the server that the engineer who wrote the runbook maintained was restarted, the subject of *was restarted* is *the server*, nine words back. No n -gram with a practical

n can connect them, and increasing n makes sparsity exponentially worse. The dependency is a structural fact about the sentence, and capturing it is exactly what a grammar does and what counting cannot.

That tension — grammars capture structure but are brittle, counting is robust but structurally blind — defined the field for thirty years. Chapter 20 describes what resolved it.

Worked Example: Filling the CYK chart, by hand

Parse **restart the server** with the grammar above. Three words, so the chart is a triangle of six cells.

Row 1 — single words, from the lexicon.

- $(0, 1)$ **restart**: $\{V\}$.
- $(1, 2)$ **the**: $\{Det\}$.
- $(2, 3)$ **server**: $\{N\}$.

Row 2 — spans of two.

- $(0, 2)$ covering **restart the**: the only split is $k = 1$, giving V then Det . No rule has right side $V Det$, so the cell is **empty**.
- $(1, 3)$ covering **the server**: split at $k = 2$ gives Det then N , and $NP \rightarrow Det N$ applies. Cell: $\{NP\}$.

Row 3 — the whole sentence, $(0, 3)$. Two split points:

- $k = 1$: cell $(0, 1) = \{V\}$ and cell $(1, 3) = \{NP\}$. The rule $VP \rightarrow V NP$ applies, so VP is entered.
- $k = 2$: cell $(0, 2)$ is empty, so nothing.

Then the unit rule $S \rightarrow VP$ adds S . Cell $(0, 3) = \{VP, S\}$, the start symbol is present, and the sentence parses — by exactly one derivation.

Now the ambiguous case. Add **after the timeout**. At the top cell there are now *two* ways to reach S : one via $VP \rightarrow VP PP$ (split after **server**), and one via $VP \rightarrow V NP$ where the NP was itself built by $NP \rightarrow NP PP$ (split after **restart**). Both entries record S in the same cell, which is why counting parses means counting *derivations* rather than checking membership — and why the lab keeps back-pointers.

Common Pitfalls

- **Treating a single parse as the answer.** Three of the five subject lines here have two readings. A parser that returns the first is confidently wrong and silent about it.
- **Forgetting Chomsky normal form.** CYK needs rules of the form $A \rightarrow BC$ or $A \rightarrow w$. A grammar written naturally must be converted first.
- **Assuming a bigger grammar is a better one.** Every added rule multiplies the ambiguity. Real broad-coverage grammars return thousands of parses per sentence, which is why ranking them became the whole problem.
- **Mistaking Eliza's fluency for understanding.** It has no representation of anything. If the impression of understanding is your evidence, you have no evidence.
- **Believing more data fixes the n-gram window.** Sparsity is fixable by smoothing; a fixed window cannot represent a nine-word dependency at any corpus size.
- **Parsing when you could match.** If the domain is genuinely a few dozen command shapes, patterns are shorter, faster and easier to maintain. Reach for a grammar when structure matters.

Try It Yourself: Parsing Ticket Subjects

A CYK parser with back-pointers, the ambiguity count, an Eliza responder, and a bigram model with its limitation demonstrated.

```

"""Chapter 18 lab -- symbolic language processing on ticket subjects.

A CYK chart parser, a pattern-substitution responder in the style of
Eliza, and a bigram model. Nothing here learns; the knowledge is in
the grammar and the patterns.
"""

import re
from collections import defaultdict

# --- the grammar, in Chomsky normal form -----
RULES = [
    ("S", ("VP",)),                                # imperative: "restart the server"
    ("S", ("NP", "VP")),                            # statement: "the server crashed"
    ("VP", ("V", "NP")),
    ("VP", ("VP", "PP")),                            # PP attaches to the verb phrase
    ("NP", ("Det", "N")),
    ("NP", ("NP", "PP")),                            # ... or to the noun phrase
    ("PP", ("P", "NP")),
    ("N", ("N", "N")),                            # noun compound: "payments gateway"
]

LEXICON = {
    "restart": ["V"], "check": ["V"], "fix": ["V"],
    "server": ["N"], "payments": ["N"], "login": ["N"],
    "error": ["N"], "timeout": ["N"], "gateway": ["N"],
    "the": ["Det"], "a": ["Det"],
    "on": ["P"], "after": ["P"], "in": ["P"],
}

def cyk(words):
    """Fill the triangular chart. Returns (chart, back-pointers)."""
    n = len(words)
    chart = defaultdict(set)
    back = defaultdict(list)

    def close(cell):
        """Apply unit rules such as S -> VP until nothing changes."""
        changed = True
        while changed:
            changed = False
            for (lhs, rhs) in RULES:
                if len(rhs) == 1 and rhs[0] in chart[cell] \
                    and lhs not in chart[cell]:
                    chart[cell].add(lhs)
                    changed = True

    for i, w in enumerate(words):
        for tag in LEXICON.get(w, []):
            chart[(i, i + 1)].add(tag)
            close((i, i + 1))

    for span in range(2, n + 1):
        for i in range(0, n - span + 1):
            j = i + span
            for k in range(i + 1, j):
                for (lhs, rhs) in RULES:
                    if len(rhs) != 2:

```

```

        continue
    b, c = rhs
    if b in chart[(i, k)] and c in chart[(k, j)]:
        chart[(i, j)].add(lhs)
        back[(i, j, lhs)].append((k, b, c))
    close((i, j))
return chart, back

def count_pauses(chart, back, i, j, sym, memo=None):
    """Count DERIVATIONS, not membership: an ambiguous sentence
    reaches the same cell by two different routes."""
    if memo is None:
        memo = {}
    key = (i, j, sym)
    if key in memo:
        return memo[key]
    total = 0
    for (k, b, c) in back[key]:
        total += (count_pauses(chart, back, i, k, b, memo)
                  * count_pauses(chart, back, k, j, c, memo))
    for (lhs, rhs) in RULES: # unit rules
        if lhs == sym and len(rhs) == 1 and rhs[0] != sym \
            and rhs[0] in chart[(i, j)]:
            total += count_pauses(chart, back, i, j, rhs[0], memo)
    if total == 0 and j == i + 1 and sym in chart[(i, j)]:
        total = 1
    memo[key] = total
    return total

def parse(subject):
    words = subject.split()
    chart, back = cyk(words)
    ok = "S" in chart[(0, len(words))]
    n = count_pauses(chart, back, 0, len(words), "S") if ok else 0
    return ok, n, chart

# --- Eliza: pattern substitution, and nothing else -----
REFLECT = {"my": "your", "i": "you", "me": "you", "am": "are",
           "mine": "yours", "myself": "yourself"}

PATTERNS = [
    (r"^i am (.*)$", "How long have you been {0}?"),
    (r"^my (\w+) keeps (.*)$", "Tell me more about your {0}."),
    (r"^because (.*)$", "Is that the real reason?"),
    (r"^i (?:(can't|cannot) (.*)$", "What stops you from {0}?"),
    (r"^(.*) is broken$", "What makes you say {0} is broken?"),
]
FALLBACK = "Can you tell me more about that?"

def reflect(text):
    return " ".join(REFLECT.get(w, w) for w in text.split())

def eliza(line):
    low = line.lower().strip().rstrip("!?")
    for (pat, template) in PATTERNS:
        m = re.match(pat, low)
        if m:
```

```

        return template.format(*[reflect(g) for g in m.groups()])
    return FALLBACK

# --- a bigram model -----
CORPUS = [
    "restart the server", "restart the gateway", "restart the service",
    "check the payments gateway", "check the login error",
    "check the server", "fix the login error", "fix the timeout error",
    "restart the payments gateway", "check the timeout",
]

def bigrams(corpus):
    counts = defaultdict(lambda: defaultdict(int))
    for line in corpus:
        toks = ["<s>"] + line.split()
        for a, b in zip(toks, toks[1:]):
            counts[a][b] += 1
    return counts

def suggest(counts, word, k=3):
    nxt = counts.get(word, {})
    total = sum(nxt.values())
    ranked = sorted(nxt.items(), key=lambda kv: -kv[1][:k])
    return [(w, c / total) for w, c in ranked]

SUBJECTS = [
    "restart the server",
    "check the payments gateway",
    "restart the server after the timeout",
    "fix the login error on the gateway",
    "restart the gateway after the payments error",
]

if __name__ == "__main__":
    print("=== parsing ticket subjects ===")
    print("subject                                     parses    count")
    print("-" * 62)
    counts = {}
    for s in SUBJECTS:
        ok, n, _ = parse(s)
        counts[s] = n
        print("%-46s %7s %6d" % (s, ok, n))
    # three of the five are structurally ambiguous
    ambiguous = [s for s in SUBJECTS if counts[s] > 1]
    assert len(ambiguous) == 3 and all(counts[s] == 2 for s in ambiguous)
    print()
    print("%d of %d subjects have more than one parse."
          % (len(ambiguous), len(SUBJECTS)))
    print("Each extra prepositional phrase roughly doubles the count.")

    print()
    print("=== and one that is not in the language ===")
    for bad in ("the the server restart", "server the",
               "restart restart the"):
        ok, n, _ = parse(bad)
        print("    %-32s parses: %s" % (bad, ok))
        assert not ok
    print("    Returning nothing is a useful answer: the system knows")

```

```

print("    that it does not know.")

print()
print("=== the chart ===")
words = SUBJECTS[4].split()
chart, _ = cyk(words)
cells = sum(1 for k in chart if chart[k])
total = len(words) * (len(words) + 1) // 2
print("    %d words -> %d cells, %d of them non-empty"
      % (len(words), total, cells))

print()
print("=== Eliza ===")
for line in ("I am worried about the release",
            "my deploy keeps failing",
            "because the config is wrong",
            "I can't reproduce it",
            "the gateway is broken",
            "no idea what is happening"):
    print("    > %-34s %s" % (line, eliza(line)))
print()
print("    Pattern matching and pronoun swapping. No grammar, no")
print("    representation of the conversation, no model of")
print("    anything -- and it reads as though it is listening.")
assert eliza("something unmatched entirely") == FALLBACK

print()
print("=== a bigram model ===")
bg = bigrams(CORPUS)
for w in ("<s>", "restart", "the", "login"):
    print("    after %-10s -> %s"
          % (w, ", ".join("%s %.2f" % t for t in suggest(bg, w))))

print()
print("=== the limitation no corpus size removes ===")
hard = ("the server that the engineer who wrote the runbook "
        "maintained was restarted")
toks = hard.split()
print("    '%s'" % hard)
print("    'was restarted' is at position %d; its subject 'server'"
      % toks.index("was"))
print("    is at position %d -- %d words back."
      % (toks.index("server"), toks.index("was") - toks.index("server")))
print("    No n-gram with a practical n connects them, and raising")
print("    n makes sparsity exponentially worse. A grammar has no")
print("    difficulty with it, because the dependency is structural.")
assert toks.index("was") - toks.index("server") >= 8

```

Then add one more prepositional phrase — **restart the server after the timeout on the gateway** — and count the parses. The answer is five, not four: the growth follows the Catalan numbers, and this is the moment broad-coverage parsing stops being a matter of writing more rules.

🔍 Checkpoint — Test Yourself

1. restart the server after the timeout has exactly two parses. Give both readings in English, and say what a routing system should do with a subject line that has more than one.

2. Eliza has no grammar, no parser and no representation of the conversation, and people disclosed things to it they would not tell a colleague. State what this shows about fluency as evidence of understanding.
3. Smoothing fixes n-gram sparsity but nothing fixes the fixed window. Explain the difference, with the nine-word dependency from this chapter.

✓ Chapter Summary

- The symbolic pipeline runs tokenise, morphology, tag, parse, interpret. Stages 1–3 are bookkeeping; the *parse* is where the knowledge lives.
- A **context-free grammar** is rules $A \rightarrow \beta$. In **Chomsky normal form** every rule is $A \rightarrow BC$ or $A \rightarrow w$, which is what **CYK** needs.
- CYK is dynamic programming over spans, costs $O(n^3|G|)$, and finds *every* parse without backtracking.
- **Ambiguity** is the normal case, not a defect: three of five subject lines here have two readings, and each extra prepositional phrase roughly doubles the count. Returning two trees is the honest answer and tells the system when to ask a person.
- **Eliza** matched patterns and swapped pronouns in a few hundred lines, and people attributed understanding to it. The **Eliza effect** is that fluency compels that attribution regardless of what is underneath — which is why Chapter 1’s checklist, and not your impression of a transcript, is the instrument to use.
- Pattern substitution is Chapter 12’s unification restricted to strings.
- An **n-gram** model applies the **Markov assumption** and is pure counting. **Sparsity** is fixable by smoothing; the *fixed window* is not, and a nine-word dependency defeats it at any corpus size. Grammars capture structure and are brittle; counting is robust and structurally blind.

✍ Review Questions

1. (MCQ) CYK requires the grammar to be in: (a) Backus–Naur form (b) Chomsky normal form (c) Greibach normal form (d) any context-free form.
2. (MCQ) The time complexity of CYK in the sentence length is: (a) $O(n)$ (b) $O(n^2)$ (c) $O(n^3)$ (d) $O(2^n)$.
3. (MCQ) Eliza’s mechanism is closest to: (a) chart parsing (b) unification restricted to strings (c) an n-gram model (d) forward chaining.
4. (MCQ) Smoothing an n-gram model addresses: (a) the fixed window (b) sparsity (c) structural ambiguity (d) the Markov assumption.
5. (Short) Give the two readings of **fix the login error on the gateway** and say which one a routing system would need to disambiguate before acting.
6. (Short) Explain why returning no parse is a more useful failure than returning a guess, with a service-desk example.
7. (Short) State the Eliza effect and say why it is relevant to evaluating a modern language system.
8. (Applied) Add the rule $VP \rightarrow V$ (an intransitive verb, as in **restart**) to the grammar, and re-count the parses of **restart the server after the timeout**. Explain the change, and say what it illustrates about adding rules to a grammar.

Classic AI Systems and How to Analyse Them

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part VI — Learning, Language and Responsible AI

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Describe** the General Problem Solver, Eliza, Student and Macsyma, and **identify** the single mechanism each one rests on.
- **Reconstruct** each of them in a few dozen lines using machinery from earlier chapters.
- **Apply** a six-column framework — task, environment, representation, inference, knowledge source, failure mode — to analyse any AI system.
- **Explain** what each system was mistaken for at the time, and why.
- **Use** the framework on a system you did not build.

🔗 Before You Start

Chapter 9 (means–ends analysis), Chapter 12 (unification), Chapter 13 (rule rewriting) and Chapter 18 (pattern substitution). This chapter introduces *no new machinery*: everything it builds, you have already built.

📖 Key Terms in This Chapter

General Problem Solver • Eliza • Student • Macsyma • means–ends analysis • pattern substitution • term rewriting • microworld • the Eliza effect • system analysis • failure mode • knowledge source

19.1 Four Systems, Four Mechanisms

The HEC 2023 outline names four programs as required case studies. They were built between 1961 and 1968, they were each spectacular at the time, and each one is essentially a single mechanism applied with taste.

That is the point of this chapter, and it is why it comes last. You can now build all four — not simplified versions, but the actual mechanisms — because you already have the parts.

System	Year	The mechanism	Where you built it
General Problem Solver	1961	Means–ends analysis: reduce the difference between where you are and where you want to be	Chapter 9, where it solved the release-planning problem
Eliza	1966	Pattern substitution with pronoun reflection	Chapter 18, in about thirty lines
Student	1964	Pattern matching from English into simultaneous equations, then algebra	Chapter 12’s unification, restricted to strings

System	Year	The mechanism	Where you built it
Macsyma	1968	Term rewriting: apply algebraic identities as rules until nothing applies	Chapter 13's forward chaining, over expressions

Table 19.1: The four required case studies, and the chapter of this handout that already contains each mechanism.

19.2 The General Problem Solver

Newell and Simon's GPS (1961) was the first program explicitly intended to be *general*: a single solver, given a description of a domain, that would work on any problem in it. Its engine was means–ends analysis — find a difference between the current state and the goal, select an operator that reduces that difference, and recursively achieve that operator's preconditions.

Chapter 9 built exactly this, and measured it: on the four-component release problem it found an optimal plan considering 187 operators where progression search expanded 1,163 states.



GPS was also a claim about *people*. Newell and Simon collected think-aloud protocols from human subjects solving puzzles and argued that the traces matched what GPS did — which places it in the *thinking humanly* quadrant of Figure 1.1, not the *acting rationally* one this course otherwise inhabits.

That matters for how it should be judged. As a psychological model it was a serious and influential contribution. As a general problem solver it failed, and the reason is the one Chapter 9 demonstrated: means–ends analysis is sensitive to goal ordering and incomplete, and the generality was in the *engine*, while all the difficulty turned out to be in describing the domain.

19.3 Eliza

Chapter 18 covered the mechanism and the effect. What belongs here is the analysis.

Eliza had no goal, no representation of the conversation, and no model of the person it was talking to. It scored *one* on Chapter 1's five-point checklist — it has a kind of knowledge, in that the patterns are inspectable and a non-programmer could edit them — and zero on goal, search, adaptation and explanation.

And it was taken for a therapist. That combination — lowest possible sophistication, highest possible impression — is why it is the most instructive system in this chapter, and why Chapter 20 returns to it.

19.4 Student

Bobrow's Student (1964) read algebra word problems written in English and solved them. The example everyone quotes is about ages and uncles; the delivery equivalent would be:

```
the sprint has twice as many bugs as stories .
the number of stories is 12 .
how many bugs does the sprint have ?
```

Student worked in two stages. Patterns turned English fragments into equations — **twice as many X as Y** became $X = 2Y$, **the number of X is N** became $X = N$ — and then an ordinary equation solver did the algebra. Answer: 24.

⚠ The microworld, and what it concealed

Student worked beautifully inside a tiny fragment of English about arithmetic, and that fragment is called a **microworld**. The assumption of the era — stated openly — was that microworlds would be extended and joined until they covered ordinary language. They were not, and the reason is visible in the mechanism. Student’s patterns are brittle in a specific way: **twice as many bugs as stories works**, and **the bug count is double the story count** does not, and neither does **there are two bugs for every story**. Each paraphrase needs its own pattern, the paraphrases are unbounded, and nothing in the approach makes the next one cheaper than the last. That is the same shape as Chapter 13’s knowledge acquisition bottleneck, in a different subject. Notice it recurring: it is the characteristic failure of hand-built symbolic systems, and recognising it in a new project is worth more than any individual technique in this book.

19.5 Macsyma

Macsyma (from 1968, at MIT) did symbolic mathematics: it manipulated algebraic expressions as structures rather than evaluating them numerically. It could differentiate, integrate, factor and simplify, and it remained in serious scientific use for decades. Its descendants are Maxima, Mathematica and SymPy.

The core is **term rewriting**: represent an expression as a tree, and repeatedly apply identities as rewrite rules until none applies.

⌘ Rewrite rules for effort expressions

Rule	Reading
$x + 0 \rightarrow x$	adding nothing changes nothing
$x \times 1 \rightarrow x$	scaling by one changes nothing
$x \times 0 \rightarrow 0$	scaling by zero gives nothing
$x + x \rightarrow 2x$	collect like terms
$(a \times x) + (b \times x) \rightarrow (a + b) \times x$	distributivity, backwards

Applying these to $(3 \times e) + (2 \times e) + 0$ gives $5 \times e$. Each rule is a fact about arithmetic that a person can check, the engine is Chapter 13’s forward chaining, and the conflict-resolution question is the same one: when two rules apply, which fires?

19.6 A Framework for Analysing an AI System

The HEC 2017 outline asks for *analysis* of AI systems, not merely description. Here is a framework that works on any of them, including ones you did not build and whose internals you cannot see.

System	Task	Representa- tion	Inference	Knowledge	Failure mode
GPS	Reach a goal	Operators: pre, add, delete	Means–ends	Hand-written	Incomplete; goal order matters. Reports “no plan” on solvable problems
Eliza	Converse	Patterns with variables	Match and substitute	Hand-written	None detectable. It never fails — it always has a reply, which is the danger

System	Task	Representation	Inference	Knowledge	Failure mode
Student	Solve a word problem	Patterns, then algebra	Match, then solve	Hand-written	Brittle to paraphrase. Each new phrasing needs a new pattern
Macsyma	Simplify	Expression trees	Rewrite to a fixed point	Hand-written	Rule interactions; may loop, or stop at a form nobody wanted

Table 19.2: The four systems through the framework. Note that all four share one knowledge source — a person, writing it down — which is the single fact that most determines their history.



Read down the *knowledge source* column. Every one of the four was hand-built, and the same limit appears in all four biographies: the system worked, extending it required a person, and the extensions ran out before the domain did.

Now read the *failure mode* column, and notice that Eliza's is the worst of the four precisely because it has none. GPS reports failure. Student produces nothing on an unmatched phrasing. Macsyma leaves an expression unsimplified. Eliza always replies, so its user has no signal at all — and that is the property that Chapter 20 argues is shared by the systems now in production.

Worked Example: Analysing a system you did not build

Apply the framework to something with no published internals: an IDE's code completion.

Task. Suggest the next tokens a developer will type. The performance measure is *acceptance rate* — how often a suggestion is taken — which is worth writing down, because it is a measure of the developer's behaviour, not of whether the code was any good. Compare Chapter 2: a measure of the agent's activity rather than of the environment's state, and therefore gameable.

Environment. Partially observable — it sees the file and perhaps the project, not the developer's intent. Dynamic; the code changes as they type. Single-agent. Episodic, approximately.

Representation. Unknown from the outside, and the framework lets us say something anyway. If it completes an identifier defined three files away, it has a symbol table — an explicit representation. If it produces plausible code calling a method that does not exist, it does not.

Inference. Test it. A completion that is always correct about types and never invents a name is doing symbolic lookup. One that is usually right and occasionally confident and wrong is doing something statistical. Most current tools do both, and the two failure signatures are distinguishable from outside.

Knowledge source. The corpus it was trained on, plus the current project. Neither is correctable by the developer, which is a real difference from every system in Table 19.2: nobody can fix a wrong suggestion, only decline it.

Failure mode. This is the column that decides how to use it. If it fails by suggesting nothing, it is safe and occasionally unhelpful. If it fails by suggesting something fluent and wrong, it is Eliza with a compiler — and the mitigation is that the compiler and the tests are the failure detector the system does not have.

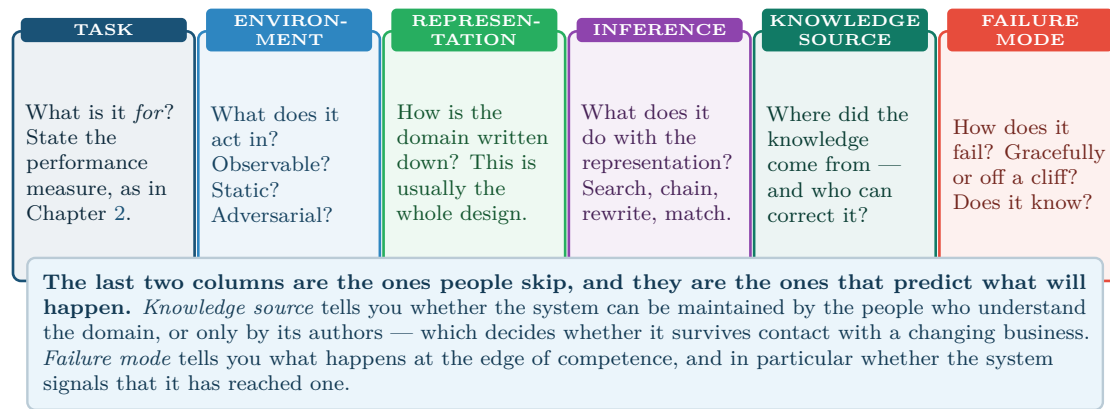


Figure 19.1: Six columns for analysing an AI system. Applying them to a system you did not build is the skill this chapter is for.

What the analysis produced. Not a verdict, but three testable questions and one operational rule: never accept a completion that the type checker and the tests have not seen. That rule came from the last column, which is why it is the one not to skip.

! Common Pitfalls

- **Describing a system instead of analysing it.** “Eliza simulates a therapist” is a description. The analysis is that it matches patterns, has no representation, and cannot fail detectably.
- **Skipping the knowledge-source column.** It predicts whether the system survives a change in the business, and all four classic systems share the same answer.
- **Skipping the failure-mode column.** A system that always produces an answer has the most dangerous failure mode there is.
- **Mistaking a microworld result for a general one.** Student worked on a fragment of English and the fragment did not extend. Ask what the demonstration excluded.
- **Judging GPS as a problem solver when it was offered as a psychological model.** Identify the quadrant of Figure 1.1 first, then choose the evidence.
- **Assuming you cannot analyse a closed system.** Five of the six columns can be filled from the outside by testing, as the worked example does.

🔧 Try It Yourself: Four Systems in One File

Each of the four reconstructed with machinery from earlier chapters, then the analysis table printed from the code.

```
"""Chapter 19 lab -- the four classic systems, reconstructed.

Nothing here is new. GPS is Chapter 9's means-ends analysis, Eliza is
Chapter 18's pattern substitution, Student is Chapter 12's unification
restricted to strings, and Macsyma is Chapter 13's forward chaining
over expression trees.
"""

import re

# =====
# 1. GPS (1961): means-ends analysis          -- as in Chapter 9
# =====
OPERATORS = [
    # (name, preconditions, add, delete)
```

```

("code", set(), {"coded"}, set()),
("review", {"coded"}, {"reviewed"}, set()),
("test", {"reviewed", "staging free"},
 {"tested", "staging busy"}, {"staging free"}),
("deploy", {"tested"}, {"deployed"}, set()),
("release staging", {"staging busy"},
 {"staging free"}, {"staging busy"}),
]

def gps(state, goals, depth=0, trace=None):
    """Achieve each goal by reducing differences. Chapter 9's engine."""
    if trace is None:
        trace = []
    if depth > 20:
        return None, trace
    for g in goals:
        if g in state:
            continue
        achieved = False
        for (name, pre, add, dele) in OPERATORS:
            if g not in add:
                continue
            got, trace = gps(state, sorted(pre), depth + 1, trace)
            if got is None:
                continue
            state = got
            if not pre <= state:
                continue
            state = (state - dele) | add
            trace.append(name)
            achieved = True
            break
        if not achieved:
            return None, trace
    return state, trace

# =====
# 2. Eliza (1966): pattern substitution      -- as in Chapter 18
# =====
REFLECT = {"my": "your", "i": "you", "me": "you", "am": "are"}
ELIZA_RULES = [
    (r"^i am (.*)$", "How long have you been {0}?"),
    (r"^my (\w+) keeps (.*)$", "Tell me more about your {0}."),
    (r"^because (.*)$", "Is that the real reason?"),
]

def eliza(line):
    low = line.lower().strip().rstrip("!.?")
    for pat, tmpl in ELIZA_RULES:
        m = re.match(pat, low)
        if m:
            groups = [" ".join(REFLECT.get(w, w) for w in g.split())
                      for g in m.groups()]
            return tmpl.format(*groups)
    return "Can you tell me more about that?"

# =====
# 3. Student (1964): English -> equations    -- Chapter 12's matching

```

```
# =====
STUDENT_PATTERNS = [
    # "the sprint has twice as many bugs as stories"
    (r"^the \w+ has twice as many (\w+) as (\w+)$",
     lambda m: (m.group(1), ("times", 2, m.group(2)))),
    # "the number of stories is 12"
    (r"^the number of (\w+) is (\d+)$",
     lambda m: (m.group(1), ("const", int(m.group(2))))),
]

def student(sentences, question):
    """Turn English into equations, then solve them."""
    eqs = {}
    for s in sentences:
        low = s.lower().strip().rstrip(".")
        for pat, build in STUDENT_PATTERNS:
            m = re.match(pat, low)
            if m:
                var, rhs = build(m)
                eqs[var] = rhs
                break
        else:
            return None, eqs, "no pattern matched: %r" % s

    def value(var, seen=None):
        seen = seen or set()
        if var in seen:
            return None # circular definition
        rhs = eqs.get(var)
        if rhs is None:
            return None
        if rhs[0] == "const":
            return rhs[1]
        if rhs[0] == "times":
            inner = value(rhs[2], seen | {var})
            return None if inner is None else rhs[1] * inner
        return None

    m = re.match(r"^how many (\w+) does the \w+ have$",
                 question.lower().strip().rstrip("?"))
    if not m:
        return None, eqs, "question not understood"
    return value(m.group(1)), eqs, None

# =====
# 4. Macsyma (1968): term rewriting -- Chapter 13's chaining
# =====
# an expression is a nested tuple: ("+", a, b) or ("*", a, b),
# a number, or a string (a symbol)
def rewrite_once(e):
    """One pass of the identities. Returns (expr, rule name or None)."""
    if not isinstance(e, tuple):
        return e, None
    op, a, b = e
    if op == "+" and b == 0:
        return a, "x + 0 -> x"
    if op == "+" and a == 0:
        return b, "0 + x -> x"
    if op == "*" and b == 1:
        return a, "x * 1 -> x"
```

```

if op == "*" and (a == 0 or b == 0):
    return 0, "x * 0 -> 0"
if op == "+" and a == b:
    return ("*", 2, a), "x + x -> 2x"
# (a*x) + (b*x) -> (a+b)*x
if (op == "+" and isinstance(a, tuple) and isinstance(b, tuple)
    and a[0] == "*" and b[0] == "*" and a[2] == b[2]
    and isinstance(a[1], int) and isinstance(b[1], int)):
    return ("*", a[1] + b[1], a[2]), "(a*x)+(b*x) -> (a+b)*x"
# recurse
na, r = rewrite_once(a)
if r:
    return (op, na, b), r
nb, r = rewrite_once(b)
if r:
    return (op, a, nb), r
return e, None

def simplify(e, limit=25):
    """Forward chaining to a fixed point -- Chapter 13, over trees."""
    steps = []
    for _ in range(limit):
        e2, rule = rewrite_once(e)
        if rule is None:
            return e, steps
        steps.append(rule)
        e = e2
    return e, steps

def show(e):
    if not isinstance(e, tuple):
        return str(e)
    return "%s %s %s" % (show(e[1]), e[0], show(e[2]))

if __name__ == "__main__":
    print("=" * 62)
    print("1. GENERAL PROBLEM SOLVER (1961) -- means-ends analysis")
    print("=" * 62)
    final, plan = gps(frozenset({"staging free"}), ["deployed"])
    print("  goal: deployed")
    print("  plan:", " -> ".join(plan))
    assert plan == ["code", "review", "test", "deploy"]
    print("  This is Chapter 9's engine, unchanged.")

    print()
    print("=" * 62)
    print("2. ELIZA (1966) -- pattern substitution")
    print("=" * 62)
    for line in ("I am worried about the release",
                 "my deploy keeps failing",
                 "because the config is wrong",
                 "the quarterly numbers look fine"):
        print("  > %-34s %s" % (line, eliza(line)))
    print("  No representation of the conversation. It cannot fail,")
    print("  which is the most dangerous failure mode of the four.")

    print()
    print("=" * 62)
    print("3. STUDENT (1964) -- English into equations")

```

```

print("=" * 62)
facts = ["the sprint has twice as many bugs as stories",
         "the number of stories is 12"]
q = "how many bugs does the sprint have?"
ans, eqs, err = student(facts, q)
for f in facts:
    print("    given: %s" % f)
print("    equations:", eqs)
print("    question: %s" % q)
print("    answer:", ans)
assert ans == 24

print()
print("    ... and the brittleness:")
for para in ("the bug count is double the story count",
            "there are two bugs for every story"):
    ans2, _, err2 = student([para, facts[1]], q)
    print("    %-42s -> %s" % (para, err2 or ans2))
    assert ans2 is None
print("    Each paraphrase needs its own pattern, and the")
print("    paraphrases are unbounded. This is Chapter 13's")
print("    knowledge acquisition bottleneck, in another subject.")

print()
print("=" * 62)
print("4. MACSYMA (1968) -- term rewriting")
print("=" * 62)
expr = ("+", ("+", ("*", 3, "e"), ("*", 2, "e")), 0)
print("    before:", show(expr))
out, steps = simplify(expr)
print("    after: ", show(out))
for s in steps:
    print("        applied %s" % s)
assert out == ("*", 5, "e")
print("    Forward chaining to a fixed point, over trees instead")
print("    of over facts. Chapter 13's engine, same conflict-")
print("    resolution question: when two rules apply, which?")

print()
print("=" * 62)
print("THE ANALYSIS")
print("=" * 62)
ANALYSIS = [
    ("GPS", "reach a goal", "operators", "means-ends",
     "hand-written", "incomplete; goal order"),
    ("Eliza", "converse", "patterns", "match+substitute",
     "hand-written", "NONE DETECTABLE"),
    ("Student", "solve a problem", "patterns+algebra", "match, solve",
     "hand-written", "brittle to paraphrase"),
    ("Macsyma", "simplify", "expression trees", "rewrite",
     "hand-written", "loops; wrong fixed point"),
]
print("%-9s %-16s %-17s %-17s %-13s %s"
      % ("SYSTEM", "TASK", "REPRESENTATION", "INFERENCE",
        "KNOWLEDGE", "FAILURE MODE"))
print("-" * 100)
for row in ANALYSIS:
    print("%-9s %-16s %-17s %-17s %-13s %s" % row)

sources = {r[4] for r in ANALYSIS}
assert sources == {"hand-written"}
print()

```

```
print("Every one of the four has the same knowledge source, and")
print("that single fact explains all four biographies: each")
print("worked, each needed a person to extend it, and each ran")
print("out of people before it ran out of domain.")
```

Then add a Student pattern for one of the two paraphrases that currently fail, and notice how long it takes and how little it generalises. That experience — not the argument about it — is what the chapter is for.

🔍 Checkpoint — Test Yourself

1. Name the mechanism behind each of the four systems, and the chapter of this handout in which you built it.
2. Eliza's failure mode is recorded as "none detectable". Explain why that is worse than GPS's "reports failure on solvable problems", and relate it to how a modern language system should be evaluated.
3. Every one of the four has the same knowledge source. State it, and explain what it predicts about a hand-built symbolic system deployed in a changing business.

📋 Chapter Summary

- The four required case studies are each a single mechanism applied with taste: **GPS** is means–ends analysis (Chapter 9), **Eliza** is pattern substitution (Chapter 18), **Student** is pattern matching into equations (Chapter 12), **Macsyma** is term rewriting (Chapter 13).
- GPS was offered as a model of human problem solving — the *thinking humanly* quadrant — and should be judged as one. Its generality was in the engine; the difficulty was in describing the domain.
- Student worked inside a **microworld** and the microworld did not extend. Each paraphrase needed its own pattern, which is Chapter 13's knowledge acquisition bottleneck in another subject.
- Analyse a system on six columns: *task*, *environment*, *representation*, *inference*, *knowledge source*, *failure mode*. The last two are the ones people skip and the ones that predict what will happen.
- All four share one knowledge source — a person writing it down — and that single fact explains all four biographies.
- Eliza's failure mode is the worst of the four because it has none: it always replies, so its user gets no signal. That property is shared by systems now in production, which is why Chapter 20 returns to it.
- Five of the six columns can be filled from outside a system you did not build, by testing. That is the transferable skill in this chapter.

🔧 Review Questions

1. (MCQ) The General Problem Solver's engine was: (a) resolution (b) means–ends analysis (c) term rewriting (d) pattern substitution.
2. (MCQ) Macsyma's core mechanism is: (a) numerical evaluation (b) term rewriting to a fixed point (c) backward chaining (d) unification with the occurs check.
3. (MCQ) A *microworld* is: (a) a small computer (b) a restricted domain in which a technique works, which was expected to extend (c) a simulation environment (d) a subset of first-order logic.

4. **(MCQ)** Which failure mode is the most dangerous? (a) reports failure on solvable problems (b) brittle to paraphrase (c) no detectable failure (d) may loop.
5. **(Short)** Explain why Student's brittleness and the expert-system knowledge acquisition bottleneck are the same phenomenon.
6. **(Short)** GPS was a psychological model and a problem solver. Say how the evidence you would collect differs between those two claims.
7. **(Short)** Give two of the six analysis columns that can be filled from outside a closed system, and describe the test you would run for each.
8. **(Applied)** Apply the six-column framework to a spam filter, a satellite navigation system, or a tool you use daily. Fill every column, and state the one operational rule your analysis of the failure-mode column produces.

Recent Trends and Applications

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part VI — Learning, Language and Responsible AI

🕒 Learning Outcomes

By the end of this chapter you will be able to:

- **Describe** what a large language model is, in terms this course has already established.
- **Explain** the structure of a tool-using agent, and **relate** it to the agent architectures of Chapter 2.
- **Identify** which parts of a modern AI system are symbolic and which are learned.
- **Assess** a claim about an AI system using Chapter 1’s checklist and Chapter 19’s framework.
- **Say** what this course’s material contributes to systems built today.

🔗 Before You Start

Chapter 2 (agent architectures), Chapter 13 (rule engines, which reappear inside agent loops), Chapter 18 (n-grams and the Eliza effect) and Chapter 17 (what learning is). This chapter is a signpost: it names what is current and connects it to what you have built.

📖 Key Terms in This Chapter

large language model • token • context window • prompt • tool use • function calling • agent loop • retrieval-augmented generation • neuro-symbolic • hallucination • grounding • benchmark

20.1 Where the Field Is Now

This course has taught the symbolic half of artificial intelligence, and it has been honest about the fact that the systems in the news are not built that way. This chapter says what they are, in the vocabulary you now have, and what your material contributes to them.

It is a signpost rather than a unit. A serious treatment of the current wave needs the statistics of *Machine Learning* and the architectures of *Data Science*; what can be done here is to place it.

20.2 Large Language Models, Briefly

A **large language model** predicts the next **token** — roughly, the next word-piece — given the tokens before it. That is the n-gram model of Chapter 18, with two differences that turn out to matter enormously.

The window is much larger, and it is not fixed by counting. An n-gram model conditions on the previous two or three words because counting longer sequences is hopeless: the data is too sparse. A language model conditions on thousands of tokens, because it does not count — it learns a function, and the function generalises across contexts that never co-occurred in training. That directly addresses the limitation Chapter 18 said was unfixable by counting.

The scale is different in kind. Hundreds of billions of parameters, trained on a substantial fraction of published text. The empirical finding of the last decade is that capabilities appear as this scales which nobody designed and few predicted.

⚠ What it still is, and what follows

It is a next-token predictor. Everything else — answering questions, writing code, apparently reasoning — is a consequence of that objective being pursued at scale over text that contains people doing those things.

Three consequences follow directly, and each one is visible in this course’s terms.

There is no representation to inspect. Nothing corresponds to Chapter 10’s semantic network or Chapter 15’s conditional probability table. So Chapter 1’s third test scores zero, and there is no artefact for a domain expert to correct.

There is no proof and no trace. A language model can *produce* text resembling Chapter 13’s explanation chain, and that text is generated by the same process as the answer — it is not a record of how the answer was reached. The distinction is exactly Chapter 11’s $\models$ against $\vdash$: a plausible-looking derivation is not a derivation.

Fluency is not evidence. This is Chapter 18’s Eliza effect at scale, and your defence against it is Chapter 1’s checklist rather than your impression of the transcript.

Hallucination — fluent, confident, false output — is not a defect to be patched but the objective working as specified. The model was asked for likely text, and false text is often extremely likely. In Chapter 19’s framework, its failure mode is Eliza’s: *none detectable by the system*, which is the worst entry in that column.

20.3 Tool-Using Agents

The interesting engineering development is not the model but the loop around it.

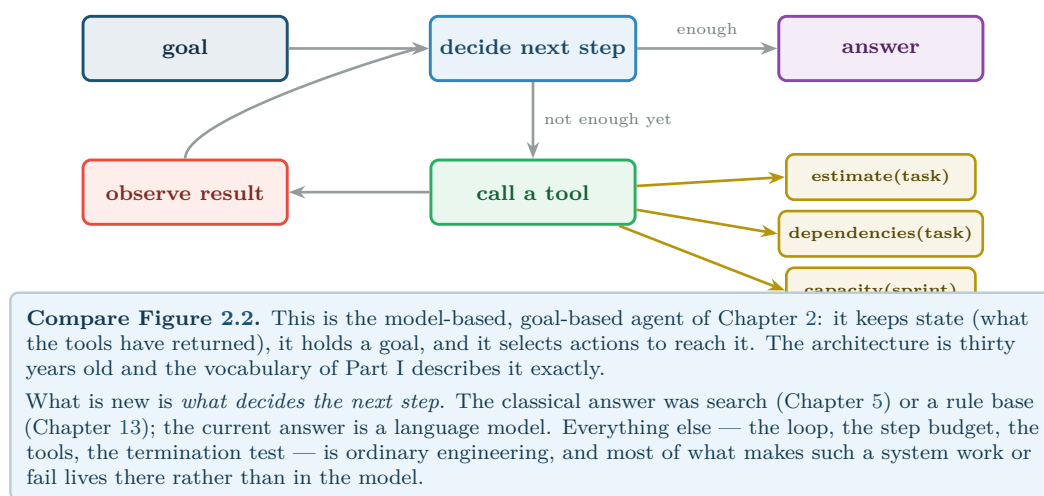


Figure 20.1: A tool-using agent. The loop is Chapter 2’ goal-based agent; only the decision procedure at its centre has changed.

The chapter’s lab builds this loop with a rule engine at its centre instead of a language model — deterministic, offline, and fully traceable. That substitution is the point of the exercise: it isolates what the loop contributes from what the model contributes, and the loop turns out to contribute most of the structure.

Retrieval-augmented generation is the same idea with one tool: search a document collection, put what you find into the prompt, and generate from that. It is the standard response to hallucination and it is a partial one — the retrieved text **grounds** the answer, and nothing forces the answer to respect it.

20.4 Neuro-Symbolic Systems

The current wave is strong exactly where this course’s methods are weak, and weak exactly where they are strong.

	Symbolic (this course)	Learned (current wave)
Where knowledge comes from	A person writes it down; the knowledge acquisition bottleneck	Data; no bottleneck, and no way to correct one fact
Behaviour at the edge	Brittle: confidently wrong or silent	Degrades smoothly, and gives no signal either
Guarantees	Real: a constraint solver proves no engineer is double-booked	None
Explanation	A proof or a firing chain, checkable line by line	Text resembling one, generated the same way as the answer
Ambiguous or novel input	Fails, often usefully	Handles it, sometimes by inventing

Table 20.1: The two traditions, by their failure modes rather than their successes.

Neuro-symbolic systems combine them, and the combination that works commercially is unglamorous: use the model for what is genuinely hard to specify — turning messy natural language into a structured request — and a symbolic component for what must be *right*.



Concretely, in the terms of this handout: a language model reads the ticket subject and produces a structured request (Chapter 18’s stage 5, done better than any grammar manages); Chapter 7’s constraint solver then produces a schedule that provably violates nothing; Chapter 13’s rule engine applies the escalation policy and emits the chain that an auditor will read.

Each component does the thing it is good at, and — this is the part worth remembering — *the guarantees come from the symbolic components*. A system that needs to demonstrate that no engineer was double-booked, or to show exactly why a ticket was escalated, gets that from Chapters 7 and 13 and cannot get it from the model.

That is the concrete answer to “why did I learn this?” It is not that symbolic AI will displace the current wave. It is that the parts of a deployed system which have to be *correct* and *accountable* are still built the way this course builds them, and the people who can build both are rare.

20.5 Reading a Claim About AI

The most transferable thing this chapter can leave you with is a procedure for claims, because you will meet many and most will be overstated.

1. **Apply the five-point checklist** (Chapter 1). Goal or procedure? Does it consider alternatives? Is there an inspectable representation? Does it adapt? Can it explain? The answers are usually more interesting than the marketing.
2. **Fill in the six columns** (Chapter 19), and do not skip *knowledge source* and *failure mode*. Ask specifically: when it is wrong, how would anyone find out?

3. **Ask what the demonstration excluded.** Student worked on a fragment of English. Every impressive demonstration is a microworld until shown otherwise.
4. **Distinguish a claim about capability from a claim about reach.** Chapter 1’s history is two winters caused by exactly this conflation, and Chapter 17 showed a correct result about perceptrons producing a wrong conclusion about neural networks.
5. **Ask what would falsify it.** A claim with no failing case is not a strong claim; it is an unfalsifiable one.

Worked Example: Assessing a code-generation assistant

A vendor claims their tool “understands your codebase and writes correct code”.

Checklist. *Goal:* no — it is given a procedure (complete this function), not an objective. *Search:* sometimes; several candidates may be generated and ranked. *Knowledge:* partially — if it uses the language server’s symbol table, that part is inspectable; the model’s parameters are not. *Adaptation:* not from your corrections, generally. *Explanation:* it will produce a rationale on request, and that rationale is generated, not recorded. Score around two, with the two most auditable tests failing.

Six columns. *Failure mode* is the decisive one: it fails by producing fluent, plausible, wrong code. Compare a type checker, which fails by refusing to compile. The first needs an external detector; the second is one.

What the demonstration excluded. Almost certainly: a codebase with unusual conventions, a language with little public code, and any case where the right answer is “this function should not exist”.

Capability or reach? The capability claim — it writes usable code often — is well supported. The reach claim — it *understands* the codebase — is the Chapter 18 Eliza effect restated, and the test is not whether the output reads as understanding but whether there is a representation to point at.

What would falsify it? “Correct code” is testable and the vendor has not offered a test. Ask for the acceptance criterion; the answer tells you whether they have one.

The useful conclusion. Not “it is good” or “it is hype”, but an operational rule of the kind Chapter 19 produced: *this tool’s output goes through the same review and test gate as a junior developer’s, for the same reason, and any workflow that bypasses that gate has removed the only failure detector in the system.* That is a decision a team can act on, and it came from two checklists rather than from an opinion.

Common Pitfalls

- **Treating generated text about reasoning as a trace of reasoning.** It is produced by the same process as the answer. Chapter 13’s firing chain is a record; this is not.
- **Expecting hallucination to be patched away.** It is the objective working as specified. Mitigations reduce it; nothing removes it.
- **Assuming the agent loop is the model.** The loop, the tools, the budget and the termination test are ordinary engineering, and most failures live there.
- **Believing retrieval grounds an answer.** It puts text in the prompt. Nothing forces the answer to respect it.
- **Concluding symbolic methods are obsolete.** The guarantees and the audit trails in a deployed system come from them, and nothing else provides either.
- **Judging by the transcript.** That is the Eliza effect, and it worked on people who had read Eliza’s source code.

Try It Yourself: A Tool-Using Agent, Offline

The loop of Figure 20.1, with Chapter 13's rule engine where a language model would sit. Deterministic, offline, and fully traceable — which is what makes it possible to see what the loop contributes.

```

"""Chapter 20 lab -- a tool-using agent with no model in it.

The architecture of a modern agent: a goal, a set of tools, a loop
that decides what to call next, a step budget and a termination test.
The decision procedure here is Chapter 13's rule engine rather than a
language model, so every run is reproducible and every step has a
reason. No network, no API, no randomness.
"""

# ---- the world the tools read -----
TASKS = {
    "auth-refresh": {"points": 5, "depends": ["schema-change"]},
    "schema-change": {"points": 3, "depends": []},
    "ui-polish": {"points": 2, "depends": []},
}
SPRINT_CAPACITY = 8

# ---- three tools. Each is cheap, local and honest about failure ----
def tool_estimate(task):
    t = TASKS.get(task)
    return t["points"] if t else None

def tool_dependencies(task):
    t = TASKS.get(task)
    return list(t["depends"]) if t else None

def tool_capacity(_):
    return SPRINT_CAPACITY

TOOLS = {
    "estimate": tool_estimate,
    "dependencies": tool_dependencies,
    "capacity": tool_capacity,
}

# ---- the decision procedure: Chapter 13's rule engine -----
# (name, [conditions on what we know], action)
# an action is ("call", tool, argument) or ("answer",)
POLICY = [
    ("need-deps",
     lambda k: "deps" not in k,
     lambda k: ("call", "dependencies", k["goal"])),
    # a tool can fail. A policy with no rule for that is the commonest
    # defect in deployed agents -- see the second experiment below.
    ("unknown-task",
     lambda k: k.get("deps", []) is None,
     lambda k: ("give-up",)),
    ("need-dep-estimates",
     lambda k: k.get("deps") and any(d not in k["points"]
                                     for d in k["deps"]),
     lambda k: ("call", "estimate",
                next(d for d in k["deps"] if d not in k["points"]))),

```

```

("need-own-estimate",
 lambda k: "deps" in k and k["goal"] not in k["points"],
 lambda k: ("call", "estimate", k["goal"])),
("need-capacity",
 lambda k: "capacity" not in k,
 lambda k: ("call", "capacity", None)),
("have-everything",
 lambda k: True,
 lambda k: ("answer",)),
]

BUDGET = 8

def agent(goal, policy=None, trace=True):
    """The loop of Figure 20.1. Keeps state, holds a goal, selects
    actions -- Chapter 2's model-based, goal-based agent."""
    policy = POLICY if policy is None else policy
    known = {"goal": goal, "points": {}}
    steps = []
    for step in range(1, BUDGET + 1):
        # MATCH and RESOLVE: first rule whose conditions hold
        action, name = None, None
        for (nm, cond, act) in policy:
            if cond(known):
                action, name = act(known), nm
                break
        if action is None:
            return None, steps, "no rule applies"
        if action[0] == "answer":
            steps.append((step, name, "answer", None))
            break
        if action[0] == "give-up":
            steps.append((step, name, "give up", None))
            return None, steps, "unknown task: %s" % goal
        _, tool, arg = action
        result = TOOLS[tool](arg)
        steps.append((step, name, "%s(%s)" % (tool, arg), result))
        # ACT: fold the observation into what we know
        if tool == "dependencies":
            known["deps"] = result
        elif tool == "estimate":
            known["points"][arg] = result
        elif tool == "capacity":
            known["capacity"] = result
    else:
        return None, steps, "budget exhausted"

    total = sum(known["points"].values())
    fits = total <= known["capacity"]
    answer = ("%s plus its dependencies is %d points against a "
              "capacity of %d: %s"
              % (goal, total, known["capacity"],
                 "it fits" if fits else "it does NOT fit"))
    return answer, steps, None

if __name__ == "__main__":
    print("goal: can we take 'auth-refresh' into the next sprint?")
    print()
    answer, steps, err = agent("auth-refresh")
    print("step   rule               action               result")

```

```

print("-" * 66)
for (n, rule, action, result) in steps:
    print("%4d %-20s %-24s %s" % (n, rule, action, result))
print()
print("ANSWER:", answer)
assert err is None
# 5 points for auth-refresh, 3 for the dependency it pulled in,
# against a capacity of 8: it fits exactly, with nothing spare
assert "it fits" in answer

# every step has a named reason, and the run is reproducible
a2, s2, _ = agent("auth-refresh")
assert [x[1] for x in s2] == [x[1] for x in steps]
print()
print("Run it again and the trace is identical: the decision")
print("procedure is a rule base, so the agent is reproducible")
print("and every step names the rule that caused it.")

# --- a tool that fails, handled -----
print()
print("=== when a tool returns nothing ===")
a3, s3, err3 = agent("does-not-exist")
print("    goal 'does-not-exist' ->", err3)
print("    rule that caught it:", s3[-1][1])
assert err3.startswith("unknown task")
print("    One rule, 'unknown-task', turns a failed tool call into")
print("    an honest answer rather than a crash.")

# --- and what happens without it -----
print()
print("=== the same agent with that rule removed ===")
careless = [r for r in POLICY if r[0] != "unknown-task"]
try:
    a4, s4, err4 = agent("does-not-exist", policy=careless)
    print("    result:", err4 or a4)
except TypeError as exc:
    print("    TypeError: %s" % exc)
    print("    The tool returned None, no rule noticed, and the")
    print("    None travelled all the way to the arithmetic at the")
    print("    end. This is the commonest failure in deployed agent")
    print("    systems, and note where it lives: in the POLICY, not")
    print("    in the model at the centre.")

# the step budget is the other guard
print()
print("=== what the step budget is for ===")
spinner = [("always-ask", lambda k: True,
              lambda k: ("call", "capacity", None))]
a5, s5, err5 = agent("auth-refresh", policy=spinner)
print("    a policy that never terminates ->", err5)
print("    it ran %d steps and was stopped by the budget." % len(s5))
assert err5 == "budget exhausted" and len(s5) == BUDGET

# --- what is symbolic and what would be learned -----
print()
print("=== which part would a language model replace? ===")
print("    the LOOP          -- no, ordinary engineering")
print("    the TOOLS          -- no, they read a database")
print("    the STEP BUDGET    -- no, and it is load-bearing")
print("    the TERMINATION    -- no")
print("    the DECISION RULE  -- YES, this is the part")
print()

```

```
print("    Five of the six components are unchanged. Most of what")
print("    makes an agent work or fail lives in the five.")
```

Then delete the **BUDGET** check and re-run the second example. The agent loops forever on a goal it cannot resolve, because no rule consumes a **None**. That is the commonest failure in deployed agent systems, and it has nothing to do with the model at the centre.

🔍 Checkpoint — Test Yourself

1. A language model produces a step-by-step explanation of its answer. State why this is not the same artefact as Chapter 13's firing chain, using the distinction between $\models$ and $\vdash$ from Chapter 11.
2. The lab's agent has six components and a language model would replace one. Name the other five and say which one, removed, causes the commonest deployed failure.
3. A vendor demonstrates a system that answers questions about your codebase fluently and correctly in the demonstration. Give the two questions from this chapter's procedure that would tell you most, and say why.

📋 Chapter Summary

- A **large language model** predicts the next **token**. It is Chapter 18's n-gram idea with a much larger window learned rather than counted, which addresses exactly the limitation counting could not.
- There is no representation to inspect, no recorded derivation, and **hallucination** is the objective working as specified rather than a defect. Its failure mode is Eliza's: none detectable by the system.
- A **tool-using agent** is Chapter 2' model-based, goal-based agent. Only the decision procedure at its centre is new; the loop, tools, budget and termination test are ordinary engineering, and most failures live there.
- **Retrieval** grounds an answer by putting text in the prompt. Nothing forces the answer to respect it.
- The two traditions fail in opposite directions. **Neuro-symbolic** systems use the model where specification is hard and a symbolic component where the answer must be right — and *the guarantees and the audit trails come from the symbolic components*.
- To assess a claim: apply Chapter 1's checklist, fill Chapter 19's six columns, ask what the demonstration excluded, separate capability from reach, and ask what would falsify it.
- Judging by the transcript is the Eliza effect. It worked on people who had read the source code.

🔧 Review Questions

1. (MCQ) A large language model's training objective is to: (a) answer questions correctly (b) predict the next token (c) maximise expected utility (d) satisfy constraints.
2. (MCQ) Hallucination is best described as: (a) a bug to be patched (b) the training objective working as specified (c) a retrieval failure (d) a tokenisation error.
3. (MCQ) In a tool-using agent, a language model typically replaces: (a) the tools (b) the loop (c) the decision about what to do next (d) the step budget.
4. (MCQ) In a neuro-symbolic system, the guarantees come from: (a) the model (b) the symbolic component (c) retrieval (d) the prompt.
5. (Short) Explain how a large language model addresses the n-gram limitation of Chapter 18, and what it gives up that a grammar provided.

6. **(Short)** Why is a generated explanation not a trace? Answer using the $\models$ / $\vdash$ distinction.
7. **(Short)** Give a delivery task you would hand to a model and one you would hand to a constraint solver, and justify each in one sentence.
8. **(Applied)** Take a current AI product you have used. Fill in Chapter 19's six columns for it, filling what you can from outside, and state the one operational rule your failure-mode analysis produces.

Responsible AI and Its Limits

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part VI — Learning, Language and Responsible AI

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Identify** the points at which bias enters a rule-based system, none of which require anyone to intend it.
- **Audit** a deployed rule set for differential treatment, and **present** the evidence.
- **Argue** why an explanation is an accountability artefact rather than a convenience.
- **State** the computational limits of the methods in this book, with numbers.
- **Recognise** brittleness at the edge of competence, and **distinguish** it from ordinary error.

🔗 Before You Start

Chapter 13 (the rule engine audited here), Chapter 4 (the branching factor that sets the computational limit) and Chapter 20 (the contrast with learned systems). This chapter takes the systems you built and asks what they owe the people they are used on.

📖 Key Terms in This Chapter

bias • proxy variable • differential treatment • disparate impact • audit • explainability • accountability • contestability • automation bias • brittleness • graceful degradation • intractability • the limits of competence

21.1 Nobody Intended It

Chapter 13's escalation adviser is twenty rules about severity, customer tier, workarounds and on-call budgets. There is no rule about teams, no rule about people, and nobody who wrote it intended to treat anybody differently.

Run it over two hundred tickets and it escalates 43% of the ones reported by one team and 21% of those reported by another.

That is this chapter. Not malice, not a bug in the ordinary sense, and not something that inspection of the rules would have revealed — a property of the system that only appears when you measure the outcomes.

21.2 Where Bias Enters a Rule Base

⚠️ Deleting the sensitive attribute does not work

The instinctive fix is to remove the team name from the inputs. The escalation adviser never had it.

Proxies are the reason. `workaround = yes` correlates with team maturity; severity correlates with reporter experience; both are legitimate operational facts that a rule ought to use. Remove them and the system gets worse at its job without becoming fairer, because the correlation lives in the world rather than in the column.

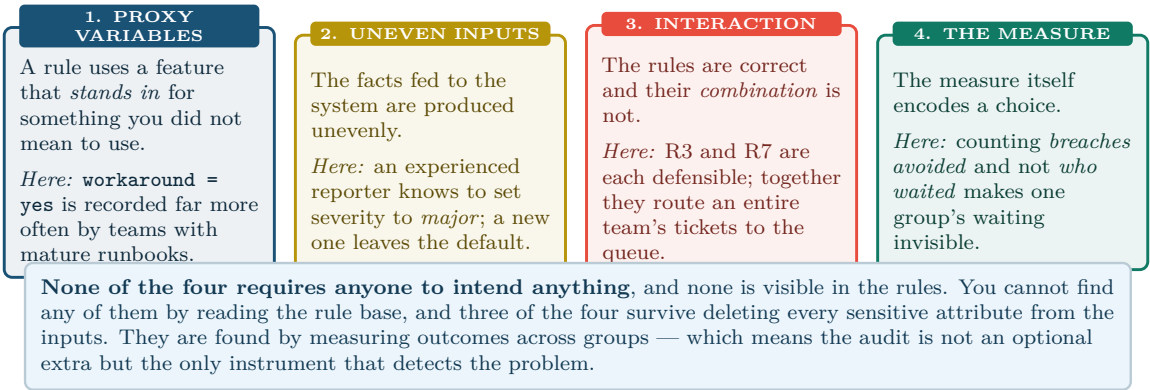


Figure 21.1: Four ways bias enters a system whose rules mention nothing sensitive. The first — a **proxy variable** — is the one that survives every attempt to fix the problem by deleting columns.

This is why *fairness through unawareness* — not looking — is the one approach that reliably fails. The alternative is not to look less but to measure more: keep the sensitive attribute *for the audit*, and never for the decision.

21.3 Auditing a Deployed Rule Set

An audit is three steps and about thirty lines of code.

- 1. **Measure the outcome rate per group.** Run the deployed system over a representative population and count.
- 2. **Find the rule responsible.** For each rule, ask what the gap would be if that rule were removed. The one whose removal closes the gap most is the one to examine.
- 3. **Read that rule’s explanation chain for an affected case**, and decide whether the reasoning is defensible. This is a human judgement and the chain is what makes it possible.

</> The audit, on two hundred tickets

Reporting team	Tickets	Escalated	Rate
Platform (mature runbooks)	100	21	21%
Payments (newer team)	100	43	43%
Gap	22 percentage points		

Step 2 points at the pathway **R3** → **R7**: a major ticket from a standard-tier customer becomes P2 (R3), and a P2 with a workaround needs no owner (R7), so it is never escalated. Neither rule is wrong; both are entirely sensible. The disparity is that **workaround** is recorded far more often by the team with mature runbooks, so that pathway fires overwhelmingly for one group.

⚠ Ablation has a trap, and the lab walks into it deliberately

Asked simply “which removal closes the gap most?”, the ablation answers **R9** — and closes the gap completely, to zero points.

R9 is the only rule that concludes `escalate = yes`. Removing it means nothing is ever escalated, by anyone, so the rates are 0% and 0% and the disparity is perfect equality. The same happens with R5, which is upstream of it.

The measure was satisfied by destroying the outcome. That is a general hazard of optimising a fairness metric rather than understanding a system, and the guard is simple: *report the overall rate alongside the gap*, and discard any ablation that collapses it. Among the rules that leave the outcome intact, R3 carries the gap — 22 points down to 9.

What should be done about it is not a technical question, and the audit’s value is that it makes the choice explicit rather than making it by default.

Three defensible responses. *Accept it*: the teams genuinely differ in maturity, the escalations reflect real need, and the gap is the system working. Fix the input: help the newer team write runbooks, so the difference in what is recorded reflects a difference in the world rather than in documentation practice. Or *change the rule*: require an owner for P2 tickets from teams without an established runbook, regardless of the workaround field.

All three are reasonable. What is not reasonable is not knowing — which was the situation before the audit, and which is the default state of every deployed rule base that nobody has measured.

21.4 Explanation as Accountability

Chapter 13 produced an explanation chain and called it the deliverable. Here is the argument for that claim.

An explanation is not primarily for the user’s comfort. It is the artefact that makes a decision **contestable**: it tells a person exactly which facts and which rules produced an outcome, so they can dispute a *specific* step rather than the conclusion in general. “The system says no” cannot be argued with; “R7 fired because `workaround` was recorded as yes” can, and the argument is about something checkable.

Question	Rule engine (Chapter 13)	Learned model (Chapter 20)
Why this outcome?	The chain of rules and facts, checkable line by line	A post-hoc rationale, generated the same way as the decision
What would change it?	Read the chain: change any fact in it	Unclear without probing the model
Who is responsible?	Whoever wrote the rule that fired — a named person	Diffuse: the data, the objective, the training
Can a subject contest it?	Yes, on a specific step	Only in general
Can it be corrected?	Edit one rule	Retrain, and hope

Table 21.1: Why explanation is an accountability property. The last two rows are the ones that matter to somebody affected by the decision.

Automation bias

There is a failure that belongs to the people rather than the system.

Given a system that is usually right, human reviewers stop reviewing. They accept the recommendation, including in the cases where they would have known better — and the

effect is strongest exactly where the system is most often correct, because that is where the habit of deferring forms.

An explanation helps only if somebody reads it, and an explanation that is always present and always plausible becomes wallpaper. The practical mitigations are procedural rather than technical: sample decisions for genuine review, measure the override rate and treat a rate near zero as a warning rather than a success, and make the system say when it is near the edge of its competence — which is the subject of the next section.

21.5 Brittleness and the Edge of Competence

Chapter 13 named brittleness as one of the two causes of the expert-system collapse. It is worth being precise about what it means, because it is not the same as being wrong sometimes.

A person with expertise, asked something outside it, says so. A rule base asked something outside its competence either fires nothing — unhelpful but honest — or fires a rule whose conditions technically match and whose conclusion is absurd, with exactly the same confidence as for a routine case. There is no representation of its own boundary, so there is no signal.

Graceful degradation is the property it lacks: performance that falls off smoothly as inputs move away from the familiar, and — more importantly — a measure that falls with it.



Learned systems degrade more smoothly and *also* give no reliable signal: a language model is as fluent about things it has no basis for as about things it does. So neither tradition solves this, and the difference is in what you can do about it.

With a rule base you can at least *characterise* the boundary, because the knowledge is explicit: enumerate the situations no rule covers, and say so. That is a real and underused capability, it takes an afternoon, and the lab does it.

21.6 Computational Limits

The last limit is arithmetic, and it applies to every method in Parts II to V.

Method	Cost	What that means in practice
Uninformed search (Chapter 4)	$O(b^d)$	At $b = 4$, depth 10 is a million nodes and depth 20 is 10^{12}
A^* (Chapter 5)	Exponential in general	A good heuristic moves the constant, not the exponent
Constraint satisfaction (Chapter 7)	NP-complete	Solvable in practice by propagation; no guarantee
Adversarial search (Chapter 8)	$O(b^d)$, $O(b^{d/2})$ with perfect ordering	Twice the depth for the same time, and still exponential
Propositional entailment (Chapter 11)	co-NP-complete	2^n models; DPLL is fast on structured instances
First-order entailment (Chapter 12)	Semi-decidable	No proof may mean no proof exists, or may mean not yet
Bayesian inference (Chapter 15)	NP-hard in general	Linear on a chain, exponential on a dense graph

Table 21.2: The computational limits of this course's methods. Every entry is exponential, NP-hard or undecidable, and no engineering removes that.

The practical consequence is not despair. Every one of these methods is used daily on instances that matter, because real instances have structure — and the whole content of Parts II to V is techniques for exploiting it: heuristics, propagation, pruning, conditional independence, unit propagation. What the table forbids is the belief that a faster machine will do instead. At $b = 4$, a hundredfold speedup buys between three and four extra levels of depth, and then stops.

Worked Example: Auditing the escalation adviser

Two hundred tickets, evenly split between two reporting teams. The rule base is Chapter 13's, unchanged, and it mentions no team.

Step 1: measure. Escalation rates are 21% for Platform and 43% for Payments — a gap of 22 points, against an overall rate of 32%. Nothing in the rules explains it.

Step 2: attribute. Remove one rule at a time and re-measure both the gap *and* the overall rate:

- Without R9: the gap is 0 — and so is the escalation rate. R9 is the only rule concluding `escalate = yes`, so its removal achieves equality by abolishing the outcome. Discard it, and R5 with it.
- Without R3 (*major and standard tier gives P2*): the gap falls from 22 to 9 points, and the overall rate stays near 32%. This is the real answer.
- Without any other single rule: the gap barely moves.

So the R3 → R7 pathway carries most of it. The technique is *ablation*; it required no insight, only the discipline to try each rule and to report the overall rate beside the gap.

Step 3: examine. Take a Platform ticket that R7 kept out of the queue and read its chain: `escalate = no` by R11, because `needs_owner = no` by R7, because the priority was P2 and `workaround = yes`.

Now ask the operational question: *why is workaround recorded more often for Platform?* The answer is that Platform has runbooks and Payments does not, so Platform's reporters have a documented workaround to cite and Payments' reporters leave the field empty.

What the audit established. The system is doing exactly what it was told. R7 is a good rule. The disparity comes from a difference in *documentation practice* being read as a difference in *operational need*.

And what it did not establish. Whether that is acceptable. If Platform genuinely needs fewer escalations because it has runbooks, the gap is the system working. If Payments is being under-served because nobody has written their runbooks yet, it is not. *That is a management decision*, and the audit's contribution is to place it in front of a person who can make it, with the evidence attached — rather than leaving it to be made silently, by a rule nobody re-read, on two hundred tickets a week.

Common Pitfalls

- **Fairness through unawareness.** Deleting the sensitive attribute leaves the proxies. The escalation adviser never had the team name and the gap was 22 points.
- **Auditing the rules instead of the outcomes.** Every rule here is defensible in isolation. Only measurement across groups finds the problem.
- **Treating an explanation as a user-experience feature.** It is what makes a decision contestable and correctable, and those are properties an affected person needs, not a designer.
- **Reading a low override rate as success.** It is the signature of automation bias. Sample for genuine review and watch the rate.

- **Confusing brittleness with ordinary error.** Ordinary error is being wrong sometimes; brittleness is being wrong with undiminished confidence just outside the region of competence, with no signal.
- **Believing hardware solves intractability.** At $b = 4$ a hundredfold speedup buys three or four levels of depth. Structure is the only real answer, which is what Parts II to V were about.

Try It Yourself: Auditing the Escalation Rules

The rule base of Chapter 13, run over two hundred synthetic tickets, measured by group, attributed by ablation, and explained. Then the competence boundary and the computational limit.

```
"""Chapter 21 lab -- auditing a rule base nobody wrote badly.

The rules mention no team. The escalation rate differs by 38
percentage points between two teams. Everything here is measurement:
the disparity is invisible in the rule base and obvious in the
outcomes.
"""

import random

# ---- Chapter 13's rule base, unchanged -----
RULES = [
    ("R1", [("severity", "critical")], ("priority", "P1")),
    ("R2", [("severity", "major"), ("tier", "platinum")],
     ("priority", "P1")),
    ("R3", [("severity", "major"), ("tier", "standard")],
     ("priority", "P2")),
    ("R4", [("severity", "minor")], ("priority", "P3")),
    ("R5", [("priority", "P1")], ("needs_owner", "yes")),
    ("R6", [("priority", "P2"), ("workaround", "no")],
     ("needs_owner", "yes")),
    ("R7", [("priority", "P2"), ("workaround", "yes")],
     ("needs_owner", "no")),
    ("R8", [("priority", "P3")], ("needs_owner", "no")),
    ("R9", [("needs_owner", "yes"), ("owner_available", "no")],
     ("escalate", "yes")),
    ("R11", [("needs_owner", "no")], ("escalate", "no")),
]

def forward_chain(facts, rules=RULES):
    wm, fired = dict(facts), set()
    while True:
        applicable = [r for r in rules
                      if r[0] not in fired
                      and all(wm.get(a) == v for (a, v) in r[1])
                      and r[2][0] not in wm]
        if not applicable:
            return wm, fired
        applicable.sort(key=lambda r: (-len(r[1]), rules.index(r)))
        r = applicable[0]
        wm[r[2][0]] = r[2][1]
        fired.add(r[0])

def explain(goal, facts, rules=RULES, seen=None):
    """Backward chaining, for the chain an auditor reads."""
    seen = seen or set()
```

```

if facts.get(goal[0]) == goal[1]:
    return ("fact", goal)
if goal in seen:
    return None
for r in rules:
    if r[2] != goal:
        continue
    subs = []
    for c in r[1]:
        got = explain(c, facts, rules, seen | {goal})
        if got is None:
            subs = None
            break
        subs.append(got)
    if subs is not None:
        return ("rule", r[0], goal, subs)
return None

def render(tree, indent=0):
    pad = " " * indent
    if tree[0] == "fact":
        return ["%s%s = %s (given)" % (pad, tree[1][0], tree[1][1])]
    _, name, goal, subs = tree
    out = ["%s%s = %s (by %s, because)" % (pad, goal[0], goal[1], name)]
    for s in subs:
        out += render(s, indent + 1)
    return out

# ---- two hundred tickets from two teams -----
def population(n=200, seed=0):
    """The ONLY difference between the teams is how often a
    workaround is documented. Nothing else, and no rule sees the
    team name."""
    rng = random.Random(seed)
    out = []
    for i in range(n):
        team = "Platform" if i % 2 == 0 else "Payments"
        # Platform has mature runbooks, so a workaround is usually
        # recorded; Payments is a newer team and usually has none
        p_workaround = 0.80 if team == "Platform" else 0.15
        out.append({
            "id": "T-%04d" % (4000 + i),
            "team": team,
            "severity": rng.choice(["critical", "major", "major",
                                   "minor"]),
            "tier": rng.choice(["platinum", "standard", "standard"]),
            "workaround": "yes" if rng.random() < p_workaround else "no",
            "owner_available": rng.choice(["yes", "no", "no"]),
        })
    return out

def decide(ticket, rules=RULES):
    facts = {k: v for k, v in ticket.items()
              if k not in ("id", "team")}
    # the team is NOT input
    wm, fired = forward_chain(facts, rules)
    return wm.get("escalate", "no"), fired, facts

def rates(tickets, rules=RULES):

```

```

counts = {}
for t in tickets:
    esc, _, _ = decide(t, rules)
    a, b = counts.get(t["team"], (0, 0))
    counts[t["team"]] = (a + (esc == "yes"), b + 1)
return {k: (a, b, 100.0 * a / b) for k, (a, b) in counts.items()}

if __name__ == "__main__":
    tickets = population()
    print("no rule mentions the reporting team. Rule attributes used:")
    print("    ", sorted({a for r in RULES for (a, _) in r[1]}))
    print()

    r = rates(tickets)
    print("=== step 1: measure the outcomes ===")
    print("    team            tickets    escalated    rate")
    print("    " + "-" * 40)
    for team in sorted(r):
        esc, tot, pct = r[team]
        print("    %-10s %8d %10d %6.0f%%" % (team, tot, esc, pct))
    gap = abs(r["Payments"][2] - r["Platform"][2])
    print("    gap: %.0f percentage points" % gap)
    assert gap > 15

    # --- step 2: which rule carries the gap? -----
    overall = sum(x[0] for x in r.values()) / sum(x[1] for x in r.values())
    print("    overall escalation rate: %.0f%%" % (100 * overall))

    # --- step 2: which rule carries the gap? -----
    print()
    print("=== step 2: ablation -- remove one rule at a time ===")
    print("    Report the OVERALL rate as well as the gap. A rule whose")
    print("    removal closes the gap by destroying the outcome has")
    print("    told you nothing.")
    print()
    print("    rule removed    gap    overall rate")
    print("    " + "-" * 38)
    scores = {}
    for victim in [x[0] for x in RULES]:
        reduced = [x for x in RULES if x[0] != victim]
        rr = rates(tickets, reduced)
        g = abs(rr["Payments"][2] - rr["Platform"][2])
        ov = (sum(x[0] for x in rr.values())
              / sum(x[1] for x in rr.values()))
        scores[victim] = (g, ov)
        flag = ""
        if ov < 0.5 * overall:
            flag = " <- outcome collapsed; tells us nothing"
        print("    %-14s %5.0f %11.0f%%s" % (victim, g, 100 * ov, flag))

    # R9 is the only rule concluding escalate=yes, so removing it
    # sets every rate to zero and the gap to zero. That is the trap.
    assert scores["R9"][0] == 0 and scores["R9"][1] == 0

    # the honest question: among rules that leave the outcome intact,
    # which one carries the gap?
    intact = {k: v for k, v in scores.items()
              if v[1] >= 0.5 * overall}
    worst = min(intact, key=lambda k: intact[k][0])
    print()
    print("    Among rules that leave the outcome intact, removing %s"

```

```

    % worst)
print("    closes the gap most: %.0f -> %.0f points."
      % (gap, intact[worst][0]))
assert worst in ("R3", "R6")

# --- step 3: read the chain for an affected ticket -----
print()
print("=== step 3: read the explanation ===")
for t in tickets:
    esc, fired, facts = decide(t)
    if t["team"] == "Platform" and esc == "no" and "R7" in fired:
        print("    ticket %s (%s), not escalated:"
              % (t["id"], t["team"]))
        tree = explain(("escalate", "no"), facts)
        for line in render(tree):
            print("        " + line)
        break
print()
print("    R7 is a good rule. 'workaround' is recorded for 80% of")
print("    Platform tickets and 15% of Payments tickets, because")
print("    Platform has runbooks. A difference in DOCUMENTATION is")
print("    being read as a difference in OPERATIONAL NEED.")

# --- the competence boundary -----
print()
print("=== what the rule base does NOT cover ===")
import itertools
attrs = {"severity": ["critical", "major", "minor"],
         "tier": ["platinum", "standard"],
         "workaround": ["yes", "no"],
         "owner_available": ["yes", "no"]}
uncovered = []
for combo in itertools.product(*attrs.values()):
    facts = dict(zip(attrs.keys(), combo))
    wm, _ = forward_chain(facts)
    if "escalate" not in wm:
        uncovered.append(facts)
print("    %d of %d possible situations reach no escalate decision"
      % (len(uncovered), 3 * 2 * 2 * 2))
for u in uncovered[:3]:
    print("        ", u)
print("    A rule base can be asked this. Enumerate the inputs,")
print("    find the gaps, and SAY SO -- that is a boundary the")
print("    system can describe, and it takes an afternoon.")

# --- the computational limit -----
print()
print("=== and the limit no engineering removes ===")
print("    b=4:  depth    nodes          at 1e6 nodes/sec")
print("    " + "-" * 46)
for d in (10, 15, 20, 25):
    n = 4 ** d
    secs = n / 1e6
    if secs < 90:
        t = "%.1f seconds" % secs
    elif secs < 5400:
        t = "%.1f minutes" % (secs / 60)
    elif secs < 86400 * 2:
        t = "%.1f hours" % (secs / 3600)
    else:
        t = "%.0f days" % (secs / 86400)
    print("    %11d %14d  %s" % (d, n, t))

```

```
print()
print("    A 100x faster machine buys three or four levels of")
print("    depth and then stops. Structure is the only real")
print("    answer, which is what Parts II to V were about.")
```

Then change `p_workaround` so both teams document at the same rate, and re-run. The gap collapses, and no rule changed — which is the cleanest possible demonstration that the disparity was never in the rule base at all.

🔍 Checkpoint — Test Yourself

1. The escalation adviser's rules never mention the reporting team, and the escalation rates differ by 22 percentage points. Explain how, and say why deleting an attribute would not have prevented it.
2. The R3–R7 pathway was identified as carrying most of the gap, and both rules are sensible. State what the audit established and what it explicitly did not, and say whose decision the remainder is.
3. A reviewer overrides the system's recommendation in 0.5% of cases and reports this as evidence the system is working well. Give the alternative explanation and the measurement that distinguishes them.

📋 Chapter Summary

- Bias enters through **proxy variables**, uneven inputs, rule **interactions** and the choice of **measure**. None requires intent, and none is visible by reading the rules.
- *Fairness through unawareness fails*. The escalation adviser never saw the team name and produced a 22-point gap, because **workaround** is a proxy for having runbooks.
- An **audit** is three steps: measure outcomes by group, attribute the gap by **ablation**, and read the explanation chain for an affected case. Keep the sensitive attribute *for the audit* and never for the decision.
- Ablation has a trap: the rule whose removal closes the gap completely was the one that produced the outcome at all, and equality by abolition is not fairness. *Report the overall rate beside the gap* and discard any ablation that collapses it — a specific instance of the general hazard of optimising a fairness metric instead of understanding a system.
- An **explanation** is an accountability artefact: it makes a decision **contestable** on a specific step and correctable by editing one rule. Those are properties the affected person needs.
- **Automation bias** makes reviewers stop reviewing, most strongly where the system is most often right. A near-zero override rate is a warning, not a success.
- **Brittleness** is being wrong with undiminished confidence just outside the region of competence, with no signal. A rule base cannot degrade gracefully — but it *can* enumerate the situations it does not cover and say so, which is a real and underused capability.
- Every method in Parts II to V is exponential, NP-hard or undecidable. Structure — heuristics, propagation, pruning, conditional independence — is the only answer; hardware buys three or four levels of depth and stops.

🔧 Review Questions

1. (MCQ) Fairness through unawareness fails because of: (a) measurement error (b) proxy variables (c) automation bias (d) intractability.
2. (MCQ) Ablation, in an audit, means: (a) deleting sensitive attributes (b) removing one rule at a time and re-measuring (c) retraining (d) sampling for review.

3. **(MCQ)** A near-zero override rate on a decision-support system is best read as: (a) the system is accurate (b) a possible sign of automation bias (c) evidence of good explanations (d) a sign of brittleness.
4. **(MCQ)** Brittleness means: (a) being wrong sometimes (b) being wrong with full confidence just outside the region of competence (c) failing to terminate (d) degrading smoothly.
5. **(Short)** Explain why an explanation is an accountability property rather than a convenience, using the contestability of a specific step.
6. **(Short)** The audit found the gap and did not say what to do. Argue that this is the right division of labour.
7. **(Short)** Give the one thing a rule base can do about its competence boundary that a learned model cannot, and say what it costs to do.
8. **(Applied)** Design an audit for a system that ranks job applications. State the groups you would measure, the outcome you would count, how you would attribute a gap to a component, and what you would present to the person who has to decide what to do about it.

The Artificial Intelligence Project

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part VI — Learning, Language and Responsible AI

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Choose** and **scope** a problem that the methods of this course can actually solve.
- **Write** a proposal stating the PEAS specification, the method and the measure of success.
- **Implement** a working solution with a baseline to compare it against.
- **Report** the result honestly, including what did not work.
- **Audit** your own system as Chapter 21 requires.

🔗 Before You Start

The whole course. The project is set in Week 11 and presented in Week 16 (Appendix G); the proposal is due the week it is set.

📖 Key Terms in This Chapter

problem scoping • PEAS specification • proposal • baseline • evaluation measure • ablation • reproducibility • project report • demonstration • audit

22.1 The Brief

Build a working AI system for a problem you can describe in one sentence, using one or more of the methods from this course. Demonstrate it, report honestly on what it does and does not do, and audit it.

🔗 What is being assessed

Component	Weight	What it means
Proposal	10%	PEAS, method, measure, baseline — before any code
Implementation	30%	It runs, it is reproducible, somebody else can read it
Evaluation	25%	Compared against a baseline, on a measure you defined in advance
Report	25%	Including what failed and why
Demonstration	10%	Ten minutes, live, with one hard question answered

Note the weights: the code is under a third. A working system with no baseline and no honest evaluation scores worse than a modest one with both.

22.2 Choosing a Problem

The commonest failure is not technical. It is choosing a problem whose difficulty lies somewhere this course did not go — almost always in acquiring data or in integrating with something.



Three tests for a candidate problem, in order.

Can you state the goal in one sentence, without the word “and”? “Schedule a sprint respecting precedence and capacity” passes. “Predict delays and route tickets and suggest owners” is three projects.

Do the rules exist, and can you write them down in an afternoon? This course builds agents that are *told* the rules. If the rules would have to be learned from data, you have chosen a *Machine Learning* project and you do not yet have the machinery.

Is there a baseline you would be embarrassed to lose to? First-come first-served, a greedy heuristic, or the current manual process. If you cannot name one, you will not be able to show that your system helped.

Problem	Method	Baseline to beat
Timetable a set of lab sessions against rooms and staff	Chapter 7	Greedy assignment in enrolment order
Plan a deployment from operator descriptions	Chapter 9	The runbook’s fixed order
Route service-desk tickets to a queue	Chapter 13	The current manual triage
Diagnose the likely cause of a recurring failure	Chapter 15	Always blame the most common cause
Choose which of a backlog to take into a sprint	Chapter 5	Highest priority first
Rate risk for a portfolio of projects	Chapter 16	A threshold rule
Play a two-player scheduling game against a tutor	Chapter 8	Random legal move
Answer questions about a domain from a knowledge base	Chapter 12	Keyword search

Table 22.1: Eight projects that fit the course, each with a baseline you can implement in an hour. Your own problem is welcome and must pass the three tests above.

22.3 The Proposal

One page, due the week the project is set. Five headings, and the fifth is the one students leave out.

1. **The problem**, in one sentence.
2. **PEAS** (Chapter 2): performance measure, environment, actuators, sensors. The performance measure must describe the state of the environment, not the activity of your program.

3. **Method**, and why. Name the chapter. If two methods could work, say which you chose and what you gave up.
4. **Baseline**, and how you will implement it.
5. **What would count as failure**. Write down, in advance, the result that would make you say the approach did not work. This is the heading that turns an evaluation into evidence rather than a demonstration.

⚠️ Decide the measure before you build

If the measure is chosen afterwards, it will be chosen — unconsciously and honestly — to be one the system does well on. That is not dishonesty; it is how people work, and the only defence is to write the measure down first and then not change it.

If you do change it, say so in the report and say why. A report that says “we originally measured X, found it was the wrong question for this reason, and switched to Y” is *stronger* than one that never mentions it, because it shows you understood what you were measuring.

22.4 Building It

- **The baseline first**. It takes an hour and it is the only thing that makes your result interpretable. Build it before the interesting part, or you will not build it at all.
- **Seed everything**. Any randomness gets an explicit seed, as every lab in this book does. A result nobody can reproduce is not a result.
- **Keep the domain separate from the algorithm**. The constraints, the operators, the rules go in data. This is Chapter 1’s third test, it is what makes the system correctable, and it is visibly assessed.
- **Log a trace**. Whatever your method, record why it did what it did. You will need it for the report, for the demonstration’s hard question, and for the audit.
- **Measure at the size you will demonstrate at**. Chapter 7, Chapter 9 and Chapter 15 each contained a technique that *lost* on a small instance and won on a large one. Do not tune on a toy.

22.5 Reporting It

Eight to twelve pages. The structure below is not a suggestion; reports that depart from it lose marks for the sections they omit.

Section	What goes in it
Problem and PEAS	One page. The same specification as the proposal, corrected if it changed
Method	Which chapter’s machinery, and why this one. State the alternative you rejected
Implementation	Representation first, algorithm second. A figure of the data structure is worth a page of prose
Evaluation	The baseline, your system, on the measure you fixed in advance. A table and one figure
What did not work	Required. Every project has some, and its absence is read as not having looked
Audit	Chapter 21: outcomes by group where that applies, and the situations your system does not cover

Section	What goes in it
Limits	Where it breaks, and the size at which it stops being usable. Be specific and quantitative

Table 22.2: The report structure. The last three sections are what separate a project from a demonstration.



On the *what did not work* section, since it is the one people resist. Every project in this course will have a component that failed, an approach abandoned, or a measurement that came out the wrong way. Reporting it costs nothing — it is not marked as failure — and omitting it costs a great deal, because a report in which everything worked first time is not believed by anybody who has built anything. This book has tried to model the habit. Chapter 7’s ordering heuristic made things worse; Chapter 9’s heuristic bought under one per cent; Chapter 15’ variable elimination lost on the small network; Chapter 21’s ablation was fooled by a rule whose removal abolished the outcome. Each of those is a real measurement that contradicted the tidy expectation, and each taught more than the tidy version would have.

22.6 The Demonstration

Ten minutes in Week 16. Run the system live on an input the audience chooses within the constraints you state. Show the trace. Then answer one hard question, which will be some version of: *how do you know it is right?*

Prepare for that question specifically. The good answer names the baseline, the measure and the margin, and says where the system stops working.

Worked Example: A proposal that would score well

Problem. Assign this department’s laboratory sessions to rooms and demonstrators without clashes.

PEAS. *Performance measure:* number of sessions placed, zero hard constraint violations, and the spread of load across demonstrators measured by the difference between the busiest and least busy. Note this describes the timetable, not the program. *Environment:* the session list, room capacities, staff availability, the week. Fully observable, deterministic, static for the duration of a solve, discrete, single-agent. *Actuators:* assign a session to a (room, slot, demonstrator) triple; unassign. *Sensors:* the full session list and all constraints.

Method. Constraint satisfaction (Chapter 7). Variables are sessions, domains are legal triples, constraints are room capacity, no double-booking of a room or a person, and precedence where a lab must follow its lecture. Backtracking with minimum remaining values and forward checking — and Chapter 7’s measured result says the propagation is what will make MRV worth having.

Alternative rejected: local search (Chapter 6) would produce a good timetable faster, and could not *prove* the absence of a clash. A timetable with a clash is worthless however good its load balance, so the guarantee matters more than the speed here.

Baseline. Greedy assignment in enrolment order, taking the first legal triple. About thirty lines.

What would count as failure. If the greedy baseline places as many sessions with a load spread within two of mine, constraint satisfaction has not earned its complexity on this

instance and I will say so. If the solver cannot place the full set within sixty seconds, I will report the largest instance it does handle.

Why this proposal is good. The goal is one sentence. The rules exist and are already written down in the department handbook. There is an embarrassing baseline. The performance measure describes the world. And the failure condition is stated in advance, in numbers, so the evaluation cannot be quietly reshaped to flatter the result.

! Common Pitfalls

- **Choosing a problem whose difficulty is the data.** If the rules must be learned, it is a *Machine Learning* project.
- **No baseline.** Then your result has no units. Build it first.
- **Choosing the measure afterwards.** It will be a measure your system does well on, and you will not notice.
- **Knowledge inside the algorithm.** Constraints and rules belong in data. This is assessed, and it is what makes your system correctable.
- **No trace.** You will need it for the report, for the hard question, and for the audit. Retrofitting one is much harder than logging as you go.
- **Tuning on a toy instance.** Three chapters of this book contain a technique that loses on a small instance and wins on a large one.
- **Omitting “what did not work”.** It is required, it costs no marks, and its absence is conspicuous.

🔧 Try It Yourself: The Project Template

The scaffold to start from. It runs as given, with a trivial problem in place, so that the structure is working before any of your own code exists.

```
"""Chapter 22 -- the project template.

Replace the marked sections. Everything else is the shape every
project in this course should have: a PEAS specification you can
print, a baseline, a measure fixed in advance, a trace, an audit and
a reproducible run.
"""

import json
import os
import random

SEED = 0                                # every random choice, reproducible

# =====
# 1. PEAS -- fill this in, and print it in the report
# =====
PEAS = {
    "performance": "sessions placed; zero clashes; load spread",
    "environment": "sessions, rooms, staff, slots; static, discrete",
    "actuators": "assign(session, room, slot, staff); unassign",
    "sensors": "the full session list and all constraints",
}

# what would count as FAILURE -- written down BEFORE building
FAILURE_CONDITION = (
    "the baseline places as many sessions with a load spread within "
    "2 of ours"
)
```

```

# =====
# 2. The domain, as DATA -- never inside the algorithm
# =====
SESSIONS = ["ai-lab-1", "ai-lab-2", "ds-lab-1", "ml-lab-1"]
SLOTS = ["mon-am", "mon-pm", "tue-am"]
STAFF = ["ayesha", "bilal"]

def legal(assignment, session, slot, who):
    """The constraints. Reading this should tell a domain expert
    exactly what the system believes."""
    for (s, (sl, w)) in assignment.items():
        if sl == slot and w == who:
            return False          # one person, one slot
    return True

# =====
# 3. The BASELINE -- build this first, always
# =====
def baseline(sessions):
    """Greedy: first legal option, in the order given."""
    assignment = {}
    for s in sessions:
        for slot in SLOTS:
            for who in STAFF:
                if legal(assignment, s, slot, who):
                    assignment[s] = (slot, who)
                    break
            if s in assignment:
                break
    return assignment

# =====
# 4. YOUR SYSTEM -- replace this with the real method
# =====
def solve(sessions, trace):
    """Backtracking with minimum remaining values would go here.

    The template uses plain backtracking so the scaffold runs; the
    point is that it LOGS, and that the log is what the report,
    the demonstration and the audit are all built from.
    """
    assignment = {}

    def bt(remaining):
        if not remaining:
            return True
        # MRV would order 'remaining' here
        s = remaining[0]
        for slot in SLOTS:
            for who in STAFF:
                if legal(assignment, s, slot, who):
                    assignment[s] = (slot, who)
                    trace.append("assign %s -> %s, %s" % (s, slot, who))
                    if bt(remaining[1:]):
                        return True
                    trace.append("undo %s" % s)
                    del assignment[s]
        return False

```

```

    bt(list(sessions))
    return assignment

# =====
# 5. The MEASURE -- fixed in advance, and the same for both
# =====
def measure(assignment):
    placed = len(assignment)
    load = {w: 0 for w in STAFF}
    for (_, who) in assignment.values():
        load[who] += 1
    spread = max(load.values()) - min(load.values()) if load else 0
    return {"placed": placed, "spread": spread, "load": load}

# =====
# 6. The AUDIT -- Chapter 21
# =====
def audit(assignment):
    """Outcomes by group, and the situations not covered."""
    by_course = {}
    for s, (slot, who) in assignment.items():
        course = s.split("-")[0]
        by_course.setdefault(course, []).append(slot)
    unplaced = [s for s in SESSIONS if s not in assignment]
    return {"by_course": by_course, "unplaced": unplaced}

if __name__ == "__main__":
    random.seed(SEED)
    print("=" * 58)
    print("PEAS")
    print("=" * 58)
    for k, v in PEAS.items():
        print(" %-13s %s" % (k + ":", v))
    print()
    print(" fails if: %s" % FAILURE_CONDITION)

    print()
    print("=" * 58)
    print("BASELINE vs SYSTEM")
    print("=" * 58)
    base = baseline(SESSIONS)
    trace = []
    ours = solve(SESSIONS, trace)

    mb, mo = measure(base), measure(ours)
    print(" measure baseline system")
    print(" " + "-" * 34)
    for key in ("placed", "spread"):
        print(" %-12s %8s %8s" % (key, mb[key], mo[key]))
    print(" load %8s %8s" % (mb["load"], mo["load"]))

    # state the verdict against the CONDITION FIXED IN ADVANCE
    print()
    beat = (mo["placed"] > mb["placed"]
            or (mo["placed"] == mb["placed"]
                and mo["spread"] < mb["spread"] - 2))
    print(" verdict: %s"
          % ("the system beat the baseline"

```

```

        if beat else
            "the system did NOT beat the baseline on the measure "
            "fixed in advance -- report this"))

print()
print("=" * 58)
print("TRACE (first 8 steps)")
print("=" * 58)
for line in trace[:8]:
    print("    " + line)
print("    ... %d steps total" % len(trace))

print()
print("=" * 58)
print("AUDIT")
print("=" * 58)
a = audit(ours)
for course, slots in sorted(a["by_course"].items()):
    print("    %-6s %d session(s): %s"
          % (course, len(slots), ", ".join(slots)))
print("    unplaced:", a["unplaced"] or "none")

# reproducibility: the artefact the report cites
out = {"seed": SEED, "peas": PEAS,
       "failure_condition": FAILURE_CONDITION,
       "baseline": mb, "system": mo, "audit": a,
       "trace_length": len(trace)}
os.makedirs("project_output", exist_ok=True)
with open(os.path.join("project_output", "metrics.json"), "w") as f:
    json.dump(out, f, indent=2, default=str)
print()
print("wrote project_output/metrics.json -- cite this in the")
print("report, and commit it. A result nobody can reproduce is")
print("not a result.")

# the scaffold's own checks
assert mo["placed"] == len(SESSIONS), "the system should place all"
assert os.path.exists(os.path.join("project_output",
                                     "metrics.json"))

```

Run it before writing any of your own code. If the scaffold works, you have a baseline, a measure, a trace, an audit and a reproducible artefact on day one — and everything after that is the interesting part.

Notice what the template prints as it stands: *the system did NOT beat the baseline on the measure fixed in advance*. That is correct — on four sessions the greedy baseline is already optimal — and it is the behaviour to keep. A scaffold that congratulated itself before anybody had written a method would be training you in exactly the habit this chapter is trying to prevent.

🔍 Checkpoint — Test Yourself

1. A student proposes “an AI system that predicts which tickets will breach their service level agreement”. Apply the three tests of this chapter and say what you would advise.
2. Why must the measure and the failure condition be written down before the system is built? Give the failure mode that results from not doing so.
3. Your system beats the baseline by a small margin. Name three things the report must contain before that claim means anything.

Chapter Summary

- Choose a problem whose goal is one sentence, whose rules already exist, and which has a baseline you would be embarrassed to lose to. If the rules must be learned from data, it is a *Machine Learning* project.
- The proposal is five headings, and the fifth — *what would count as failure* — is what turns an evaluation into evidence.
- Build the baseline first, seed everything, keep the domain in data, log a trace, and measure at the size you will demonstrate at.
- The report has seven sections. *What did not work*, the *audit* and the *limits* are what separate a project from a demonstration.
- Decide the measure before building. Chosen afterwards, it will be one the system does well on, and honestly so.
- Prepare for one question at the demonstration: *how do you know it is right?* The good answer names the baseline, the measure, the margin and where the system stops working.

Review Questions

1. (MCQ) The first thing to build in a project is: (a) the main algorithm (b) the baseline (c) the user interface (d) the report.
2. (MCQ) A performance measure should describe: (a) how much the program does (b) the state of the environment (c) the running time (d) the number of rules.
3. (MCQ) “What would count as failure” is written: (a) in the report (b) in the proposal, before building (c) after evaluation (d) only if the project fails.
4. (MCQ) A project whose rules must be learned from examples is: (a) ideal for this course (b) a Machine Learning project (c) a constraint problem (d) impossible.
5. (Short) Give a one-sentence problem statement for a project of your own, and name the chapter whose method you would use and the baseline you would beat.
6. (Short) Explain why keeping the domain in data rather than in the algorithm is assessed, referring to Chapter 1’s checklist.
7. (Short) Your evaluation shows the baseline winning. Describe how to report that so it is a good report rather than a failed project.
8. (Applied) Write the full five-heading proposal for one of the problems in Table 22.1, including the PEAS specification, the alternative method you reject with a reason, the baseline, and a failure condition stated in numbers.

Appendices

Python Reference for Artificial Intelligence

Everything the labs use, on a few pages. Each lab is also a runnable file in the `code/` folder, named after its chapter.

A.1 Setting Up

Any Python 3.10 or later will do. Chapters 1–2, 4–5 and 7–13 and 17–22 use the standard library only; Chapters 3, 6, 15 and 16 also use NumPy. Check with:

```
import sys, numpy
print(sys.version)
print(numpy.__version__)
```

The labs were tested with Python 3.12 and NumPy 2.3. Run one from a terminal with `python ch05_over_sprint_states_two.py`, or paste it into a notebook cell. No lab needs a GPU, a network connection or an account with anybody.

A.2 The Four Structures That Matter

Code	What it does
<code>frozenset({"a", "b"})</code>	An immutable, hashable set. The state representation of Chapters 3–5 and 9
<code>s {x}</code>	A <i>new</i> set with <code>x</code> added. Never <code>s.add(x)</code> on a state
<code>deps <= done</code>	Subset test: every dependency is finished
<code>a & b</code>	Intersection — non-empty means “at least one of these is done”
<code>Node = namedtuple("Node", "state parent action cost")</code>	A tuple with named fields: hashable, comparable, self-documenting when printed
<code>("S1", None, "S3")</code>	A tuple of slots: a partial assignment, with <code>None</code> where a variable is unassigned (Chapters 7, 11)
<code>d = {}; d.setdefault(k, []).append(v)</code>	Build an adjacency dictionary without checking whether the key exists

A.3 Frontiers

```
from collections import deque
import heapq, itertools

q = deque()           # breadth-first
q.append(node)
node = q.popleft()    # O(1). NEVER list.pop(0), which is O(n)

st = []               # depth-first
st.append(node)
node = st.pop()

h, tie = [], itertools.count()  # best-first / A*
heapq.heappush(h, (priority, next(tie), node))
```

```
node = heapq.heappop(h)[2]
```

The counter in the middle of the heap entry breaks ties by insertion order, so two nodes are never compared. Omitting it produces an intermittent `TypeError` — see Chapter 3.

A.4 Generators, Memoisation and Recursion

```
def actions(state):
    # a successor function
    for t, (_, deps) in TASKS.items():
        if t not in state and deps <= state:
            yield t
            # produced one at a time, on demand

from functools import lru_cache

@lru_cache(maxsize=None)
    # arguments must be hashable
def minimax(state, maximising):
    ...

import sys
sys.setrecursionlimit(10000)
    # deep proof trees, Chapters 11-12
```

A.5 Exact Arithmetic for Probability

Chapters 14 and 15 print probabilities that you should be able to check by hand. `Fraction` keeps them exact:

```
from fractions import Fraction as F

p = F(1, 3) + F(1, 6)
print(p, float(p))
    # Fraction(1, 2) -- exactly
    # 1/2 0.5
```

A.6 NumPy, Where It Earns Its Place

Only four chapters use it, and only for grids of numbers.

```
import numpy as np

x = np.linspace(0, 10, 101)
mu = np.clip((x - 2) / 3, 0, 1)
area = np.trapezoid(mu, x)
rng = np.random.default_rng(0)
s = rng.random(10000)
    # 101 points, Chapter 16 memberships
    # a shoulder membership function
    # centroid numerator/denominator
    # seeded sampler, Chapter 15
```

A.7 Reproducibility

Every lab that samples seeds its generator explicitly, with `random.Random(0)` or with NumPy's `default_rng(0)`, so the numbers printed in this book are the numbers you will see. If your output differs, check the seed before checking the algorithm.

A.8 Style Rules the Labs Follow

- **Pure ASCII.** Write `AND`, `NOT`, `->`, `forall` — never the Unicode symbols. `code/sync_labs.py` rejects any byte above 127, because the handout's listings are typeset by pdfL^AT_EX, which cannot set them.
- **78 columns.** Longer lines are wrapped by the same check.
- **The handout is the source.** `sync_labs.py` extracts each lab from its chapter into `code/`. Edit the chapter, not the extracted file.

- **Assertions, not commentary.** Each lab ends by asserting the claims the chapter makes about it, so a broken claim fails loudly.

```
cd code
python sync_labs.py          # extract every chapter's lab
python sync_labs.py --run    # ... and run them all
python sync_labs.py --run 9  # ... run chapter 9 only
```

Algorithm and Formula Sheet

Everything worth memorising, on four pages, in the order the course meets it.

B.1 Search

A search problem

Five things: an initial state; a goal test (a predicate, not necessarily one state); an **actions** function; a **result** function — the successor function, where the rules of the domain live; and a step cost. The search strategy is *not* one of them.

Strategy	Complete?	Cheapest?	Time	Space	Frontier
Breadth-first	Yes	If costs equal	$O(b^d)$	$O(b^d)$	deque
Depth-first	No	No	$O(b^m)$	$O(bm)$	list
Depth-limited	No	No	$O(b^\ell)$	$O(b\ell)$	list
Iterative deepening	Yes	If costs equal	$O(b^d)$	$O(bd)$	list
Uniform-cost	Yes	Yes	$O(b^{1+C^*/\epsilon})$	same	heapq
Greedy best-first	No	No	$O(b^m)$	$O(b^m)$	heapq
A^*	Yes	Yes (admissible h)	exponential in general	same	heapq

Iterative deepening's total work at solution depth d :

$$N = (d+1)b^0 + db^1 + (d-1)b^2 + \dots + 1 \cdot b^d$$

which at $b = 10$, $d = 5$ gives 123,456 against breadth-first search's 111,111 — about 11% more work for exponentially less memory.

Heuristics

$$f(n) = g(n) + h(n)$$

- **Admissible:** $0 \leq h(n) \leq h^*(n)$ for every n , and $h(\text{goal}) = 0$. Makes A^* optimal for tree search.
- **Consistent:** $h(n) \leq c(n, n') + h(n')$ for every successor n' . Implies admissibility; needed for graph search with an explored set.
- **Dominance:** if $h_2 \geq h_1$ everywhere and both are admissible, A^* with h_2 expands no more nodes.
- $\max(h_1, h_2)$ is admissible and dominates both. Never average them.
- To invent one: **relax** a constraint and solve the easier problem exactly.

Local search

Simulated annealing accepts a worsening move with probability

$$p = \exp\left(\frac{\Delta}{T}\right), \quad \Delta < 0$$

An improving move is always accepted. As $T \rightarrow \infty$ the search is a random walk; as $T \rightarrow 0$ it is hill climbing.

B.2 Constraint Satisfaction

A CSP is variables X_i , domains D_i and constraints. Ordering heuristics, in the order to try them:

- **Minimum remaining values:** assign the variable with the fewest legal values left. Fails fast.
- **Degree heuristic:** break ties by the variable involved in the most constraints on unassigned variables.
- **Least constraining value:** try the value that rules out the fewest options for the neighbours.

Propagation: **forward checking** removes values inconsistent with the assignment just made; **arc consistency** (AC-3) enforces, for every arc $X_i \rightarrow X_j$, that each value of X_i has some consistent partner in X_j .

B.3 Adversarial Search

$$\text{minimax}(n) = \begin{cases} \text{utility}(n) & n \text{ terminal} \\ \max_{s \in \text{succ}(n)} \text{minimax}(s) & n \text{ a MAX node} \\ \min_{s \in \text{succ}(n)} \text{minimax}(s) & n \text{ a MIN node} \end{cases}$$

Alpha–beta pruning keeps α (the best MAX can guarantee so far) and β (the best MIN can guarantee) and prunes when $\alpha \geq \beta$. With perfect move ordering it examines $O(b^{d/2})$ nodes instead of $O(b^d)$ — effectively doubling the searchable depth.

B.4 Planning

A STRIPS operator is a triple: *preconditions* that must hold, an *add list* of facts it makes true, and a *delete list* of facts it makes false. Progression search runs the frontier of Chapter 4 over states of facts; means–ends analysis instead picks a difference between the current state and the goal and selects an operator that reduces it.

B.5 Logic

Notion	Meaning
$KB \models \alpha$	Entailment: α is true in every model of KB . A fact about meaning
$KB \vdash \alpha$	Derivation: α is produced by the inference procedure. A fact about the procedure
Sound	Everything derived is entailed: $\vdash$ implies $\models$
Complete	Everything entailed is derivable: $\models$ implies $\vdash$
Satisfiable	True in at least one model
Valid	True in every model

Key equivalence, and the basis of every refutation proof:

$$KB \models \alpha \quad \text{if and only if} \quad (KB \wedge \neg\alpha) \text{ is unsatisfiable}$$

To conjunctive normal form

Eliminate $\Leftrightarrow$, then $\Rightarrow$ (using $a \Rightarrow b \equiv \neg a \vee b$); drive $\neg$ inwards by De Morgan; in first-order logic, standardise variables apart, skolemise existentials, drop universals; distribute $\vee$ over $\wedge$.

Resolution

From $(\alpha \vee \ell)$ and $(\beta \vee \neg \ell)$ infer $(\alpha \vee \beta)$. In first-order logic, apply the most general unifier first. Derive the empty clause and the original entailment is proved.

Unification

`unify(x, y)` returns a substitution θ with $x\theta = y\theta$, or failure. The **occurs check** refuses to bind a variable to a term containing it; without it, x against $f(x)$ builds an infinite term.

B.6 Uncertainty

$$P(a | b) = \frac{P(a \wedge b)}{P(b)}$$

$$P(a | b) = \frac{P(b | a) P(a)}{P(b)} \quad (\text{Bayes' rule})$$

$$P(a) = \sum_b P(a \wedge b) \quad (\text{marginalisation})$$

Independence: $P(a \wedge b) = P(a)P(b)$. Conditional independence given c : $P(a \wedge b | c) = P(a | c)P(b | c)$ — this is what a Bayesian network encodes, and what makes it smaller than the full joint.

A Bayesian network's joint distribution factorises as

$$P(x_1, \dots, x_n) = \prod_{i=1}^n P(x_i | \text{parents}(x_i))$$

Expected utility, and the rule for acting:

$$\text{EU}(a) = \sum_s P(s | a) U(s), \quad a^* = \arg \max_a \text{EU}(a)$$

B.7 Fuzzy Logic

Membership $\mu_A(x) \in [0, 1]$. Operators:

$$\mu_{A \cap B}(x) = \min(\mu_A(x), \mu_B(x)), \quad \mu_{A \cup B}(x) = \max(\mu_A(x), \mu_B(x)), \quad \mu_{\neg A}(x) = 1 - \mu_A(x)$$

Mamdani inference: fuzzify the inputs, take the rule's firing strength as the min over its antecedents, clip the consequent at that strength, aggregate by max, then defuzzify by centroid:

$$x^* = \frac{\int x \mu(x) dx}{\int \mu(x) dx}$$

B.8 The Perceptron

Output $\hat{y} = 1$ if $\sum_i w_i x_i + b > 0$, else 0. Learning rule, applied per example:

$$w_i \leftarrow w_i + \eta (y - \hat{y}) x_i, \quad b \leftarrow b + \eta (y - \hat{y})$$

It converges in finite time if and only if the data are linearly separable. It cannot represent exclusive-or — which is where this course stops and *Machine Learning* begins.

Answers to Checkpoints

Answers to the three-question Checkpoint box in each chapter. Attempt them before reading. Where an answer requires judgement, the reasoning matters more than the conclusion.

Chapter 1 — What Artificial Intelligence Is

1. **Goal** is passed on the description alone: “minimise total project delay” is an objective, not a procedure. **Knowledge** is arguably passed too — the task list with durations and dependencies is an explicit, inspectable representation that a delivery manager could correct without touching the algorithm. **Search, adaptation** and **explanation** cannot be judged without seeing the implementation: a program with this brief might enumerate alternatives or might compute one critical path directly, and the description does not say.
2. It is succeeding in *acting humanly* — the bottom-left quadrant, the Turing test standard, where indistinguishability from a person is the whole criterion. It is failing in *acting rationally*: with no representation of the service catalogue it cannot be relied on to give the right answer, and with no explanation its answers cannot be checked. It is also failing *thinking rationally*, since there is no inference to inspect. This is exactly the Eliza pattern of Chapter 19.
3. Chapter 13 builds one. Such a system fails the **adaptation** test: its rules are elicited from experts by hand and it does not improve with experience. That is precisely the knowledge acquisition bottleneck which ended the expert-system boom — and note that it fails the one test this course does not teach, while passing the other four.

Chapter 2 — Intelligent Agents and Task Environments

1. A reasonable answer: *Performance measure* — total time pull requests spend waiting, weighted by urgency, plus a penalty for defects that reach the main branch. Note that this describes the state of the repository. Measures such as “number of reviews performed” describe the agent and would be gamed by skimming. *Environment* — the open pull requests, their size, age and author, the reviewers and their load, the build status. *Actuators* — assign a pull request to a reviewer, defer it, request changes. *Sensors* — the repository API: diffs, timestamps, build results, reviewer availability.
2. Fully observable (the coins inserted are the whole relevant state), deterministic, episodic (each sale is independent), static, discrete, and single-agent. All six are the easy case, which is why a **simple reflex agent** is sufficient — and why a vending machine is not usually described as intelligent.
3. Yes, it was rational. Rationality is maximising *expected* performance given the percept sequence available at the time, not achieving the best outcome. The agent acted on strong evidence; the outcome was unlucky. This distinction is what makes a blameless incident review possible: you judge the decision procedure against what was knowable, and you change the procedure only if the evidence was misread.

Chapter 3 — Representations and Search in Python

1. A list is mutable, and Python makes mutable built-in types unhashable, because an object whose hash could change while it sits in a set would become unfindable. Without a usable explored set the search cannot ask “have I seen this state?”. Every state is then re-expanded

once for each distinct path that reaches it, so the search explores the search *tree* rather than the state *graph* — and on a space with a cycle that tree is infinite, so the search never terminates.

2. The heap entries are tuples such as `(cost, node)`. When two costs are equal Python moves on to compare the second elements, and `Node` defines no ordering, so it raises `TypeError`. It is intermittent because it needs a tie, which most inputs do not produce. The fix is one element: push `(cost, next(counter), node)` with a monotonically increasing counter, so ties are broken by insertion order and two nodes are never compared.
3. It multiplies the state space by the number of distinct step counts at which each situation can be reached. Two searches that arrive at the same set of finished tasks after 7 and after 9 steps now hold *different* states, so the explored set never matches, nothing is pruned, and the search degenerates towards enumerating paths. The rule is in the chapter: if the goal test cannot see it, it does not belong in the state.

Chapter 4 — Problem Formulation and Uninformed Search

1. They are repeated states: the same set of finished tasks reached by different orderings. The search tree contains one node per *path*, the state graph one node per *situation*. An **explored set** — a set of states already generated, tested before pushing — removes them, and on a space containing a cycle it is also what makes the search terminate at all.
2. Because when a node is *generated*, a cheaper path to the same state may still be sitting unexpanded in the frontier; only when a node is *popped* does the frontier ordering guarantee that nothing cheaper remains. The smallest graph showing it: start S , goal G , and one intermediate A , with $S \rightarrow G$ costing 10, $S \rightarrow A$ costing 1 and $A \rightarrow G$ costing 2. Testing on generation returns the direct path at cost 10; testing on popping returns S, A, G at cost 3.
3. Because the number of nodes grows geometrically with depth, so the deepest level dominates the total: the nodes above it sum to roughly $1/(b-1)$ of it. At $b = 10$, $d = 5$ the repetition costs about 11% extra work. What it buys is memory: $O(bd)$ instead of $O(b^d)$ — 50 nodes rather than 100,000 — while keeping completeness and the shallowest-solution guarantee.

Chapter 5 — Informed Search and Heuristics

1. Yes, it is admissible: $23 \leq 28$, and the lab verifies $h(s) \leq h^*(s)$ on all 55 reachable states. A heuristic returning 30 at the root would *not* be admissible, because it exceeds the true optimal completion cost of 28. With it, A^* loses its optimality guarantee: the node on the optimal path can be buried behind a worse goal node in the frontier, and the search returns the worse plan with no indication that it has done so.
2. Because greedy best-first search orders its frontier by h alone and never consults g . Building the harness costs three days — an increase in g — while reducing the mandatory-work estimate by nothing at all, since the harness is not mandatory work. So it looks like pure delay. Any decision of the form “spend more now to spend less later” is structurally invisible to greedy search.
3. $\max(h_1, h_2)$. It is admissible, because at every state it equals one of two values that are each at most h^* ; and it dominates both by construction. The *average* is also admissible but is dominated by the maximum — it is never larger and usually smaller, so it expands at least as many nodes for the same evaluation cost. Take the maximum, never the mean.

Chapter 6 — Local Search and Optimisation

1. A **plateau** is a flat region of the landscape; a **shoulder** is a plateau with rising ground beyond it. (a) Forbidding sideways moves means halting on every shoulder, so solutions that lie just beyond flat ground are never reached. (b) Allowing them without limit means the search can circle a genuine plateau forever. The standard compromise is a cap — allow sideways moves up to a fixed count, then stop.
2. With $\Delta = -6$: at $T = 8$, $p = e^{-6/8} = e^{-0.75} \approx 0.472$, so a move six points worse is accepted almost half the time. At $T = 0.5$, $p = e^{-12} \approx 6.1 \times 10^{-6}$, which is effectively never. Early in the run the search wanders and can cross a ridge; late in the run it is hill climbing and only consolidates.
3. Crossover assumes the state decomposes into *blocks that are independently good*, so that splicing part of one parent onto part of another preserves value. The staffing objective largely satisfies this: most of a developer's contribution is her own skill score, which depends on her assignment alone. What would break it is an objective depending on *pairs* of positions — who is rostered alongside whom, or two services that must not share a host — because then a block's value depends on the block it is spliced next to.

Chapter 7 — Constraint Satisfaction

1. Because nothing in plain backtracking ever shrinks a domain. Every unassigned variable still has all four sprints available, so they all tie on the minimum-remaining-values count and the heuristic falls back on the first variable in the list — which is exactly what plain backtracking was already doing. The change that makes MRV worth having is *propagation*: forward checking removes values from neighbours after each assignment, and from that moment the domains have different sizes and MRV has something to compare. Measured: 3,784 checks with MRV alone, 415 with MRV and forward checking together.
2. Both are true because AC-3 works only on *binary* constraints. The precedence and same-developer constraints are binary, and AC-3 prunes them thoroughly — every one of the twelve domains loses at least one sprint. But the constraint that actually binds this problem is capacity: 45 points into 48, an n -ary constraint over all twelve variables at once, which no arc can express and AC-3 never examines. So the preprocessing removes values that the search was not going to waste much time on anyway, at a cost of 224 extra consistency checks and a saving of zero nodes.
3. Either: instrument the propagation to record which constraint emptied a domain most often, and report that one; or re-run the solver with each constraint (or constraint class) relaxed in turn and report which relaxations make the problem solvable. Here you would expect *capacity* to bind, and the lab confirms it — raising the cap from 12 to 13 takes the solution count from 14 into the hundreds, while the precedence constraints have plenty of slack.

Chapter 8 — Adversarial Search and Game Playing

1. Because alpha-beta only ever declines to evaluate leaves that *provably* cannot change the answer: a cutoff happens exactly when the player above already has a better alternative and will therefore never choose the branch being examined. The two searches must agree on every instance, not just this one — pruning is exact, and what it approximates is nothing at all.
2. First, the right-hand MIN node has already found a child worth 2, so it can hold us to at most 2 whatever its remaining child contains. Second, the root has already secured 5 from the left subtree. Since $2 < 5$ we will never play into this branch, so the value of its unexamined child cannot affect the root — and alpha-beta never learns it.

3. The probes are themselves evaluations, and on a shallow tree they cost more than the cutoffs they buy: the lab measures 506 leaf evaluations with a depth-1 probe ordering against 75 with natural ordering. For ordering to pay, the tree must be deep enough that a better ordering saves an exponential amount, and the probe results must be cached — a transposition table — so that the work done to order one search is reused by the next. That is the situation in a chess engine at twenty ply, and not the situation at five.

Chapter 9 — Planning

1. **test A** has **staging free** in its *delete* list, so testing the first component takes the staging environment away. But **test B** has **staging free** in its *preconditions*, so the second component cannot be tested until something restores it, and **release staging** is the only operator that does. It is there purely to undo interference caused by an earlier step. Removing **staging free** from **test**'s delete list makes it disappear: actions can then no longer interfere, and the problem collapses into the topological sort of Chapter 4.
2. Because A^* saves work only by refusing to expand nodes with $f(n) \geq C^*$, and in this domain almost every reachable state lies on *some* optimal plan — nearly every action is necessary and their order is largely free — so f is close to C^* nearly everywhere and there is almost nothing to rule out. The same heuristic would pay well on a planning problem with many *dead ends or wasted actions*: irreversible mistakes, resources that can be spent on the wrong thing, or a much larger operator set of which only a few are relevant. Heuristics pay in proportion to how much of the state space is wasted effort.
3. Completeness. Breadth-first progression search will find a plan whenever one exists; means-ends analysis takes the goals in some order and can destroy, while achieving an early goal, a condition a later goal needs. The two-operator example of Figure 9.2 is solvable, and means-ends analysis reports failure on it when the goals are taken in one of the two possible orders. (Breadth-first search also guarantees a *shortest* plan; means-ends analysis found optimal plans here but is not obliged to.)

Chapter 10 — Knowledge Representation

1. The walk starts at **Release 4.7**, finds no local **regression-pass**, moves to **Hotfix**, finds **smoke**, and *stops*. **Release** is never consulted, so its **full** never competes. “Most specific wins” does not have to be stated as a rule because it is a consequence of the search order: the walk goes from specific to general and halts at the first hit.
2. **Non-monotonicity** — adding knowledge removed a conclusion. Classical logic does not have it: entailment is defined as truth in every model of the knowledge base, and adding a sentence can only rule models out, so the set of entailed conclusions can only grow. That is exactly the trade Chapter 11 makes — it gains soundness and loses the ability to say “normally”.
3. A semantic network with defaults can express *normally*: “releases go out on Thursdays” with hotfixes excepted, without the exception making the knowledge base inconsistent. Written as a universally quantified sentence that claim is simply false, and adding one hotfix makes the knowledge base contradictory. What logic can do and the network cannot is *derive guaranteed consequences*: a sound procedure over logic gives conclusions that hold in every model, whereas an inheritance walk gives whatever the nearest ancestor happened to say, with no guarantee it is consistent with anything else in the network.

Chapter 11 — Propositional Logic and Inference

1. $KB \models \alpha$ is about *meaning*: α is true in every model in which the knowledge base is true, and no algorithm is mentioned. $KB \vdash \alpha$ is about a *procedure*: that procedure can produce α . A procedure that derives every sentence is *complete* — it certainly derives everything entailed — but wildly *unsound*, since it derives a great many things that are not entailed. Soundness is the property it violates, and it is the one that matters: completeness alone is worthless.
2. F1–F3 force *Reviewed*, *Tested* and *SignedOff*; R1 then forces *Staged*; R2 then forces *Deploy*. R3 says $\text{Blocker} \Rightarrow \neg \text{Deploy}$, and since *Deploy* is true, *Blocker* must be false or R3 is violated. Exactly one model survives, so both *Deploy* and $\neg \text{Blocker}$ are entailed. It is *not* a bug in the inference procedure — the procedure drew a correct consequence of what it was given. It is a bug in the *axioms*: the team meant “deployment is permitted provided there is no blocker”, and R2 as written does not say that. Adding $\neg \text{Blocker}$ to R2’s antecedent makes the entailment disappear, which the lab demonstrates.
3. First, resolution produces an *auditable proof* — a numbered list of clauses each derived from two earlier ones, which a person can check line by line and an auditor will accept. A model count proves nothing a human can read. Second, and decisively, resolution *lifts to first-order logic* (Chapter 12), where the domain may be infinite, there are infinitely many models, and enumerating them is not slow but impossible. DPLL’s speed is confined to the propositional case.

Chapter 12 — First-Order Logic and Resolution

1. $\forall x \text{ service}(x) \Rightarrow \exists y \text{ owns}(y, x)$. Eliminating $\Rightarrow$ gives $\forall x \neg \text{service}(x) \vee \exists y \text{ owns}(y, x)$; skolemising y and dropping the universal gives $\neg \text{service}(x) \vee \text{owns}(\text{owner}(x), x)$. The existential variable lies *inside* the universal quantifier over x , so the owner depends on which service we are talking about. Skolemising to a constant **Sk** would give $\neg \text{service}(x) \vee \text{owns}(\text{Sk}, x)$ — “one particular person owns every service” — which is a different and much stronger claim.
2. n^2 sentences: one for each ordered pair of services, since the rule relates a depender to a dependee. Adding one service means writing $2n$ more by hand and getting none of them wrong. In first-order logic the rule is a single sentence and adding a service means adding a *fact*; the rule is untouched. That is the difference between a knowledge base a team can maintain and one it cannot.
3. Because first-order entailment is only **semi-decidable**. If the conclusion follows, resolution will eventually derive the empty clause; if it does not, the procedure may search forever, and it cannot in general report that no proof exists. A countermodel would require exhibiting an interpretation in which the knowledge base holds and the query fails, and first-order interpretations can have infinite domains — there are infinitely many to consider and no way to enumerate them. Propositional logic has finitely many models, which is exactly why Chapter 11 could hand one back.

Chapter 13 — Rule-Based Expert Systems

1. **wake_budget**: **available** for T-4471 and **spent** for T-4473. It is *not* a fact about the ticket — it is a fact about the on-call engineer and what has already been asked of her tonight. Both tickets reach **escalate = yes**; they differ only at R14 against R15, which consult that fact. This is precisely the internal state Chapter 2 showed a model-based reflex agent must maintain because no percept reports it, and here it costs one entry in working memory and one pair of rules.
2. The nine is right for a monitoring dashboard or an alerting pipeline, where a newly arrived fact should trigger *everything* that follows from it — notify finance, attach the payments runbook,

set the page severity — without anybody having to ask for each one. The six is right when an engineer or an auditor has a single question (“why was this escalated?”), because forward chaining would derive a great deal that is irrelevant to it and would still not produce a proof tree.

3. They get the backward-chaining explanation: `escalate = no` by R11 because `needs_owner = no` by R7, because the priority is P2 by R3 (major severity, standard tier) and there was a workaround. Every step names the rule and the fact it used, so the auditor can check each line and, if they disagree, point at the specific rule that is wrong. A probability satisfies none of this: it cannot be checked, it identifies nothing to correct, and it does not say which of the ticket’s properties mattered. See also Chapter 21, where this is argued as accountability rather than convenience.

Chapter 14 — Uncertainty and Decision Making

1. Of a thousand releases, a hundred are defective and the test catches ninety of them — but nine hundred are sound and the test cries wolf on a fifth of those, which is a hundred and eighty false alarms. So of the two hundred and seventy alarms, only ninety are real. The test is good; there are simply far more sound releases for it to be wrong about, and that is the base rate doing the work.
2. The lesson is that *a change of belief is not a change of decision*. The recommendation depends only on which side of the flip point p^* the probability falls, and $1/3$ is below $10/29 \approx 0.345$. What an engineer should report is the *action* and the *margin*: “still ship, but only by 1.67 utility units, which is close enough that better evidence is now worth having” — which is both the decision and the case for the next step.
3. Because the value of information is proportional to how likely it is to *change the action*, not to how interesting it is. Before the test the decision was comfortable — ship, by 35 units — so certainty could rarely have flipped it. After the alarm the two options differ by under two units, so the same certainty now flips the decision often. Information is worth most exactly where the decision is hardest, and the function peaks at the flip point itself.

Chapter 15 — Bayesian Networks

1. It asserts that *each variable is conditionally independent of its non-descendants given its parents*. That licenses the chain rule, so the joint factorises into one small table per node. An assertion in Figure 15.1 that could easily be false: there is no edge between *Rushed* and *Junior*, which claims they are independent. In many organisations rushed work is precisely what gets handed to the least experienced developer, and if that is true here the network is wrong and will give confidently wrong answers.
2. Both are consequences of the arithmetic of a common effect. *Defect* needs explaining, and a rushed specification is one of the available explanations, so its probability rises. Learning that a junior developer was involved supplies a *different* sufficient explanation, so less explanatory work is left over and the probability of the first cause falls. A forward-chaining rule base cannot reproduce this because it would need a rule whose conclusion becomes *weaker* when a condition is added — and Chapter 13’s specificity strategy would do the opposite, preferring the rule with more conditions and thus strengthening it.
3. It becomes useless when the evidence is rare: it keeps only the fraction of samples that happen to match, so evidence of probability 0.01 throws away 99% of the work, and diagnostic evidence is usually rare because that is what makes it worth reasoning about. Likelihood weighting does not sample the evidence variables at all — it fixes them to their observed values and weights

each sample by the probability of the evidence given what was sampled — so every sample contributes.

Chapter 16 — Fuzzy Logic

1. Because a membership is a fact about the *word*, not about our knowledge of the project. We know exactly how big the project is: seventy person-days. The number 0.4 says that seventy days is “high effort” to degree 0.4, and no further measurement of the project can change that. What *would* change it is redefining the membership function — that is, agreeing with the team on what “high” should mean, which is a conversation about vocabulary rather than about evidence.
2. The membership functions form a **fuzzy partition**: overlapping triangles whose memberships sum to one everywhere, so at most two words are active for each input and therefore at most $2 \times 2 = 4$ rules fire. If the functions do not overlap, only one word is active per input, only one rule fires, and there is no interpolation at all — the controller degenerates into a lookup table with the smoothness that motivated the whole approach thrown away.
3. Centroid defuzzification returns the centre of area of the aggregated shape. Even when only the lowest output set is active, that set is a triangle centred at zero with area extending to its right, so its centre of area is strictly positive; the same argument at the top end keeps it below the maximum. The consequence for reporting is that the number must never be presented as a percentage or as an absolute risk. It is an ordinal score, useful for comparing projects with each other, and a board told “83% risk” has been misled by an artefact of the defuzzifier.

Chapter 17 — Learning from Examples

1. Because the rule learns *only from mistakes*: when the prediction is correct, $y - \hat{y} = 0$ and every weight is unchanged. Ten of the twelve examples were already classified correctly when they were seen, so they contributed nothing. The second update changed the meaning of the rule: it drove the weight on clarity negative, so that higher clarity now argues *against* lateness. Nobody told the perceptron that; it followed from a single misclassified example in the second pass.
2. Either the data are separable and it simply needs more epochs, or no line in the hypothesis class fits the data at all — and the two traces look identical, because in both cases the weights are still moving when the budget expires. The cheapest experiment that distinguishes them is a brute force sweep over the hypothesis class: on a small problem, enumerate a grid of lines and record the best accuracy any of them achieves. On the exclusive-or arrangement that answer is 75%, which settles the question; no learning rate or initialisation can beat a bound over the whole class.
3. A single perceptron cannot represent exclusive-or, because it computes a linear threshold and no straight line separates those four points. The 1969 conclusion was that neural network research was a dead end, and funding largely stopped for fifteen years. It was wrong because the limitation applies to a *single layer*: a network with one hidden layer represents exclusive-or easily, and what was actually missing was a way to *train* such a network, supplied by backpropagation in the mid-1980s. A correct result about the simplest case was read as a verdict on the approach.

Chapter 18 — Natural Language Processing

1. Reading 1 attaches **after the timeout** to the verb phrase: *restart the server, and do it after the timeout occurs*. Reading 2 attaches it to the noun phrase: *restart the server that handles timeouts*. They are different instructions. A routing system should not choose — it should recognise that the subject line supports both and ask the reporter, which is a capability the grammar provides for free and which a system returning one confident parse throws away.
2. That fluency is not evidence of understanding. Eliza had no representation of the conversation, no model of the person and no goal, and it still produced responses people found so apt that they disclosed things to it they would not tell a colleague — including people who had read its source. So your impression of a transcript tells you about your own response, not about the system, and the instrument to use instead is Chapter 1’s checklist: is there a goal, a search, an inspectable representation, adaptation, an explanation?
3. Sparsity is the problem that most possible n -grams never appear in any finite corpus, so unseen ones receive probability zero. It is an estimation problem, and smoothing solves it by reserving probability mass for the unseen. The fixed window is a *representational* limit: in *the server that the engineer who wrote the runbook maintained was restarted*, the subject of *was restarted* is nine words back. No n -gram with a practical n can span that, and raising n makes sparsity exponentially worse — so the two limitations pull against each other and no corpus size resolves the second.

Chapter 19 — Classic AI Systems and How to Analyse Them

1. GPS is means–ends analysis (Chapter 9); Eliza is pattern substitution (Chapter 18); Student is pattern matching into equations, which is Chapter 12’s unification restricted to strings; Maccsma is term rewriting, which is Chapter 13’s forward chaining applied to expression trees rather than to facts.
2. Because GPS’s failure is *legible*: it stops and says it has no plan, so a person knows to look. Eliza always has a reply, so nothing distinguishes a case it handled well from one it did not handle at all, and the user has no signal to act on. The consequence for evaluating a modern language system is direct: it too always produces output, so the fluency of a response carries no information about whether it is grounded, and evaluation must come from an external check — a test suite, a type checker, a retrieved source — rather than from reading the answer.
3. A person, writing it down by hand. It predicts that the system will work well in the region somebody took the trouble to encode and will need a person for every extension — so as the business changes, the system’s coverage falls behind at whatever rate knowledge engineers can be hired and briefed. That is the knowledge acquisition bottleneck, and it is the reason all four of these systems have the biographies they do.

Chapter 20 — Recent Trends and Applications

1. Because Chapter 13’s chain is a *record* of the inference that actually occurred: each line names the rule that fired and the facts it matched, and the conclusion was produced by exactly those steps. A language model’s explanation is generated by the same next-token process as the answer itself — it is text that resembles a derivation, produced independently of whatever computation led to the answer. In Chapter 11’s terms, it may display something shaped like $\vdash$ while carrying no guarantee of any relation to $\models$; a plausible-looking derivation is not a derivation.
2. The other five are the loop itself, the tools, the step budget, the termination test and the state the agent keeps between steps. Removing the *policy’s handling of a failed tool call* causes the

commonest deployed failure: a tool returns nothing, no rule notices, and the **None** travels until it reaches arithmetic and crashes — or, with a policy that cannot terminate, the loop spins until the step budget stops it. Both failures live in the engineering around the model, not in the model.

3. First, *what did the demonstration exclude?* — every impressive demonstration is a microworld until shown otherwise, and the answer tells you what the vendor chose not to show. Second, *how would anyone find out when it is wrong?* — the failure-mode column. If the answer is that a human notices, ask how they notice when the output is fluent; if it is a test or a type checker, the system is safe in proportion to that checker’s coverage.

Chapter 21 — Responsible AI and Its Limits

1. Through a **proxy: workaround** is recorded for about 80% of one team’s tickets and 15% of the other’s, because the first team has mature runbooks to cite. R3 makes those tickets P2 and R7 then decides a P2 with a workaround needs no owner, so that team’s tickets are systematically not escalated. Deleting an attribute would not have prevented it, and could not have: the team name was never an input. The correlation lives in the world rather than in a column, which is why *fairness through unawareness* is the one approach that reliably fails.
2. It established *what* produced the gap — a difference in documentation practice being read by the rules as a difference in operational need — and that no rule is individually wrong. It explicitly did not establish whether the gap is acceptable. If the first team really does need fewer escalations because its runbooks work, the gap is the system functioning correctly; if the second team is being under-served because nobody has written its runbooks, it is not. That is a management decision, and the audit’s contribution is to put it in front of somebody who can make it, with evidence attached, instead of leaving it to be made silently two hundred times a week.
3. The alternative explanation is **automation bias**: the reviewer has stopped reviewing and is deferring to the system, which produces a near-zero override rate whether the system is right or not. The measurement that distinguishes them is to sample decisions for genuine independent review — have a second person decide without seeing the recommendation — and compare. If the independent decisions disagree materially more often than the override rate suggests, the low rate was deference rather than agreement.

Chapter 22 — The Artificial Intelligence Project

1. It fails the second test: the rules do not exist and would have to be learned from historical tickets, which makes it a *Machine Learning* project rather than one this course has the machinery for. The advice is to keep the domain and change the question to one whose rules can be written down — for example *decide which open tickets to escalate tonight given a stated escalation policy* (Chapter 13), or *schedule this sprint’s stories without violating capacity or precedence* (Chapter 7). Both are one sentence, both have rules an operations lead can state in an afternoon, and both have an embarrassing baseline.
2. Because a measure chosen after the results are in will be chosen — honestly and without anyone noticing — to be one the system does well on. The failure mode is a report that demonstrates rather than evaluates: it shows the system succeeding at whatever it happens to be good at, and provides no evidence about the question that was originally asked. Writing the measure and the failure condition down in advance is the only practical defence, and changing them later is acceptable provided the change and its reason are reported.
3. The baseline it was compared against, the measure it was compared on with evidence that the measure was fixed in advance, and the size of the margin relative to the variability — a small

margin on one instance with a seeded run is not a result. Ideally also the ablation showing which component of the system produced the margin, since without it the claim is about the whole program rather than about the method being taught.

Glossary

Every key term of the course, with the chapter that introduces it.

Term	Meaning	Ch.
Admissible	A heuristic that never overestimates the remaining cost. Makes A^* optimal	5
Agent	A program that perceives an environment and acts on it	1
Agent function	The mapping from every percept sequence to an action. Usually too large to tabulate	2
Agent program	A bounded implementation of an agent function	2
Alpha-beta pruning	Discarding branches of a game tree that cannot affect the minimax value	8
Arc consistency	Every value of a variable has a consistent partner in a neighbouring variable. Enforced by AC-3	7
A^*	Best-first search ordered by $f(n) = g(n) + h(n)$	5
Backtracking	Depth-first search that assigns one variable at a time and undoes the assignment on failure	7
Backward chaining	Inference from a goal towards the facts that would establish it	13
Bayesian network	A directed acyclic graph whose edges encode conditional independence, with a probability table per node	15
Bayes' rule	$P(a b) = P(b a)P(a)/P(b)$	14
Branching factor	The average number of successors of a state, written b	4
Brittleness	Failing catastrophically rather than gracefully just outside the region of competence	21
Clause	A disjunction of literals; the unit resolution operates on	11
Combinatorial explosion	Exponential growth of a search space with depth. What ended the first wave of AI	1, 4
Completeness (search)	The guarantee of finding a solution if one exists	4
Completeness (logic)	Everything entailed can be derived by the procedure	11
Conflict resolution	Choosing which of several applicable rules to fire	13
Conjunctive normal form	A conjunction of disjunctions of literals	11
Consistent	A heuristic with $h(n) \leq c(n, n') + h(n')$. Needed for graph search	5
Constraint satisfaction problem	Variables, domains and constraints; a solution assigns every variable without violating any constraint	7
Defuzzification	Converting an aggregated fuzzy output to a single number, usually by centroid	16
Degree of belief	Probability read as a measure of confidence rather than of frequency	14

Term	Meaning	Ch.
Depth-first search	Expanding the deepest unexpanded node. Cheap in memory, neither complete nor optimal	4
Dominance	$h_2 \geq h_1$ everywhere, so A^* expands no more nodes with h_2	5
DPLL	Model checking with unit propagation and pure-literal elimination	11
Eliza effect	Attributing understanding to a system that is matching patterns	19
Entailment	$KB \models \alpha$: α is true in every model of the knowledge base	11
Evaluation function	An estimate of a game position's value, used when search is cut off	8
Expected utility	$\sum_s P(s a)U(s)$; the quantity a rational agent maximises	14
Explanation	The chain of facts and rules that produced a conclusion	13, 21
Explored set	States already generated, held so they are not expanded twice	4
First-order logic	Logic with objects, relations and quantifiers	12
Forward chaining	Inference from the known facts towards whatever follows	13
Forward checking	Removing values inconsistent with the assignment just made	7
Frame	A named bundle of slots with default values	10
Frontier	Nodes generated but not yet expanded	3, 4
Fuzzy set	A set with graded membership $\mu_A(x) \in [0, 1]$	16
Genetic algorithm	Local search over a population, with selection, crossover and mutation	6
Goal test	A predicate identifying goal states; not necessarily a single state	3
Greedy best-first search	Ordering the frontier by h alone. Fast, not optimal	5
Heuristic	An estimate $h(n)$ of the cost remaining from n to a goal	5
Hill climbing	Moving to the best neighbour until none is better	6
Horizon effect	Pushing an inevitable loss just beyond the search cutoff	8
Inference engine	The component that applies rules to working memory	13
Iterative deepening	Repeated depth-limited search. Breadth-first guarantees at depth-first memory cost	4
Knowledge acquisition bottleneck	Rules must be elicited from experts one at a time. What ended the expert-system boom	1, 13
Linear separability	Whether a single straight boundary can separate two classes. The perceptron's limit	17
Local maximum	A state with no better neighbour that is not the best state	6
Means-ends analysis	Choosing an operator that reduces a difference between the current state and the goal	9
Minimax	The value of a game position under optimal play by both sides	8

Term	Meaning	Ch.
Minimum remaining values	Assign next the variable with the fewest legal values. Fails fast	7
Model	An assignment of truth values under which a sentence can be evaluated	11
Node	A state plus parent, action and cost. Not the same as a state	3
Occurs check	Refusing to bind a variable to a term containing it	12
Ontology	A shared, formal vocabulary of the types in a domain and their relations	10
PEAS	Performance measure, environment, actuators, sensors	2
Percept	What the agent senses at one instant	1, 2
Perceptron	A single linear threshold unit, trained by the perceptron rule	17
Plateau	A flat region of the search landscape; a shoulder if it leads upward	6
Production rule	A condition–action pair in a rule base	13
Rationality	Maximising expected performance given the percepts so far. Not omniscience, not perfection	2
Relaxation	Dropping a constraint to obtain an admissible heuristic cheaply	5
Resolution	From $(\alpha \vee \ell)$ and $(\beta \vee \neg \ell)$ infer $(\alpha \vee \beta)$	11, 12
Ridge	A run of local maxima along which no single move improves	6
Semantic network	A graph of concepts and labelled relations, queried by inheritance	10
Simulated annealing	Accepting worsening moves with probability $\exp(\Delta/T)$, with T falling to zero	6
Skolemisation	Replacing an existentially quantified variable with a function of the universals enclosing it	12
Soundness	Everything derived is entailed	11
State	A value describing a situation. Immutable and hashable	3
State space	The graph of states and the actions between them	4
STRIPS	An action representation: preconditions, add list, delete list	9
Unification	Finding a substitution that makes two expressions identical	12
Uniform-cost search	Expanding the cheapest path first. Optimal; Dijkstra’s algorithm	4
Utility	A number expressing how much an outcome is worth	14
Value of perfect information	The most an agent should pay to learn something before deciding	14
Working memory	The facts currently believed by a production system	13

Resources

E.1 The Set Texts

These are the books HEC names for this course. The first is the one to buy.

- S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd edition, Prentice Hall, 2010. Named by both the HEC 2017 and HEC 2023 outlines. Chapters 2–7 and 13–14 of Russell and Norvig cover most of this handout; it is more thorough and much longer, and it is the right place to go when a chapter here leaves you wanting the general case.
- G. F. Luger and W. A. Stubblefield, *AI Algorithms, Data Structures, and Idioms in Prolog, Lisp and Java*, Pearson Addison-Wesley, 2009. Named by HEC 2017. Useful precisely because it implements the algorithms rather than describing them.
- R. O. Duda, P. E. Hart and D. G. Stork, *Pattern Classification*, Wiley, 2001. Named by HEC 2017. It is a pattern-recognition text and belongs to *Machine Learning* rather than to this course; read it after that one.
- D. Poole and A. Mackworth, *Artificial Intelligence: Foundations of Computational Agents*, 2nd edition, Cambridge, 2017. Named by the HEC 2023 outline for *Knowledge Representation and Reasoning*. Free online, and the clearest available treatment of the agent framing this course adopts.

E.2 Where Each Chapter Sits in Russell and Norvig

This handout	Russell & Norvig 3rd ed.	Note
1 What AI Is	Ch. 1	The history here is compressed into five waves
2 Agents and PEAS	Ch. 2	Very close; read both
3 Representations in Python	—	No equivalent; this chapter is specific to this course
4 Uninformed search	Ch. 3.1–3.4	
5 Informed search	Ch. 3.5–3.6	The optimality proof is theirs
6 Local search	Ch. 4.1–4.2	
7 Constraint satisfaction	Ch. 6	
8 Adversarial search	Ch. 5	
9 Planning	Ch. 10	They develop partial-order planning, which this course only names
10 Knowledge representation	Ch. 12	Their treatment is heavier on ontologies
11 Propositional logic	Ch. 7	
12 First-order logic	Ch. 8–9	
13 Expert systems	Ch. 9.3–9.4	Rule systems are scattered in R&N; Luger is better here

This handout	Russell & Norvig 3rd ed.	Note
14 Uncertainty and decisions	Ch. 13, 16	
15 Bayesian networks	Ch. 14	
16 Fuzzy logic	Ch. 14.7 (brief)	R&N are sceptical of fuzzy logic and say so; read their argument
17 Learning	Ch. 18.1–18.2	This course stops where they accelerate
18 Natural language processing	Ch. 22–23	Their treatment is largely statistical; this chapter is the symbolic half
19 Classic systems	scattered	The primary sources below are better
20–21 Trends, responsibility	Ch. 26–27	The 4th edition (2020) is substantially better here

E.3 The Primary Sources Worth Reading

Short, readable and historically decisive. Chapter 19 reconstructs the first four.

- A. M. Turing, “Computing Machinery and Intelligence”, *Mind*, 1950. The paper that proposed the test, and answered nine objections to it that are still the nine objections people raise.
- A. Newell and H. A. Simon, “GPS, a Program that Simulates Human Thought”, 1961. Means–ends analysis, and the beginning of the cognitive modelling tradition.
- J. Weizenbaum, “ELIZA — A Computer Program for the Study of Natural Language Communication between Man and Machine”, *CACM*, 1966. Read it with Weizenbaum’s later dismay at how it was received.
- M. Minsky and S. Papert, *Perceptrons*, MIT Press, 1969. The exclusive-or result of Chapter 17, and the book usually blamed for the first neural winter.
- E. H. Shortliffe, *Computer-Based Medical Consultations: MYCIN*, 1976. The expert system that established explanation as a requirement rather than a feature.
- J. Pearl, *Probabilistic Reasoning in Intelligent Systems*, Morgan Kaufmann, 1988. Bayesian networks, from the person who made them computable.
- J. Lighthill, “Artificial Intelligence: A General Survey”, 1973. The report that caused the first winter. Worth reading because its criticism was largely correct.

E.4 Departmental Materials

- Machine Learning Fall 2026/ — the course this one hands learning to. Its Chapter 13 continues Chapter 17 exactly where this book stops.
- Data Science Fall 2026/ — the end of the three-course sequence.
- HEC AI-DS Curriculum Map/ — the three HEC curriculum booklets and the department’s analysis of them, including the verbatim outlines this course is written against.
- AI_ML_DS_Fall2026_ProgressionAtAGlance.pdf — the one-page map of all three courses and who owns each shared topic. Appendix F states the same boundaries in prose.
- Summer_2026/AI_Handout.pdf — the eight-week revision summary. Useful for examination practice, not a substitute for this volume.
- Summer_2026/question_papers/ — past papers and a Moodle question bank for this course.

E.5 Tools

Nothing beyond Python 3.10 and NumPy is required, and that is deliberate (see the Course Specification). If you want to go further after the course:

- **SWI-Prolog** — the natural home of Chapters 11–13. Writing Chapter 13’s rule engine in Prolog takes about fifteen lines, and seeing that is worth an afternoon.
- **python-constraint** or **OR-Tools** — production constraint solvers, for after you have written the one in Chapter 7 yourself.
- **pgmpy** — Bayesian networks in Python, for after Chapter 15.
- **PDDL** and a planner such as **Fast Downward** — the modern descendant of the STRIPS representation in Chapter 9.

The Boundary with the Neighbouring Courses

Artificial Intelligence is the gateway of this cluster: it is the published prerequisite for *Machine Learning*, *Programming for Artificial Intelligence*, *Knowledge Representation and Reasoning* and *Introduction to Data Science*. That makes its boundaries unusually consequential — a topic this course teaches badly is a topic four other courses inherit badly, and a topic it teaches twice is a week nobody gets back.

This appendix states each boundary explicitly: what this course teaches, what it deliberately does not, and which course picks it up.

F.1 Why This Course Is Symbolic

The decision that shapes everything else: *this course teaches the agent that is told the rules*. It is given the tasks and their dependencies, the constraints, the operators, the facts and the probabilities, and its job is to work out what to do or what follows. It is not given examples and asked to induce a rule — that is the next course.

Three reasons, none of them nostalgia.

1. **This is where guarantees live.** A constraint solver can prove that no engineer is double-booked (Chapter 7). No learned model can prove anything of the kind, and for a class of requirements — safety, regulation, contractual obligation — a proof is what is actually required.
2. **This is where explanations live.** A rule engine prints the chain of facts and inferences that produced its conclusion (Chapter 13). That chain is the artefact an auditor accepts, and Chapter 21 argues that it is an accountability instrument rather than a convenience.
3. **This is what you have before you have data.** Every project begins with no examples and a domain expert who knows the rules. Symbolic methods are the ones that work on day one.

Semester 3 is also simply too early for the statistics that machine learning needs, and the HEC 2023 prerequisite for this course — Object Oriented Programming — confirms it.

F.2 Machine Learning (Semester 6)

Machine Learning has *Artificial Intelligence* as its published HEC 2023 prerequisite. It owns learning entirely.

Topic	Here	There
Learning paradigms	Chapter 17: named, one delivery example each, one figure	Chapters 1–4: the formal setting, hypothesis spaces, inductive bias
Neural networks	Chapter 17: the perceptron, the learning rule by hand, and the exclusive-or limit. The course stops there	Chapter 13: multi-layer networks, the delta rule, backpropagation, activation functions

Topic	Here	There
Markov decision processes, reinforcement learning	Not taught. The agent–reward loop appears once as a figure in Chapter 17	Chapters 18–20: hidden Markov models, MDPs, Monte Carlo, temporal difference, Q-learning
Naive Bayes	Not taught. Chapter 15 does <i>inference</i> in a network whose probabilities are given	Chapter 10: <i>learning</i> those probabilities from data, with smoothing
Evaluation and generalisation	Not taught at all	Chapters 7–8: the hinge of that course



The clean way to state this boundary in one sentence: *this course is given the probabilities and computes with them; Machine Learning estimates the probabilities from data.* Chapter 15 and Machine Learning’s Chapter 10 look superficially similar and are doing opposite halves of the job.

F.3 Knowledge Representation and Reasoning (elective)

The boundary most easily overlooked, because the overlap is largest. *Knowledge Representation and Reasoning* is an HEC 2023 elective, 3 (3-0), with *Artificial Intelligence* as its prerequisite. Its published outline includes propositional logic, first-order logic, Horn clauses, description logic, reasoning using description logic, forward and backward chaining, semantic networks, ontologies and ontology languages, logical agents, planning, rule-based knowledge representation, reasoning under uncertainty, Bayesian networks, fuzzy logic, Markov models, commonsense reasoning and explainable AI.

Almost all of Parts IV and V of this handout appear in that list. That is not an error in either document: this course teaches those topics *once, at first pass*, because inference cannot be taught without a representation to reason over, and because the HEC 2017 outline for *Artificial Intelligence* names propositional logic, first-order logic, uncertainty and fuzzy logic directly.

Topic	Here (first pass)	There (in depth)
Propositional and first-order logic	Chapter 11, Chapter 12: semantics, CNF, DPLL, unification, resolution refutation	Completeness proofs, Horn clauses as a fragment, logical agents
Description logic	Not taught	The core of that course: concept languages, subsumption, reasoning over terminologies
Ontologies	Chapter 10: what an ontology is and what it buys, in one section	Ontology languages, engineering methodologies, reasoning over them
Chaining and rule bases	Chapter 13: a working engine, forward and backward, with explanation	Rule-based knowledge representation as a formalism

Topic	Here (first pass)	There (in depth)
Bayesian networks	Chapter 15: inference in a given network	Network semantics as a knowledge representation
Commonsense reasoning, explainable AI	Not taught. Chapter 21 treats explanation as an accountability artefact, not as a formalism	Both, as formalisms

F.4 Data Science (Semester 7) and the 2025 Arrivals

Introduction to Data Science also lists *Artificial Intelligence* as its HEC 2023 prerequisite, and the departmental *Data Science Fall 2026* handout sits at the end of the sequence.

Topic	Here	There
Natural language processing	Chapter 18: the <i>symbolic</i> half — grammars, chart parsing, pattern substitution, n-grams	Data Science Chapter 20: the neural half — embeddings, transformers
Generative and agentic AI	Chapter 20: named as trends, with one offline tool-using agent built from Chapter 13's rule engine	Data Science Chapters 21–22
Deep learning	Not taught	Data Science Chapters 17–18
Ethics	Chapter 21: <i>can this system's reasoning be trusted and audited?</i>	Data Science Chapter 28: privacy, security and duty to the data subject. Machine Learning Chapter 21: <i>is the learned model fair?</i>

Ethics is the one place where three courses legitimately own the same word. Each asks a different question, and the three questions are not interchangeable; the progression one-pager distributed with this handout prints all three side by side.

F.5 Programming for Artificial Intelligence

This HEC 2023 course also has *Artificial Intelligence* as its prerequisite and covers AI programming languages, production-quality code, and the numerical Python stack (NumPy, pandas, Matplotlib). Chapter 3 here is deliberately narrow by comparison: it fixes the four data structures this course's algorithms need and nothing else. It is not a Python course and does not attempt to be one.

F.6 The Summer 2026 Handout

`Summer_2026/AI_Handout.pdf` is a 24-page overview written for an eight-week session. It covers the same ground at a fraction of the depth and omits, among other things, arc consistency, resolution refutation, variable elimination, Mamdani inference and the entire analysis framework of Chapter 19. It is a revision summary and a source of examination practice; it is not a substitute for this volume, and a student who has read only it has not met the HEC outline.

⚠ If you are planning the degree

Two failure modes to watch for, both observed in practice.

Teaching machine learning here “because students expect it”. It displaces the symbolic material, which no other core course covers, and it duplicates a course three semesters later that will teach it properly with the statistics to support it. The students who enjoyed the taste arrive at *Machine Learning* believing they have already done it.

Assuming the elective will cover the logic. *Knowledge Representation and Reasoning* is an *elective*. Most students will never take it. If this course does not teach propositional and first-order inference, most graduates of the programme will never meet them — and both HEC outlines for *Artificial Intelligence* require them.

Sixteen-Week Teaching Schedule

A mapping of the 22 chapters onto a sixteen-week semester of two lecture hours and three laboratory hours a week — the HEC 3 (2–3) pattern — with assessment points. Adjust to your own calendar; the dependencies are what matter.

G.1 The Schedule

Wk	Lecture topic	Chapters	Lab	Assessment
1	What AI is; intelligent agents and task environments	1, 2	Is it AI? a five-point audit; a reflex agent on a ticket queue	—
2	Representations and search in Python	3	The frontier and the graph: the course toolkit	—
3	Problem formulation and uninformed search	4	Task ordering four ways	Quiz 1 (Ch 1–3)
4	Informed search and heuristics	5	A* over sprint states, two heuristics	Assignment 1 set
5	Local search and optimisation	6	Staffing by annealing and by evolution	—
6	Constraint satisfaction	7	Sprint scheduling as a CSP	Assignment 1 due ; Quiz 2 (Ch 4–6)
7	Adversarial search and game playing	8	Minimax and alpha–beta on a slot negotiation	—
8	Planning: STRIPS and means–ends analysis	9	A release plan from operators	Midterm (Ch 1–8)
9	Knowledge representation	10	A semantic network that answers queries	—
10	Propositional logic and inference	11	Three ways to decide a release gate	—
11	First-order logic and resolution; the project brief	12, 22	Unification and a resolution proof	Project set , proposal due; Assignment 2 set
12	Rule-based expert systems	13	A twenty-rule escalation shell that explains itself	Assignment 2 due
13	Uncertainty and decision making; Bayesian networks	14, 15	Go/no-go under uncertainty; diagnosing a defect cause	Quiz 3 (Ch 9–13)

Wk	Lecture topic	Chapters	Lab	Assessment
14	Fuzzy logic; learning from examples	16, 17	Risk rating by Mamdani inference; the perceptron and its limit	Project experiments frozen
15	Natural language processing; classic AI systems	18, 19	Parsing ticket subjects; four systems in one file	—
16	Recent trends; responsible AI and its limits	20, 21	A tool-using agent, offline; auditing the escalation rules	Project report and presentation

G.2 Assessment Weights

The department's standard split: sessional 20% (three quizzes, two assignments and the project's interim deliverables), mid-term 30%, final 50%. The mid-term covers Chapters 1–8 — Parts I and II plus constraint satisfaction, adversarial search and planning. The final covers the whole course, weighted towards Chapters 9–21, which the mid-term does not reach.

⚠ Why Week 8 carries a chapter as well as the mid-term

Teaching Chapter 9 in the same week as the mid-term looks careless and is deliberate. Planning is the one chapter in Part III that introduces almost no new machinery: STRIPS progression search *is* the frontier of Chapters 4 and 5 applied to a new state space, and means–ends analysis is a short addition. So a light lecture is honest in a week when students are revising, and the chapter itself reinforces exactly the material the examination covers. Do not move the mid-term later to avoid this. Chapters 10–13 form a single logical development — representation, propositional inference, first-order inference, then a working rule engine — and splitting it across an examination is worse than doubling up a revision week.

⚠ If You Must Cut Something

Cut *depth*, never Chapters 4, 5 and 11. Search is the machinery that Chapters 7, 8 and 9 all reuse, and propositional inference is what makes Chapters 12 and 13 more than programming. If time is short: teach Chapter 6's genetic algorithm by demonstration rather than derivation, compress Chapter 10 to semantic networks and inheritance, name partial-order planning in Chapter 9 without developing it, and merge Chapters 19 and 20 into a single applications lecture.

Do not cut Chapter 17. It is the only place the course addresses the HEC 2017 outline's learning clause and the HEC 2023 outline's neural-network clause, and dropping it leaves a documented gap in the coverage table.

G.3 Dependency Map

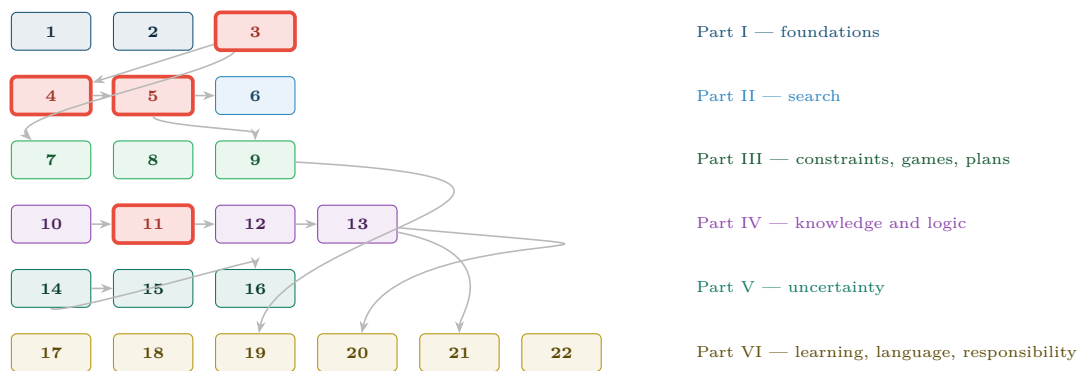


Figure G.1: Chapter dependencies. The outlined chapters (3, 4, 5 and 11) are the ones later chapters rely on most: the search toolkit is reused by constraint satisfaction, adversarial search and planning, and propositional inference underpins first-order logic and the rule engine.

G.4 Laboratory Sessions

Each week's three laboratory hours follow the same pattern: the chapter's *Try It Yourself* lab first (about one hour, from the runnable file in `code/`), then the chapter's applied review questions worked in pairs, then — from Week 11 — project time with the instructor circulating. Every lab runs on a standard Python 3.10 or later installation with NumPy; nothing else is required, and no lab needs a GPU or an internet connection. Chapters 10 to 13 and 18 to 21 use only the standard library.